

FM-Agent 自验证 Bug 清单（主文件： 契约不确定与可能有 Bug）

本文件是可操作的实现复核队列，仅保留“契约不确定”和“可能有 Bug”。SPEC 错误与推理误判已拆分到子文件。最终运行结果：457 个函数；MATCH 145、MISMATCH 312、ERROR 0；confirmed 262、not_confirmed 50。原唯一 `_split_into_blocks` ERROR 已通过删除旧 SPEC/INFO/结果后用原始验证器 resume，恢复为 MISMATCH/confirmed。原始函数与 prompt 未修改；报告仍是 PARTIAL / INCONCLUSIVE，因为预检测测试失败、旧索引混用和 trace 损坏问题尚未重跑消除。

过滤统计

过滤结果	数量
保留： 契约不确定	22
保留： 可能有 Bug	40
删除： 大概率不是 Bug 或影响较小	80
保留合计	62

原始全局分类统计（供参考）

四类结论	数量	文件
契约不确定	91	主文件
可能有 Bug	51	主文件
SPEC 错误	94	SPEC 错误子文件
推理误判	76	推理误判子文件
合计	312	每项恰好出现一次

分类口径

- **可能有 Bug**：probe 复现具体实现风险，但仍是候选，需确认真实调用前置条件。
 - **契约不确定**：差异可复现，但现有证据不足以决定应改实现还是改 SPEC。
 - **SPEC 错误**：生成 SPEC 添加了源码、调用者或文档未证明的格式、类型、异常或边界要求。
 - **推理误判**：Validator 未确认，或 probe 直接推翻 Reasoner 的行为描述/反例。
- 165 项沿用旧 `fm_agent/bug_list.md` 的逐条审计结论；147 项按本轮 Validator、trigger、probe 与 SPEC 证据重新分流。分类是复核队列，不是最终产品裁决。

主文件索引

编号	分类	Phase / 模块	条目	Validator
014	契约不确定	P1 / configuration	src--configure_llm-py-- apply_llm_settings_update	confirmed
028	契约不确定	P1 / configuration	src--configure_llm-py--update_env_text	confirmed
030	可能有Bug	P1 / configuration	src--configure_llm-py-- update_llm_settings_toml_text	confirmed
031	契约不确定	P1 / configuration	src--configure_llm-py--validate_base_url	confirmed
033	契约不确定	P1 / configuration	src--configure_llm-py--validate_llm_setting	confirmed
038	可能有Bug	P1 / environment	src--env_check-py--_check_oh_my_openagent	confirmed
045	契约不确定	P1 / git_utils	src--git-py--frozen_worktree	confirmed
054	可能有Bug	P2 / call_graph_edges	src--call_graph_edges-py-- _normalize_endpoint_label	confirmed
058	可能	P2 / call_graph_edges	src--call_graph_edges-py--load_call_edges	confirmed

编号	分类	Phase / 模块	条目	Validator
	有 Bug			
061	契约不确定	P2 / codegraph	src--languages--codegraph-py--CodeGraphExtractor::get_call_edges	confirmed
063	可能有 Bug	P2 / codegraph	src--languages--codegraph-py--_bare_function_name	confirmed
066	可能有 Bug	P2 / codegraph	src--languages--codegraph-py--_fqn_for	confirmed
070	契约不确定	P2 / codegraph	src--languages--codegraph-py--try_codegraph_init	confirmed
071	可能有 Bug	P2 / extraction	src--extract-py--_extract_func_name_brace	confirmed
073	可能有 Bug	P2 / extraction	src--extract-py--_extract_functions_indent	confirmed
076	可能有 Bug	P2 / extraction	src--extract-py--_strip_angle_brackets	confirmed
077	可能有 Bug	P2 / extraction	src--extract-py--extract_functions_from_file	confirmed
084	契约不确定	P2 / language_handlers	src--languages--erlang-py--ElpClient::_send	confirmed

编号	分类	Phase / 模块	条目	Validator
086	契约不确定	P2 / language_handlers	src--languages--erlang-py--ElpClient::close	confirmed
091	契约不确定	P2 / language_handlers	src--languages--erlang-py--_SourceIndex::build	confirmed
092	契约不确定	P2 / language_handlers	src--languages--erlang-py--_SourceIndex::position_to_offset	confirmed
097	可能有Bug	P2 / language_handlers	src--languages--erlang-py--_elp_argv	confirmed
098	可能有Bug	P2 / language_handlers	src--languages--erlang-py--_escape_component	confirmed
099	可能有Bug	P2 / language_handlers	src--languages--erlang-py--_function_id	confirmed
102	可能有Bug	P2 / language_handlers	src--languages--erlang-py--_project_fingerprint	confirmed
106	可能有Bug	P2 / language_handlers	src--languages--erlang-py--batch_extract	not_confirmed
107	可能有Bug	P2 / language_handlers	src--languages--erlang-py--call_edges	confirmed
114	可能	P2 / language_handlers	src--languages--python-py--batch_extract	confirmed

编号	分类	Phase / 模块	条目	Validator
	有 Bug			
128	可能有 Bug	P3 / pipeline_setup	src--pipeline_setup-py--_collapse_phases_to_one	confirmed
131	可能有 Bug	P3 / pipeline_setup	src--pipeline_setup-py--_domain_context_complete	confirmed
132	可能有 Bug	P3 / pipeline_setup	src--pipeline_setup-py--_ensure_source_files_in_phases	confirmed
139	可能有 Bug	P3 / pipeline_setup	src--pipeline_setup-py--_run_generate_phases	confirmed
146	可能有 Bug	P3 / plugins	src--plugin-py--run_plugin_command	confirmed
157	契约不确定	P4 / llm_client	src--llm_client-py--_matches_inject_target	confirmed
159	可能有 Bug	P4 / llm_client	src--llm_client-py--_parse_json_response	confirmed
164	可能有 Bug	P4 / opencode_trace	src--opencode_trace-py--_copy_opencode_output	confirmed
165	契约不确定	P4 / opencode_trace	src--opencode_trace-py--_opencode_env	confirmed

编号	分类	Phase / 模块	条目	Validator
166	可能有Bug	P4 / opencode_trace	src--opencode_trace-py--_opencode_log_path	confirmed
167	可能有Bug	P4 / opencode_trace	src--opencode_trace-py--_opencode_trace_path	confirmed
171	可能有Bug	P4 / opencode_trace	src--opencode_trace-py--_wait_opencode_process	confirmed
177	可能有Bug	P4 / opencode_trace	src--opencode_trace-py--run_opencode_traced	confirmed
182	可能有Bug	P4 / trace_writer	src--trace_writer-py--append_event	confirmed
188	契约不确定	P5 / batch_prompts	src--generate_batch_prompts-py--main	not_confirmed
192	可能有Bug	P5 / file_utils	src--file_utils-py--_get_incomplete_verification_files	confirmed
193	可能有Bug	P5 / file_utils	src--file_utils-py--_get_phase_files	confirmed
198	可能有Bug	P5 / file_utils	src--file_utils-py--_json_file_is_valid	confirmed
202	契约不确定	P5 / parser	src--parser-py--format_info_for_reasoner	confirmed

编号	分类	Phase / 模块	条目	Validator
218	契约不确定	P6 / incremental_pipeline	src--incremental_reasoner-py--_codegraph_legacy_coverage	confirmed
222	契约不确定	P6 / incremental_pipeline	src--incremental_reasoner-py--_collect_changed_functions::_is_workspace_file	confirmed
225	可能有Bug	P6 / incremental_pipeline	src--incremental_reasoner-py--_llm_check_caller_info_update	confirmed
227	契约不确定	P6 / incremental_pipeline	src--incremental_reasoner-py--_llm_select_json	confirmed
238	契约不确定	P6 / incremental_pipeline	src--incremental_reasoner-py--_update_specs_for_intent::_plan_spec_update	confirmed
239	契约不确定	P6 / incremental_pipeline	src--incremental_reasoner-py--_update_specs_for_intent::_reconcile_caller	confirmed
244	可能有Bug	P6 / incremental_pipeline	src--incremental_reasoner-py--check_last_run_existence	confirmed
248	可能有Bug	P6 / orchestration	dashboard-py--State::_ingest_event	confirmed
252	可能有Bug	P6 / orchestration	dashboard-py--State::scan_bugs	confirmed

编号	分类	Phase / 模块	条目	Validator
270	契约不确定	P6 / orchestration	main-py--_clean_previous_run	confirmed
272	可能有 Bug	P6 / reasoner	src--reasoner-py--_compute_brace_depth_per_line	confirmed
284	可能有 Bug	P6 / scope_analysis	src--scope-py--_llm_rerank	confirmed
291	契约不确定	P6 / scope_analysis	src--scope-py--_score_class	confirmed
301	可能有 Bug	P6 / topdown_layers	src--generate_topdown_layers-py--_get_call_regex	confirmed
310	可能有 Bug	P6 / verification	src--verification-py--_validate_single_bug	not_confirmed

契约不确定（22 项）

阶段 1 — 初始化与配置

模块：configuration

FMA-MISMATCH-014 — src--configure_llm-py--apply_llm_settings_update

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator： confirmed；尝试次数 1
- Phase / 模块： Phase 1 — Initialization & Configuration / configuration；原源码 src/configure_llm.py
- 证据： 抽取源码 · SPEC · INFO · Logic · Validator 结果 · probe · 详细报告

冲突摘要

- **SPEC claim:** `toml_path` 处的文件被覆盖，使得 `updates` 中的每个键的值都写入到 `[llm]` 节中相应的字段路径，而所有其他节、字段以及更新字段之外的整个 TOML 结构均保持不变。在写入之前，原始文件内容被复制到一个与 `toml_path` 相邻的带时间戳后缀的备份文件中。当创建了备份时，返回备份文件 `Path`；当未创建备份时，返回 `None`。当 `toml_path` 不指向现有文件，或者更新后的 TOML 内容无法被标准库解析器解析为有效的 TOML 时，抛出 `ConfigWizardError`。
- **Actual behavior:** 函数执行后，以下情况之一成立：(1) 函数抛出了 `ConfigWizardError`（消息为 `"fm-agent.toml not found at {toml_path}; refusing to guess a new project config."` 或 `"Generated fm-agent.toml is invalid TOML."`）。此时，位于 `toml_path` 的文件保持不变，且未创建备份文件。(2) 函数正常返回了值 `r`。那么，位于 `toml_path` 的文件已被原子地替换为将给定的 `updates` 应用于原始文件的 `[llm]` 节后得到的 TOML 文本，同时保留文档的所有其他部分。如果 `r` 是一个 `Path` 对象，则在路径 `r`（与 `toml_path` 相邻，带时间戳后缀的名称）存在原始文件内容的备份副本；位于 `r` 的内容等于调用前原始文件的内容。如果 `r` 是 `None`，则未创建备份副本。

正式地，令 `orig` 为函数入口处 `toml_path` 的内容，令 `updated = update_llm_settings_toml_text(orig, updates)`。令 `F_before` 和 `F_after` 分别为紧接调用之前和之后的文件系统状态（将路径映射到内容）。

- 如果函数抛出 `ConfigWizardError`： `F_after = F_before`（未更改或创建文件）。
- 如果函数正常返回 `r`： `F_after(toml_path) = updated (r = None p adjacent to toml_path, F_after(p) = F_before(p) no new timestamp-suffixed backup file exists) (isinstance(r, Path) r is a path adjacent to toml_path r did not exist in F_before F_after(r) = orig all other files unchanged)`。
- **Code evidence:** Line 6: `if not toml_text`:
- **Trigger condition:** 代码针对现有的空文件抛出 `ConfigWizardError`，并声称找不到该文件，这违反了规范。规范仅允许在 `toml_path` 不指向现有文件或更新后的 TOML 内容无法解析时抛出 `ConfigWizardError`。空文件存在，因此代码不应抛出“文件未找到”错误，而应继续生成更新后的 TOML。
- **Validator trigger:** `apply_llm_settings_update` 针对现有的空 `fm-agent.toml` 文件抛出 `ConfigWizardError`，因为 `_read_text_if_exists` 对空文件返回 `"`，而 `'if not toml_text'` 守卫将其与缺失文件同等对待。
- **Probe stdout:** CONFIRMED -- 实际： `ConfigWizardError` 引发： `fm-agent.toml` 未在 `/tmp/tmp5zk0ey_k/fm-agent.toml` 找到；拒绝猜测新项目配置。| 预期：对于现有的空文件，函数应正常进行。

► SPEC sidecar（完整）

```
{
  "signature": "apply_llm_settings_update(updates: dict[str, str], toml_path: Path) -> Path | None",
  "pre_condition": "updates 是一个非空字典，其键是 [llm] 节中由点分隔的字段路径，其值为非空字符串或布尔值，代表新的配置值。toml_path 标识一个存在、可读、可写的文件，其内容是标准库 TOML 解析器可解析的有效 TOML。",
  "post_condition": "toml_path 处的文件被覆盖，使得 updates 中的每个键的值都写入到 [llm] 节中相应的字段路径，而所有其他节、字段以及更新字段之外的整个 TOML 结构均保持不变。在写入之前，原始文件内容被复制到一个与 toml_path 相邻的带时间戳后缀的备份文件中。当创建了备份时，返回备份文件 Path；当未创建备份时，返回 None。当 toml_path 不指向现有文件，或者更新后的 TOML 内容无法被标准库解析器解析为有效的 TOML 时，抛出 ConfigWizardError。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "_read_text_if_exists",
      "signature": "_read_text_if_exists(path: Path) -> str",

```

```

    "pre_condition": "path 是一个标识文件系统位置的 Path 对象。",
    "post_condition": "返回 path 处文件的 UTF-8 解码文本内容。当 path 处不存在文件时返回空字符串。"
  },
  {
    "name": "update_llm_settings_toml_text",
    "signature": "update_llm_settings_toml_text(toml_text: str, updates: dict[str, str]) -> str",
    "pre_condition": "toml_text 是一个非空字符串，包含包含可解析 [llm] 节的有效 TOML。updates 是一个非空字典，其键是 [llm] 节中由点分隔的字段路径，其值为字符串。",
    "post_condition": "返回 TOML 文本，其中 updates 中的每个字段路径键的值已替换为 [llm] 节中对应的更新值。所有其他节、字段以及更新字段之外的 TOML 结构在返回的文本中均保持不变。"
  },
  {
    "name": "backup_file",
    "signature": "backup_file(path: Path) -> Path | None",
    "pre_condition": "path 标识一个存在、可读的文件。",
    "post_condition": "将 path 处的文件复制到与 path 相邻且带时间戳后缀名称的新文件。当复制成功时返回新备份文件的 Path，当无法创建备份时返回 None。"
  },
  {
    "name": "atomic_write",
    "signature": "atomic_write(path: Path, content: str) -> None",
    "pre_condition": "path 是一个可写的文件系统位置。content 是一个字符串。",
    "post_condition": "path 处的文件被 content 覆盖，使得文件在任何时刻都不处于部分写入的状态。调用后，path 处的文件内容等于 content。"
  }
]
}

```

► Logic verification result（完整）

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/apply_llm_settings_update.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "toml_path 处的文件被覆盖，使得 updates 中的每个键的值都写入到 [llm] 节中相应的字段路径，而所有其他节、字段以及更新字段之外的整个 TOML 结构均保持不变。在写入之前，原始文件内容被复制到一个与 toml_path 相邻的带时间戳后缀的备份文件中。当创建了备份时，返回备份文件 Path；当未创建备份时，返回 None。当 toml_path 不指向现有文件，或者更新后的 TOML 内容无法被标准库解析器解析为有效的 TOML 时，抛出 ConfigWizardError。",
    "actual_behavior": "函数执行后，以下情况之一成立：(1) 函数抛出了 `ConfigWizardError`（消息为 `\"fm-agent.toml not found at {toml_path}; refusing to guess a new project config.\"` 或 `\"Generated fm-agent.toml is invalid TOML.\"`）。此时，位于 `toml_path` 的文件保持不变，且未创建备份文件。(2) 函数正常返回了值 `r`。那么，位于 `toml_path` 的文件已被原子地替换为将给定的 `updates` 应用于原始文件的 `[llm]` 节后得到的 TOML 文本，同时保留文档的所有其他部分。如果 `r` 是一个 `Path` 对象，则在路径 `r`（与 `toml_path` 相邻，带时间戳后缀的名称）存在原始文件内容的备份副本；位于 `r` 的内容等于调用前原始文件的内容。如果 `r` 是 `None`，则未创建备份副本。\\n\\n正式地，令 `orig` 为函数入口处 `toml_path` 的内容，令 `updated = update_llm_settings_toml_text(orig, updates)`。令 `F_before` 和 `F_after` 分别为紧接调用之前和之后的文件系统状态（将路径映射到内容）。\\n\\n- 如果函数抛出 `ConfigWizardError`：\\n `F_after = F_before`（未更改或创建文件）。\\n\\n- 如果函数正常返回 `r`：\\n `F_after(toml_path) = updated` \\n `(r = None & p adjacent to toml_path, F_after(p) = F_before(p) no new timestamp-suffixed backup file exists)` \\n `(isinstance(r, Path) & r is a path adjacent to toml_path & r did not exist in F_before & F_after(r) = orig all other files unchanged)`。",
    "code_evidence": "Line 6: if not toml_text:",
    "trigger_condition": "代码针对现有的空文件抛出 ConfigWizardError，并声称找不到该文件，这违反了规范。规范仅允许在 toml_path 不指向现有文件或更新后的 TOML 内容无法解析时抛出 ConfigWizardError。空文件存在，因此代码不应抛出“文件未找到”错误，而应继续生成更新后的 TOML。"
  }
}

```

► Bug Validator result（完整）

```

{
  "id": "src--configure_llm-py--apply_llm_settings_update",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/apply_llm_settings_update.py",
  "function_name": "apply_llm_settings_update",
  "confirmation_status": "confirmed",

```

```
"attempts": 1,
"probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--apply_llm_settings_update.py",
"detail_file": "fm_agent/bug_validation/src--configure_llm-py--apply_llm_settings_update.md",
"probe_stdout": "CONFIRMED -- 实际: ConfigWizardError 引发: fm-agent.toml 未在 /tmp/tmp5zk0ey_k/fm-agent.toml 找到; 拒绝猜测新项目配置。| 预期: 对于现有的空文件, 函数应正常进行。\\n",
"trigger_summary": "apply_llm_settings_update 针对现有的空 fm-agent.toml 文件抛出 ConfigWizardError, 因为 _read_text_if_exists 对空文件返回 '', 而 'if not toml_text' 守卫将其与缺失文件同等对待。"
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌, 使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T15:52:57.153471Z	5017
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:54:04.055816Z	9550

事件 1 — [llm_ad914ce7efae43d2a4f031f367e53fcc](#)

► parsed_json (完整)

```
{
  "post_condition": "函数执行后, 以下情况之一成立: (1) 函数抛出了 `ConfigWizardError` (消息为 `\"fm-agent.toml not found at {toml_path}; refusing to guess a new project config.\"` 或 `\"Generated fm-agent.toml is invalid TOML.\"`)。此时, 位于 `toml_path` 的文件保持不变, 且未创建备份文件。(2) 函数正常返回了值 `r`。那么, 位于 `toml_path` 的文件已被原子地替换为将给定的 `updates` 应用于原始文件的 `[llm]` 节后得到的 TOML 文本, 同时保留文档的所有其他部分。如果 `r` 是一个 `Path` 对象, 则在路径 `r` (与 `toml_path` 相邻, 带时间戳后缀的名称) 存在原始文件内容的备份副本; 位于 `r` 的内容等于调用前原始文件的内容。如果 `r` 是 `None`, 则未创建备份副本。\\n\\n正式地, 令 `orig` 为函数入口处 `toml_path` 的内容, 令 `updated = update_llm_settings_toml_text(orig, updates)`。令 `F_before` 和 `F_after` 分别为紧接调用之前和之后的文件系统状态 (将路径映射到内容)。\\n\\n- 如果函数抛出 `ConfigWizardError`: \\n  `F_after = F_before` (未更改或创建文件)。\\n\\n- 如果函数正常返回 `r`: \\n  `F_after(toml_path) = updated` \\n  `(r = None => \\forall adjacent to toml_path, F_after(p) = F_before(p) \\wedge no new timestamp-suffixed backup file exists)` \\n  `(isinstance(r, Path) => r is a path adjacent to toml_path \\wedge r did not exist in F_before \\wedge F_after(r) = orig \\wedge all other files unchanged)`。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_ad914ce7efae43d2a4f031f367e53fcc_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_ad914ce7efae43d2a4f031f367e53fcc_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_ad914ce7efae43d2a4f031f367e53fcc_response.txt)

事件 2 — [llm_1937feee8eb4459ea24da1dc27796dc1](#)

► parsed_json (完整)

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_1937feee8eb4459ea24da1dc27796dc1_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_1937feee8eb4459ea24da1dc27796dc1_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_1937feee8eb4459ea24da1dc27796dc1_response.txt) **复核动作**

- 先由维护者确认预期契约, 再决定修改实现或重写 SPEC; 当前不应直接登记为真实 Bug。

FMA-MISMATCH-028 — [src--configure_llm-py--update_env_text](#)

- 四类结论: 契约不确定

- **分类依据：** probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator：** [confirmed](#)；attempts [1](#)
- **Phase / 模块：** Phase 1 — Initialization & Configuration / [configuration](#)；原源码 [src/configure_llm.py](#)
- **证据：** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim：** 返回一个字符串，其中 (1) 恰好一行将 ENV_SECRET_KEY 赋值为 api_key，其位置为已存在的 ENV_SECRET_KEY 行出现处，若不存在则追加到末尾；(2) 那些在去除可选的 'export' 前缀后，其键属于一组预定义的传统 LLM 配置键的行，将被排除在输出之外；(3) 其他所有行——空行、注释或赋值行——按其原始相对顺序逐字出现。当已有的 ENV_SECRET_KEY 行带有 'export' 前缀时，重写的行保留该前缀。当追加键且输入不包含任何非空、非注释行时，头部注释行会出现在追加的键行之前。
- **实际行为：** 函数返回一个字符串结果，通过以下方式转换输入文本（.env 文件内容）得到。令 $L = \text{text.splitlines}(keepends=True)$ 。按顺序处理每一行 $\in L$ ：令 $s = \text{.lstrip}()$ 。若 s 为空或 $s.startswith('#')$ ，则将复制到输出。否则，尝试解析键值赋值。确定前导空白和可选的 'export' 前缀：令 $\text{leading} = [: \text{len}() - \text{len}(s)]$ ；若 $\text{_ENV_EXPORT_PREFIX_RE.match}(s)$ 成功，则令 $\text{export_prefix} = \text{leading} + \text{match.group}()$ ，令 $\text{working} = \text{leading} + s[\text{match.end}():]$ ；否则 $\text{export_prefix} = ''$ ， $\text{working} = s$ 。在第一个 '=' 处拆分 working： $\text{key_candidate}, \text{sep}, _ = \text{working.partition}('=')$ 。若 $\text{sep} == ''$ （没有 '='），将复制到输出。否则，令 $\text{env_key} = \text{key_candidate.strip}()$ 。若 $\text{env_key} == \text{ENV_SECRET_KEY}$ ，输出行 $f'\{\text{export_prefix}\}\text{ENV_SECRET_KEY}=\{\text{api_key}\}\n'$ 并设置标志 $\text{key_written} = \text{True}$ 。若 env_key 在 ENV_LEGACY_LLM_KEYS 中，则忽略该行（不复制）。否则，将原样复制。处理完所有行后，若 key_written 为 False，追加秘密行：若输出列表为空，首先预置两行头部注释行：'# fm-agent secrets gitignored, do not commit.\n'、'# Only the LLM API key belongs here.\n'；然后追加行 $\text{ENV_SECRET_KEY}=\{\text{api_key}\}\n$ 。否则，若输出中最后一行的去除前后空白后内容非空，则先追加一个换行符 '\n'，然后追加秘密行；若最后一行已经是空白（仅包含空白字符），则仅追加秘密行。最后，返回所有输出行的拼接结果。从输入中复制的行的相对顺序得以保留。正式地： $\exists \text{key_written} \in \{\text{true}, \text{false}\}$ 使得 $\text{result} = \text{''.join}(N)$ ，其中 N 是一个字符串列表且满足以下条件：(1) 对每个 i ， $_ = L[i]$ ，令 $\text{c}(i)$ 为该行的类别：若 $\text{.strip}() == ''$ 则为 'blank'；若 $\text{.lstrip}().startswith('#')$ 则为 'comment'；剥离前导空白及可能的 'export' 前缀后不含 '=' 则为 'no_eq'；若提取的键等于 ENV_SECRET_KEY 则为 'secret'；若提取的键 $\in \text{ENV_LEGACY_LLM_KEYS}$ 则为 'legacy'；否则为 'other'。(2) 序列 N 由每个类别非 'legacy' 的 i 对应的字符串 $\text{v}(i)$ 构成， $\text{v}(i)$ 定义为：若 $\text{c}(i)$ 为 'blank'、'comment'、'no_eq' 或 'other'，则 $\text{v}(i) = _$ ；若 $\text{c}(i)$ 为 'secret'，则 $\text{v}(i) = \text{export_prefix} + \text{ENV_SECRET_KEY} + '=' + \text{api_key} + '\n'$ 。(3) 若 $\text{key_written} = \text{true}$ ，则 $N = [\text{v}(i) \text{ 对于所有 } \text{c}(i) \neq \text{'legacy' 的 } i]$ 。(4) 若 $\text{key_written} = \text{false}$ ，则令 $M = [\text{v}(i) \text{ 对于所有 } \text{c}(i) \neq \text{'legacy' 的 } i]$ ；若 M 为空， $N = [\text{'# fm-agent secrets gitignored, do not commit.\n'}, \text{'# Only the LLM API key belongs here.\n'}, \text{ENV_SECRET_KEY} + '=' + \text{api_key} + '\n']$ ；否则若 $M[-1].strip() != ''$ ， $N = M + [\text{'\n'}, \text{ENV_SECRET_KEY} + '=' + \text{api_key} + '\n']$ ；否则 $N = M + [\text{ENV_SECRET_KEY} + '=' + \text{api_key} + '\n']$ 。(5) N 中 $\text{v}(i)$ 的顺序与递增的 i 一致。
- **代码证据：** 第 21 行：if env_key == ENV_SECRET_KEY: 第 22 行：new_lines.append(f'{export_prefix}{ENV_SECRET_KEY}={api_key}\n') 第 23 行：key_written = True 第 24 行：continue
- **触发条件：** 代码会替换每一次出现的 ENV_SECRET_KEY，当输入包含多个时，会为秘密密钥产生多行输出。而规范要求输出中该密钥恰好只有一行。
- **Validator 触发：** 当输入包含多个 ENV_SECRET_KEY 行时，函数会为每一次出现输出一个替换，违反规范中恰好一行该密钥的要求。
- **Probe stdout：** CONFIRMED — LLM_API_KEY= 出现 2 次（预期恰好 1 次）。实际输出：'LLM_API_KEY=new-test-api-key\nLLM_API_KEY=new-test-api-key\nOTHER_VAR=keep_me\n'

► SPEC sidecar（完整）

```
{
  "signature": "update_env_text(text: str, api_key: str) -> str",
  "pre_condition": "text 是一个字符串，其内容遵循 .env 文件约定——每行是空行、以 '#' 开头的注释，或是可选地以 'export' 为前缀的 KEY=VALUE 赋值；api_key 是一个非空字符串。",
  "post_condition": "text 是一个字符串，其内容遵循 .env 文件约定——每行是空行、以 '#' 开头的注释，或是可选地以 'export' 为前缀的 KEY=VALUE 赋值；api_key 是一个非空字符串。"
```



```
"post_condition": "返回一个字符串，其中（1）恰好 一行将 ENV_SECRET_KEY 赋值为 api_key，其位置为已存在的 ENV_SECRET_KEY 行出现处，若不存在则追加到末尾；（2）那些在去除可选的 'export ' 前缀后，其键属于一组预定义的传统 LLM 配置键的行，将被排除在输出之外；（3）其他所有行——空行、注释或赋值行——按其原始相对顺序逐字出现。当已有的 ENV_SECRET_KEY 行带有 'export ' 前缀时，重写的行保留该前缀。当追加键且输入不包含任何非空、非注释行时，头部注释行会出现在追加的键行之前。"
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/update_env_text.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个字符串，其中（1）恰好 一行将 ENV_SECRET_KEY 赋值为 api_key，其位置为已存在的 ENV_SECRET_KEY 行出现处，若不存在则追加到末尾；（2）那些在去除可选的 'export ' 前缀后，其键属于一组预定义的传统 LLM 配置键的行，将被排除在输出之外；（3）其他所有行——空行、注释或赋值行——按其原始相对顺序逐字出现。当已有的 ENV_SECRET_KEY 行带有 'export ' 前缀时，重写的行保留该前缀。当追加键且输入不包含任何非空、非注释行时，头部注释行会出现在追加的键行之前。",
    "actual_behavior": "函数返回一个字符串结果，通过以下方式转换输入文本（.env 文件内容）得到。令 L = text.splitlines(keepends=True)。按顺序处理每一行 ∈ L：令 s = .rstrip()。若 s 为空或 s.startswith('#')，则将 复制到输出。否则，尝试解析键值赋值。确定前导空白和可选的 'export ' 前缀：令 leading = [:len()-len(s)]；若 _ENV_EXPORT_PREFIX_RE.match(s) 成功，则令 export_prefix = leading + match.group()，令 working = leading + s[match.end():]；否则 export_prefix = ''，working = 。在第一个 '=' 处拆分 working：key_candidate, sep, _ = working.partition('='）。若 sep == ''（没有 '='），将 复制到输出。否则，令 env_key = key_candidate.strip()。若 env_key == ENV_SECRET_KEY，输出行 f\"{export_prefix}ENV_SECRET_KEY={api_key}\\n\\n\" 并设置标志 key_written = True。若 env_key 在 ENV_LEGACY_LLM_KEYS 中，则忽略该行（不复制）。否则，将 原样复制。处理完所有行后，若 key_written 为 False，追加秘密行：若输出列表为空，首先预置两行头部注释行：'# fm-agent secrets gitignored, do not commit.\\n'、'# Only the LLM API key belongs here.\\n'；然后追加行 ENV_SECRET_KEY={api_key}\\n。否则，若输出中最后一行的去除前后空白后内容非空，则先追加一个换行符 '\\n'，然后追加秘密行；若最后一行已经是空白（仅包含空白字符），则仅追加秘密行。最后，返回所有输出行的拼接结果。从输入中复制的行的相对顺序得以保留。正式地：∃ key_written ∈ {true, false} 使得 result = ''.join(N)，其中 N 是一个字符串列表且满足以下条件：（1）对每个 i， = L[i]，令 c(i) 为该行的类别：若 .strip() == '' 则为 'blank'；若 .rstrip().startswith('#') 则为 'comment'；剥离前导空白及可能的 'export ' 前缀后不含 '=' 则为 'no_eq'；若提取的键等于 ENV_SECRET_KEY 则为 'secret'；若提取的键 ∈ ENV_LEGACY_LLM_KEYS 则为 'legacy'；否则为 'other'。（2）序列 N 由每个类别非 'legacy' 的 i 对应的字符串 v(i) 构成，v(i) 定义为：若 c(i) 为 'blank'、'comment'、'no_eq' 或 'other'，则 v(i)=；若 c(i) 为 'secret'，则 v(i)=export_prefix + ENV_SECRET_KEY + '=' + api_key + '\\n'。（3）若 key_written = true，则 N = [v(i) 对于所有 c(i)≠'legacy' 的 i]。（4）若 key_written = false，则令 M = [v(i) 对于所有 c(i)≠'legacy' 的 i]；若 M 为空，N = ['# fm-agent secrets gitignored, do not commit.\\n', '# Only the LLM API key belongs here.\\n', ENV_SECRET_KEY + '=' + api_key + '\\n']；否则若 M[-1].strip() != ''，N = M + ['\\n', ENV_SECRET_KEY + '=' + api_key + '\\n']；否则 N = M + [ENV_SECRET_KEY + '=' + api_key + '\\n']。（5）N 中 v(i) 的顺序与递增的 i 一致。",
    "code_evidence": "Line 21: if env_key == ENV_SECRET_KEY:\\nLine 22: new_lines.append(f\"{export_prefix}{ENV_SECRET_KEY}={api_key}\\n\\n\")\\nLine 23: key_written = True\\nLine 24: continue",
    "trigger_condition": "代码会替换每一次出现的 ENV_SECRET_KEY，当输入包含多个时，会为秘密密钥产生多行输出。而规范要求输出中该密钥恰好只有一行。"
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--configure_llm-py--update_env_text",
  "source_file": "src/configure_llm.py",
  "function_name": "update_env_text",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--update_env_text.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--update_env_text.md",
  "probe_stdout": "CONFIRMED - LLM_API_KEY= 出现 2 次（预期恰好 1 次）。实际输出：\\n'LLM_API_KEY=new-test-api-
```

```
key\\nLLM_API_KEY=new-test-api-key\\nOTHER_VAR=keep_me\\n",
"trigger_summary": "当输入包含多个 ENV_SECRET_KEY 行时，函数会为每一次出现输出一个替换，违反规范中恰好一行该密钥的要求。"
}
```

► Reasoner 流程（3 条事件）

共 3 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	format_error	1	0/1	2026-07-27T15:59:45.503363Z	3839
2	generate_block_post_condition	success	2	0/1	2026-07-27T16:00:36.665004Z	3648
3	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:01:02.271196Z	7307

事件 1 — [llm_82c46a339d0f4f78ab682daaa111343f](#)

- parse_error: LLM response contains multiple JSON values
- 载荷: [system_prompt](#) · [user_prompt](#) · [assistant_output](#)

事件 2 — [llm_ee09c09f74de4b27be95a14cb98021f8](#)

► parsed_json（完整）

- 载荷: [\[system_prompt\]](#)

(fm_agent/trace/payloads/llm_ee09c09f74de4b27be95a14cb98021f8_message_00_system.txt) · [\[user_prompt\]](#)
(fm_agent/trace/payloads/llm_ee09c09f74de4b27be95a14cb98021f8_message_01_user.txt) · [\[assistant_output\]](#)
(fm_agent/trace/payloads/llm_ee09c09f74de4b27be95a14cb98021f8_message_02_assistant.txt) · [\[user_prompt\]](#)
(fm_agent/trace/payloads/llm_ee09c09f74de4b27be95a14cb98021f8_message_03_user.txt) · [\[assistant_output\]](#)
(fm_agent/trace/payloads/llm_ee09c09f74de4b27be95a14cb98021f8_response.txt)

事件 3 — [llm_18c54f5bdfbf44d08c8728b123988441](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "text = `FM_AGENT_SECRET_KEY=old\\nOTHER=val\\nFM_AGENT_SECRET_KEY=old2\\n\\n\\napi_key = `newkey`",
  "offending_statements": "Line 21: if env_key == ENV_SECRET_KEY:\\nLine 22: new_lines.append(f`{export_prefix}{ENV_SECRET_KEY}={api_key}\\n\\n`)\\nLine 23: key_written = True\\nLine 24: continue",
  "reason": "代码替换每个出现的 ENV_SECRET_KEY，当输入包含多个时会产生多行输出。规约要求输出中该键恰好只有一行。"
}
```

- 载荷: [\[system_prompt\]](#)

(fm_agent/trace/payloads/llm_18c54f5bdfbf44d08c8728b123988441_message_00_system.txt) · [\[user_prompt\]](#)
(fm_agent/trace/payloads/llm_18c54f5bdfbf44d08c8728b123988441_message_01_user.txt) · [\[assistant_output\]](#)
(fm_agent/trace/payloads/llm_18c54f5bdfbf44d08c8728b123988441_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: `confirmed`; attempts 1
- **Phase / 模块**: Phase 1 — Initialization & Configuration / `configuration`; 原源码 `src/configure_llm.py`
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: 当 url 不是语法上有效的绝对 URL，且其 scheme 既不是 'http' 也不是 'https' 时，应抛出 ConfigWizardError。当 url 是语法上有效的绝对 URL，且 scheme 为 'http' 或 'https' 时，返回而不产生任何效果。
- **Actual behavior**: 如果函数正常返回（未抛出异常），则解析后的 URL 的 scheme 为 'http' 或 'https' 且 network location（netloc）非空。否则，抛出 ConfigWizardError，其消息指示 URL 无效。形式化表示：(url: str, url " url == url.strip() (after execution: (raised_exception urlparse(url).scheme {'http','https'}) urlparse(url).netloc " (raised_exception raised_exception is ConfigWizardError raised_exception.args[0] starts with 'Base URL must be an absolute http(s) URL, got:')))
- **Code evidence**: Line 2: parsed = urlparse(url) Line 3: if parsed.scheme not in ("http", "https") or not parsed.netloc:
- **Trigger condition**: 代码仅检查 scheme 是否为 http/https 且 netloc 非空，但未验证 netloc 是否构成语法上有效的 authority（例如，端口必须是数字，主机名不能包含空格）。像 'http://example.com:abc'（端口含字母）这样的 URL 根据 RFC 3986 并非语法上有效，但代码却接受它，因为 urlparse 将 netloc 设置为 'example.com:abc'，该值非空。SPEC 要求对任何非语法有效的绝对 URL 抛出 ConfigWizardError，因此该输入违反了 SPEC 规定。
- **Validator trigger**: 带非数字端口的 URL（http://example.com:abc）通过了 urlparse 的 netloc 检查，但根据 RFC 3986 语法上无效；SPEC 要求应抛出 ConfigWizardError。
- **Probe stdout**: CONFIRMED — validate_base_url 接受了一个无效 URL（url='http://example.com:abc'），但 SPEC 要求抛出 ConfigWizardError

► SPEC sidecar（完整）

```
{
  "signature": "validate_base_url(url: str) -> None",
  "pre_condition": "url 是一个非空字符串，且已移除首尾空白字符。",
  "post_condition": "当 url 不是语法上有效的绝对 URL，且其 scheme 既不是 'http' 也不是 'https' 时，抛出 ConfigWizardError。当 url 是语法上有效的绝对 URL 且 scheme 为 'http' 或 'https' 时，返回而不产生任何效果。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/validate_base_url.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "当 url 不是语法上有效的绝对 URL，且其 scheme 既不是 'http' 也不是 'https' 时，抛出 ConfigWizardError。当 url 是语法上有效的绝对 URL，且 scheme 为 'http' 或 'https' 时，返回而不产生任何效果。",
```

```
"actual_behavior": "如果函数正常返回（未抛出异常），则解析后的 URL 的 scheme 为 'http' 或 'https' 且 network location(netloc) 非空。否则，抛出 ConfigWizardError，其消息指示 URL 无效。形式化表示：( url: str, url != url.strip() (after execution: (raised_exception urlparse(url).scheme {'http','https'} urlparse(url).netloc '')) (raised_exception raised_exception is ConfigWizardError raised_exception.args[0] starts with 'Base URL must be an absolute http(s) URL, got:')))",
"code_evidence": "Line 2: parsed = urlparse(url)\nLine 3: if parsed.scheme not in ('http',\n'https') or not parsed.netloc:",
"trigger_condition": "代码仅检查 scheme 是否为 http/https 且 netloc 非空，但未验证 netloc 是否构成语法上有效的 authority（例如，端口必须是数字，主机名不能包含空格）。像 'http://example.com:abc'（端口含字母）这样的 URL 根据 RFC 3986 并非语法上有效，但代码却接受它，因为 urlparse 将 netloc 设置为 'example.com:abc'，该值非空。SPEC 要求对任何非语法有效的绝对 URL 抛出 ConfigWizardError，因此该输入违反了 SPEC 规定。"
}
```

► Bug Validator result（完整）

```
{
  "id": "src--configure_llm-py--validate_base_url",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/validate_base_url.py",
  "function_name": "validate_base_url",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--validate_base_url.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--validate_base_url.md",
  "probe_stdout": "CONFIRMED - validate_base_url 接受了一个无效 URL (url='http://example.com:abc')，但 SPEC 要求抛出 ConfigWizardError\n",
  "trigger_summary": "带非数字端口的 URL (http://example.com:abc) 通过了 urlparse 的 netloc 检查，但根据 RFC 3986 语法上无效；SPEC 要求应抛出 ConfigWizardError。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:30:04.731168Z	1122
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:30:19.618859Z	3438

事件 1 — [llm_7d13764a8c8e488087cd3c4a95899cc3](#)

► parsed_json（完整）

```
{
  "post_condition": "如果函数正常返回（未抛出异常），则解析后的 URL 的 scheme 为 'http' 或 'https' 且 network location(netloc) 非空。否则，抛出 ConfigWizardError，其消息指示 URL 无效。形式化表示：(∀ url: str, url ≠ '' ∧ url == url.strip() ⇒ (after execution: (¬raised_exception ⇒ urlparse(url).scheme ∈ {'http','https'} ∧ urlparse(url).netloc ≠ '')) ∧ (raised_exception ⇒ raised_exception is ConfigWizardError ∧ raised_exception.args[0] starts with 'Base URL must be an absolute http(s) URL, got:')))"
}
```

- 载荷：[system_prompt]

(fm_agent/trace/payloads/llm_7d13764a8c8e488087cd3c4a95899cc3_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_7d13764a8c8e488087cd3c4a95899cc3_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_7d13764a8c8e488087cd3c4a95899cc3_response.txt)

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "http://example.com:abc",
  "offending_statements": "Line 2: parsed = urlparse(url)\nLine 3: if parsed.scheme not in (\"http\", \"https\") or not parsed.netloc:",
  "reason": "代码仅检查 scheme 是否为 http/https 且 netloc 非空,但未验证 netloc 是否构成语法上有效的 authority (例如,端口必须是数字,主机名不能包含空格)。像 'http://example.com:abc' (端口含字母) 这样的 URL 根据 RFC 3986 并非语法上有效,但代码却接受它,因为 urlparse 将 netloc 设置为 'example.com:abc',该值非空。SPEC 要求对任何非语法有效的绝对 URL 抛出 ConfigWizardError,因此该输入违反了 SPEC 规定。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_6d2f29f9c232497fa8688b3fa7a5d302_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_6d2f29f9c232497fa8688b3fa7a5d302_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_6d2f29f9c232497fa8688b3fa7a5d302_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-033 — src--configure_llm-py--validate_llm_setting

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator：confirmed；attempts 1
- Phase / 模块：Phase 1 — Initialization & Configuration / configuration；原源码 src/configure_llm.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim：** 当 key 不是可识别的 LLM 配置设置键时，抛出 ConfigWizardError。当 key 可识别时，如果 value 未能通过与该键关联的验证约束，则抛出 ConfigWizardError：要求非空修剪后字符串的约束（value 为空或去除前后空白后仅包含空白字符）；要求语法有效的、使用 http 或 https 协议的绝对 URL 的约束（value 不满足）；要求属于固定可识别标识符集合的约束（value 不在该集合中）；或委托给 API 风格适配器的约束（适配器拒绝 value）。当 key 可识别且 value 满足所有与之关联的约束时，返回且无副作用。
- **Actual behavior：** 函数 validate_llm_setting(key, value) 要么返回 None，要么抛出异常。如果 key 不在 _LLM_TOML_KEYS 中，抛出 ConfigWizardError("Unsupported LLM setting: {key}")。如果 key == 'provider' 且 value.strip() 为空，抛出 ConfigWizardError("Provider ID must not be empty.")。如果 key == 'base_url'，调用 validate_base_url(value.strip())，当 value.strip() 不是语法有效的绝对 URL（http 或 https 协议）时可能抛出异常；否则不抛出。如果 key == 'backend' 且 value 不在 _BACKENDS 中，抛出 ConfigWizardError 并给出列出受支持后端的消息。如果 key == 'api_style'，调用 adapter_for_api_style(value)，当 value 不是可识别的 API 风格时可能抛出异常；否则不抛出。若上述条件均未导致异常，函数返回 None，无副作用。
- **Code evidence：** Line 7: if key == "base_url": Line 8: validate_base_url(value.strip()) Line 14: elif key == "api_style": Line 15: adapter_for_api_style(value)
- **Trigger condition：** 对于 base_url 和 api_style，规范要求当 value 验证失败时抛出 ConfigWizardError。然而，代码委托 validate_base_url 和 adapter_for_api_style，它们可能抛出非 ConfigWizardError 的异常（例如 ValueError）。因此，代码行为不符合所需的异常类型。
- **Validator trigger：** 可识别的键 'name' 和 'effort' 没有验证——validate_llm_setting('name', ") 和 validate_llm_setting('effort', 'invalid') 返回 None，而非按照规范要求抛出 ConfigWizardError。
- **Probe stdout：** CONFIRMED — 发现 6 个规范违规：

- 'name' 键使用 ':': 返回 None —— 规范要求对空/仅空白的修剪后值抛出 ConfigWizardError
- 'name' 键使用 ' ': 返回 None —— 规范要求对空/仅空白的修剪后值抛出 ConfigWizardError
- 'effort' 键使用 'invalid': 返回 None —— 规范要求对不在 ('', 'low', 'medium', 'high') 内的值抛出 ConfigWizardError
- 'effort' 键使用 'x': 返回 None —— 规范要求对不在 ('', 'low', 'medium', 'high') 内的值抛出 ConfigWizardError
- 'effort' 键使用 'super_high': 返回 None —— 规范要求对不在 ('', 'low', 'medium', 'high') 内的值抛出 ConfigWizardError
- 'effort' 键使用 'unknown_effort': 返回 None —— 规范要求对不在 ('', 'low', 'medium', 'high') 内的值抛出 ConfigWizardError (base_url 和 api_style 正确抛出 ConfigWizardError; bug 在于 'name' 和 'effort' 键没有针对其规范约束的验证)

► SPEC sidecar (完整)

```
{
  "signature": "validate_llm_setting(key: str, value: str) -> None",
  "pre_condition": "key 是一个非空字符串; value 是一个字符串。",
  "post_condition": "当 key 不是可识别的 LLM 配置设置键时, 抛出 ConfigWizardError。当 key 可识别时, 如果 value 未能通过与该键关联的验证约束, 则抛出 ConfigWizardError: 要求非空修剪后字符串的约束 (value 为空或去除前后空白后仅包含空白字符); 要求语法有效的、使用 http 或 https 协议的绝对 URL 的约束 (value 不满足); 要求属于固定可识别标识符集合的约束 (value 不在该集合中); 或委托给 API 风格适配器的约束 (适配器拒绝 value)。当 key 可识别且 value 满足所有与之关联的约束时, 返回且无副作用。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "validate_base_url",
      "signature": "validate_base_url(url: str) -> None",
      "pre_condition": "url 是一个非空的、已去除空白的字符串。",
      "post_condition": "当 url 不是语法有效的、协议为 http 或 https 的绝对 URL 时, 抛出异常。当 url 是有效的、协议为 http 或 https 的绝对 URL 时, 返回且无副作用。"
    },
    {
      "name": "adapter_for_api_style",
      "signature": "adapter_for_api_style(style: str) -> None",
      "pre_condition": "style 是一个字符串。",
      "post_condition": "当 style 不是可识别的 API 风格标识符时, 抛出异常。当 style 是可识别的 API 风格标识符时, 返回且无副作用。"
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/validate_llm_setting.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "当 key 不是可识别的 LLM 配置设置键时, 抛出 ConfigWizardError。当 key 可识别时, 如果 value 未能通过与该键关联的验证约束, 则抛出 ConfigWizardError: 要求非空修剪后字符串的约束 (value 为空或去除前后空白后仅包含空白字符); 要求语法有效的、使用 http 或 https 协议的绝对 URL 的约束 (value 不满足); 要求属于固定可识别标识符集合的约束 (value 不在该集合中); 或委托给 API 风格适配器的约束 (适配器拒绝 value)。当 key 可识别且 value 满足所有与之关联的约束时, 返回且无副作用。",
    "actual_behavior": "函数 validate_llm_setting(key, value) 要么返回 None, 要么抛出异常。如果 key 不在 _LLM_TOML_KEYS 中, 抛出 ConfigWizardError(\"Unsupported LLM setting: {key}\")。如果 key == 'provider' 且 value.strip() 为空, 抛出 ConfigWizardError(\"Provider ID must not be empty.\")。如果 key == 'base_url', 调用
```

```
validate_base_url(value.strip()), 当 value.strip() 不是语法有效的绝对 URL (http 或 https 协议) 时可能抛出异常; 否则不抛出。如果 key == 'backend' 且 value 不在 _BACKENDS 中, 抛出 ConfigWizardError 并给出列出受支持后端的消息。如果 key == 'api_style', 调用 adapter_for_api_style(value), 当 value 不是可识别的 API 风格时可能抛出异常; 否则不抛出。若上述条件均未导致异常, 函数返回 None, 无副作用。",
    "code_evidence": "Line 7:         if key == \"base_url\":\nLine 8:\nvalidate_base_url(value.strip())\nLine 14:         elif key == \"api_style\":\nLine 15:\nadapter_for_api_style(value)",
    "trigger_condition": "对于 base_url 和 api_style, 规范要求当 value 验证失败时抛出 ConfigWizardError。然而, 代码委托 validate_base_url 和 adapter_for_api_style, 它们可能抛出非 ConfigWizardError 的异常 (例如 ValueError)。因此, 代码行为不符合所需的异常类型。"
}
}
```

► Bug Validator result (完整)

```
{
  "id": "src--configure_llm-py--validate_llm_setting",
  "source_file": "src/configure_llm.py",
  "function_name": "validate_llm_setting",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--validate_llm_setting.py",
  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--validate_llm_setting.md",
  "probe_stdout": "CONFIRMED - 发现 6 个规范违规:\n - 'name' 键使用 ':': 返回 None — 规范要求对空/仅空白的修剪后值抛出 ConfigWizardError\n - 'name' 键使用 ' ': 返回 None — 规范要求对空/仅空白的修剪后值抛出 ConfigWizardError\n - 'effort' 键使用 'invalid': 返回 None — 规范要求对不在 ('', 'low', 'medium', 'high') 内的值抛出 ConfigWizardError\n - 'effort' 键使用 'x': 返回 None — 规范要求对不在 ('', 'low', 'medium', 'high') 内的值抛出 ConfigWizardError\n - 'effort' 键使用 'super_high': 返回 None — 规范要求对不在 ('', 'low', 'medium', 'high') 内的值抛出 ConfigWizardError\n - 'effort' 键使用 'unknown_effort': 返回 None — 规范要求对不在 ('', 'low', 'medium', 'high') 内的值抛出 ConfigWizardError\n(base_url 和 api_style 正确抛出 ConfigWizardError; bug 在于 'name' 和 'effort' 键没有针对其规范约束的验证)",
  "trigger_summary": "可识别的键 'name' 和 'effort' 没有验证—validate_llm_setting('name', '') 和 validate_llm_setting('effort', 'invalid') 返回 None, 而非按照规范要求抛出 ConfigWizardError。"
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌, 使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:22:40.450123Z	2811
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:23:12.537778Z	3910

事件 1 — [llm_c2dbc8e24dd44e828275b3479a23288f](#)

► parsed_json (完整)

```
{
  "post_condition": "函数 validate_llm_setting(key, value) 要么返回 None, 要么抛出异常。如果 key 不在 _LLM_TOML_KEYS 中, 抛出 ConfigWizardError(\"Unsupported LLM setting: {key}\")。如果 key == 'provider' 且 value.strip() 为空, 抛出 ConfigWizardError(\"Provider ID must not be empty.\")。如果 key == 'base_url', 调用 validate_base_url(value.strip()), 当 value.strip() 不是语法有效的绝对 URL (http 或 https 协议) 时可能抛出异常; 否则不抛出。如果 key == 'backend' 且 value 不在 _BACKENDS 中, 抛出 ConfigWizardError 并给出列出受支持后端的消息。如果 key == 'api_style', 调用 adapter_for_api_style(value), 当 value 不是可识别的 API 风格时可能抛出异常; 否则不抛出。若上述条件均未导致异常, 函数返回 None, 无副作用。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_c2dbc8e24dd44e828275b3479a23288f_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_c2dbc8e24dd44e828275b3479a23288f_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_c2dbc8e24dd44e828275b3479a23288f_response.txt)

事件 2 — llm_00550c05051a41dfa4de1d3f0fdb0ca6

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "key='base_url', value='invalid'",
  "offending_statements": "Line 7:      if key == \"base_url\":\nLine 8:      validate_base_url(value.strip())\nLine 14:      elif key == \"api_style\":\nLine 15:      adapter_for_api_style(value)",
  "reason": "对于 base_url 和 api_style, 规范要求当 value 验证失败时抛出 ConfigWizardError。然而, 代码委托 validate_base_url 和 adapter_for_api_style, 它们可能抛出非 ConfigWizardError 的异常 (例如 ValueError)。因此, 代码行为不符合所需的异常类型。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_00550c05051a41dfa4de1d3f0fdb0ca6_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_00550c05051a41dfa4de1d3f0fdb0ca6_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_00550c05051a41dfa4de1d3f0fdb0ca6_response.txt) **复核动作**

- 先由维护者确认预期契约, 再决定修改实现或重写 SPEC; 当前不应直接登记为真实 Bug。

模块: git_utils

FMA-MISMATCH-045 — src--git-py--frozen_worktree

- 四类结论: 契约不确定
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块: Phase 1 — Initialization & Configuration / git_utils; 原源码 src/git.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** 生成一个绝对路径 wt 指向一个目录, 该目录在 yield 时刻的内容是进入 frozen_worktree 时 proj_dir 文件系统树的忠实快照。对 proj_dir 的后续修改不会反映在 wt 处的快照中。快照构建永远不会修改原始的 proj_dir 树、索引或 git 状态。如果 proj_dir 是一个至少有一次提交的 git 仓库, 则快照是一个分离的 git 工作树, 包含 HEAD、所有未提交的已跟踪编辑以及在进入时 proj_dir 中存在的所有未跟踪文件, 并且名称与 exclude 匹配的条目将从 git 提交中排除。如果 proj_dir 不是一个具有有效 HEAD 的 git 仓库, 则快照是 proj_dir 的递归目录副本, 名称与 exclude 匹配的条目将被排除。当 copy_excluded 为 true 时, exclude 中指定的每个存在于 proj_dir 中的目录, 在 git 工作树或目录副本构建完成后, 会递归复制到 wt 处的快照中。wt 处的快照在上下文管理器退出后仍然存在, 并且不会自动移除。
- **Actual behavior:** 正常执行时, 生成器产生 wt, 一个位于 <temp_dir>/snapshot 的目录, 其中 <temp_dir> = tempfile.mkdtemp(prefix="fm_agent_wt-<repo_name>-") 和 <repo_name> 派生自 proj_dir。如果 proj_dir 是一个有 HEAD 提交的 git 仓库 (is_git 为 true), 则 wt 是一个指向新提交的分离工作树, 该提交捕获了 git add -A 运行时的该工作树 (已跟踪的修改 + 未跟踪文件), 其中 exclude 中的目录通过 git rm --cached 从提交中移除。原始的 git 状态 (HEAD、索引、工作树) 保持不变。如果 copy_excluded 为 true, 则每个存在于 proj_dir 但在 wt 中不存在的被排除目录, 会作为未跟踪目录递归复制

到 `wt` 中。如果 `proj_dir` 不是一个有 `HEAD` 的有效 `git` 仓库 (`is_git` 为 `false`)，则 `wt` 是 `proj_dir` 的普通副本 (`exclude` 中的目录通过 `shutil.ignore_patterns` 跳过)，并且如果 `copy_excluded`，随后将这些目录复制到 `wt` 中。`<temp_dir>` 在 `yield` 后仍然存在 (不会自动删除)。`proj_dir` 未被修改。包含 `wt` 和清理指令的消息会打印到 `stdout`。异常执行时 (`yield` 前发生异常)，`proj_dir` 不变，`<temp_dir>` 存在但可能包含不完整的产物，`wt` 可能不存在或不完整。

形式化描述： 令 $P = \text{os.path.abspath}(\text{proj_dir})$ ， $R = \text{os.path.basename}(P.\text{rstrip}(\text{os.sep}))$ 或 `"repo"`， $B = \text{tempfile.mkdtemp}(\text{prefix}="fm_agent_wt_"+R+"_")$ ， $W = \text{os.path.join}(B, \text{"snapshot"})$ ， $G = (\text{subprocess.run}([\text{"git"}, "-C", P, \text{"rev-parse"}, "--verify", \text{"HEAD"}], \text{check}=\text{True}, \text{capture_output}=\text{True}, \text{text}=\text{True})$ 未引发 `CalledProcessError`)。则结果满足： (正常产出 W) $\vee$ (引发异常 $\rightarrow$ 无产出)。若正常产出： G (存在 $I = \text{os.path.join}(B, \text{"index"})$ ，它是一个有效的 `git` 索引 $\text{tree} = _git(\text{"write-tree"}, \text{env}=\text{env})$ 的输出 $\text{snap} = _git(\text{"commit-tree"}, \text{tree}, "-p", \text{"HEAD"}, "-m", \text{"fm_agent snapshot"}, \text{env}=\text{env})$ 的输出 $_git(\text{"worktree"}, \text{"add"}, "--detach", W, \text{snap})$ 成功 P 的 `git` 状态 (`HEAD`、索引、工作树) 与进入时状态相同 $\vee \text{name} \in \text{exclude}$ ，令 $S = \text{os.path.join}(P, \text{name})$ ， $D = \text{os.path.join}(W, \text{name})$ ： ($\text{copy_excluded} \wedge \text{os.path.isdir}(S) \wedge \neg \text{os.path.exists}(D) \rightarrow D$ 是工作树创建后 `shutil.copytree`($S, D, \text{symlinks}=\text{True}$) 的结果)) $\neg G$ (W 是 `shutil.copytree`($P, W, \text{ignore}=\text{shutil.ignore_patterns}(*\text{exclude}), \text{symlinks}=\text{True}$) 的结果 $\vee \text{name} \in \text{exclude}$ ： ($\text{copy_excluded} \wedge \text{os.path.isdir}(\text{os.path.join}(P, \text{name})) \wedge \text{os.path.exists}(\text{os.path.join}(W, \text{name})) \rightarrow \text{shutil.copytree}(\text{os.path.join}(P, \text{name}), \text{os.path.join}(W, \text{name}), \text{symlinks}=\text{True})$ 在初始 `copytree` 后执行)) 产出后 B 存在，且 `stdout` 包含 W 。若引发异常： P 及其所有内容与进入时状态相同 $\wedge B$ 存在 $\wedge (W$ 可能不存在或不完整)。

- **代码证据：** Line 47: `_git("rm", "-r", "--cached", "--quiet", "--ignore-unmatch", "--",` Line 48: `*exclude, env=env)`
- **触发条件：** 代码仅使用顶层的排除名称执行 `'git rm --cached'`，因此同名的嵌套目录/文件仍保留在提交中。规范要求“名称与 `exclude` 匹配的条目被排除”，这意味着需要排除所有深度下的此类条目。
- **Validator 触发：** `frozen_worktree()` 仅使用顶层的排除名称执行 `git rm --cached`，因此同名的嵌套目录（如 `testdata/fm_agent/`）会留在提交中并泄漏到快照中。
- **Probe 标准输出：** [Pipeline] Snapshot created at: /tmp/fm_agent_wt_repo_54som_02/snapshot [Pipeline]
Snapshot is kept after the run. Remove with: `git -C /tmp/probe_fwt_frax15ct/repo worktree remove --force /tmp/fm_agent_wt_repo_54som_02/snapshot` 确认 —— 快照中存在嵌套的 `fm_agent/`，位于 `/tmp/fm_agent_wt_repo_54som_02/snapshot/testdata/fm_agent`（包含：`['nested.txt']`）—— 规范要求在所有深度下排除

► SPEC sidecar（完整）

```
{
  "signature": "frozen_worktree(proj_dir, exclude=('fm_agent',), copy_excluded=True) -> yields str",
  "pre_condition": "proj_dir 是一个指向现有目录的字符串路径。exclude 是一个相对目录名的可迭代对象。copy_excluded 是一个布尔值。",
  "post_condition": "生成一个绝对路径 wt 指向一个目录，该目录在 yield 时刻的内容是进入 frozen_worktree 时 proj_dir 文件系统树的忠实快照。对 proj_dir 的后续修改不会反映在 wt 处的快照中。快照构建永远不会修改原始的 proj_dir 树、索引或 git 状态。如果 proj_dir 是一个至少有一次提交的 git 仓库，则快照是一个分离的 git 工作树，包含 HEAD、所有未提交的已跟踪编辑以及在进入时 proj_dir 中存在的所有未跟踪文件，并且名称与 exclude 匹配的条目将从 git 提交中排除。如果 proj_dir 不是一个具有有效 HEAD 的 git 仓库，则快照是 proj_dir 的递归目录副本，名称与 exclude 匹配的条目将被排除。当 copy_excluded 为 true 时，exclude 中指定的每个存在于 proj_dir 中的目录，在 git 工作树或目录副本构建完成后，会递归复制到 wt 处的快照中。wt 处的快照在上下文管理器退出后仍然存在，并且不会自动移除。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "os.path.abspath",
      "signature": "os.path.abspath(path: str) -> str",
    }
  ]
}
```

```

"pre_condition": "path 是一个字符串。",
"post_condition": "返回一个规范化的绝对路径，等价于 path，不访问文件系统。"
},
{
  "name": "tempfile.mkdtemp",
  "signature": "tempfile.mkdtemp(prefix: str | None) -> str",
  "pre_condition": "父临时目录存在且可写。",
  "post_condition": "创建一个唯一目录，当 prefix 非 None 时其名称以 prefix 开头。返回所创建目录的绝对路径。"
},
{
  "name": "subprocess.run",
  "signature": "subprocess.run(args: list[str], *, check: bool, capture_output: bool, text: bool, env: dict | None) -> CompletedProcess",
  "pre_condition": "args 是一个非空的字符串列表，指定可执行文件及其参数。当指定 env 时，它将环境变量名映射到值。",
  "post_condition": "在子进程中执行 args 指定的命令。当 capture_output 为 true 时，将 stdout 和 stderr 捕获到结果的 stdout 和 stderr 属性中。当 text 为 true 时，捕获的输出被解码为字符串。当 check 为 true 时，如果子进程以非零状态退出，则引发 CalledProcessError。返回一个 CompletedProcess，其 stdout 属性为去除首尾空格的捕获标准输出。"
},
{
  "name": "shutil.copytree",
  "signature": "shutil.copytree(src: str, dst: str, *, ignore: callable | None, symlinks: bool) -> str",
  "pre_condition": "src 是一个存在的目录。dst 不存在。",
  "post_condition": "递归地将 src 为根的目录树复制到 dst。当 symlinks 为 true 时，符号链接作为链接保留。当提供 ignore 时，会被 ignore 可调用对象匹配的条目将从副本中排除。返回 dst。"
},
{
  "name": "os.path.isdir",
  "signature": "os.path.isdir(path: str) -> bool",
  "pre_condition": "path 是一个字符串。",
  "post_condition": "如果 path 存在且引用一个目录，则返回 True；否则返回 False。"
},
{
  "name": "os.path.exists",
  "signature": "os.path.exists(path: str) -> bool",
  "pre_condition": "path 是一个字符串。",
  "post_condition": "如果 path 在文件系统中存在，则返回 True；否则返回 False，包括断开的符号链接。"
}
]
}

```

► Logic verification result（完整）

► Bug Validator 结果（完整）

```

{
  "id": "src--git-py--frozen_worktree",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/git-py/frozen_worktree.py",
  "function_name": "frozen_worktree",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--git-py--frozen_worktree.py",
  "detail_file": "fm_agent/bug_validation/src--git-py--frozen_worktree.md",
  "probe_stdout": "[Pipeline] 快照创建于：/tmp/fm_agent_wt_repo_54som_02/snapshot\n[Pipeline] 运行后快照被保留。通过以下命令移除：git -C /tmp/probe_fwt_frax15ct/repo worktree remove --force /tmp/fm_agent_wt_repo_54som_02/snapshot\nCONFIRMED - 嵌套的 fm_agent/ 存在于快照 /tmp/fm_agent_wt_repo_54som_02/snapshot/testdata/fm_agent 中（包含：['nested.txt']）- 规范要求在所有深度均排除",
  "trigger_summary": "frozen_worktree() 使用仅带有顶层排除名称的 git rm --cached，因此具有相同名称的嵌套目录（例如 testdata/fm_agent/）仍保留在提交中并泄漏到快照中。"
}

```

► Reasoner 流程（3 条事件）

共 3 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	format_error	1	0/1	2026-07-27T15:37:22.417794Z	8614
2	generate_block_post_condition	success	2	0/1	2026-07-27T15:39:02.286524Z	5559
3	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:39:41.039217Z	11504

事件 1 — llm_42242808c65246bf948fa3f52184c058

- parse_error: LLM response is not valid JSON: Invalid \\escape: line 2 column 66 (char 67)
- 载荷: system_prompt · user_prompt · assistant_output

事件 2 — llm_01aca045e7fa434eab75c67be53662bf

► parsed_json (完整)

```
{
  "post_condition": "正常执行时，生成器 yield `wt`，这是位于 `/snapshot` 的目录，其中 `` =  
tempfile.mkdtemp(prefix=`fm_agent_wt_<repo_name>_`), `` 从 `proj_dir` 派生。如果 `proj_dir` 是  
一个具有 HEAD 提交的 git 仓库 (`is_git` 为 true)，则 `wt` 是为一个新提交而创建的分离 worktree，该提交捕获了运行 `git  
add -A` 时的工作树（受追踪的编辑 + 未被追踪的文件），其中 `exclude` 中的目录通过 `git rm --cached` 从提交中移除。原始  
git 状态 (`HEAD`、索引、工作树) 保持不变。如果 `copy_excluded` 为 true，则曾经存在于 `proj_dir` 但不存在于 `wt` 的每个  
被排除的目录，会被递归复制到 `wt` 中作为未被追踪的目录。如果 `proj_dir` 不是一个具有 HEAD 的有效 git 仓库 (`is_git`  
为 false)，则 `wt` 是一个 `proj_dir` 的普通副本（通过 `shutil.ignore_patterns` 跳过 `exclude` 中的目录），且若  
`copy_excluded` 为 true，这些目录随后会被复制到 `wt` 中。`<temp_dir>` 在 yield 之后仍然存在（无自动删除）。  
`proj_dir` 未被修改。包含 `wt` 和清理说明的消息被打印到标准输出。异常执行时 (yield 前发生异常)，`proj_dir` 不变，  
`<temp_dir>` 存在但可能包含部分产物，且 `wt` 可能不存在或不完整。\\n\\n形式化地：\\n令 P = os.path.abspath(proj_dir), R  
= os.path.basename(P.rstrip(os.sep)) 或 `repo`, B = tempfile.mkdtemp(prefix=`fm_agent_wt_` + R +  
`_`), W = os.path.join(B, `snapshot`), G = (subprocess.run([`git`, `--C`, P, `rev-parse`, `--  
verify`, `HEAD`], check=True, capture_output=True, text=True) 未引发 CalledProcessError)。那么结果满足：\\n  
(正常 yield W) ^ (引发异常且未 yield)。\\n 若正常 yield：\\n      G => (exists I = os.path.join(B, `index`))  
是一个有效的 git 索引 ^\\n      exists tree = _git(`write-tree`, env=env) 的输出 ^\\n      exists snap =  
_git(`commit-tree`, tree, `--p`, `HEAD`, `--m`, `fm_agent snapshot`, env=env) 的输出 ^\\n  
_git(`worktree`, `add`, `--detach`, W, snap) 成功 ^\\n      P 的 git 状态 (HEAD、索引、工作树) 与进入时的  
状态相同 ^\\n      forall name in exclude, 令 S = os.path.join(P, name), D = os.path.join(W, name): \\n  
(copy_excluded ^ os.path.isdir(S) ^ !os.path.exists(D) -> D 是 worktree 创建后 shutil.copytree(S, D,  
symlinks=True) 的结果) ^\\n      !G => (W 是 shutil.copytree(P, W,  
ignore=shutil.ignore_patterns(*exclude), symlinks=True) 的结果 ^\\n      forall name in exclude: (copy_excluded  
^ os.path.isdir(os.path.join(P, name)) ^ !os.path.exists(os.path.join(W, name)) -> \\n      在初始  
copytree 之后执行了 shutil.copytree(os.path.join(P, name), os.path.join(W, name), symlinks=True)) ^\\n      ^  
在 yield 之后 B 存在，且 stdout 包含 W。\\n 若引发异常：\\n      P 及其所有内容进入时的状态相同 ^ B 存在 ^ (W 可能不存在或不完整)。"
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_01aca045e7fa434eab75c67be53662bf_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_01aca045e7fa434eab75c67be53662bf_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_01aca045e7fa434eab75c67be53662bf_message_02_assistant.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_01aca045e7fa434eab75c67be53662bf_message_03_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_01aca045e7fa434eab75c67be53662bf_response.txt)

事件 3 — llm_11f0918b468947608ab50c754b1dc97b

► parsed_json (完整)


```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir 是一个至少有一个提交的 git 仓库。其工作树包含一个顶层目录 'fm_agent' 以及一个子目录 'subdir/fm_agent/important.txt'。使用默认 exclude=('fm_agent',) 调用 frozen_worktree(proj_dir)。快照 worktree 将包含 'subdir/fm_agent/important.txt'，但规范要求与 exclude 中名称匹配的所有条目都被从快照中省略。",
  "offending_statements": "Line 47:          _git(\"rm\", \"-r\", \"--cached\", \"--quiet\", \"--ignore-unmatch\", \"--\", \"\\nLine 48:          *exclude, env=env)\",
  "reason": "代码使用仅带有顶层排除名称的 'git rm --cached'，因此具有相同名称的嵌套目录/文件仍保留在提交中。规范要求“与 exclude 中名称匹配的条目被省略”，这要求在任何深度的所有此类条目都被省略。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_11f0918b468947608ab50c754b1dc97b_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_11f0918b468947608ab50c754b1dc97b_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_11f0918b468947608ab50c754b1dc97b_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

Phase 2 — Language Backend & Function Extraction

模块: **codegraph**

FMA-MISMATCH-061 — [src--languages--codegraph-py--CodeGraphExtractor::get_call_edges](#)

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator:** [confirmed](#); attempts [1](#)
- **Phase / 模块:** Phase 2 — Language Backend & Function Extraction / [codegraph](#); 原源码 [src/languages/codegraph.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** 返回一个字典，将每个调用者 FQN（字符串）映射到一个非空的被调用者 FQN 集合（set[str]），涵盖在索引项目中给定语言的源文件中发现的所有直接调用关系。键和值均使用规范化 FQN，并以 :: 作为组件分隔符。调用边既包含函数调用边，也包含由类实例化关系合成的构造函数调用。FQN 身份通过 codegraph 节点 ID 解析，保留不同文件中同名函数的精确调用者/被调用者关联。当 lang_key 不是已识别语言或给定语言不存在调用边时，返回空字典。
- **Actual behavior:** 若未抛出异常，则：如果 _CG_LANG.get(lang_key) 得出一个非空列表 cg_langs，则该方法打开到 self._db 的连接，获取游标，计算 fqn_of = _node_fqn_map(cur, cg_langs)，然后构建这样一个映射 result：对于每个 edges.kind='calls' 且源语言在 cg_langs 中的 (src, tgt)，若 fqn_of[src] 和 fqn_of[tgt] 均不为 None，则 result[fqn_of[src]] 包含 fqn_of[tgt]；此外，如果存在 ctor_filter = _CONSTRUCTOR_FILTER.get(lang_key)，则对于每个来自第二个查询（edges.kind='instantiates'，源语言在 cg_langs 中，目标类包含与 ctor_filter 匹配的方法/函数）的 (src, ctor_id)，若 fqn_of[src] 和 fqn_of[ctor_id] 均不为 None，则 result[fqn_of[src]] 包含 fqn_of[ctor_id]。连接在返回前关闭。该方法返回 dict(result)。如果 _CG_LANG.get(lang_key) 得到假值，则该方法返回 {}，且无任何副作用。在所有情况下，self 保持未修改，self._db 不变，无资源泄漏。
- **Code evidence:** Line 40: caller, callee = fqn_of.get(src_id), fqn_of.get(tgt_id) Line 41: if caller and callee: Line 42: result[caller].add(callee)
- **Trigger condition:** 代码会遗漏被调用者的 FQN 不在 fqn_of 中的调用边，而 fqn_of 仅覆盖所请求语言的节点。然而，规范要求所有在给定语言的源文件中发现的直接调用关系，包括对其他语言中定义的函数的调用。这将导致

输出字典中缺失被调用者。

- **Validator trigger:** 从所请求语言调用其他语言中的函数的调用会被丢弃，因为 `_node_fqn_map` 仅映射所请求语言的节点，因此 `fqn_of.get` 对跨语言被调用者返回 `None`。
- **Probe stdout:** CONFIRMED — Bug reproduced in attempt 1: cross-language call edge from Python to C was dropped. `actual={} | expected={'src::py_code-py::py_func': {'src::c_code-c::c_func'}}`

► SPEC sidecar (完整)

```
{
  "signature": "CodeGraphExtractor.get_call_edges(lang_key: str) -> dict",
  "pre_condition": "self 是一个 CodeGraphExtractor 实例，已使用包含调用边数据的有效且未损坏的 codegraph 数据库初始化；lang_key 是一个字符串值。",
  "post_condition": "返回一个字典，将每个调用者 FQN（字符串）映射到一个非空的被调用者 FQN 集合（set[str]），涵盖在索引项目中给定语言的源文件中发现的所有直接调用关系。键和值均使用规范化 FQN，并以 :: 作为组件分隔符。调用边既包含函数调用边，也包含由类实例化关系合成的构造函数调用。FQN 身份通过 codegraph 节点 ID 解析，保留不同文件中同名函数的精确调用者/被调用者关联。当 lang_key 不是已识别语言或给定语言不存在调用边时，返回空字典。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-py/CodeGraphExtractor::get_call_edges.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个字典，将每个调用者 FQN（字符串）映射到一个非空的被调用者 FQN 集合（set[str]），涵盖在索引项目中给定语言的源文件中发现的所有直接调用关系。键和值均使用规范化 FQN，并以 :: 作为组件分隔符。调用边既包含函数调用边，也包含由类实例化关系合成的构造函数调用。FQN 身份通过 codegraph 节点 ID 解析，保留不同文件中同名函数的精确调用者/被调用者关联。当 lang_key 不是已识别语言或给定语言不存在调用边时，返回空字典。",
    "actual_behavior": "若未抛出异常，则：如果 _CG_LANG.get(lang_key) 得出一个非空列表 cg_langs，则该方法打开通到 self._db 的连接，获取游标，计算 fqn_of = _node_fqn_map(cur, cg_langs)，然后构建这样一个映射 result：对于每个 edges.kind='calls' 且源语言在 cg_langs 中的 (src, tgt)，若 fqn_of[src] 和 fqn_of[tgt] 均不为 None，则 result[fqn_of[src]] 包含 fqn_of[tgt]；此外，如果存在 ctor_filter = _CONSTRUCTOR_FILTER.get(lang_key)，则对于每个来自第二个查询（edges.kind='instantiates'，源语言在 cg_langs 中，目标类包含与 ctor_filter 匹配的方法/函数）的 (src, ctor_id)，若 fqn_of[src] 和 fqn_of[ctor_id] 均不为 None，则 result[fqn_of[src]] 包含 fqn_of[ctor_id]。连接在返回前关闭。该方法返回 dict(result)。如果 _CG_LANG.get(lang_key) 得到假值，则该方法返回 {}，且无任何副作用。在所有情况下，self 保持未修改，self._db 不变，无资源泄漏。",
    "code_evidence": "Line 40: caller, callee = fqn_of.get(src_id), fqn_of.get(tgt_id)\nLine 41: if caller and callee:\nLine 42: result[caller].add(callee)",
    "trigger_condition": "代码会遗漏被调用者的 FQN 不在 fqn_of 中的调用边，而 fqn_of 仅覆盖所请求语言的节点。然而，规范要求所有在给定语言的源文件中发现的直接调用关系，包括对其他语言中定义的函数的调用。这将导致输出字典中缺失被调用者。"
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--languages--codegraph-py--CodeGraphExtractor::get_call_edges",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-py/CodeGraphExtractor::get_call_edges.py",
  "function_name": "CodeGraphExtractor::get_call_edges",
  "confirmation_status": "confirmed",
  "attempts": "1",
}
```

```
"probe_script": "fm_agent/bug_validation/probe_src--languages--codegraph-py--
CodeGraphExtractor::get_call_edges.py",
"detail_file": "fm_agent/bug_validation/src--languages--codegraph-py--
CodeGraphExtractor::get_call_edges.md",
"probe_stdout": "CONFIRMED - Bug reproduced in attempt 1: cross-language call edge from Python to C was
dropped. actual={} | expected={'src::py_code-py::py_func': {'src::c_code-c::c_func'}}",
"trigger_summary": "从所请求语言调用其他语言中的函数的调用会被丢弃，因为 _node_fqn_map 仅映射所请求语言的节点，因此
fqn_of.get 对跨语言被调用者返回 None。"
}
```

► Reasoner 流程（3 条事件）

共 3 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	format_error	1	0/1	2026-07-27T16:54:34.609052Z	2666
2	generate_block_post_condition	success	2	0/1	2026-07-27T16:54:56.276908Z	3284
3	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:55:14.298556Z	5818

事件 1 — llm_0dbae84127b946c290dc6217ff8e1961

► parsed_json（完整）

```
{}
```

- parse_error: `post-condition JSON requires a non-empty string field: post\_condition` - 载荷: [system_prompt] (fm_agent/trace/payloads/llm_0dbae84127b946c290dc6217ff8e1961_message_00_system.txt) · [user_prompt] (fm_agent/trace/payloads/llm_0dbae84127b946c290dc6217ff8e1961_message_01_user.txt) · [assistant_output] (fm_agent/trace/payloads/llm_0dbae84127b946c290dc6217ff8e1961_response.txt)

事件 2 — llm_75e849652bbb432b96f4e7dcae0059cc

► parsed_json（完整）

```
{
  "post_condition": "若未抛出异常，则：如果 _CG_LANG.get(lang_key) 得出一个非空列表 cg_langs，则该方法打开到
self._db 的连接，获取游标，计算 fqn_of = _node_fqn_map(cur, cg_langs)，然后构建这样一个映射 result：对于每个
edges.kind='calls' 且源语言在 cg_langs 中的 (src, tgt)，若 fqn_of[src] 和 fqn_of[tgt] 均不为 None，则
result[fqn_of[src]] 包含 fqn_of[tgt]；此外，如果存在 ctor_filter = _CONSTRUCTOR_FILTER.get(lang_key)，则对于每
个来自第二个查询（edges.kind='instantiates'，源语言在 cg_langs 中，目标类包含与 ctor_filter 匹配的方法/函数）的
(src, ctor_id)，若 fqn_of[src] 和 fqn_of[ctor_id] 均不为 None，则 result[fqn_of[src]] 包含 fqn_of[ctor_id]。连
接在返回前关闭。该方法返回 dict(result)。如果 _CG_LANG.get(lang_key) 得到假值，则该方法返回 {}，且无任何副作用。在所有
情况下，self 保持未修改，self._db 不变，无资源泄漏。"
}
```

- 载荷: [system_prompt] (fm_agent/trace/payloads/llm_75e849652bbb432b96f4e7dcae0059cc_message_00_system.txt) · [user_prompt] (fm_agent/trace/payloads/llm_75e849652bbb432b96f4e7dcae0059cc_message_01_user.txt) · [assistant_output] (fm_agent/trace/payloads/llm_75e849652bbb432b96f4e7dcae0059cc_message_02_assistant.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_75e849652bbb432b96f4e7dcae0059cc_message_03_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_75e849652bbb432b96f4e7dcae0059cc_response.txt)

事件 3 — llm_f74a6de9292c40bda7c8ff45e97964d8

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "考虑 lang_key='python', 且 _CG_LANG['python']=['python']。数据库包含一条从源节点 1 (language='python', FQN='src::file::foo') 到目标节点 2 (language='cpp', FQN='other::file::bar') 的 'calls' 调用边。代码仅为 ['python'] 中的节点构建 fqn_of, 因此 fqn_of.get(2) 为 None。该边被跳过, 方法返回 {}。规范要求返回 {'src::file::foo': {'other::file::bar'}}。",
  "offending_statements": "Line 40:             caller, callee = fqn_of.get(src_id),
fqn_of.get(tgt_id)\nLine 41:             if caller and callee:\nLine 42:                 result[caller].add(callee)",
  "reason": "代码会遗漏被调用者的 FQN 不在 fqn_of 中的调用边, 而 fqn_of 仅覆盖所请求语言的节点。然而, 规范要求所有在给定语言的源文件中发现的直接调用关系, 包括对其他语言中定义的函数的调用。这将导致输出字典中缺失被调用者。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_f74a6de9292c40bda7c8ff45e97964d8_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_f74a6de9292c40bda7c8ff45e97964d8_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_f74a6de9292c40bda7c8ff45e97964d8_response.txt) **复核动作**

- 先由维护者确认预期契约, 再决定修改实现或重写 SPEC; 当前不应直接登记为真实 Bug。

FMA-MISMATCH-070 — src--languages--codegraph-py--try_codegraph_init

- 四类结论: 契约不确定
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 2
- Phase / 模块: Phase 2 — Language Backend & Function Extraction / codegraph; 原源码 src/languages/codegraph.py
- 证据: 抽取源码] · SPEC] · INFO] · Logic] · Validator result] · probe] · 详细报告]

冲突摘要

- SPEC claim:** 如果 force 为真, 则在尝试构建新的索引之前, 先移除位于 <proj_dir>/.codegraph/ 的任何既有 CodeGraph 索引目录。如果 force 为假且索引数据库 <proj_dir>/.codegraph/codegraph.db 已经存在, 则函数立即返回而不修改文件系统。否则, 函数以 proj_dir 为工作目录调用 'codegraph init' 子命令。函数永不抛出异常: 若在系统 PATH 中找不到 codegraph 可执行文件, 函数静默返回; 若 'init' 子命令以非零状态码退出, 则记录一条警告并正常返回, 无错误。
- Actual behavior:** 执行后, 函数已尝试为 proj_dir 构建或重建 codegraph 索引。不会抛出异常; 所有内部错误均被处理 (缺少 codegraph 时静默返回, 非零退出时发出警告)。具体效果取决于初始状态和 force 参数:
- 令 db_path = os.path.join(os.path.join(proj_dir, '.codegraph'), 'codegraph.db')。
- 令 E_before = 初始时 os.path.exists(db_path)。
- 令 CMD_EXIST 为真, 当 codegraph 命令 (由 _codegraph_cmd() 返回) 存在且可执行, 否则为假。
- 令 INIT_SUCCEED 为真, 当子进程 [cmd, 'init'] 运行并返回退出码 0, 否则为假 (包括命令缺失的情况)。

后置条件：

1. 若 $E_before \wedge \neg force$ ：函数立即返回。文件系统不发生变化。不打印任何输出。`db_path` 仍存在。
2. 若 $(E_before \wedge force) \vee \neg E_before$ ：
 - 打印一条消息：
 - 若 $E_before \wedge force$ ：打印 "[Pipeline] Rebuilding codegraph index for current working tree..."，并将包含 `db_path` 的目录删除（通过 `shutil.rmtree` 调用 `ignore_errors=True` 删除 `codegraph_dir`）。
 - 若 $\neg E_before$ ：打印 "[Pipeline] Building codegraph index..."，不执行事先删除。
 - 调用函数 `_warn_on_codegraph_version_mismatch(cmd)`，该函数可能发出版本不匹配警告。
 - 然后尝试 `subprocess.run([cmd, 'init'], cwd=proj_dir, capture_output=True, text=True)`。
 - 若 `CMD_EXIST` 为假（`FileNotFoundError`），函数静默返回；不产生进一步输出。
 - 若 `CMD_EXIST`：
 - 若 `INIT_SUCCEED`：打印 "[Pipeline] codegraph index built."。
 - 若 `¬INIT_SUCCEED`：记录一条警告，包含 `stderr` 的前 300 个字符。

正式地，`db_path` 的最终存在性由下式给出： $exists(db_path) \Leftrightarrow (E_before \wedge \neg force) \vee (INIT_SUCCEED \wedge CMD_EXIST)$ 。

特别地，以下情况下数据库文件保证在调用后存在：

- 数据库原本已存在且 `force` 为假，或
- `codegraph` 命令可用且其 `init` 子命令成功（无论之前目录是否被删除）。

所有打印输出在 `stdout` 上；警告通过 `logging` 模块发出。

- **Code evidence**：第19行：`if os.path.exists(db_path):`
- **Trigger condition**：规范要求当 `force` 为 `True` 时，在构建之前应移除任何既有的 `.codegraph/` 目录。而代码仅在 `codegraph.db` 存在时（第19行）才删除目录。如果 `.codegraph/` 存在但没有 `codegraph.db`，代码不会将其删除，从而违反规范。
- **Validator trigger**：当 `.codegraph/` 目录存在但内部没有 `codegraph.db` 时，`force=True` 不会将其删除（规范要求删除任何既有目录）。
- **Probe stdout**：[Pipeline] Building codegraph index... WARNING:root:codegraph '1.3.0-fmagent.1' does not match the pinned '1.5.0-fmagent.1' (fm-agent.toml [codegraph].version); re-run install.sh to update. [Pipeline] codegraph index built. 已确认 — `.codegraph/` 目录在 `force=True` 重建后仍然存在（预期：按规范应被删除，实际：依然存在）。`db_exists_before=False`, `cgdir_exists_before=True`, `cgdir_exists_after=True`, `db_exists_after=True`

► SPEC sidecar（完整）

```
{
  "signature": "try_codegraph_init(proj_dir: str, force: bool = True) -> None",
  "pre_condition": "proj_dir 是表示一个已存在文件系统目录的字符串。",
  "post_condition": "如果 force 为真，则在尝试构建新的索引之前，先移除位于 <proj_dir>/.codegraph/ 的任何既有 CodeGraph 索引目录。如果 force 为假且索引数据库 <proj_dir>/.codegraph/codegraph.db 已经存在，则函数立即返回而不修改文件系统。否则，函数以 proj_dir 为工作目录调用 'codegraph init' 子命令。函数永不抛出异常：若在系统 PATH 中找不到 codegraph 可执行文件，函数静默返回；若 'init' 子命令以非零状态码退出，则记录一条警告并正常返回，无错误。"
}
```

► INFO / callees sidecar（完整）


```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-
py/try_codegraph_init.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "如果 force 为真,则在尝试构建新的索引之前,先移除位于 <proj_dir>/codegraph/ 的任何既有
CodeGraph 索引目录。如果 force 为假且索引数据库 <proj_dir>/codegraph/codegraph.db 已经存在,则函数立即返回而不修改
文件系统。否则,函数以 proj_dir 为工作目录调用 'codegraph init' 子命令。函数永不抛出异常:若在系统 PATH 中找不到
codegraph 可执行文件,函数静默返回;若 'init' 子命令以非零状态码退出,则记录一条警告并正常返回,无错误。",
    "actual_behavior": "执行后,函数已尝试为 proj_dir 构建或重建 codegraph 索引。不会抛出异常;所有内部错误均被处理
(缺少 codegraph 时静默返回,非零退出时发出警告)。具体效果取决于初始状态和 force 参数:\n\n- 令 db_path =
os.path.join(os.path.join(proj_dir, '.codegraph'), 'codegraph.db')。 \n- 令 E_before = 初始时
os.path.exists(db_path)。 \n- 令 CMD_EXIST 为真,当 codegraph 命令(由 `_codegraph_cmd()` 返回)存在且可执行,否则
为假。 \n- 令 INIT_SUCCEED 为真,当子进程 `[cmd, 'init']` 运行并返回退出码 0,否则为假(包括命令缺失的情况)。 \n\n后置条
件:\n1. 若 E_before ^ ¬force:函数立即返回。文件系统不发生变化。不打印任何输出。db_path 仍存在。 \n2. 若 (E_before ^
force) v ¬E_before:\n    - 打印一条消息:\n        * 若 E_before ^ force:打印 `[Pipeline] Rebuilding
codegraph index for current working tree...`,并将包含 db_path 的目录删除(通过 `shutil.rmtree` 调用
`ignore_errors=True` 删除 codegraph_dir)。 \n        * 若 ¬E_before:打印 `[Pipeline] Building codegraph
index...`,不执行事先删除。 \n    - 调用函数 `_warn_on_codegraph_version_mismatch(cmd)`,该函数可能发出版本不匹配警
告。 \n    - 然后尝试 `subprocess.run([cmd, 'init'], cwd=proj_dir, capture_output=True, text=True)`。 \n    - 若
CMD_EXIST 为假(FileNotFoundError),函数静默返回;不产生进一步输出。 \n    - 若 CMD_EXIST:\n        * 若
INIT_SUCCEED:打印 `[Pipeline] codegraph index built.`。 \n        * 若 ¬INIT_SUCCEED:记录一条警告,包含
stderr 的前 300 个字符。 \n\n正式地,db_path 的最终存在性由下式给出:\n    exists(db_path) ⇔ (E_before ^ ¬force) v
(INIT_SUCCEED ^ CMD_EXIST)。 \n\n特别地,以下情况下数据库文件保证在调用后存在:\n    - 数据库原本已存在且 force 为假,或
\n    - codegraph 命令可用且其 `init` 子命令成功(无论之前目录是否被删除)。 \n\n所有打印输出在 stdout 上;警告通过
logging 模块发出。",
    "code_evidence": "Line 19: if os.path.exists(db_path):",
    "trigger_condition": "规范要求当 force 为 True 时,在构建之前应移除任何既有的 `codegraph/` 目录。而代码仅在
`codegraph.db` 存在时(第19行)才删除目录。如果 `codegraph/` 存在但没有 `codegraph.db`,代码不会将其删除,从而违反规
范。"
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--languages--codegraph-py--try_codegraph_init",
  "source_file": "/home/fancy/Projects_Vault/FM-
Agent/fm_agent/extracted_functions/src/languages/codegraph-py/try_codegraph_init.py",
  "function_name": "try_codegraph_init",
  "confirmation_status": "confirmed",
  "attempts": 2,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--codegraph-py--try_codegraph_init.py",
  "detail_file": "fm_agent/bug_validation/src--languages--codegraph-py--try_codegraph_init.md",
  "probe_stdout": "[Pipeline] Building codegraph index...\nWARNING:root:codegraph '1.3.0-fmagent.1' does
not match the pinned '1.5.0-fmagent.1' (fm-agent.toml [codegraph].version); re-run install.sh to
update.\n[Pipeline] codegraph index built.\n已确认 - .codegraph/ 目录在 force=True 重建后仍然存在(预期:按规范应
被删除,实际:依然存在)。db_exists_before=False, cgdir_exists_before=True, cgdir_exists_after=True,
db_exists_after=True",
  "trigger_summary": "当 .codegraph/ 目录存在但内部没有 codegraph.db 时,force=True 不会将其删除(规范要求删除任何
既有目录)。"
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌,使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:34:17.847918Z	3927
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:35:12.412468Z	6054

事件 1 — llm_3623b0a19ca94dcb86a57ffb018ce69c

► parsed_json (完整)

```
{
  "post_condition": "执行后，函数已尝试为 proj_dir 构建或重建 codegraph 索引。不会抛出异常；所有内部错误均被处理（缺少 codegraph 时静默返回，非零退出时发出警告）。具体效果取决于初始状态和 force 参数：\n\n- 令 db_path = os.path.join(os.path.join(proj_dir, '.codegraph'), 'codegraph.db')。 \n- 令 E_before = 初始时 os.path.exists(db_path)。 \n- 令 CMD_EXIST 为真，当 codegraph 命令（由 `_codegraph_cmd()` 返回）存在且可执行，否则为假。 \n- 令 INIT_SUCCEED 为真，当子进程 `[cmd, 'init']` 运行并返回退出码 0，否则为假（包括命令缺失的情况）。 \n\n后置条件：\n1. 若 E_before ^ ¬force：函数立即返回。文件系统不发生变化。不打印任何输出。db_path 仍存在。 \n2. 若 (E_before ^ force) v ¬E_before：\n    - 打印一条消息：\n        * 若 E_before ^ force：打印 `[Pipeline] Rebuilding codegraph index for current working tree...`，并将包含 db_path 的目录删除（通过 `shutil.rmtree` 调用 `ignore_errors=True` 删除 codegraph_dir）。 \n        * 若 ¬E_before：打印 `[Pipeline] Building codegraph index...`，不执行事先删除。 \n    - 调用函数 `_warn_on_codegraph_version_mismatch(cmd)`，该函数可能发出版本不匹配警告。 \n    - 然后尝试 `subprocess.run([cmd, 'init'], cwd=proj_dir, capture_output=True, text=True)`。 \n    - 若 CMD_EXIST 为假（FileNotFoundError），函数静默返回；不产生进一步输出。 \n    - 若 CMD_EXIST：\n        * 若 INIT_SUCCEED：打印 `[Pipeline] codegraph index built.`。 \n        * 若 ¬INIT_SUCCEED：记录一条警告，包含 stderr 的前 300 个字符。 \n\n正式地，db_path 的最终存在性由下式给出：\nexists(db_path) ⇔ (E_before ^ ¬force) v (INIT_SUCCEED ^ CMD_EXIST)。 \n\n特别地，以下情况下数据库文件保证在调用后存在：\n    - 数据库原本已存在且 force 为假，或 \n    - codegraph 命令可用且其 `init` 子命令成功（无论之前目录是否被删除）。 \n\n所有打印输出在 stdout 上；警告通过 logging 模块发出。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_3623b0a19ca94dcb86a57ffb018ce69c_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_3623b0a19ca94dcb86a57ffb018ce69c_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_3623b0a19ca94dcb86a57ffb018ce69c_response.txt)]

事件 2 — llm_b30729f439174a43813d172dbb643773

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir = '/tmp/testproj'; 其中, '.codegraph' 目录存在但不包含 'codegraph.db' 文件; force = True",
  "offending_statements": "Line 19: if os.path.exists(db_path):",
  "reason": "规范要求当 force 为 True 时, 构建前会移除任何已存在的 `.codegraph/` 目录。而代码仅在 `codegraph.db` 存在时才移除该目录 (第 19 行)。如果 `.codegraph/` 存在但不含 `codegraph.db`, 代码不会移除它, 违反了规范。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_b30729f439174a43813d172dbb643773_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_b30729f439174a43813d172dbb643773_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_b30729f439174a43813d172dbb643773_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

- 四类结论： 契约不确定
- 分类依据： probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: [confirmed](#); attempts 1
- **Phase / 模块**: Phase 2 — Language Backend & Function Extraction / [language_handlers](#); 原源码 [src/languages/erlang.py](#)
- **证据**: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: 如果服务器进程未运行或其 `stdin` 流不可写，则引发 `RuntimeError`。否则：消息以 UTF-8 JSON 字节序列序列化，使用最小分隔符（逗号和冒号之间无空白），且不对非 ASCII 字符进行 ASCII 转义；序列化后的字节序列前面加上 LSP Content-Length 头（Content-Length: <octet_count>\r\n\r\n），构成完整帧；在独占获取写锁的情况下将完整帧写入服务器进程的 `stdin` 并刷新流，确保帧作为单个原子单元发送到服务器。成功发送后返回 `None`。当 `stdin` 写入或 `flush` 操作失败时引发 `OSError`（或其子类，如 `BrokenPipeError`）。
- **Actual behavior**: 如果函数未引发异常，则 `self._proc` 和 `self._proc.stdin` 均不为 `None`，并且由 `message` 构造的 JSON-RPC 帧（UTF-8 编码的 JSON，确保非 ASCII 字符并以紧凑分隔符）已在 `with self._write_lock` 块内写入并刷新到服务器的 `stdin`，因此锁现已释放。帧内容确切为 `Content-Length: <payload_length>\r\n\r\n<payload_utf8>`。 `message` 参数保持不变。如果函数引发 `RuntimeError`，则 `self._proc is None or self._proc.stdin is None` 为真，未写入任何帧，锁不受影响，服务器 `stdin` 未被修改。形式逻辑： $(\neg \text{Exception} \Rightarrow (\text{self._proc} \neq \text{None} \wedge \text{self._proc.stdin} \neq \text{None}) \wedge \text{sent}(\text{frame}) \wedge \text{lock_released}(\text{self._write_lock})) \wedge (\text{Exception}(\text{RuntimeError}) \Rightarrow (\text{self._proc} = \text{None} \vee \text{self._proc.stdin} = \text{None}) \wedge \neg \text{sent}(\text{frame}) \wedge \text{lock_unacquired}(\text{self._write_lock}) \wedge \text{stdin_unchanged}(\text{self._proc.stdin}))$ 。
- **Code evidence**: 第 2 行: `if self._proc is None or self._proc.stdin is None:`
- **Trigger condition**: 规范要求当服务器进程未运行或其 `stdin` 流不可写时引发 `RuntimeError`。代码仅检查 `None` 值，遗漏了进程已退出但 `Popen` 对象和 `stdin` 仍然存在的情况。在这种情况下，代码引发 `OSError` 而非 `RuntimeError`，违反了契约。
- **Validator trigger**: 代码仅检查 `self._proc is None`，遗漏了进程已退出但 `Popen` 对象仍然存在的情况；向已失效的 `stdin` 写入会引发 `BrokenPipeError` 而非 `RuntimeError`。
- **Probe stdout**: CONFIRMED — 实际: 引发 `BrokenPipeError` | 预期: `RuntimeError` | 进程已退出 (`poll()=0`) 但 `self._proc` 不为 `None`，因此 `None` 检查通过；向已失效管道写入引发 `BrokenPipeError` 而非 `RuntimeError`

► SPEC sidecar (完整)

```
{
  "signature": "_send(self, message: dict) -> None",
  "pre_condition": "self 持有到一个 ELP 服务器进程的活动连接，该服务器进程的 stdin 流可写。self._write_lock 是一个锁，用于序列化并发写入服务器 stdin 的操作。message 是一个构成 JSON-RPC 2.0 消息的 dict，其顶层值均为 JSON 可序列化。",
  "post_condition": "如果服务器进程未运行或其 stdin 流不可写，则引发 RuntimeError。否则：消息以 UTF-8 JSON 字节序列序列化，使用最小分隔符（逗号和冒号之间无空白），且不对非 ASCII 字符进行 ASCII 转义；序列化后的字节序列前面加上 LSP Content-Length 头（Content-Length: <octet_count>\\r\\n\\r\\n），构成完整帧；在独占获取写锁的情况下将完整帧写入服务器进程的 stdin 并刷新流，确保帧作为单个原子单元发送到服务器。成功发送后返回 None。当 stdin 写入或 flush 操作失败时引发 OSError（或其子类，如 BrokenPipeError）。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/ElpClient::_send.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "如果服务器进程未运行或其 stdin 流不可写，则引发 RuntimeError。否则：消息以 UTF-8 JSON 字节序列序列化，使用最小分隔符（逗号和冒号之间无空白），且不对非 ASCII 字符进行 ASCII 转义；序列化后的字节序列前面加上 LSP Content-Length 头（Content-Length: <octet_count>\\r\\n\\r\\n），构成完整帧；在独占获取写锁的情况下将完整帧写入服务器进程的 stdin 并刷新流，确保帧作为单个原子单元发送到服务器。成功发送后返回 None。当 stdin 写入或 flush 操作失败时引发 OSError（或其子类，如 BrokenPipeError）。",
    "actual_behavior": "如果函数未引发异常，则 self._proc 和 self._proc.stdin 均不为 None，并且由 `message` 构造的 JSON-RPC 帧（UTF-8 编码的 JSON，确保非 ASCII 字符并以紧凑分隔符）已在 `with self._write_lock` 块内写入并刷新到服务器的 stdin，因此锁现已释放。帧内容确切为 `Content-Length: <payload_length>\\r\\n\\r\\n<payload_utf8>`。message 参数保持不变。如果函数引发 `RuntimeError`，则 `self._proc is None or self._proc.stdin is None` 为真，未写入任何帧，锁不受影响，服务器 stdin 未被修改。形式逻辑：(¬Exception ⇒ (self._proc ≠ None ∧ self._proc.stdin ≠ None) ∧ sent(frame) ∧ lock_released(self._write_lock)) ∧ (Exception(RuntimeError) ⇒ (self._proc = None ∨ self._proc.stdin = None) ∧ ¬sent(frame) ∧ lock_unacquired(self._write_lock) ∧ stdin_unchanged(self._proc.stdin)).",
    "code_evidence": "Line 2:         if self._proc is None or self._proc.stdin is None:",
    "trigger_condition": "规范要求当服务器进程未运行或其 stdin 流不可写时引发 RuntimeError。代码仅检查 None 值，遗漏了进程已退出但 Popen 对象和 stdin 仍然存在的情况。在这种情况下，代码引发 OSError 而非 RuntimeError，违反了契约。"
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--erlang-py--ElpClient::_send",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/ElpClient::_send.py",
  "function_name": "ElpClient::_send",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--ElpClient::_send.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--ElpClient::_send.md",
  "probe_stdout": "CONFIRMED — 实际：引发 BrokenPipeError | 预期：RuntimeError | 进程已退出 (poll()==0) 但 self._proc 不为 None，因此 None 检查通过；向已失效管道写入引发 BrokenPipeError 而非 RuntimeError\n",
  "trigger_summary": "代码仅检查 self._proc is None，遗漏了进程已退出但 Popen 对象仍然存在的情况；向已失效的 stdin 写入会引发 BrokenPipeError 而非 RuntimeError。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:01:21.906983Z	2345
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:01:55.844847Z	3971

事件 1 — [llm_2ba1a5ae88914fa09916c958cd2935b5](#)

► parsed_json（完整）

```
{
  "post_condition": "如果函数未引发异常，则 self._proc 和 self._proc.stdin 均不为 None，并且由 `message` 构造的"
```

```
JSON-RPC 帧 (UTF-8 编码的 JSON, 确保非 ASCII 字符并以紧凑分隔符) 已在 `with self._write_lock` 块内写入并刷新到服务器的 stdin, 因此锁现已释放。帧内容确切为 `Content-Length: <payload_length>\r\n\r\n<payload_utf8>`。message 参数保持不变。如果函数引发 `RuntimeError`, 则 `self._proc is None or self._proc.stdin is None` 为真, 未写入任何帧, 锁不受影响, 服务器 stdin 未被修改。形式逻辑:  $(\neg \text{Exception} \Rightarrow (\text{self._proc} \neq \text{None} \wedge \text{self._proc.stdin} \neq \text{None}) \wedge \text{sent}(\text{frame}) \wedge \text{lock\_released}(\text{self._write\_lock})) \wedge (\text{Exception}(\text{RuntimeError}) \Rightarrow (\text{self._proc} = \text{None} \vee \text{self._proc.stdin} = \text{None}) \wedge \neg \text{sent}(\text{frame}) \wedge \text{lock\_unacquired}(\text{self._write\_lock}) \wedge \text{stdin\_unchanged}(\text{self._proc.stdin}))$ 。"
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_2ba1a5ae88914fa09916c958cd2935b5_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_2ba1a5ae88914fa09916c958cd2935b5_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_2ba1a5ae88914fa09916c958cd2935b5_response.txt)

事件 2 — llm_5297329890924aaeb8db853985e24db4

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "假设 self._proc 是一个 Popen 对象, 其进程已经终止, 但 self._proc 不为 None 且 self._proc.stdin 不为 None (管道仍然打开)。使用任意消息 dict 调用 _send, 例如, {\"method\": \"test\"}。代码不会引发 RuntimeError, 继续写入, 并引发 BrokenPipeError (属于 OSError), 而非规范要求的 RuntimeError。",
  "offending_statements": "第 2 行:         if self._proc is None or self._proc.stdin is None:",
  "reason": "规范要求当服务器进程未运行或其 stdin 流不可写时引发 RuntimeError。代码仅检查 None 值, 遗漏了进程已退出但 Popen 对象和 stdin 仍然存在的情况。在这种情况下, 代码引发 OSError 而非 RuntimeError, 违反了契约。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_5297329890924aaeb8db853985e24db4_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_5297329890924aaeb8db853985e24db4_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_5297329890924aaeb8db853985e24db4_response.txt) **复核动作**

- 先由维护者确认预期契约, 再决定修改实现或重写 SPEC; 当前不应直接登记为真实 Bug。

FMA-MISMATCH-086 — src -- languages -- erlang-py -- ElpClient::close

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异, 但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: confirmed; attempts 2
- Phase / 模块: Phase 2 — Language Backend & Function Extraction / language_handlers; 原源码 src/languages/erlang.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** 由 self._proc 引用的 ELP 子进程被终止, 其 stdin 和 stdout 管道被关闭。self._proc 被设置为 None。后台读取线程 self._reader 被停止, 不再从子进程 stdout 消费。与该子进程关联的所有 OS 级资源 (管道、进程句柄) 均被释放。
- **Actual behavior:** 执行后, self._proc 为 None。原始子进程对象 (如果存在) 已被正确处理: 如果它仍在运行, 会尝试优雅地关闭 (通过 request('shutdown')、notify('exit') 以及带超时的等待), 必要时会终止或杀死; 最终该进程不再运行, 其标准输入和输出流已被关闭 (关闭过程中的任何 OSError 被捕获)。self._reader 属性保持不变, 仍引用一个活跃的背景读取线程。该方法没有未处理的异常传播。形式化表述: $(\text{self._proc} = \text{None}) \wedge (\forall s \in \{$

proc.stdin, proc.stdout } : s 已关闭) $\wedge$ (self._reader 是活跃线程) $\wedge$ (最初引用的子进程不再运行, 其终止已被正确等待)。

- **Code evidence:** 第 1-30 行: 整个 close() 方法中没有任何代码用于停止或 join self._reader。
- **Trigger condition:** 规范要求后台读取线程 self._reader 被停止, 不再从子进程 stdout 消费。代码仅终止子进程并关闭其流, 使 self._reader 未被触及且仍活跃, 这违反了规范。
- **Validator trigger:** ElpClient::close() 终止子进程并关闭其流, 但从未停止、join 或将后台读取线程 self._reader 设置为 None, 使其保持存活并仍引用已关闭的子进程 stdout。
- **Probe stdout:** CONFIRMED — _proc is None=True, _reader is alive=True, _reader is None=False

► SPEC sidecar (完整)

```
{
  "signature": "ElpClient.close(self)",
  "pre_condition": "self._proc 引用一个活跃的 ELP 子进程, self._reader 引用一个活跃的后台读取线程。",
  "post_condition": "由 self._proc 引用的 ELP 子进程被终止, 其 stdin 和 stdout 管道被关闭。self._proc 被设置为 None。后台读取线程 self._reader 被停止, 不再从子进程 stdout 消费。与该子进程关联的所有 OS 级资源 (管道、进程句柄) 均被释放。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/ElpClient::close.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "由 self._proc 引用的 ELP 子进程被终止, 其 stdin 和 stdout 管道被关闭。self._proc 被设置为 None。后台读取线程 self._reader 被停止, 不再从子进程 stdout 消费。与该子进程关联的所有 OS 级资源 (管道、进程句柄) 均被释放。",
    "actual_behavior": "执行后, self._proc 为 None。原始子进程对象 (如果存在) 已被正确处理: 如果它仍在运行, 会尝试优雅地关闭 (通过 request('shutdown')、notify('exit') 以及带超时的等待), 必要时会终止或杀死; 最终该进程不再运行, 其标准输入和输出流已被关闭 (关闭过程中的任何 OSError 被捕获)。self._reader 属性保持不变, 仍引用一个活跃的后台读取线程。该方法没有未处理的异常传播。形式化表述: (self._proc = None)  $\wedge$  ( $\forall s \in \{ \text{proc.stdin}, \text{proc.stdout} \} : s$  已关闭)  $\wedge$  (self._reader 是活跃线程)  $\wedge$  (最初引用的子进程不再运行, 其终止已被正确等待)。",
    "code_evidence": "Line 1-30: the entire close() method does not contain any code to stop or join self._reader.",
    "trigger_condition": "规范要求后台读取线程 self._reader 被停止, 不再从子进程 stdout 消费。代码仅终止子进程并关闭其流, 使 self._reader 未被触及且仍活跃, 这违反了规范。"
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--languages--erlang-py--ElpClient::close",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/ElpClient::close.py",
  "function_name": "close",
  "confirmation_status": "confirmed",
  "attempts": 2,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--ElpClient::close.py",
}
```



```
"detail_file": "fm_agent/bug_validation/src--languages--erlang-py--ElpClient::close.md",
"probe_stdout": "CONFIRMED - _proc is None=True, _reader is alive=True, _reader is None=False\n",
"trigger_summary": "ElpClient::close() 终止子进程并关闭其流，但从未停止、join 或将后台读取线程 self._reader 设置为 None，使其保持存活并仍引用已关闭的子进程 stdout。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:47:42.200995Z	2577
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:48:15.423432Z	2346

事件 1 — llm_779cf3cf1cfc4a2bbdce4b1c6e306f36

► parsed_json（完整）

```
{
  "post_condition": "执行后，self._proc 为 None。原始子进程对象（如果存在）已被正确处理：如果它仍在运行，会尝试优雅地关闭（通过 request('shutdown')、notify('exit') 以及带超时的等待），必要时会终止或杀死；最终该进程不再运行，其标准输入和输出流已被关闭（关闭过程中的任何 OSError 被捕获）。self._reader 属性保持不变，仍引用一个活跃的后台读取线程。该方法没有未处理的异常传播。形式化表述：(self._proc = None) ∧ (∀ s ∈ { proc.stdin, proc.stdout } : s 已关闭) ∧ (self._reader 是活跃线程) ∧ (最初引用的子进程不再运行，其终止已被正确等待)。"
}
```

- 载荷：[system_prompt](fm_agent/trace/payloads/llm_779cf3cf1cfc4a2bbdce4b1c6e306f36_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_779cf3cf1cfc4a2bbdce4b1c6e306f36_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_779cf3cf1cfc4a2bbdce4b1c6e306f36_response.txt)

事件 2 — llm_e1c4ad391d9540bbbf8307819b216a41

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "对一个 self._proc 为正在运行的 subprocess.Popen 对象且 self._reader 为循环读取 self._proc.stdout 的活跃 threading.Thread 的实例调用 close()。close() 返回后，self._proc 为 None 且管道已关闭，但 self._reader 仍在运行，可能仍从已关闭 stdout 管道的缓冲区消费。",
  "offending_statements": "第 1-30 行：整个 close() 方法中没有任何代码用于停止或 join self._reader。",
  "reason": "规范要求后台读取线程 self._reader 被停止，不再从子进程 stdout 消费。代码仅终止子进程并关闭其流，使 self._reader 未被触及且仍活跃，这违反了规范。"
}
```

- 载荷：[system_prompt](fm_agent/trace/payloads/llm_e1c4ad391d9540bbbf8307819b216a41_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_e1c4ad391d9540bbbf8307819b216a41_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_e1c4ad391d9540bbbf8307819b216a41_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

- **四类结论：** 契约不确定
- **分类依据：** probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator：** **confirmed**；attempts **1**
- **Phase / 模块：** 阶段 2 — 语言后端与函数提取 / **language_handlers**；原源码 **src/languages/erlang.py**
- **证据：** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator 结果](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC 声明：** 返回一个 `_SourceIndex` 实例，该实例按行边界索引源码。返回的实例包含完整的源码文本逐字副本（保留所有行结束符）以及一个从零开始的字节偏移量列表，其中每个偏移量是每一行在源码中的起始位置。第一个偏移量始终为 0。后续偏移量单调递增——每个偏移量等于其前面所有行（包括它们的行结束符）的字节长度之和。偏移量的数量等于源码中由换行符分隔的行数。返回的实例使调用者能够将任何有效的行索引和字符位置映射到源码中对应的字节偏移量，并提取从起始字节偏移量（含）到结束字节偏移量（不含）的连续子串。
- **实际行为：** 返回一个 `_SourceIndex` 实例 `r`，满足 `r.source == source`，`r.lines == source.splitlines(keepends=True)`，且对于所有 `i` 在 `0..len(r.lines)-1`：`r.line_offsets[i] == sum(len(r.lines[j]) for j in range(i))`。
- **代码证据：** 第 7 行：`offset += len(line)`
- **触发条件：** 规范明确要求使用指向源文本的字节偏移量，但代码使用了 `len(line)`，该函数返回 `Unicode` 码位（字符）的数量。对于包含多字节字符（如非 `ASCII` 字符）的字符串，字符长度与字节长度不同，导致字节偏移量不正确。反例使用包含双字节字符 " 的字符串演示了这一点。
- **Validator 触发：** 多字节 UTF-8 字符 (é) 导致 `len()` 返回字符数 (3) 而非字节长度 (4)，在 `line_offsets` 中产生不正确的字节偏移量。
- **Probe stdout：** 已确认 — 实际（字符偏移量）：`[0, 3]` | 预期（字节偏移量）：`[0, 4, 6]`

► SPEC sidecar（完整）

```
{
  "signature": "_SourceIndex.build(cls, source: str) -> _SourceIndex",
  "pre_condition": "source 是一个包含源码文本的非空字符串。",
  "post_condition": "返回一个 _SourceIndex 实例，该实例按行边界索引源码。返回的实例包含完整的源码文本逐字副本（保留所有行结束符）以及一个从零开始的字节偏移量列表，其中每个偏移量是每一行在源码中的起始位置。第一个偏移量始终为 0。后续偏移量单调递增——每个偏移量等于其前面所有行（包括它们的行结束符）的字节长度之和。偏移量的数量等于源码中由换行符分隔的行数。返回的实例使调用者能够将任何有效的行索引和字符位置映射到源码中对应的字节偏移量，并提取从起始字节偏移量（含）到结束字节偏移量（不含）的连续子串。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_SourceIndex::build.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个 _SourceIndex 实例，该实例按行边界索引源码。返回的实例包含完整的源码文本逐字副本（保留所有行结束符）以及一个从零开始的字节偏移量列表，其中每个偏移量是每一行在源码中的起始位置。第一个偏移量始终为 0。后续偏移量单调递增——每个偏移量等于其前面所有行（包括它们的行结束符）的字节长度之和。偏移量的数量等于源码中由换行符分隔的行数。返回的实例使调用者能够将任何有效的行索引和字符位置映射到源码中对应的字节偏移量，并提取从起始字节偏移量（含）到结束字节偏移量（不含）的连续子串。",
    "actual_behavior": "返回一个 _SourceIndex 实例 r，满足 r.source == source，r.lines == source.splitlines(keepends=True)，且对于所有 i 在 0..len(r.lines)-1：r.line_offsets[i] == "
  }
}
```

```
sum(len(r.lines[j]) for j in range(i)).",
    "code_evidence": "Line 7: offset += len(line)",
    "trigger_condition": "规范明确要求使用指向源文本的字节偏移量，但代码使用了 len(line)，该函数返回 Unicode 码位（字
符）的数量。对于包含多字节字符（如非 ASCII 字符）的字符串，字符长度与字节长度不同，导致字节偏移量不正确。反例使用包含双字节字
符 ' ' 的字符串演示了这一点。"
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--erlang-py--_SourceIndex::build",
  "source_file": "/home/fancy/Projects/Vault/FM-Agent/src/languages/erlang.py",
  "function_name": "_SourceIndex.build",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--_SourceIndex::build.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--_SourceIndex::build.md",
  "probe_stdout": "已确认 - 实际（字符偏移量）：[0, 3] | 预期（字节偏移量）：[0, 4, 6]\n",
  "trigger_summary": "多字节 UTF-8 字符（é）导致 len() 返回字符数（3）而非字节长度（4），在 line_offsets 中产生不正
确的字节偏移量。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:29:08.385080Z	1438
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:29:24.499742Z	3228

事件 1 — [llm_023b2ff038a04aa9bf1b5fb4dcc6605b](#)

► parsed_json（完整）

```
{
  "post_condition": "返回一个 _SourceIndex 实例 r，满足 r.source == source，r.lines ==
source.splitlines(keepends=True)，且对于所有 i 在 0..len(r.lines)-1：r.line_offsets[i] ==
sum(len(r.lines[j]) for j in range(i))."
}
```

- 载荷：[\[system_prompt\]](#)

[\(fm_agent/trace/payloads/llm_023b2ff038a04aa9bf1b5fb4dcc6605b_message_00_system.txt\)](#) · [\[user_prompt\]](#)
[\(fm_agent/trace/payloads/llm_023b2ff038a04aa9bf1b5fb4dcc6605b_message_01_user.txt\)](#) · [\[assistant_output\]](#)
[\(fm_agent/trace/payloads/llm_023b2ff038a04aa9bf1b5fb4dcc6605b_response.txt\)](#)

事件 2 — [llm_c42243a98c914b0e8d492b350cccace5](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "输入 source = \"café\\nhello\"。在 UTF-8 中，第一行 'café\\n' 为 6 字节，但
```

```
len('café\n') 返回 5。代码计算 line_offsets = [0, 5]，而规范要求字节偏移量，因此第二行的正确起始字节偏移量为 6。返回的实例无法提供正确的字节偏移量。",
"offending_statements": "第 7 行: offset += len(line)",
"reason": "规范明确要求使用指向源文本的字节偏移量，但代码使用了 len(line)，该函数返回 Unicode 码位（字符）的数量。对于包含多字节字符（如非 ASCII 字符）的字符串，字符长度与字节长度不同，导致字节偏移量不正确。反例使用包含双字节字符 'é' 的字符串演示了这一点。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_c42243a98c914b0e8d492b350cccace5_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_c42243a98c914b0e8d492b350cccace5_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_c42243a98c914b0e8d492b350cccace5_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-092 — [src--languages--erlang-py--_SourceIndex::position_to_offset](#)

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: [confirmed](#); attempts 1
- Phase / 模块：Phase 2 — Language Backend & Function Extraction / [language_handlers](#)；原源码 [src/languages/erlang.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: 返回一个在闭区间 $[0, \text{len}(\text{self.source})]$ 内的整数，表示与给定位置对应的 `self.source` 中的字节偏移量。当行号等于或超过已索引行数时，返回 `len(self.source)`。当 UTF-16 代码单元列偏移量达到或超过目标行上 UTF-16 代码单元的数量时，返回该行之后第一个字节的字节偏移量。Unicode 码点高于 U+FFFF 的字符将 UTF-16 代码单元列偏移量增加 2；码点等于或低于 U+FFFF 的字符将其增加 1。对于任意两个位置 p_1 和 p_2 ，如果 p_1 在相对于已索引源的文档顺序中不晚于 p_2 ，则 $\text{position_to_offset}(\text{self}, p_1) \leq \text{position_to_offset}(\text{self}, p_2)$ 。
- **Actual behavior**: 方法返回一个整数偏移量 r ，且无副作用。令 $\text{line_number} = \max(0, \text{int}(\text{position.get('line', 0)}))$ 且 $\text{utf16_target} = \max(0, \text{int}(\text{position.get('character', 0)}))$ 。若 $\text{line_number} \geq \text{len}(\text{self.lines})$ ，则 $r = \text{len}(\text{self.source})$ 。否则，令 $\text{base} = \text{self.line_offsets}[\text{line_number}]$ 且 $L = \text{self.lines}[\text{line_number}]$ 。定义 $\text{unit}(c) = 2$ 若 $\text{ord}(c) > 0xFFFF$ ，否则 1。令 $\text{index} = \max \{ k \in [0, \text{len}(L)] \mid \bigwedge_{i=0}^{k-1} \text{unit}(L[i]) \leq \text{utf16_target} \}$ 。则 $r = \text{base} + \text{index}$ 。从位置到偏移量的映射在行上使用基于字符的索引，加上一个字节偏移量基址，如果源包含多字节字符，结果可能并不对应真实的字节偏移量。
- **Code evidence**: 第 16 行: `return base + index`
- **Trigger condition**: 代码将偏移量视为加到字节偏移量基址上的基于字符的索引。只有当所有字符都是单字节（ASCII）时，才能得到正确的字节偏移量。存在多字节字符时，返回值不是 `self.source` 中所需的字节偏移量。对于高于 U+FFFF 的字符，例如 U+1F600 (😄)，`len(line)=1`，但其字节长度为 4，因此 $\text{base} + \text{index} = 0 + 1 = 1$ ，违反了规范要求的 4。
- **Validator trigger**: 使用源 `'😄\n'` 和位置 `{line: 0, character: 2}` 调用 `_position_to_offset`；返回字符索引 1 而非字节偏移量 4，因为对于多字节 UTF-8 字符，`index += 1` 按字符计数而非字节计数。
- **Probe stdout**: CONFIRMED — 实际值: 1 | 期望值: 4

► SPEC sidecar（完整）

```
{
  "signature": "position_to_offset(self, position: dict) -> int",
  "pre_condition": "self 是一个 _SourceIndex 实例，其 source、lines 和 line_offsets 属性由同一源文本初始化；
line_offsets 中的每个条目是对应行在 source 中的字节偏移量。position 是一个字典，包含可选的 'line' 条目（默认为 0），表示从
零开始的行号；以及可选的 'character' 条目（默认为 0），表示该行内从零开始的 UTF-16 代码单元列偏移量。",
```

```
"post_condition": "返回一个在闭区间 [0, len(self.source)] 内的整数，表示与给定位置对应的 self.source 中的字节偏移量。当行号等于或超过已索引行数时，返回 len(self.source)。当 UTF-16 代码单元列偏移量达到或超过目标行上 UTF-16 代码单元的数量时，返回该行之后第一个字节的字节偏移量。Unicode 码点高于 U+FFFF 的字符将 UTF-16 代码单元列偏移量增加 2；码点等于或低于 U+FFFF 的字符将其增加 1。对于任意两个位置 p1 和 p2，如果 p1 在相对于已索引源的文档顺序中不晚于 p2，则 position_to_offset(self, p1) <= position_to_offset(self, p2)。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_SourceIndex::position_to_offset.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个在闭区间 [0, len(self.source)] 内的整数，表示与给定位置对应的 self.source 中的字节偏移量。当行号等于或超过已索引行数时，返回 len(self.source)。当 UTF-16 代码单元列偏移量达到或超过目标行上 UTF-16 代码单元的数量时，返回该行之后第一个字节的字节偏移量。Unicode 码点高于 U+FFFF 的字符将 UTF-16 代码单元列偏移量增加 2；码点等于或低于 U+FFFF 的字符将其增加 1。对于任意两个位置 p1 和 p2，如果 p1 在相对于已索引源的文档顺序中不晚于 p2，则 position_to_offset(self, p1) <= position_to_offset(self, p2)。",
    "actual_behavior": "方法返回一个整数偏移量 r，且无副作用。令 line_number = max(0, int(position.get('line', 0))) 且 utf16_target = max(0, int(position.get('character', 0)))。若 line_number >= len(self.lines)，则 r = len(self.source)。否则，令 base = self.line_offsets[line_number] 且 L = self.lines[line_number]。定义 unit(c) = 2 若 ord(c) > 0xFFFF，否则 1。令 index = max { k [0, len(L)] | _{i=0}^{k-1} unit(L[i]) <= utf16_target }。则 r = base + index。从位置到偏移量的映射在行上使用基于字符的索引，加上一个字节偏移量基址，如果源包含多字节字符，结果可能并不对应真实的字节偏移量。",
    "code_evidence": "Line 16: return base + index",
    "trigger_condition": "代码将偏移量视为加到字节偏移量基址上的基于字符的索引。只有当所有字符都是单字节（ASCII）时，才能得到正确的字节偏移量。存在多字节字符时，返回值不是 self.source 中所需的字节偏移量。对于高于 U+FFFF 的字符，例如 U+1F600 ('')，len(line)=1，但其字节长度为 4，因此 base + index = 0 + 1 = 1，违反了规范要求的 4。"
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--erlang-py--_SourceIndex::position_to_offset",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_SourceIndex::position_to_offset.py",
  "function_name": "_SourceIndex::position_to_offset",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--_SourceIndex::position_to_offset.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--_SourceIndex::position_to_offset.md",
  "probe_stdout": "CONFIRMED - 实际值: 1 | 期望值: 4\n",
  "trigger_summary": "使用源 '😄\\n' 和位置 {line: 0, character: 2} 调用 _position_to_offset；返回字符索引 1 而非字节偏移量 4，因为对于多字节 UTF-8 字符，index += 1 按字符计数而非字节计数。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	目的	状态	尝试	块	开始	令牌数
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:39:06.866213Z	2851
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:39:54.207265Z	4402

事件 1 — llm_b8564e5e13df42298ef40fa2f7d79850

► parsed_json (完整)

```
{
  "post_condition": "方法返回一个整数偏移量 r ,且无副作用。令 line_number = max(0, int(position.get('line', 0))) 且 utf16_target = max(0, int(position.get('character', 0)))。若 line_number >= len(self.lines), 则 r = len(self.source)。否则, 令 base = self.line_offsets[line_number] 且 L = self.lines[line_number]。定义 unit(c) = 2 若 ord(c) > 0xFFFF, 否则 1。令 index = max { k ∈ [0, len(L)] | Σ_{i=0}^{k-1} unit(L[i]) <= utf16_target }。则 r = base + index。从位置到偏移量的映射在行上使用基于字符的索引, 加上一个字节偏移量基址, 如果源包含多字节字符, 结果可能并不对应真实的字节偏移量。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_b8564e5e13df42298ef40fa2f7d79850_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_b8564e5e13df42298ef40fa2f7d79850_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_b8564e5e13df42298ef40fa2f7d79850_response.txt)

事件 2 — llm_d882e6f0c15347e3a2ad18773a09a4af

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "self.lines = ['\\U0001F600']; self.line_offsets = [0]; self.source = '\\U0001F600'; position = {'line': 0, 'character': 2}; self.position_to_offset(position) 返回 1, 但规范要求该行之后的字节偏移量, 对于 UTF-8 编码此处应为 4。",
  "offending_statements": "第 16 行: return base + index",
  "reason": "代码将偏移量视为加到字节偏移量基址上的基于字符的索引。只有当所有字符都是单字节 (ASCII) 时, 才能得到正确的字节偏移量。存在多字节字符时, 返回值不是 self.source 中所需的字节偏移量。对于高于 U+FFFF 的字符, 例如 U+1F600 ('😄'), len(line)=1, 但其字节长度为 4, 因此 base + index = 0 + 1 = 1, 违反了规范要求的 4。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_d882e6f0c15347e3a2ad18773a09a4af_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_d882e6f0c15347e3a2ad18773a09a4af_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_d882e6f0c15347e3a2ad18773a09a4af_response.txt) **复核动作**

- 先由维护者确认预期契约, 再决定修改实现或重写 SPEC; 当前不应直接登记为真实 Bug。

Phase 4 — LLM Communication & Tracing

模块: llm_client

FMA-MISMATCH-157 — src--llm_client-py--_matches_inject_target

- 四类结论: 契约不确定
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。

- **Validator:** [confirmed](#); attempts [1](#)
- **Phase / 模块:** Phase 4 — LLM Communication & Tracing / [llm_client](#); 原源码 [src/llm_client.py](#)
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** 当判定 url 属于 target 的地址空间时返回 True; 否则返回 False。当 target 以 'http://' 或 'https://' 开头 (不区分大小写) 时, 需要 url 以 target 作为大小写敏感的前缀才认为属于。当 target 没有 scheme 时, 需要从 url 中提取的主机名组件与 target 完全相等, 或者以 '.' 加上 target 结尾 (表示子域名) 才认为属于。当无法从 url 中提取主机名时返回 False。
- **实际行为:** 该函数返回一个布尔值。如果 target 的小写版本以 'http://' 或 'https://' 开头, 则当 True 为真时函数返回 url.startswith(target), 否则返回 False。否则, 函数尝试使用 url 从 urllib.parse.urlparse 解析主机名。如果解析过程中发生异常, 则返回 False。如果解析成功, 则提取 host = urlparse(url).hostname or '' (如果主机名为 None 则为空字符串)。然后, 如果 True 或 host == target 则返回 host.endswith('.') + target, 否则返回 False。不修改其他状态。形式化定义为, 设 low = target.lower(); scheme = low.startswith('http://') or low.startswith('https://'); 则返回值 r 满足: r = (scheme 且 url.startswith(target)) 或 (非 scheme 且 [如果没有异常且 host = (urlparse(url).hostname or "") 则 (host = target 或 host.endsWith('.') + target)) 否则 False])。
- **代码证据:** 第 5 行: host = urllib.parse.urlparse(url).hostname or "" 第 8 行: return host == target or host.endswith('.') + target
- **触发条件:** 主机名不区分大小写, 但代码执行的是大小写敏感的相等性检查。当 target 为 "example.com" (无 scheme) 且 url 为 "http://EXAMPLE.COM" 时, urllib.parse.urlparse(url).hostname 返回 "EXAMPLE.COM"。然后代码比较 "EXAMPLE.COM" == "example.com" (False) 以及 "EXAMPLE.COM".endswith(".example.com") (False), 返回 False。根据规约, 主机名 "EXAMPLE.COM" 属于 "example.com" 的地址空间, 因此函数应返回 True。
- **Validator 触发条件:** 当 target 包含大写 ASCII 字母时 (例如, 'EXAMPLE.COM' 与 urlparse 得到的主机名 'example.com' 比较), 第 8 行的大小写敏感 host == target 比较会失败, 这违反了 DNS 不区分大小写的规则。
- **Probe stdout:** CONFIRMED — url='http://example.com/some/path', target='EXAMPLE.COM', actual=False, expected=True

► SPEC sidecar (完整)

```
{
  "signature": "_matches_inject_target(url, target) -> bool",
  "pre_condition": "url 是一个非 None 字符串 (假值视为空), 其尾部的斜杠已被去除。target 是一个非空字符串, 表示要匹配的主机模式。",
  "post_condition": "当判定 url 属于 target 的地址空间时返回 True; 否则返回 False。当 target 以 'http://' 或 'https://' 开头 (不区分大小写) 时, 需要 url 以 target 作为大小写敏感的前缀才认为属于。当 target 没有 scheme 时, 需要从 url 中提取的主机名组件与 target 完全相等, 或者以 '.' 加上 target 结尾 (表示子域名) 才认为属于。当无法从 url 中提取主机名时返回 False。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► 逻辑验证结果 (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/llm_client-
py/_matches_inject_target.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "当判定 url 属于 target 的地址空间时返回 True;否则返回 False。当 target 以 'http://' 或
'https://' 开头（不区分大小写）时，需要 url 以 target 作为大小写敏感的前缀才认为属于。当 target 没有 scheme 时，需要从
url 中提取的主机名组件与 target 完全相等，或者以 '.' 加上 target 结尾（表示子域名）才认为属于。当无法从 url 中提取主机名
时返回 False。",
    "actual_behavior": "该函数返回一个布尔值。如果 `target` 的小写版本以 'http://' 或 'https://' 开头，则当
`url.startswith(target)` 为真时函数返回 `True`，否则返回 `False`。否则，函数尝试使用 `urllib.parse.urlparse` 从
`url` 解析主机名。如果解析过程中发生异常，则返回 `False`。如果解析成功，则提取 `host = urlparse(url).hostname or ''`
（如果主机名为 `None` 则为空字符串）。然后，如果 `host == target` 或 `host.endswith('.') + target` 则返回 `True`，
否则返回 `False`。不修改其他状态。形式化定义为，设 low = target.lower(); scheme = low.startswith('http://') or
low.startswith('https://');则返回值 r 满足:r = ( scheme 且 url.startswith(target) ) 或 ( 非 scheme 且 [如果没有
异常且 host = (urlparse(url).hostname or '') 则 (host = target 或 host.endswith('.') + target)) 否则 False]
)。",
    "code_evidence": "Line 5: host = urllib.parse.urlparse(url).hostname or \"\"\nLine 8: return host ==
target or host.endswith(\".\" + target)",
    "trigger_condition": "主机名不区分大小写，但代码执行的是大小写敏感的相等性检查。当 target 为 \"example.com\" (无
scheme) 且 url 为 \"http://EXAMPLE.COM\" 时，urllib.parse.urlparse(url).hostname 返回 \"EXAMPLE.COM\"。然后代
码比较 \"EXAMPLE.COM\" == \"example.com\" (False) 以及 \"EXAMPLE.COM\".endswith(\".example.com\") (False)，
返回 False。根据规约，主机名 \"EXAMPLE.COM\" 属于 \"example.com\" 的地址空间，因此函数应返回 True。"
  }
}
```

► Bug Validator 结果 (完整)

```
{
  "id": "src--llm_client-py--_matches_inject_target",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/llm_client-
py/_matches_inject_target.py",
  "function_name": "_matches_inject_target",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--llm_client-py--_matches_inject_target.py",
  "detail_file": "fm_agent/bug_validation/src--llm_client-py--_matches_inject_target.md",
  "probe_stdout": "CONFIRMED - url='http://example.com/some/path', target='EXAMPLE.COM', actual=False,
expected=True\n",
  "trigger_summary": "当 target 包含大写 ASCII 字母时（例如，'EXAMPLE.COM' 与 urlparse 得到的主机名
'example.com' 比较），第 8 行的大小写敏感 host == target 比较会失败，这违反了 DNS 不区分大小写的规则。"
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:19:31.041222Z	2408
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:20:10.164167Z	8048

事件 1 — llm_29d4f31220fb4fd6a6d9681c82c2d8da

► parsed_json (完整)

```
{
  "post_condition": "该函数返回一个布尔值。如果 `target` 的小写版本以 'http://' 或 'https://' 开头，则当
```

```
url.startswith(target)`为真时函数返回`True`, 否则返回`False`。否则, 函数尝试使用`urllib.parse.urlparse`从`url`解析主机名。如果解析过程中发生异常, 则返回`False`。如果解析成功, 则提取`host = urlparse(url).hostname or ''` (如果主机名为`None`则为空字符串)。然后, 如果`host == target`或`host.endswith('.') + target`则返回`True`, 否则返回`False`。不修改其他状态。形式化定义为, 设`low = target.lower(); scheme = low.startswith('http://') or low.startswith('https://');`则返回值`r`满足:  $r = (scheme \wedge url.startswith(target)) \vee (\neg scheme \wedge [\text{如果没有异常且 } host = (urlparse(url).hostname \text{ or } '') \text{ 则 } (host = target \vee host.endswith('.') + target)])$  否则`False`。"
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_29d4f31220fb4fd6a6d9681c82c2d8da_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_29d4f31220fb4fd6a6d9681c82c2d8da_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_29d4f31220fb4fd6a6d9681c82c2d8da_response.txt)

事件 2 — llm_ec6e7b3ef0014c1a87a2226b1a80e17e

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "url = \"http://EXAMPLE.COM\", target = \"example.com\"",
  "offending_statements": "第 5 行: host = urllib.parse.urlparse(url).hostname or \"\"\\n第 8 行: return host == target or host.endswith(\".\" + target)",
  "reason": "主机名不区分大小写, 但代码执行的是大小写敏感的相等性检查。当 target 为 \"example.com\" (无 scheme) 且 url 为 \"http://EXAMPLE.COM\" 时, urllib.parse.urlparse(url).hostname 返回 \"EXAMPLE.COM\"。然后代码比较 \"EXAMPLE.COM\" == \"example.com\" (False) 以及 \"EXAMPLE.COM\".endswith(\".example.com\") (False), 返回 False。根据规约, 主机名 \"EXAMPLE.COM\" 属于 \"example.com\" 的地址空间, 因此函数应返回 True。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_ec6e7b3ef0014c1a87a2226b1a80e17e_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_ec6e7b3ef0014c1a87a2226b1a80e17e_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_ec6e7b3ef0014c1a87a2226b1a80e17e_response.txt) **复核动作**

- 先由维护者确认预期契约, 再决定修改实现或重写 SPEC; 当前不应直接登记为真实 Bug。

模块: opencode_trace

FMA-MISMATCH-165 — src--opencode_trace-py--_opencode_env

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异, 但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator:** **confirmed**; attempts 1
- **Phase / 模块:** Phase 4 — LLM Communication & Tracing / **opencode_trace**; 原源码 **src/opencode_trace.py**
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** 返回一个字符串键到字符串值的字典, 适合用作子进程环境。返回的字典包含当前进程环境的每一个条目。它包含一个 TRACE_DIR 条目, 其值为一个绝对文件系统路径, 该路径指定一个在调用后存在于文件系统上、且位于以 work_dir 为根的 trace 结构下的目录。它包含一个 TRACE_FILENAME 条目, 其值等于 event_id。它包含一个 PWD 条目, 其值为直接包含 work_dir 的目录的绝对路径。当所有已配置的 LLM API key、base URL、model name 和 provider 均可用 (即 provider 配置完整) 时, 返回的字典额外包含一个 LLM_API_KEY 条目, 其值为配置的 API key, 以及一个 OPENCODE_CONFIG_CONTENT 条目, 其值为一个有效的 JSON 字符

串，编码一个字典，该字典的顶级键是环境中任何已存在的 `OPENCODE_CONFIG_CONTENT` 值的顶级键与解析的 `provider` 配置的顶级键的并集，其中 `provider` 定义的键覆盖同键的预先存在的条目；当 `provider` 配置不完整时，既不会出现 `LLM_API_KEY`，也不会出现修改过的 `OPENCODE_CONFIG_CONTENT`。

- **Actual behavior:** 在正常完成时（即函数返回一个字典而没有引发异常），以下成立：

1. 文件系统副作用

位于

```
td = os.path.abspath(os.path.join(_trace_dir(work_dir), 'opencode'))
```

的目录存在。如果在调用前不存在，则被创建。

2. 返回的环境字典

设 `E0` 是函数入口时对 `os.environ` 的快照。返回的映射 `env` 满足：

- `env['TRACE_DIR'] = td`
- `env['TRACE_FILENAME'] = event_id`
- `env['PWD'] = os.path.dirname(os.path.abspath(work_dir))`
- 如果 `cfg = _opencode_provider_config()` 不是 `None`，那么：
 - `env['LLM_API_KEY'] = settings.llm.api_key`
 - 令 `raw = E0.get('OPENCODE_CONFIG_CONTENT')`。如果 `raw` 是一个可以被 `json.loads` 解析为字典的字符串，那么 `base = that dictionary`；否则 `base = {}`。然后 `env['OPENCODE_CONFIG_CONTENT'] = json.dumps(_deep_merge(base, cfg))`。否则（当 `cfg` 为 `None` 时），`'LLM_API_KEY'` 和 `'OPENCODE_CONFIG_CONTENT'` 在 `env` 中的值正好是 `E0` 中存在的值（如果有的话）。
- 对于 `k` 中存在的每一个其他键 `E0`，`env[k] = E0[k]`。`env` 的键的集合正好是 `keys(E0) - {'TRACE_DIR', 'TRACE_FILENAME', 'PWD'}`（如果 `cfg` 为 `None`），没有额外的键。

3. 没有其他可观察的状态变化

除了目录创建，文件系统没有变化；没有网络或其他副作用发生。

形式化逻辑（针对返回值与文件系统表达）：

令 `fs` 为调用后的文件系统状态。后置条件 `Post(env, fs)` 成立当且仅当：

```
td, base, raw, cfg :
  td = abspath(join(_trace_dir(work_dir), 'opencode'))
  directory_exists(td, fs)
  cfg = _opencode_provider_config()
  env = E0 {
    'TRACE_DIR': td,
    'TRACE_FILENAME': event_id,
    'PWD': dirname(abspath(work_dir))
  }
  ( if cfg None then {
    'LLM_API_KEY': settings.llm.api_key,
    'OPENCODE_CONFIG_CONTENT': json.dumps(_deep_merge(base, cfg))
  } else )
  ( if cfg None then
    raw = E0.get('OPENCODE_CONFIG_CONTENT')
    ( if (raw is str json.loads(raw) succeeds the result is a dict) then
      base = that dict
    else
      base = {}
    )
  else true
  )
```

(此处 `` 表示映射的函数覆盖：对于给定的基础映射 M 和更新集合 U , $M \cup U$ 将每个键 k 映射到 $U[k]$, 如果 $k \in \text{dom}(U)$ 为真, 否则映射到 $M[k]$ 。)

- **Code evidence:** 第 25 行: `env["OPENCODE_CONFIG_CONTENT"] = json.dumps(_deep_merge(existing, provider_config))`
- **Trigger condition:** 规范指出“provider 定义的键覆盖同键的预先存在的条目”(即浅覆盖)。代码使用了 `_deep_merge`, 它会递归合并嵌套字典, 导致应被替换的预先存在的嵌套键仍然存留。在一个 `OPENCODE_CONFIG_CONTENT` 包含的嵌套键也出现在 `provider` 配置中的具体环境中, 会暴露该差异。
- **Validator trigger:** 预先存在的 `OPENCODE_CONFIG_CONTENT` 含有嵌套的 `provider` 键: `deep_merge` 保留了应按照规范被 `provider` 配置覆盖的现有嵌套键 (例如 `'plugins'`)。
- **Probe stdout:** `CONFIRMED -- deep_merge` 保留了现有嵌套键 `"plugins": ['plugin-a', 'plugin-b']` 完整的 `provider` 块: `{ "npm": "@ai-sdk/openai-compatible", "plugins": [ "plugin-a", "plugin-b" ], "options": { "baseUrl": "https://test.example.com/api/v1", "apiKey": "{env:LLM_API_KEY}" }, "models": { "test-model": {} } }`

► SPEC sidecar (完整)

```
{
  "signature": "_opencode_env(work_dir, event_id) -> dict[str, str]",
  "pre_condition": "work_dir 是一个有效的文件系统目录路径; event_id 是一个非空字符串",
  "post_condition": "返回一个字符串键到字符串值的字典, 适合作为子进程环境。返回的字典包含当前进程环境的每一个条目。它包含一个 TRACE_DIR 条目, 其值为一个绝对文件系统路径, 该路径指定一个在调用后存在于文件系统上、且位于以 work_dir 为根的 trace 结构下的目录。它包含一个 TRACE_FILENAME 条目, 其值等于 event_id。它包含一个 PWD 条目, 其值为直接包含 work_dir 的目录的绝对路径。当所有已配置的 LLM API key、base URL、model name 和 provider 均可用 (即 provider 配置完整) 时, 返回的字典额外包含一个 LLM_API_KEY 条目, 其值为配置的 API key, 以及一个 OPENCODE_CONFIG_CONTENT 条目, 其值为一个有效的 JSON 字符串, 编码一个字典, 该字典的顶级键是环境中任何已存在的 OPENCODE_CONFIG_CONTENT 值的顶级键与解析的 provider 配置的顶级键的并集, 其中 provider 定义的键覆盖同键的预先存在的条目; 当 provider 配置不完整时, 既不会出现 LLM_API_KEY, 也不会出现修改过的 OPENCODE_CONFIG_CONTENT。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_trace_dir",
      "signature": "_trace_dir(work_dir) -> str",
      "pre_condition": "work_dir 是一个有效的文件系统目录路径",
      "post_condition": "返回一个文件系统路径字符串, 该字符串是 work_dir 下名为 'trace' 的子目录"
    },
    {
      "name": "_opencode_provider_config",
      "signature": "_opencode_provider_config() -> dict | None",
      "pre_condition": "无 (不接受参数)",
      "post_condition": "当已配置的 LLM 设置 (api_key、base_url、name 和 provider) 全部存在且非空时, 返回一个表示 OpenCode provider 块的字典; 当任何必需设置缺失或为 falsy 时, 返回 None, 表示不应注入 any provider 配置"
    },
    {
      "name": "_deep_merge",
      "signature": "_deep_merge(base, overlay) -> dict",
      "pre_condition": "base 和 overlay 是字典",
      "post_condition": "返回一个字典, 其顶级键是 base 和 overlay 的顶级键的并集。对于在 base 和 overlay 中值均为 dict 的任何键, 返回的值是这些嵌套 dict 的递归合并。对于在两个字典中值不都是 dict 的键, 或者只在一个输入中存在的键, 当键存在于 overlay 时, 返回的值是 overlay 值, 否则是 base 值"
    }
  ]
}
```

► Logic verification result (完整)


```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-
py/_opencode_env.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个字符串键到字符串值的字典，适合用作子进程环境。返回的字典包含当前进程环境的每一个条目。它包含一个
TRACE_DIR 条目，其值为一个绝对文件系统路径，该路径指定一个在调用后存在于文件系统上、且位于以 work_dir 为根的 trace 结构下
的目录。它包含一个 TRACE_FILENAME 条目，其值等于 event_id。它包含一个 PWD 条目，其值为直接包含 work_dir 的目录的绝对路
径。当所有已配置的 LLM API key、base URL、model name 和 provider 均可用（即 provider 配置完整）时，返回的字典额外包含
一个 LLM_API_KEY 条目，其值为配置的 API key，以及一个 OPENCODE_CONFIG_CONTENT 条目，其值为一个有效的 JSON 字符串，编
码一个字典，该字典的顶级键是环境中任何已存在的 OPENCODE_CONFIG_CONTENT 值的顶级键与解析的 provider 配置的顶级键的并集，
其中 provider 定义的键覆盖同键的预先存在的条目；当 provider 配置不完整时，既不会出现 LLM_API_KEY，也不会出现修改过的
OPENCODE_CONFIG_CONTENT。",
    "actual_behavior": "在正常完成时（即函数返回一个字典而没有引发异常），以下成立：\n\n1. **文件系统副作用** \n  位于 \n
    \td = os.path.abspath(os.path.join(_trace_dir(work_dir), 'opencode'))` \n  的目录存在。如果在调
用前不存在，则被创建。\n\n2. **返回的环境字典** \n  设 `E0` 是函数入口时对 `os.environ` 的快照。返回的映射 `env` 满
足：\n\n  - `env['TRACE_DIR'] = td`\n  - `env['TRACE_FILENAME'] = event_id`\n  - `env['PWD'] =
os.path.dirname(os.path.abspath(work_dir))`\n  - 如果 `cfg = _opencode_provider_config()` 不是 `None`，那
么：\n    * `env['LLM_API_KEY'] = settings.llm.api_key`\n    * 令 `raw =
E0.get('OPENCODE_CONFIG_CONTENT')`。如果 `raw` 是一个可以被 `json.loads` 解析为字典的字符串，那么 `base = that
dictionary`；否则 `base = {}`。然后 \n
    \n      `env['OPENCODE_CONFIG_CONTENT'] =
json.dumps(_deep_merge(base, cfg))`。 \n
    \n      否则（当 `cfg` 为 `None` 时），`'LLM_API_KEY'` 和
`'OPENCODE_CONFIG_CONTENT'` 在 `env` 中的值正好是 `E0` 中存在的值（如果有的话）。 \n
    \n      - 对于 `E0` 中存在的每一个其他
键 `k`，`env[k] = E0[k]`。 `env` 的键的集合正好是 `keys(E0) - {'TRACE_DIR', 'TRACE_FILENAME', 'PWD'}`
（`{'LLM_API_KEY', 'OPENCODE_CONFIG_CONTENT'}` if `cfg` None），没有额外的键。 \n\n3. **没有其他可观察的状态变化** \n
除了目录创建，文件系统没有变化；没有网络或其他副作用发生。 \n\n**形式化逻辑**（针对返回值与文件系统表达）： \n\n令 `fs` 为调
用后的文件系统状态。后置条件 `Post(env, fs)` 成立当且仅当： \n\n```\n td, base, raw, cfg : \n  td =
abspath(join(_trace_dir(work_dir), 'opencode')) \n  directory_exists(td, fs) \n  cfg =
_opencode_provider_config() \n  env = E0 - { \n    'TRACE_DIR': td, \n    'TRACE_FILENAME':
event_id, \n    'PWD': dirname(abspath(work_dir)) \n  } \n  ( if cfg None then { \n
'LLM_API_KEY': settings.llm.api_key, \n
'OPENCODE_CONFIG_CONTENT': json.dumps(_deep_merge(base,
cfg)) \n  } else ) \n  ( if cfg None then \n    raw = E0.get('OPENCODE_CONFIG_CONTENT') \n
( if (raw is str & json.loads(raw) succeeds the result is a dict) then \n      base = that dict \n
else \n      base = {} \n    ) \n    else true \n  ) \n```\n\n（此处 `` 表示映射的函数覆盖：对于给
定的基础映射 `M` 和更新集合 `U`，`M U` 将每个键 `k` 映射到 `U[k]`，如果 `k ∈ dom(U)` 为真，否则映射到 `M[k]`。）",
    "code_evidence": "Line 25: env[\"OPENCODE_CONFIG_CONTENT\"] = json.dumps(_deep_merge(existing,
provider_config))",
    "trigger_condition": "规范指出“provider定义的键覆盖同键的预先存在的条目”（即浅覆盖）。代码使用了 _deep_merge，它
会递归合并嵌套字典，导致应被替换的预先存在的嵌套键仍然存留。在一个 OPENCODE_CONFIG_CONTENT 包含的嵌套键也出现在 provider
配置中的具体环境中，会暴露该差异。"
  }
}
```

- Bug Validator result（完整）
- Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T19:07:43.379185Z	4762
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T19:08:43.328018Z	10409

事件 1 — [llm_767bd496a4a84621adc4f684aa1e6e91](#)

- parsed_json（完整）

```
{
  "post_condition": "在正常完成（即函数返回一个字典且未抛出异常）时，以下条件成立：\n\n1. **文件系统副作用** \n  位于 \n
  \td = os.path.abspath(os.path.join(_trace_dir(work_dir), 'opencode'))` 的目录存在。如果调用前该目录不存在，则会被
创建。 \n\n2. **返回的环境字典** \n  设 `E0` 为函数进入时对 `os.environ` 的快照。返回的映射 `env` 满足： \n\n  -
```

```

env['TRACE_DIR'] = td\n - `env['TRACE_FILENAME'] = event_id`\n - `env['PWD']` =
os.path.dirname(os.path.abspath(work_dir))\n - 如果 `cfg = _opencode_provider_config()` 不为 `None`, 则:
\n * `env['LLM_API_KEY']` = settings.llm.api_key\n * 令 `raw` =
E0.get('OPENCODE_CONFIG_CONTENT')`. 如果 `raw` 是一个能由 `json.loads` 成功解析为字典的字符串, 那么 `base` = 那个字典; 否则 `base` = `{}`. 然后 `env['OPENCODE_CONFIG_CONTENT']` = json.dumps(_deep_merge(base, cfg)).\n 否则 (当 `cfg` 为 `None` 时), `env` 中 `LLM_API_KEY` 和 `OPENCODE_CONFIG_CONTENT` 的值就是 `E0` 中原有的值 (如果有的话).\n - 对于 `E0` 中存在的所有其他键 `k`, `env[k]` = `E0[k]`. `env` 的键集合恰好为 `keys(E0) ∪ {'TRACE_DIR', 'TRACE_FILENAME', 'PWD'} ∪ (如果 cfg ≠ None 则为 {'LLM_API_KEY', 'OPENCODE_CONFIG_CONTENT'})`, 无多余键.\n\n3. **无其他可观察状态变化** \n 除目录创建外, 文件系统不变; 不发生网络或其他副作用.\n\n**形式逻辑** (基于返回值和文件系统表述): \n\n令 `fs` 为调用后的文件系统状态. 后置条件 `Post(env, fs)` 成立当且仅当: \n\n`∃ td, base, raw, cfg : \n td = abspath(join(_trace_dir(work_dir), 'opencode')) \n ∧ directory_exists(td, fs) \n ∧ cfg = _opencode_provider_config() \n ∧ env = E0 ⊗ { \n 'TRACE_DIR': td, \n 'TRACE_FILENAME': event_id, \n 'PWD': dirname(abspath(work_dir)) \n } \n ⊗ (如果 cfg ≠ None 则 { \n 'LLM_API_KEY': settings.llm.api_key, \n 'OPENCODE_CONFIG_CONTENT': json.dumps(_deep_merge(base, cfg)) \n } 否则 ∅) \n ∧ (如果 cfg ≠ None 则 \n raw = E0.get('OPENCODE_CONFIG_CONTENT') \n ∧ (如果 (raw 是字符串 ∧ json.loads(raw) 成功且结果为字典) 则 \n base = 那个字典 \n 否则 \n base = {} \n ) \n 否则 true \n ) \n` (这里 `⊗` 表示映射的函数覆写: 对于给定的基映射 `M` 和更新集合 `U`, `M ⊗ U` 将每个键 `k` 映射为 `U[k]` (如果 `k ∈ dom(U)`), 否则映射为 `M[k]`. )"
}

```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_767bd496a4a84621adc4f684aa1e6e91_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_767bd496a4a84621adc4f684aa1e6e91_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_767bd496a4a84621adc4f684aa1e6e91_response.txt)

事件 2 — llm_be3ae1659dd04fe2a9664710df99f748

► parsed_json (完整)

```

{
  "verdict": "MISMATCH",
  "counterexample": "已存在的环境变量 OPENCODE_CONFIG_CONTENT = '{\"plugins\": {\"p1\": true}}' 且 _opencode_provider_config() 返回 {\"plugins\": {\"p2\": true}}. 规范要求 provider 定义的键覆盖已有的条目, 即, 得到的 OPENCODE_CONFIG_CONTENT 应为 '{\"plugins\": {\"p2\": true}}'. 而代码执行的是深度合并, 产生了 '{\"plugins\": {\"p1\": true, \"p2\": true}}', 这违反了规范.",
  "offending_statements": "第 25 行: env[\"OPENCODE_CONFIG_CONTENT\"] = json.dumps(_deep_merge(existing, provider_config))",
  "reason": "规范规定“provider 定义的键覆盖同名的已有条目” (即浅层覆盖)。而代码使用了 _deep_merge, 对嵌套字典进行递归合并, 导致本应被替换的已有嵌套键得以保留。当 OPENCODE_CONFIG_CONTENT 中包含一个也在 provider 配置里出现的嵌套键时, 具体环境便会暴露出这一不匹配。"
}

```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_be3ae1659dd04fe2a9664710df99f748_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_be3ae1659dd04fe2a9664710df99f748_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_be3ae1659dd04fe2a9664710df99f748_response.txt) **复核动作**

- 先由维护者确认预期契约, 再决定修改实现或重写 SPEC; 当前不应直接登记为真实 Bug。

Phase 5 — 规范与验证工具

模块: batch_prompts

FMA-MISMATCH-188 — src--generate_batch_prompts-py--main

- 四类结论: 契约不确定
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: not_confirmed; attempts 1

- **Phase / 模块:** Phase 5 — Specification & Verification Utilities / [batch_prompts](#); 原源码

[src/generate_batch_prompts.py](#)

- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** 返回 0。在 dry-run 模式下: 将批处理计划 (阶段、层范围、函数/批处理计数及每个批处理的详细信息) 打印到标准输出, 而不写入任何文件。在非 dry-run 模式下: 在输出目录下写入批处理提示 .txt 文件, 为指定阶段和层范围内的每个函数批处理生成一个文件, 每个文件都包含该批处理中每个函数的代码、签名和被调用方规格; 写入一个 manifest.json 文件, 描述所有批处理 (在恢复时包括已经编写了规格的函数); 从输出目录中清除不属于本次运行的过期批处理文件。在恢复时, 那些 .spec.json 和 .info.json 都已经准备好的函数被排除在提示生成之外。若 batch_size 不是严格正数, 或者请求的层范围超出 [0, total_layers - 1], 则引发 ValueError。
- **实际行为:** 若 args.batch_size <= 0, 则引发 ValueError, 消息为 "--batch-size must be > 0", 且不再执行后续语句。否则, 如果在读取 topdown_layers.json 并解析层规格后, 出现 start_layer < 0 或 end_layer >= total_layers 的情况, 则引发 ValueError, 消息为 "layer range {args.layers} out of bounds [0, {total_layers - 1}]", 且不再进行后续的赋值。否则 (正常终止): args 已定义, 其所有属性反映了命令行, 且 args.batch_size > 0; work_dir 是解析到 fm_agent/ 工作目录 (即包含本脚本的目录的父目录) 的一个 Path 对象; repo_root 为 work_dir.parent; fm_agent_prefix 是从 repo_root 到 work_dir 的相对路径字符串, 后跟 '/'; phases_json 是 phases.json 解析后的字典, 且 project = phases_json['project']; languages 和 exts 是 phases_json 中的列表值 (若缺失则默认为 []); ext_to_lang 是一个字典, 将 exts 中的每一个扩展名 (小写并去除前缀点) 映射到 languages 中对应的语言; topdown_path 是请求阶段的 topdown_layers.json 的路径, topdown 是其解析后的字典, layers = topdown.get('layers', []), 且 total_layers = len(layers); start_layer 和 end_layer 是从解析 args.layers 得到的包含性边界, 并满足 $0 \leq \text{start_layer} \leq \text{end_layer} < \text{total_layers}$; output_dir 是一个 Path 对象, 若 args.output_dir 是真值, 则等于 Path(args.output_dir), 否则为 work_dir / 'spec_prompts' / f'batch_prompts_{project}_phase{args.phase:02d}'; func_to_layer 将所有层中的每个函数名映射到其层索引 (如果文件路径包含 fm_agent_prefix, 则先移除该前缀); all_funcs 将同样的函数名映射到其函数元数据字典 (可能修改了 file 字段); manifest_batches、total_functions、skipped_functions、batch_index、write_targets 分别初始化为空列表、0、0、0 和空列表。除读取两个 JSON 文件外, 不发生其他文件系统副作用。
- **代码证据:** 第 1 行 - 第 40 行: 代码仅解析参数、验证参数、读取 JSON 配置并初始化数据结构。它从未到达规范要求的用于生成批处理提示、写出输出文件、创建 manifest.json、移除过期文件或处理恢复/dry-run 的逻辑。
- **触发条件:** 代码块在初始化后 (第 40 行) 结束。对于通过所有验证的有效输入, 代码不再做任何事, 这违反了要求生成文件、创建清单、清理过期文件以及 dry-run 输出的规范。任何 batch_size > 0 且层规格在范围内的合规输入都会导致完全缺少所需的行为。
- **Validator 触发:** 逻辑验证在第 40 行错误地截断了分析; main() 有 138 行, 包含了完整的批处理生成逻辑。
- **Probe 标准输出:** NOT CONFIRMED — main() 源代码有 138 行 (超出了声明的 40 行截断), 并包含所有必需的批处理生成模式: build_prompt=True、output_dir.mkdir=True、manifest=True、dry_run=True、write_targets=True、stale=True、resume=True

► SPEC sidecar (完整)

```
{
  "signature": "main() -> int",
  "pre_condition": "进程在一个有效的 FM-Agent 工作目录中被调用, 该目录包含 phases.json 和用于所请求阶段编号的 spec_prompts/phase_NN_topdown_layers.json。",
  "post_condition": "返回 0。在 dry-run 模式下: 将批处理计划 (阶段、层范围、函数/批处理计数及每个批处理的详细信息) 打印到标准输出, 而不写入任何文件。在非 dry-run 模式下: 在输出目录下写入批处理提示 .txt 文件, 为指定阶段和层范围内的每个函数批处理生成一个文件, 每个文件都包含该批处理中每个函数的代码、签名和被调用方规格; 写入一个 manifest.json 文件, 描述所有批处理 (在恢复时包括已经编写了规格的函数); 从输出目录中清除不属于本次运行的过期批处理文件。在恢复时, 那些 .spec.json 和 .info.json 都已经准备好的函数被排除在提示生成之外。若 batch_size 不是严格正数, 或者请求的层范围超出 [0, total_layers - 1], 则引发 ValueError。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "parse_args",
      "signature": "parse_args() -> Namespace",
      "pre_condition": "进程在调用时带有命令行参数, 包括 --phase、--layers、--batch-size、--output-dir、--resume 和 --dry-run。",
      "post_condition": "返回一个 Namespace 对象, 其属性包括 phase (int)、layers (str)、batch_size (int)、output_dir (str 或 None)、resume (bool) 和 dry_run (bool), 每个属性都反映了对应的命令行参数。"
    },
    {
      "name": "read_json",
      "signature": "read_json(filepath) -> dict",
      "pre_condition": "filepath 是文件系统上存在且语法正确的 JSON 文件的路径。",
      "post_condition": "以 Python 字典的形式返回该 JSON 文件的完整解析内容。若文件不存在或包含无效的 JSON, 则引发错误。"
    },
    {
      "name": "parse_layers_spec",
      "signature": "parse_layers_spec(layers_spec) -> tuple[int, int]",
      "pre_condition": "layers_spec 是以下两种格式之一的字符串: 单个非负整数 N, 或形如 N-M 的范围, 其中 N 和 M 均为非负整数。",
      "post_condition": "返回一个由两个整数组成的 (start, end) 元组, 表示包含性层范围。对于单个整数 N, 返回 (N, N)。若格式无效或 start > end, 则引发错误。"
    },
    {
      "name": "is_file_ready",
      "signature": "is_file_ready(file_path) -> bool",
      "pre_condition": "file_path 是提取出的函数文件的路径。",
      "post_condition": "当相邻存在一个 .spec.json 文件 (包含且仅包含 signature、pre_condition、post_condition 字段, 均为字符串) 以及一个 .info.json 文件 (包含且仅包含 callees 字段, 该字段是一个数组, 其中每个元素为一个包含 name、signature、pre_condition、post_condition 字段的字典, 均为字符串) 时, 返回 True; 否则返回 False。"
    },
    {
      "name": "chunked",
      "signature": "chunked(iterable, chunk_size) -> Iterator[list]",
      "pre_condition": "iterable 是一个有限可迭代对象。chunk_size 是一个正整数。",
      "post_condition": "生成连续的、非重叠的输入可迭代对象的子列表, 每个子列表的长度恰好等于 chunk_size, 最后一个子列表可能更短。子列表内及子列表间的元素顺序与原始可迭代对象顺序一致。"
    },
    {
      "name": "build_prompt",
      "signature": "build_prompt(phase, layer_idx, is_cycle, prompt_funcs, func_to_layer, all_funcs, work_dir, fm_agent_prefix, ext_to_lang) -> str",
      "pre_condition": "phase 为正整数。layer_idx 为非负整数。is_cycle 为布尔值。prompt_funcs 为函数元数据字典的列表, 每个字典至少包含 'file' 和 'name' 键。func_to_layer 和 all_funcs 是以函数名为键的字典。work_dir 是指向 fm_agent/ 根目录的 Path 对象。fm_agent_prefix 是用于去除前缀的相对路径字符串。ext_to_lang 是将小写文件扩展名映射到语言名称的字典。",
      "post_condition": "返回一个字符串, 其中包含完整的批处理提示文本, 包括领域上下文 (引擎概述、阶段类型)、每个函数的提取源代码 (从 work_dir 读取)、每个函数的签名, 以及来自每个函数调用者的较早层 .info.json 文件的被调用方规格。提示文本是独立的, 可直接供 LLM 使用。"
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/generate_batch_prompts-py/main.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回 0。在 dry-run 模式下: 将批处理计划 (阶段、层范围、函数/批处理计数及每个批处理的详细信息) 打印到标准输出, 而不写入任何文件。在非 dry-run 模式下: 在输出目录下写入批处理提示 .txt 文件, 为指定阶段和层范围内的每个函数批处理生成一个文件, 每个文件都包含该批处理中每个函数的代码、签名和被调用方规格; 写入一个 manifest.json 文件, 描述所有批处理 (在恢复时包括已经编写了规格的函数); 从输出目录中清除不属于本次运行的过期批处理文件。在恢复时, 那些 .spec.json 和 .info.json 都已
```



```
经准备好的函数被排除在提示生成之外。若 batch_size 不是严格正数，或者请求的层范围超出 [0, total_layers - 1]，则引发 ValueError。",
    "actual_behavior": "若 args.batch_size <= 0，则引发 ValueError，消息为 \"--batch-size must be > 0\"，且不再执行后续语句。否则，如果在读取 topdown_layers.json 并解析层规格后，出现 start_layer < 0 或 end_layer >= total_layers 的情况，则引发 ValueError，消息为 \"layer range {args.layers} out of bounds [0, {total_layers - 1}]\"，且不再进行后续的赋值。否则（正常终止）：args 已定义，其所有属性反映了命令行，且 args.batch_size > 0；work_dir 是解析到 fm_agent/ 工作目录（即包含本脚本的目录的父目录）的一个 Path 对象；repo_root 为 work_dir.parent；fm_agent_prefix 是从 repo_root 到 work_dir 的相对路径字符串，后跟 '/'；phases_json 是 phases.json 解析后的字典，且 project = phases_json['project']；languages 和 exts 是 phases_json 中的列表值（若缺失则默认为 []）；ext_to_lang 是一个字典，将 exts 中的每一个扩展名（小写并去除前缀点）映射到 languages 中对应的语言；topdown_path 是请求阶段的 topdown_layers.json 的路径，topdown 是其解析后的字典，layers = topdown.get('layers', [])，且 total_layers = len(layers)；start_layer 和 end_layer 是从解析 args.layers 得到的包含性边界，并满足 0 ≤ start_layer ≤ end_layer < total_layers；output_dir 是一个 Path 对象，若 args.output_dir 是真值，则等于 Path(args.output_dir)，否则为 work_dir / 'spec_prompts' / f'batch_prompts_{project}_phase{args.phase:02d}'；func_to_layer 将所有层中的每个函数名映射到其层索引（如果文件路径包含 fm_agent_prefix，则先移除该前缀）；all_funcs 将同样的函数名映射到其函数元数据字典（可能修改了 file 字段）；manifest_batches、total_functions、skipped_functions、batch_index、write_targets 分别初始化为空列表、0、0、0 和空列表。除读取两个 JSON 文件外，不发生其他文件系统副作用。",
    "code_evidence": "Line 1 - Line 40: The code only parses arguments, validates them, reads JSON configs, and initialises data structures. It never reaches the specification-required logic for generating batch prompts, writing output files, creating manifest.json, removing stale files, or handling resume/dry-run.",
    "trigger_condition": "代码块在初始化后（第 40 行）结束。对于通过所有验证的有效输入，代码不再做任何事，这违反了要求生成文件、创建清单、清理过期文件以及 dry-run 输出的规范。任何 batch_size > 0 且层规格在范围内的合规输入都会导致完全缺少所需的行为。"
  }
}
```

- Bug Validator result（完整）
- Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/3	2026-07-28T03:29:35.658908Z	4096
2	check_post_implies_spec	mismatch	1	0/3	2026-07-28T03:30:18.503085Z	4175

事件 1 — [llm_1c66647475db438193b6da5cad612243](#)

- parsed_json（完整）

```
{
  "post_condition": "如果 args.batch_size <= 0，将引发带有消息\"--batch-size must be > 0\"的 ValueError，且不会执行后续语句。否则，如果在读取 topdown_layers.json 并解析 layers 规范后，满足 start_layer < 0 或 end_layer >= total_layers 的条件，将引发带有消息\"layer range {args.layers} out of bounds [0, {total_layers - 1}]\"的 ValueError，且不会进行进一步的赋值。否则（正常终止）：args 已定义，所有属性反映命令行且 args.batch_size > 0；work_dir 是一个 Path 对象，解析为 fm_agent/ 工作目录（包含此脚本所在目录的父目录）；repo_root 是 work_dir.parent；fm_agent_prefix 是 repo_root 到 work_dir 的相对路径字符串加上 '/'；phases_json 是 phases.json 的解析字典，project = phases_json['project']；languages 和 exts 是来自 phases_json 的列表值（若缺失则默认为 []）；ext_to_lang 是一个字典，将 exts 中的每个扩展名（转为小写并去除前导点）映射到 languages 中对应语言；topdown_path 是所请求阶段的 topdown_layers.json 的路径，topdown 是其解析字典，layers = topdown.get('layers', [])，total_layers = len(layers)；start_layer 和 end_layer 是从解析 args.layers 得到的包含边界，且满足 0 ≤ start_layer ≤ end_layer < total_layers；output_dir 是一个 Path 对象，若 args.output_dir 为真则为 Path(args.output_dir)，否则为 work_dir / 'spec_prompts' / f'batch_prompts_{project}_phase{args.phase:02d}'；func_to_layer 将各层中每个函数名称映射到其层索引（如果文件路径中存在 fm_agent_prefix 则先将其剥离）；all_funcs 将相同名称映射到其函数元数据字典（可能包含已修改的 file 字段）；manifest_batches、total_functions、skipped_functions、batch_index、write_targets 分别初始化为空列表、0、0、0 和空列表。除两个 JSON 文件的读取外，不产生其他文件系统副作用。"
}
```


- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_1c66647475db438193b6da5cad612243_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_1c66647475db438193b6da5cad612243_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_1c66647475db438193b6da5cad612243_response.txt)

事件 2 — llm_7166b8a0015d44948facac575d1aa501

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "命令行参数: --phase 1 --layers 0-0 --batch-size 5 --output-dir 未提供 --resume false --dry-run false。文件系统状态: work_dir/phases.json 包含 {\"project\": \"test\", \"languages\": [\"python\"], \"file_extensions\": [\".py\"]}, work_dir/spec_prompts/phase_01_topdown_layers.json 包含 {\"layers\": [{\"layer\": 0, \"functions\": [{\"name\": \"foo\", \"file\": \"src/foo.py\"}]}]}. 所有其他文件按需存在。代码将验证 batch_size > 0 且层范围 [0,0] 在边界内, 在到达第 40 行之前完成初始化, 然后终止, 不写入任何批量提示文件、不创建 manifest.json、不删除过期文件, 也不输出演练计划。",
  "offending_statements": "第 1 行 - 第 40 行: 代码仅解析参数、验证参数、读取 JSON 配置并初始化数据结构。它从未触达规范要求的批量提示生成、输出文件写入、manifest.json 创建、过期文件删除或 resume/dry-run 处理逻辑。",
  "reason": "代码块在初始化 (第 40 行) 后结束。对于通过所有验证的有效输入, 代码没有进一步的动作, 违反了要求文件生成、manifest 创建、过期文件清理和演练输出的规范。任何具有 batch_size > 0 且在范围内的层规范的符合性输入, 都会导致所需行为的完全缺失。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_7166b8a0015d44948facac575d1aa501_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_7166b8a0015d44948facac575d1aa501_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_7166b8a0015d44948facac575d1aa501_response.txt) **复核动作**

- 先由维护者确认预期契约, 再决定修改实现或重写 SPEC; 当前不应直接登记为真实 Bug。

模块: parser

FMA-MISMATCH-202 — src--parser-py--format_info_for_reasoner

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异, 但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator:** **confirmed**; attempts 1
- **Phase / 模块:** Phase 5 — Specification & Verification Utilities / **parser**; 原源码 **src/parser.py**
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** 返回一个 FunctionSpecMap, 其中包含输入列表中每个被调用者对应的一项。每一项将该被调用者的“name”值映射为一个两行字符串, 格式为“前置条件: <pre_condition>\n后置条件: <post_condition>”。返回的 FunctionSpecMap 的 signatures 属性是一个字典, 将每个被调用者的“name”值映射到其“signature”值。当输入列表为空时, 返回一个空的 FunctionSpecMap (不包含任何项且 signatures 字典为空)。
- **Actual behavior:** 函数返回一个新创建的 FunctionSpecMap 对象。设 callees = info['callees'] (一个字典列表, 每个字典包含键 'name'、'signature'、'pre_condition'、'post_condition')。对 callees 中的每个 d, 定义 name = d['name'], sig = d['signature'], spec_str = '前置条件: ' + d['pre_condition'] + '\n后置条件: ' + d['post_condition']。生成的 FunctionSpecMap 存储: 对每个不同的 name, 映射 name → spec_str 和 name → sig 取自 callees 迭代顺序中该 name 的最后一次出现。如果 callees 为空, 映射中不含任何项。不抛出异常, 不修改外部状态。

- **Code evidence:** Line 4: for callee in info.get("callees", []): Line 9: knowledge_map.add_entry(Line 10: callee.get("name", ""), Line 11: callee.get("signature", ""), Line 12: callee_spec, Line 13:)
- **Trigger condition:** 规范要求输入列表中的每个被调用者对应一项，但代码使用以 name 为键的映射并覆盖重复项。对于存在两个都名为 'foo' 的被调用者的反例，生成的 FunctionSpecMap 只有一项（第二个被调用者的 spec 和 signature），而不是按要求为每个被调用者保留一项。
- **Validator trigger:** 两个具有相同名称 'foo' 的被调用者导致基于字典的 FunctionSpecMap 覆盖了第一个被调用者的条目，返回 1 项而非规范要求的 2 项。
- **Probe stdout:** 已确认 —— 实际条目数: 1（仅最后一个被调用者幸存）| 期望条目数: 2（每个被调用者对应一个）保留的规范来自被调用者 #2: ‘前置条件: x 非空\n后置条件: 返回 x.upper()’

► SPEC sidecar（完整）

```
{
  "signature": "format_info_for_reasoner(info) -> FunctionSpecMap",
  "pre_condition": "info 是一个字典，包含键 \"callees\"，其值是一个字典列表，其中每个字典恰好包含字符串类型的键 \"name\"、\"signature\"、\"pre_condition\" 和 \"post_condition\"，",
  "post_condition": "返回一个 FunctionSpecMap，其中包含输入列表中每个被调用者对应的一项。每一项将该被调用者的 \"name\" 值映射为一个两行字符串，格式为“前置条件: <pre_condition>\n\n后置条件: <post_condition>”。返回的 FunctionSpecMap 的 signatures 属性是一个字典，将每个被调用者的 \"name\" 值映射到其 \"signature\" 值。当输入列表为空时，返回一个空的 FunctionSpecMap（不包含任何项且 signatures 字典为空）。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "FunctionSpecMap.add_entry",
      "signature": "add_entry(function_name, signature, spec) -> None",
      "pre_condition": "function_name 和 signature 是字符串；spec 是包含格式化前置条件和后置条件文本的字符串",
      "post_condition": "将 function_name 作为键存储在 FunctionSpecMap 中，spec 作为其值，并将 function_name 映射到签名字典中的 signature"
    }
  ]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/parser-py/format_info_for_reasoner.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个 FunctionSpecMap，其中包含输入列表中每个被调用者对应的一项。每一项将该被调用者的 \"name\" 值映射为一个两行字符串，格式为“前置条件: <pre_condition>\n\n后置条件: <post_condition>”。返回的 FunctionSpecMap 的 signatures 属性是一个字典，将每个被调用者的 \"name\" 值映射到其 \"signature\" 值。当输入列表为空时，返回一个空的 FunctionSpecMap（不包含任何项且 signatures 字典为空）。",
    "actual_behavior": "函数返回一个新创建的 FunctionSpecMap 对象。设 callees = info['callees']（一个字典列表，每个字典包含键 'name'、'signature'、'pre_condition'、'post_condition'）。对 callees 中的每个 d，定义 name = d['name']，sig = d['signature']，spec_str = '前置条件: ' + d['pre_condition'] + '\n\n后置条件: ' + d['post_condition']。生成的 FunctionSpecMap 存储: 对每个不同的 name，映射 name spec_str 和 name sig 取自 callees 迭代顺序中该 name 的最后一次出现。如果 callees 为空，映射中不含任何项。不抛出异常，不修改外部状态。",
    "code_evidence": "Line 4: for callee in info.get(\"callees\", []):\nLine 9: knowledge_map.add_entry(\nLine 10:     callee.get(\"name\", \"\"),\nLine 11:     callee.get(\"signature\", \"\"),\nLine 12:     callee_spec,\nLine 13: )",
    "trigger_condition": "规范要求输入列表中的每个被调用者对应一项，但代码使用以 name 为键的映射并覆盖重复项。对于存在两个都名为 'foo' 的被调用者的反例，生成的 FunctionSpecMap 只有一项（第二个被调用者的 spec 和 signature），而不是按要求为每个被调用者保留一项。"
  }
}
```

```
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--parser-py--format_info_for_reasoner",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/parser-py/format_info_for_reasoner.py",
  "function_name": "format_info_for_reasoner",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--parser-py--format_info_for_reasoner.py",
  "detail_file": "fm_agent/bug_validation/src--parser-py--format_info_for_reasoner.md",
  "probe_stdout": "已确认 — 实际条目数：1（仅最后一个被调用者幸存） | 期望条目数：2（每个被调用者对应一个）\n 保留的规范来自被调用者 #2：'前置条件：x 非空\n后置条件：返回 x.upper()' ",
  "trigger_summary": "两个具有相同名称 'foo' 的被调用者导致基于字典的 FunctionSpecMap 覆盖了第一个被调用者的条目，返回 1 项而非规范要求的 2 项。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:35:54.023748Z	2032
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:36:17.776589Z	6839

事件 1 — [llm_adcb2f97416423cb2b4b455c4cfc82b](#)

► parsed_json（完整）

```
{
  "post_condition": "函数返回一个新创建的 FunctionSpecMap 对象。设 callees = info['callees']（一个字典列表，每个字典包含键 'name'、'signature'、'pre_condition'、'post_condition'）。对 callees 中的每个 d，定义 name = d['name'], sig = d['signature'], spec_str = '前置条件：' + d['pre_condition'] + '\\n后置条件：' + d['post_condition']。生成的 FunctionSpecMap 存储：对每个不同的 name，映射 name ↦ spec_str 和 name ↦ sig 取自 callees 迭代顺序中该 name 的最后一次出现。如果 callees 为空，映射中不含任何项。不抛出异常，不修改外部状态。"
}
```

- 载荷：[\[system_prompt\]](#)

[\(fm_agent/trace/payloads/llm_adcb2f97416423cb2b4b455c4cfc82b_message_00_system.txt\)](#) · [\[user_prompt\]](#)
[\(fm_agent/trace/payloads/llm_adcb2f97416423cb2b4b455c4cfc82b_message_01_user.txt\)](#) · [\[assistant_output\]](#)
[\(fm_agent/trace/payloads/llm_adcb2f97416423cb2b4b455c4cfc82b_response.txt\)](#)

事件 2 — [llm_38c885f853444243b72f5aaf90db597c](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "info = {'callees': [{'name': 'foo', 'signature': '()', 'pre_condition': 'a', 'post_condition': 'b'}], {'name': 'foo', 'signature': '(int)', 'pre_condition': 'c', 'post_condition': 'd'}]"
}
```

```
d'']}]"]",
  "offending_statements": "Line 4: for callee in info.get(\"callees\", []):\nLine 9:
knowledge_map.add_entry(\nLine 10:      callee.get(\"name\", \"\"),\nLine 11:      callee.get(\"signature\",
\"\"),\nLine 12:      callee_spec,\nLine 13: )",
  "reason": "规范要求输入列表中的每个被调用者对应一项，但代码使用以 name 为键的映射并覆盖重复项。对于存在两个都名为 'foo'
的被调用者的反例，生成的 FunctionSpecMap 只有一项（第二个被调用者的 spec 和 signature），而不是按要为每个被调用者保留
一项。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_38c885f853444243b72f5aaf90db597c_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_38c885f853444243b72f5aaf90db597c_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_38c885f853444243b72f5aaf90db597c_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

Phase 6 — 推理与验证引擎

模块: incremental_pipeline

FMA-MISMATCH-218 — [src--incremental_reasoner-py--codegraph_legacy_coverage](#)

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator:** [confirmed](#)；尝试次数 1
- **Phase / 模块:** Phase 6 — Reasoning & Verification Engine / [incremental_pipeline](#)；原源码 [src/incremental_reasoner.py](#)
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** 返回一个字典，其键为 `file_languages` 中列出的文件的规范化相对路径（针对 `proj_dir` 进行相对化处理并执行大小写规范化），其值为布尔值。对于在 `proj_dir` 下对应的磁盘路径不存在的文件，值为 `True`。对于已存在的文件，当且仅当旧版提取器根据该文件的语言键产生的有序 `(func_name, source)` 元组序列是 `CodeGraph` 中针对该文件的条目的有序子序列时（两个条目匹配的条件是：它们具有相同的无限定函数名，且经过行结束符规范化后其源代码文本完全相同），并且每个匹配的 `CodeGraph` 条目出现的位置索引都严格大于前一次匹配的位置索引，值才为 `True`。当有序子序列条件不满足时（即至少存在一个旧版发现的函数在所需顺序下找不到匹配的 `CodeGraph` 条目），值为 `False`，并通过日志系统发出一个识别出未匹配函数名及其文件路径的警告。该函数不对文件系统做任何修改。
- **Actual behavior:** 该函数返回一个字典 `coverage`，其中针对输入 `rel_path` 中的每个 `file_languages` 包含一个键值对。对于给定 `rel_path` 及其关联的语言键 `lang_key`，令 `rel_key = _normalized_relative_path(proj_dir, rel_path)` 和 `abs_path = os.path.join(proj_dir, rel_path)` 分别表示相关值。如果 `os.path.exists(abs_path)` 为 `False`，则 `coverage[rel_key]` 为 `True`。否则，令 `legacy_funcs` 为 `(name, source)` 返回的 `extract_functions_from_file(abs_path, lang_key)` 元组列表，令 `codegraph_items` 为 `(identifier, source)` 中按字典项顺序排列的 `codegraph_functions.get(rel_key, {})` 对列表。当且仅当存在一个严格递增的整数序列 `coverage[rel_key]`（其中 `True`）且每个 `j_0 < j_1 < ... < j_{L-1}` 都是 `L = len(legacy_funcs)` 的有效索引，使得对于每个 `j_i` 有：`codegraph_items` 且 `i` 时，值 `_bare_function_name(codegraph_items[j_i][0]) == _bare_function_name(legacy_funcs[i][0])` 才为 `_normalized_function_source(codegraph_items[j_i][1]) == _normalized_function_source(legacy_funcs[i][1])`。如果不存在这样的序列，

`coverage[rel_key]` 为 `False`。对 `file_languages` 的迭代顺序决定了插入 `coverage` 的顺序，但除了 Python 字典原生的插入顺序（Python 3.7+）外，不保证返回字典的任何进一步排序。

- **Code evidence:** Line 28: `_bare_function_name(identifier) == legacy_bare_name`
- **Trigger condition:** 代码使用 `_bare_function_name` 来比较标识符，该函数会去除末尾的去重后缀（例如 `'_1'`）。规范要求“无限定函数名”仅指去除了限定符的名称，应保留旧版提取器添加的去重后缀。因此，旧版名称如 `'foo_1'` 和 CodeGraph 名称如 `'foo()'` 在代码中视为相等（二者均变为 `'foo'`），但根据规范应被视为不相等，从而导致在需要 `False` 判定时错误地给出 `True` 判定。
- **Validator trigger:** `_bare_function_name` 会去除末尾的去重后缀（例如 `'_1'`），导致旧版名称 `'my_func_1'` 错误地匹配 CodeGraph 名称 `'my_func'`，而规范要求将它们视为不同。
- **Probe stdout:** CONFIRMED — actual: `True` | expected: `False`（旧版 `'my_func_1'` 不应匹配 CodeGraph `'my_func'`）

► SPEC sidecar（完整）

```
{
  "signature": "_codegraph_legacy_coverage(proj_dir, codegraph_functions, file_languages) -> dict",
  "pre_condition": "proj_dir 是一个目录路径。codegraph_functions 是一个将相对文件路径映射到 {function_name_string: source_text_string} 字典的字典。file_languages 是一个将相对文件路径字符串映射到 extract_functions_from_file 可识别的语言键字符串的字典。",
  "post_condition": "返回一个字典，其键为 file_languages 中列出的文件的规范化相对路径（针对 proj_dir 进行相对化处理并执行大小写规范化），其值为布尔值。对于在 proj_dir 下对应的磁盘路径不存在的文件，值为 True。对于已存在的文件，当且仅当旧版提取器根据该文件的语言键产生的有序（func_name, source）元组序列是 CodeGraph 中针对该文件的条目的有序子序列时（两个条目匹配的条件是：它们具有相同的无限定函数名，且经过行结束符规范化后其源代码文本完全相同），并且每个匹配的 CodeGraph 条目出现的位置索引都严格大于前一次匹配的位置索引，值才为 True。当有序子序列条件不满足时（即至少存在一个旧版发现的函数在所需顺序下找不到匹配的 CodeGraph 条目），值为 False，并通过日志系统发出一个识别出未匹配函数名及其文件路径的警告。该函数不对文件系统做任何修改。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "_normalized_relative_path",
      "signature": "_normalized_relative_path(proj_dir, path) -> str",
      "pre_condition": "proj_dir 是一个目录路径。path 是一个绝对路径或相对于 proj_dir 的路径。",
      "post_condition": "返回相对于 proj_dir 规范化的路径：折叠 . 和 .. 段，规范化路径分隔符，并针对不区分大小写的文件系统应用大小写规范化。结果是一个没有前导目录分隔符的字符串。"
    },
    {
      "name": "extract_functions_from_file",
      "signature": "extract_functions_from_file(filepath, lang_key) -> list[tuple[str, str]]",
      "pre_condition": "filepath 指向一个存在且可读的文件。lang_key 是提取配置中存在的一个语言键。",
      "post_condition": "返回一个（func_name, source_text）元组列表，每个元组对应文件中通过语言特定提取规则发现的一个函数，按源代码顺序出现。每个 func_name 经过规范化：不安全字符被转译，重复名称按首次出现顺序添加数字后缀去重。每个 source_text 包含从头到尾的完整函数体，行结束符规范化为 '\\n' 并带有一个尾部换行符。"
    },
    {
      "name": "_bare_function_name",
      "signature": "_bare_function_name(identifier) -> str",
      "pre_condition": "identifier 是一个可能包含 :: 或 . 限定符分隔符以及末尾 _<digits> 去重后缀的字符串。",
      "post_condition": "返回 identifier 中最后一个 :: 或 . 分隔符之后最右侧的片段，并移除末尾下划线后跟数字的部分。当 identifier 不包含限定符分隔符时，返回移除了末尾去重后缀的 identifier 本身。"
    },
    {
      "name": "_normalized_function_source",
      "signature": "_normalized_function_source(source) -> str",
      "pre_condition": "source 是一个包含函数源代码文本的字符串。",
      "post_condition": "返回将所有行结束符规范化为单个 '\\n' 字符的源代码文本。回车换行对 (\\r\\n) 和独立的回车符 (\\r) 均被替换为 '\\n'。"
    }
  ]
}
```



```
]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_codegraph_legacy_coverage.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个字典，其键为 file_languages 中列出的文件的规范化相对路径（针对 proj_dir 进行相对化处理并执行大小写规范化），其值为布尔值。对于在 proj_dir 下对应的磁盘路径不存在的文件，值为 True。对于已存在的文件，当且仅当旧版提取器根据该文件的语言键产生的有序（func_name, source）元组序列是 CodeGraph 中针对该文件的条目的有序子序列时（两个条目匹配的条件是：它们具有相同的无限定函数名，且经过行结束符规范化后其源代码文本完全相同），并且每个匹配的 CodeGraph 条目出现的位置索引都严格大于前一次匹配的位置索引，值才为 True。当有序子序列条件不满足时（即至少存在一个旧版发现的函数在所需顺序下找不到匹配的 CodeGraph 条目），值为 False，并通过日志系统发出一个识别出未匹配函数名及其文件路径的警告。该函数不对文件系统做任何修改。",
    "actual_behavior": "该函数返回一个字典 `coverage`，其中针对输入 `file_languages` 中的每个 `rel_path` 包含一个键值对。对于给定 `rel_path` 及其关联的语言键 `lang_key`，令 `rel_key = _normalized_relative_path(proj_dir, rel_path)` 和 `abs_path = os.path.join(proj_dir, rel_path)` 分别表示相关值。如果 `os.path.exists(abs_path)` 为 False，则 `coverage[rel_key]` 为 `True`。否则，令 `legacy_funcs` 为 `extract_functions_from_file(abs_path, lang_key)` 返回的 `(name, source)` 元组列表，令 `codegraph_items` 为 `codegraph_functions.get(rel_key, {})` 中按字典项顺序排列的 `(identifier, source)` 对列表。当且仅当存在一个严格递增的整数序列 `j_0 < j_1 < ... < j_{L-1}`（其中 `L = len(legacy_funcs)`）且每个 `j_i` 都是 `codegraph_items` 的有效索引，使得对于每个 `i` 有：`_bare_function_name(codegraph_items[j_i][0]) == _bare_function_name(legacy_funcs[i][0])` 且 `_normalized_function_source(codegraph_items[j_i][1]) == _normalized_function_source(legacy_funcs[i][1])` 时，值 `coverage[rel_key]` 才为 `True`。如果不存在这样的序列，`coverage[rel_key]` 为 `False`。对 `file_languages` 的迭代顺序决定了插入 `coverage` 的顺序，但除了 Python 字典原生的插入顺序（Python 3.7+）外，不保证返回字典的任何进一步排序。",
    "code_evidence": "Line 28:         _bare_function_name(identifier) == legacy_bare_name",
    "trigger_condition": "代码使用 _bare_function_name 来比较标识符，该函数会去除末尾的去重后缀（例如 '_1'）。规范要求“无限定函数名”仅指去除了限定符的名称，应保留旧版提取器添加的去重后缀。因此，旧版名称如 'foo_1' 和 CodeGraph 名称如 'foo()' 在代码中视为相等（二者均变为 'foo'），但根据规范应被视为不相等，从而导致在需要 False 判定时错误地给出 True 判定。"
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--incremental_reasoner-py--_codegraph_legacy_coverage",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_codegraph_legacy_coverage.py",
  "function_name": "_codegraph_legacy_coverage",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_codegraph_legacy_coverage.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_codegraph_legacy_coverage.md",
  "probe_stdout": "CONFIRMED - actual: True | expected: False (旧版 'my_func_1' 不应匹配 CodeGraph 'my_func') \n",
  "trigger_summary": "_bare_function_name 会去除末尾的去重后缀（例如 '_1'），导致旧版名称 'my_func_1' 错误地匹配 CodeGraph 名称 'my_func'，而规范要求将它们视为不同。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	目的	状态	尝试	块	开始时间	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:48:14.474482Z	3174
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:48:46.457753Z	18858

事件 1 — `llm_95a1d0cbc6dc403e80f561f75f7464c2`

► parsed_json (完整)

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_95a1d0cbc6dc403e80f561f75f7464c2_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_95a1d0cbc6dc403e80f561f75f7464c2_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_95a1d0cbc6dc403e80f561f75f7464c2_response.txt)

事件 2 — `llm_b80a1c312ce142bfa13e3e66cded5627`

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir='/proj', file_languages={'src/f.cpp': 'cpp'}, codegraph_functions=
{'src/f.cpp': {'foo()': 'int foo() { return 0; }'}}. 该文件存在, 且 extract_functions_from_file 返回
[('foo_1', 'int foo() { return 0; }')] (遗留提取器添加了去重后缀)。代码通过 _bare_function_name 去除 '_1', 将裸名
称 'foo' 与 codegraph 的 'foo()' 进行匹配, 从而产生 coverage True。规范要求提取器产生的非限定函数名称上进行匹配, 即
'foo_1' ≠ 'foo()', 因此正确结果应为 False。",
  "offending_statements": "Line 28:                 _bare_function_name(identifier) == legacy_bare_name",
  "reason": "代码使用 _bare_function_name 去除尾部的去重后缀 (例如 '_1') 来比较标识符。规范要求“非限定函数名称”仅指去除
限定符后的名称, 保留遗留提取器添加的任何去重后缀。因此, 对于像 'foo_1' 这样的遗留名称和像 'foo()' 这样的 CodeGraph 名称,
代码认为它们相等 (两者都变成 'foo'), 但按照规范应视为不相等, 从而在要求 False 判定时产生了错误的 True 判定。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_b80a1c312ce142bfa13e3e66cded5627_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_b80a1c312ce142bfa13e3e66cded5627_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_b80a1c312ce142bfa13e3e66cded5627_response.txt) **复核动作**

- 先由维护者确认预期契约, 再决定修改实现或重写 SPEC; 当前不应直接登记为真实 Bug。

FMA-MISMATCH-222 — `src--incremental_reasoner.py--_collect_changed_functions::is_workspace_file`

- 四类结论: 契约不确定
- 分类依据: probe 已复现 SPEC 与实现的差异, 但现有证据不足以决定应修改实现还是修改 SPEC。
- Validator: `confirmed`; attempts `1`
- Phase / 模块: Phase 6 — Reasoning & Verification Engine / `incremental_pipeline`; 原源码 `src/incremental_reasoner.py`
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** 返回 True 当 `rel_path` 在将所有反斜杠替换为正斜杠后, 等于精确字符串 `"fm_agent"` 或以 `"fm_agent/"` 为前缀。在所有其他情况下返回 False, 包括 `rel_path` 为空、仅包含 `fm_agent` 目录名但带有额外的路径组件且不构成 `fm_agent/` 下的后代路径, 或表示归一化路径落在 `fm_agent/` 工作空间树之外的文件。
- **Actual behavior:** 函数 `_is_workspace_file` 在 `rel_path` 的归一化版本 (通过将所有反斜杠 `"` 替换为正斜杠 `'` 得到) 等于字符串 `'fm_agent'` 或以 `'fm_agent/'` 开头时返回 True; 否则返回 False。该函数无副作用且不修改 `rel_path`。形式化表示: $\forall rel_path \in \text{Strings}, _is_workspace_file(rel_path) \Leftrightarrow (rel_path.replace(\backslash, /) = 'fm_agent') \vee (rel_path.replace(\backslash, /).startswith('fm_agent/'))$ 。
- **Code evidence:** 第 3 行: `return norm == "fm_agent" or norm.startswith("fm_agent/")`
- **Trigger condition:** 代码在替换反斜杠后使用了简单的字符串前缀检查。对于输入 `'fm_agent/.'`, 归一化结果为 `'fm_agent/.'`, 它确实以 `'fm_agent/'` 开头, 因此代码返回 True。然而, 规范要求路径不构成 `fm_agent/` 下的后代

路径时返回 `False`，而 `'..'` 会逃逸出目录，使之成为非后代路径。单纯的字符串前缀无法检测此类逃逸，违反了规范。

- **Validator trigger:** 以 `fm_agent/` 开头且包含 `'..'` 组件的路径（例如 `fm_agent/..`）能通过简单的 `startswith` 检查，但实际上会逃逸出工作空间目录。
- **Probe stdout:** `[PASS] rel_path='fm_agent' expected=True actual=True (精确匹配) [PASS]`
`rel_path='fm_agent/some_file.py' expected=True actual=True (工作空间内的后代) [PASS]`
`rel_path='fm_agent/nested/deep/file.c' expected=True actual=True (深层嵌套的后代) [PASS]`
`rel_path='fm_agent\subdir\file.rs' expected=True actual=True (Windows 反斜杠归一化) [PASS]`
`rel_path='src/main.py' expected=False actual=False (完全在工作空间之外) [PASS]` `rel_path="" expected=False actual=False (空路径) [PASS]` `rel_path='fm' expected=False actual=False (部分前缀匹配，不是 fm_agent) [FAIL]`
`rel_path='fm_agent/..' expected=False actual=True (.. 逃逸工作空间目录) [FAIL]` `rel_path='fm_agent/../../escape.py' expected=False actual=True (.. 逃逸并带有后续文件) [FAIL]` `rel_path='fm_agent/../../outside' expected=False actual=True (双重 .. 逃逸) [FAIL]` `rel_path='fm_agent/subdir/../../root.txt' expected=False actual=True (嵌套然后 .. 逃逸) [FAIL]` `rel_path='fm_agent\..\escape.rs' expected=False actual=True (Windows 反斜杠 .. 逃逸)`

CONFIRMED — 5 个测试用例暴露出缺陷：`rel_path='fm_agent/..':` 期望 `False`，得到 `True` (`..` 逃逸工作空间目录)
`rel_path='fm_agent/../../escape.py':` 期望 `False`，得到 `True` (`..` 逃逸并带有后续文件) `rel_path='fm_agent/../../outside':` 期望 `False`，得到 `True` (双重 `..` 逃逸) `rel_path='fm_agent/subdir/../../root.txt':` 期望 `False`，得到 `True` (嵌套然后 `..` 逃逸)
`rel_path='fm_agent\..\escape.rs':` 期望 `False`，得到 `True` (Windows 反斜杠 `..` 逃逸)

► SPEC sidecar（完整）

```
{
  "signature": "_is_workspace_file(rel_path) -> bool",
  "pre_condition": "rel_path 是表示一个相对文件路径的字符串。",
  "post_condition": "返回 True 当 rel_path 在将所有反斜杠替换为正斜杠后，等于精确字符串 \"fm_agent\" 或以 \"fm_agent/\" 为前缀。在所有其他情况下返回 False，包括 rel_path 为空、仅包含 fm_agent 目录名但带有额外的路径组件且不构成 fm_agent/ 下的后代路径，或表示归一化路径落在 fm_agent/ 工作空间树之外的文件。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_collect_changed_functions::_is_workspace_file.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回 True 当 rel_path 在将所有反斜杠替换为正斜杠后，等于精确字符串 \"fm_agent\" 或以 \"fm_agent/\" 为前缀。在所有其他情况下返回 False，包括 rel_path 为空、仅包含 fm_agent 目录名但带有额外的路径组件且不构成 fm_agent/ 下的后代路径，或表示归一化路径落在 fm_agent/ 工作空间树之外的文件。",
    "actual_behavior": "函数 _is_workspace_file 在 rel_path 的归一化版本（通过将所有反斜杠 '\\' 替换为正斜杠 '/' 得到）等于字符串 'fm_agent' 或以 'fm_agent/' 开头时返回 True；否则返回 False。该函数无副作用且不修改 rel_path。形式化表示： $\forall rel\_path \in \text{Strings}, \_is\_workspace\_file(rel\_path) \Leftrightarrow (rel\_path.replace('\\', '/') = 'fm\_agent') \vee (rel\_path.replace('\\', '/').startswith('fm\_agent/'))$ 。",
    "code_evidence": "Line 3: return norm == \"fm_agent\" or norm.startswith(\"fm_agent/\"),",
    "trigger_condition": "代码在替换反斜杠后使用了简单的字符串前缀检查。对于输入 'fm_agent/..'，归一化结果为 'fm_agent/..'，它确实以 'fm_agent/' 开头，因此代码返回 True。然而，规范要求路径不构成 fm_agent/ 下的后代路径时返回 False，而 '..' 会逃逸出目录，使之成为非后代路径。单纯的字符串前缀无法检测此类逃逸，违反了规范。"
  }
}
```

```
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--incremental_reasoner-py--_collect_changed_functions::_is_workspace_file",
  "source_file": "src/incremental_reasoner.py",
  "function_name": "_is_workspace_file",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_collect_changed_functions::_is_workspace_file.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_collect_changed_functions::_is_workspace_file.md",
  "probe_stdout": "[PASS] rel_path='fm_agent' expected=True actual=True （精确匹配）\n[PASS] rel_path='fm_agent/some_file.py' expected=True actual=True （工作空间内的后代）\n[PASS] rel_path='fm_agent/nested/deep/file.c' expected=True actual=True （深层嵌套的后代）\n[PASS] rel_path='fm_agent\\\\\\\\subdir\\\\\\\\file.rs' expected=True actual=True （Windows 反斜杠归一化）\n[PASS] rel_path='src/main.py' expected=False actual=False （完全在工作空间之外）\n[PASS] rel_path='' expected=False actual=False （空路径）\n[PASS] rel_path='fm' expected=False actual=False （部分前缀匹配，不是 fm_agent）\n[FAIL] rel_path='fm_agent/..' expected=False actual=True （.. 逃逸工作空间目录）\n[FAIL] rel_path='fm_agent/../../escape.py' expected=False actual=True （.. 逃逸并带有后续文件）\n[FAIL] rel_path='fm_agent/../../outside' expected=False actual=True （双重 .. 逃逸）\n[FAIL] rel_path='fm_agent/subdir/../../root.txt' expected=False actual=True （嵌套然后 .. 逃逸）\n[FAIL] rel_path='fm_agent\\\\\\\\..\\\\\\\\escape.rs' expected=False actual=True （Windows 反斜杠 .. 逃逸）\n\nCONFIRMED - 5 个测试用例暴露出缺陷：\n rel_path='fm_agent/..': 期望 False，得到 True （.. 逃逸工作空间目录）\n rel_path='fm_agent/../../escape.py': 期望 False，得到 True （.. 逃逸并带有后续文件）\n rel_path='fm_agent/../../outside': 期望 False，得到 True （双重 .. 逃逸）\n rel_path='fm_agent/subdir/../../root.txt': 期望 False，得到 True （嵌套然后 .. 逃逸）\n rel_path='fm_agent\\\\\\\\..\\\\\\\\escape.rs': 期望 False，得到 True （Windows 反斜杠 .. 逃逸）",
  "trigger_summary": "以 fm_agent/ 开头且包含 '..' 组件的路径（例如 fm_agent/..）能通过简单的 startswith 检查，但实际上会逃逸出工作空间目录。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:23:26.102254Z	1641
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:23:50.700100Z	2797

事件 1 — [llm_d5d4d398d71c4b98bddd614ddd4cfb98](#)

► parsed_json（完整）

```
{
  "post_condition": "函数 _is_workspace_file 在 rel_path 的归一化版本（通过将所有反斜杠 '\\' 替换为正斜杠 '/' 得到）等于字符串 'fm_agent' 或以 'fm_agent/' 开头时返回 True；否则返回 False。该函数无副作用且不修改 rel_path。形式化表示：∀ rel_path ∈ Strings, _is_workspace_file(rel_path) ⇔ (rel_path.replace('\\\\\\\\', '/') = 'fm_agent') ∨ (rel_path.replace('\\\\\\\\', '/').startswith('fm_agent/'))."
}
```

- 载荷：[system_prompt]

(fm_agent/trace/payloads/llm_d5d4d398d71c4b98bddd614ddd4cfb98_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_d5d4d398d71c4b98bddd614ddd4cfb98_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_d5d4d398d71c4b98bddd614ddd4cfb98_response.txt)

事件 2 — [llm_de7c3366c9ff47c484567d05c09146e6](#)

► [parsed_json](#) (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "rel_path = 'fm_agent/..'",
  "offending_statements": "第 3 行: return norm == \"fm_agent\" or norm.startswith(\"fm_agent/\"),",
  "reason": "代码在替换反斜杠后使用了简单的字符串前缀检查。对于输入 'fm_agent/..'，归一化结果为 'fm_agent/..'，它确实以 'fm_agent/' 开头，因此代码返回 True。然而，规范要求路径不构成 fm_agent/ 下的后代路径时返回 False，而 '..' 会逃逸出目录，使之成为非后代路径。单纯的字符串前缀无法检测此类逃逸，违反了规范。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_de7c3366c9ff47c484567d05c09146e6_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_de7c3366c9ff47c484567d05c09146e6_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_de7c3366c9ff47c484567d05c09146e6_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-227 — [src--incremental_reasoner-py--_llm_select_json](#)

- **四类结论：** 契约不确定
- **分类依据：** probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator：** [confirmed](#)；attempts 1
- **Phase / 模块：** Phase 6 — Reasoning & Verification Engine / [incremental_pipeline](#)；原源码 [src/incremental_reasoner.py](#)
- **证据：** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim：** 将 prompt_content 发送到配置的 LLM。当 LLM 产生可解析为 JSON、符合 schema_description 并通过 validator 的输出时，返回解析并验证后的结果。当 LLM 未产生任何输出、输出无法解析为 JSON、输出不符合 schema_description 或输出未通过 validator 时，返回 None。交互过程在 work_dir 下进行追踪，并以 stage 和 trace_meta 作为标识标签。在返回 None 时，会生成一条记录 stage 的 error 级别日志条目。
- **Actual behavior：** 在 _llm_select_json 的执行之后，函数要么以返回值 v 正常终止，要么抛出异常并传播给调用者。

正常终止的后置条件： (1) (v 为 None) (v 不为 None validator(v)[0] = True v 是一个符合 schema_description 的 JSON 解析对象)。 (2) 目录 os.path.join(work_dir, 'trace') 中包含了来自 LLM 交互的追踪文件，并记录了元数据 {'stage': stage, 'summary': 'LLM ' + stage, **trace_meta}。 (3) 如果 v 为 None，则会发出包含 stage 的 logging.error 消息。

异常终止（由 _llm_json_call 抛出的异常）：异常传播；没有返回值，且对追踪数据的完整性或存在性不作任何保证。

- **Code evidence：** Line 11: result = _llm_json_call(_llm_provider_client, LLM_MODEL, messages, validator, schema_description, trace_dir=os.path.join(work_dir, "trace"), trace_meta=meta,)
- **Trigger condition：** 规范声明，当 LLM 未产生有效且符合规范的 JSON 结果时，函数始终返回 None；其中并未规定抛出异常。代码将 _llm_json_call 抛出的任何异常（例如网络或客户端错误）直接传播给调用者，违反了预期契约。

- **Validator trigger:** 来自 `_llm_json_call` 的异常（例如网络或客户端错误）传播给调用者，而非如规范要求那样被捕获并返回 `None`。
- **Probe stdout:** CONFIRMED — exception propagated to caller (expected: None returned)

► SPEC sidecar（完整）

```
{
  "signature": "_llm_select_json(work_dir, prompt_content, stage, validator, schema_description,
trace_meta=None) -> Any | None",
  "pre_condition": "work_dir 是一个有效的目录路径。prompt_content 是一个非空字符串。stage 是一个非空字符串。
validator 是一个可调用对象，接受解析后的值并返回一个 (bool, str | None) 元组，其中 bool 表示是否有效。
schema_description 是一个描述预期输出形状的非空字符串。如果提供，trace_meta 是一个字典，其键为字符串，值为 JSON 可序列化的
内容。",
  "post_condition": "将 prompt_content 发送到配置的 LLM。当 LLM 产生可解析为 JSON、符合 schema_description 并通过
validator 的输出时，返回解析并验证后的结果。当 LLM 未产生任何输出、输出无法解析为 JSON、输出不符合 schema_description
或输出未通过 validator 时，返回 None。交互过程在 work_dir 下进行追踪，并以 stage 和 trace_meta 作为标识标签。在返回
None 时，会生成一条记录 stage 的 error 级别日志条目。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "_llm_json_call",
      "signature": "_llm_json_call(client, model, messages, validator, schema_description, trace_dir,
trace_meta) -> Any | None",
      "pre_condition": "client 是已配置的 LLM 提供商客户端。model 是有效的模型标识符。messages 是一个非空的消息字典列
表，每个字典包含 'role' 和 'content' 字符串键。validator 是一个可调用对象，接受解析后的值并返回 (bool, str | None)。
schema_description 是一个非空字符串。trace_dir 是一个可写的目录路径。trace_meta 是一个字典，其键为字符串，值为 JSON 可
序列化的内容。",
      "post_condition": "通过 client 将 messages 发送到 LLM。当 LLM 产生可解析为 JSON 且符合 schema_description
并通过 validator 的输出时，返回解析并验证后的结果。当 LLM 未产生有效输出或验证失败时，返回 None。处理协议级别的重试。原始交
互过程被追踪到 trace_dir，并以 trace_meta 作为标识元数据。"
    }
  ]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-
py/_llm_select_json.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "将 prompt_content 发送到配置的 LLM。当 LLM 产生可解析为 JSON、符合 schema_description 并通过
validator 的输出时，返回解析并验证后的结果。当 LLM 未产生任何输出、输出无法解析为 JSON、输出不符合 schema_description
或输出未通过 validator 时，返回 None。交互过程在 work_dir 下进行追踪，并以 stage 和 trace_meta 作为标识标签。在返回
None 时，会生成一条记录 stage 的 error 级别日志条目。",
    "actual_behavior": "在 _llm_select_json 的执行之后，函数要么以返回值 v 正常终止，要么抛出异常并传播给调用者。\\n\\n
正常终止的后置条件：\\n(1) (v 为 None) (v 不为 None validator(v)[0] = True v 是一个符合 schema_description 的
JSON 解析对象)。\\n(2) 目录 os.path.join(work_dir, 'trace') 中包含了来自 LLM 交互的追踪文件，并记录了元数据
{'stage': stage, 'summary': 'LLM ' + stage, **trace_meta}。\\n(3) 如果 v 为 None，则会发出包含 stage 的
logging.error 消息。\\n\\n异常终止（由 _llm_json_call 抛出的异常）：异常传播；没有返回值，且对追踪数据的完整性或存在性不作
任何保证。",
    "code_evidence": "Line 11: result = _llm_json_call(\\n                _llm_provider_client,\\n
LLM_MODEL,\\n                messages,\\n                validator,\\n                schema_description,\\n
trace_dir=os.path.join(work_dir, '\\ntrace\\n'),\\n                trace_meta=meta,\\n                )",
    "trigger_condition": "规范声明，当 LLM 未产生有效且符合规范的 JSON 结果时，函数始终返回 None；其中并未规定抛出异
常。代码将 _llm_json_call 抛出的任何异常（例如网络或客户端错误）直接传播给调用者，违反了预期契约。"
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--incremental_reasoner-py--_llm_select_json",
  "source_file": "src/incremental_reasoner-py/_llm_select_json.py",
  "function_name": "_llm_select_json",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_llm_select_json.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_llm_select_json.md",
  "probe_stdout": "CONFIRMED - exception propagated to caller (expected: None returned)",
  "trigger_summary": "来自 _llm_json_call 的异常 (例如网络或客户端错误) 传播给调用者, 而非如规范要求那样被捕获并返回 None。"
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌, 使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:24:06.066810Z	3286
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:25:00.772588Z	4268

事件 1 — [llm_cec4a2348ef744339713bbac7eeefdb5](#)

► parsed_json (完整)

```
{
  "post_condition": "在 _llm_select_json 的执行之后, 函数要么以返回值 v 正常终止, 要么抛出异常并传播给调用者。\\n\\n正常终止的后置条件:\\n(1) (v 为 None) v (v 不为 None & validator(v)[0] = True & v 是一个符合 schema_description 的 JSON 解析对象)。\\n(2) 目录 os.path.join(work_dir, 'trace') 中包含了来自 LLM 交互的追踪文件, 并记录了元数据 {'stage': stage, 'summary': 'LLM ' + stage, **trace_meta}。\\n(3) 如果 v 为 None, 则会发出包含 stage 的 logging.error 消息。\\n\\n异常终止 (由 _llm_json_call 抛出的异常): 异常传播; 没有返回值, 且对追踪数据的完整性或存在性不作任何保证。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_cec4a2348ef744339713bbac7eeefdb5_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_cec4a2348ef744339713bbac7eeefdb5_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_cec4a2348ef744339713bbac7eeefdb5_response.txt)

事件 2 — [llm_a9c42dbb33174a909370b4d8652293e3](#)

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "将 _llm_provider_client 设置为一个模拟客户端, 其 chat.completions.create 抛出 ValueError('invalid API key')。调用 _llm_select_json(work_dir='/tmp', prompt_content='test', stage='test', validator=lambda x: (True, None), schema_description='desc')。代码执行 _llm_json_call, 后者抛出 ValueError; 该异常从 _llm_select_json 传播出来, 导致出现异常。规范要求当 LLM 未产生有效输出时 (例如由于客户端错误) 返回 None 并记录错误日志。",
  "offending_statements": "Line 11: result = _llm_json_call(\\n          _llm_provider_client,\\n LLM_MODEL,\\n          messages,\\n          validator,\\n          schema_description,\\n trace_dir=os.path.join(work_dir, \\\"trace\\\"),\\n          trace_meta=meta,\\n          ),\\n"
```

```
"reason": "规范声明，当 LLM 未产生有效且符合规范的 JSON 结果时，函数始终返回 None；其中并未规定抛出异常。代码将 _llm_json_call 抛出的任何异常（例如网络或客户端错误）直接传播给调用者，违反了预期契约。"
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_a9c42dbb33174a909370b4d8652293e3_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_a9c42dbb33174a909370b4d8652293e3_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_a9c42dbb33174a909370b4d8652293e3_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-238 — `src--incremental_reasoner.py--_update_specs_for_intent::_plan_spec_update`

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: `confirmed`; attempts 1
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / `incremental_pipeline`; 原源码 `src/incremental_reasoner.py`
- **证据**: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: 当以下任一条件成立时返回 None: (a) `fqn` 在 `file_map` 中无条目或映射的文件在磁盘上不存在, (b) 文件扩展名在 `EXT_TO_LANG` 中对应不到任何已识别的语言, 或 (c) 函数已有有效规范且该规范在 `developer_intent` 下仍然正确。否则返回一个计划字典, 包含: `'fqn'` (原样 FQN)、`'fpath'` (绝对文件路径)、`'spec_dict'` (一个包含 `'signature'`、`'pre_condition'`、`'post_condition'` 键的字典, 描述函数的预期行为契约)、`'info_dict'` (一个包含 `'callees'` 键的字典, 列出预期的被调用者契约)、`'info_updated'` (一个布尔值, 当被调用者信息字典已生成或与先前版本相比发生了变化时为 true)、`'updated_callees'` (一个被调用者 FQN 字符串列表, 其预期契约与上一次运行不同)。返回的字典是纯数据记录 — 此函数不会写入或修改任何文件。
- **Actual behavior**: `_plan_spec_update` 完成后, 以下之一成立:

1. 函数返回 `None`, 因为:

- `file_map.get(fqn)` 为 `None` 或相应的路径不存在或不是常规文件, 或
- 文件扩展名无法通过 `EXT_TO_LANG` 映射到语言键, 或
- 在读取源代码以及可能已存在的 `.spec.json/.info.json` 文件后, 调用 `_opencode_generate_spec` (当没有先前规范时) 或 `_llm_check_spec_update` (当先前规范和 `info` 存在时) 产生 `falsy` 结果或 `result.get('spec_updated')` 不是 `True` 的结果, 或
- `result.get('new_spec')` 不是字典。

2. 函数返回一个字典 `plan`, 包含以下键:

- `'fqn'`: 输入的 `fqn`,
- `'fpath'`: 解析后的源文件路径,
- `'spec_dict'`: `_normalize_spec_dict(new_spec)` 的结果, 其中 `new_spec` 是更新后的规范字典,
- `'info_dict'`: `_normalize_info_dict(new_info)` 的结果, 其中 `new_info` 是 (可能是新生成的) 被调用者信息字典, 按以下方式确定:
 - 如果 `old_info` 为 `None`, 则 `new_info = result.get('new_info')` (如果它是字典), 否则 `{'callees': []}`, 并且 `info_updated = True`;
 - 否则 `info_updated = bool(result.get('info_updated'))`; 如果 `info_updated` 为 `true`, 则 `new_info = result.get('new_info')` (如果它是字典), 否则 `new_info = old_info`; 如果 `info_updated` 为 `false`, 则 `new_info = old_info`。
- `'info_updated'`: 按上述确定的布尔值,

◦ `'updated_callees': result.get('updated_callees')` 或 `[]`。

3. 异常（例如，读取源文件时抛出 `OSError`，或由 `_collect_caller_context`、LLM 函数或规范化助手抛出的任何异常）被引发且未被捕获，传播到调用者。在这种情况下不会产生返回值。

该函数仅执行读取 I/O；它不会写入文件系统或修改全局映射 `file_map`、`callers_map`、`callees_map`、`edge_aliases_map`、`EXT_TO_LANG`、`proj_dir`、`work_dir` 或 `developer_intent`。

- **Code evidence:** 第 42 行: `if not result or not result.get("spec_updated")`: 第 43 行: `return None`
- **Trigger condition:** 规范要求仅当条件 (a)、(b) 或 (c) 成立时返回 `None`。当没有先前规范时，条件 (c) 不适用，且 (a) 和 (b) 为 `false`。因此函数必须返回一个计划字典。代码因为 `result` 为 `falsy` 而返回 `None`，违反了规范。
- **Validator trigger:** 当 `_opencode_generate_spec` 对一个没有先前规范的函数返回 `falsy` 值时，`_plan_spec_update` 返回 `None` 而不是计划字典，从而静默地跳过了该函数。
- **Probe stdout:** CONFIRMED — 两种方法都复现了该错误。方法 1（真实代码）: CONFIRMED — `_update_specs_for_intent` 返回了一个空列表，意味着 `_plan_spec_update` 返回了 `None` 且函数被静默跳过。当 (a) `fpath` 存在，(b) 扩展名有效（`'py' -> 'python'`），且 (c) 不存在先前规范（`old_spec` 为 `None`）时，规范要求返回一个计划字典，而不是 `None`。错误位于第 1843-1844 行: `'if not result or not result.get("spec_updated"): return None'` 即使 `_opencode_generate_spec` 返回了 `None` (`falsy`)，也返回 `None`，违反了规范。方法 2（镜像逻辑）: CONFIRMED — 第 1843-1844 行的镜像逻辑在 `_opencode_generate_spec` 返回 `falsy` 值 (`None`) 时返回 `None`，但规范要求返回计划字典，因为条件 (a)、(b) 和 (c) 均为 `false`。静默的 `None` 返回导致该函数在调用者第 1961 行的过滤中被跳过: `'applied = [p for p in plans if p]'`。

► SPEC sidecar（完整）

```
{
  "signature": "_plan_spec_update(fqn, idx) -> dict | None",
  "pre_condition": "fqn 是一个完全限定的函数名字符串；idx 是一个非负整数；模块级 file_map 将 FQN 映射到绝对文件路径；callees_map 将 FQN 映射到其被调用者 FQN；callers_map 将 FQN 映射到其调用者 FQN；developer_intent 是一个描述预期代码修改的字符串；EXT_TO_LANG 将文件扩展名映射到已识别的语言键；proj_dir 和 work_dir 是有效的目录路径。",
  "post_condition": "当以下任一条件成立时返回 None：(a) fqn 在 file_map 中无条目或映射的文件在磁盘上不存在，(b) 文件扩展名在 EXT_TO_LANG 中对应不到任何已识别的语言，或 (c) 函数已有有效规范且该规范在 developer_intent 下仍然正确。否则返回一个计划字典，包含：'fqn'（原样 FQN）、'fpath'（绝对文件路径）、'spec_dict'（一个包含 'signature'、'pre_condition'、'post_condition' 键的字典，描述函数的预期行为契约）、'info_dict'（一个包含 'callees' 键的字典，列出预期的被调用者契约）、'info_updated'（一个布尔值，当被调用者信息字典已生成或与先前版本相比发生了变化时为 true）、'updated_callees'（一个被调用者 FQN 字符串列表，其预期契约与上一次运行不同）。返回的字典是纯数据记录 — 此函数不会写入或修改任何文件。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "_collect_caller_context",
      "signature": "_collect_caller_context(fqn, callers_map, file_map, edge_aliases_map)",
      "pre_condition": "fqn 是 file_map 中的一个 FQN，指向一个存在的文件；callers_map 将 FQN 映射到其调用者列表；file_map 将 FQN 映射到文件路径；edge_aliases_map 将被调用者名称映射到别名条目。",
      "post_condition": "返回一个数据结构，描述 fqn 的调用者对函数的期望 — 用于指导行为规范生成的调用方上下文。"
    },
    {
      "name": "_opencode_generate_spec",
      "signature": "_opencode_generate_spec(proj_dir, work_dir, idx, fqn, lang_key, phase_label, developer_intent, callee_names, source, caller_context)",
      "pre_condition": "proj_dir 和 work_dir 是有效的目录路径；fqn 标识一个已存在的函数；lang_key 是一个已识别的语言标识符；source 是该函数的完整源代码文本；callee_names 是一个被调用函数名称字符串列表；caller_context 描述调用者期望。",
      "post_condition": "返回一个字典。当 'spec_updated' 为 true 时：'new_spec' 包含一个具有 'signature'、'pre_condition'、'post_condition' 键的规范字典，'new_info' 包含一个具有 'callees' 键的被调用者规范字典。当无法生成时，返回一个 falsy 或不完整的结果。"
    }
  ]
}
```



```
"name": "_llm_check_spec_update",
"signature": "_llm_check_spec_update(proj_dir, work_dir, idx, fn, lang_key, phase_label,
developer_intent, old_spec, old_info, callee_names, source)",
"pre_condition": "old_spec 是一个来自先前规范运行、具有 'signature'、'pre_condition'、'post_condition' 键
的字典；old_info 是一个来自先前运行、具有 'callees' 键的被调用者规范字典；developer_intent 是一个非空字符串，描述代码变
更；所有其他参数与 _opencode_generate_spec 中相同。",
"post_condition": "返回一个字典。当 'spec_updated' 为 true 时：'new_spec' 包含更新后的规
范，'info_updated'（布尔值）和 'new_info'（字典）指示被调用者期望是否发生了变化。当 'spec_updated' 为 false 时，现有
规范在 developer_intent 下仍然正确，无需更新。键 'updated_callees' 列出了预期契约发生变化的被调用者 FQN 列表。"
},
{
"name": "_normalize_spec_dict",
"signature": "_normalize_spec_dict(spec_dict) -> dict",
"pre_condition": "spec_dict 是一个至少包含 'signature'、'pre_condition' 和 'post_condition' 键的字典。",
"post_condition": "返回一个字典，其中所有必需的规范键都存在，且值符合下游消费者期望的规范 .spec.json 模式。"
},
{
"name": "_normalize_info_dict",
"signature": "_normalize_info_dict(info_dict) -> dict",
"pre_condition": "info_dict 是一个包含 'callees' 键的字典，其值是一个被调用者规范对象列表。",
"post_condition": "返回一个符合规范 .info.json 模式的字典，所有被调用者条目均已规范化为预期结构。"
}
]
```

- Logic verification result（完整）
- Bug Validator 结果（完整）

```
{
  "id": "src--incremental_reasoner-py--_update_specs_for_intent::_plan_spec_update",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_update_specs_for_intent::_plan_spec_update.py",
  "function_name": "_plan_spec_update",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_update_specs_for_intent::_plan_spec_update.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_update_specs_for_intent::_plan_spec_update.md",
  "probe_stdout": "已确认 - 两种方法均重现此 bug。\\n 方法 1（真实代码）：已确认 - `_update_specs_for_intent` 返回了空列表，这意味着 `_plan_spec_update` 返回了 None，该函数被静默跳过。当（a）fpath 存在、（b）扩展名有效（'py' -> 'python'）且（c）不存在先前规约（old_spec 为 None）时，规约要求返回 plan 字典，而非 None。Bug 位于第 1843-1844 行：'if not result or not result.get(\"spec_updated\")：return None'，即使在 `_opencode_generate_spec` 返回 None（假值）时也会返回 None，违反了规约。\\n 方法 2（镜像逻辑）：已确认 - 第 1843-1844 行的镜像逻辑在 `_opencode_generate_spec` 返回假值（None）时返回 None，但规约要求返回 plan 字典，因为条件（a）、（b）和（c）均为假。静默的 None 返回导致该函数在调用方第 1961 行的过滤中被跳过：'applied = [p for p in plans if p]'。",
  "trigger_summary": "当 `_opencode_generate_spec` 对没有先前规约的函数返回假值时，`_plan_spec_update` 返回 None 而非 plan 字典，导致该函数被静默跳过。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:59:32.082508Z	5713
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:00:25.097874Z	5555

► parsed_json (完整)

```
{
  "post_condition": "`_plan_spec_update` 完成后，将出现以下情况之一：\n1. 函数返回 `None`，因为：\n    - `file_map.get(fqn)` 为 `None`，或者对应的路径不存在或不是常规文件，或\n    - 文件扩展名无法通过 `EXT_TO_LANG` 映射到语言键，或\n    - 在读取源文件以及可能存在的 `.spec.json`/`.info.json` 文件后，调用 `_opcode_generate_spec`（当不存在先前规约时）或 `_llm_check_spec_update`（当先前规约和信息存在时）产生假值结果，或者结果中 `result.get('spec_updated')` 不为 `True`，或\n    - `result.get('new_spec')` 不是字典。
  \n2. 函数返回一个字典 `plan`，其键包括：\n    - ``fqn``：输入的 `fqn`，\n    - ``fpath``：解析后的源文件路径，\n    - ``spec_dict``：`_normalize_spec_dict(new_spec)` 的结果，其中 `new_spec` 是更新后的规约字典，\n    - ``info_dict``：`_normalize_info_dict(new_info)` 的结果，其中 `new_info` 是（可能新生成的）被调用方信息字典，确定方式如下：\n    * 如果 `old_info` 为 `None`，那么 `new_info = result.get('new_info')`（如果它是字典），否则为 `{'callees': []}`，同时 `info_updated = True`；\n    * 否则 `info_updated = bool(result.get('info_updated'))`；如果 `info_updated` 为 true，那么 `new_info = result.get('new_info')`（如果它是字典），否则 `new_info = old_info`；如果 `info_updated` 为 false，那么 `new_info = old_info`。
  \n    - ``info_updated``：如上确定的布尔值，\n    - ``updated_callees``：`result.get('updated_callees')` 或 `[]`。
  \n3. 异常（例如，读取源文件时的 `OSError`，或由 `_collect_caller_context`、LLM 函数或规范化辅助函数抛出的任何异常）被引发且未被捕获，传播给调用方。这种情况下不产生返回值。
  \n\n函数仅执行读 I/O；它不会写入文件系统，也不会修改全局映射 `file_map`、`callers_map`、`callees_map`、`edge_aliases_map`、`EXT_TO_LANG`、`proj_dir`、`work_dir` 或 `developer_intent`。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_102314e876aa4d7799022d9aa635f924_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_102314e876aa4d7799022d9aa635f924_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_102314e876aa4d7799022d9aa635f924_response.txt)

事件 2 — llm_43d761d32f5f46b5812bffeef94c01ee

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "fqn = 'mylib.myfunc'; file_map 包含 'mylib.myfunc' -> '/path/to/myfunc.py' 且文件存在；扩展名 'py' 映射到 EXT_TO_LANG 中已识别的语言；文件 myfunc.py.spec.json 和 myfunc.py.info.json 不存在（因此 old_spec 和 old_info 为 None）；_opcode_generate_spec 返回 None（生成失败）。",
  "offending_statements": "Line 42:         if not result or not result.get(\"spec_updated\"):
  \nLine 43: return None",
  "reason": "规约要求仅当条件（a）、（b）或（c）成立时才返回 None。当不存在先前规约时，条件（c）不适用，且（a）和（b）为假。因此函数必须返回 plan 字典。代码因为 result 为假值而返回 None，违反了规约。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_43d761d32f5f46b5812bffeef94c01ee_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_43d761d32f5f46b5812bffeef94c01ee_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_43d761d32f5f46b5812bffeef94c01ee_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

FMA-MISMATCH-239 — src--incremental_reasoner-py--update_specs_for_intent::_reconcile_caller

- 四类结论：契约不确定
- 分类依据：probe 已复现 SPEC 与实现的差异，但现有证据不足以决定应修改实现还是修改 SPEC。
- **Validator**: confirmed；尝试次数 1
- 阶段 / 模块：Phase 6 — 推理与验证引擎 / incremental_pipeline；原源码 [src/incremental_reasoner.py](#)
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** updates 中的每个 (callee_name, callee_new_spec) 对被依次处理。对每个对，判断调用方的现有 .info.json 中的 callee 条目是否需要修订以与 callee_new_spec 保持一致——该判断考虑调用方的源代码、当前 .info.json 以及被调用方的新规约。当需要修订时，调用方的整个 .info.json 将被替换为一个协调后的版本，其中 callee 条目与 callee_new_spec 以及所有之前已协调的 callee 条目一致。当任何对都不需要修订时，.info.json 保持不变。如果至少发生了一次 .info.json 替换，则返回调用方绝对源文件路径的字符串；当调用方源文件不存在、无法确定其编程语言、其 .info.json 不可读或未应用任何替换时，返回 None。
- **实际行为:** 函数正常完成（即没有引发异常）后，以下情况成立：

设 cpath = file_map.get(caller_fqn)。如果 cpath 不是指向现有文件的非空字符串，或语言无法确定（clang 为假值），则函数返回 None，且辅助文件 cpath + '.info.json' 未被修改。

否则，设 n = len(updates)。对于 i 从 0 到 n-1，令 (cname_i, cspec_i) = updates[i]。在第 i 次迭代期间：

- 从 cpath 读取源代码。如果发生 OSError，函数终止并抛出该异常。
- 加载辅助信息。如果发生 OSError 或 JSONDecodeError，迭代继续到 i+1。
- cresult_i = _llm_check_caller_info_update(...)。如果 cresult_i 为 None 或 cresult_i.get('info_updated') 不为真，继续。
- c_new_info_i = cresult_i.get('new_info')；如果不是 isinstance(..., dict)，继续。
- 辅助文件被 _normalize_info_dict(c_new_info_i) 的输出覆盖。如果发生 OSError，函数终止并抛出该异常。
- changed 被设为 True。

定义 success_i 当且仅当在第 i 次迭代写入之前没有引发异常且 cresult_i 不是 None 且 cresult_i.info_updated 为真且 cresult_i.new_info 是一个 dict。如果存在任意 i 满足 success_i，则 changed = True；否则 changed = False。

正常返回值 = 若 changed 为真则为 cpath，否则为 None。

正常返回后辅助文件的状态：

- 如果 changed = False：文件未更改（与调用前状态相同，除了临时文件系统元数据）。
- 如果 changed = True：令 j = max{ i | success_i }。文件包含 _normalize_info_dict(c_new_info_j) 的 JSON 序列化输出。其结构是一个字典，具有单个键 'callees'，其值为一个字典列表，每个字典按顺序包含键 'name'、'signature'、'pre_condition'、'post_condition'，缺失字段用空字符串填充，多余键被移除。

如果引发任何异常（例如 OSError，或来自 _llm_check_caller_info_update），函数不会正常返回，且辅助文件可能处于任意状态（损坏、部分写入或未更改）。

正式后置条件（对于正常终止）： exists cpath = file_map.get(caller_fqn), cext, clang . ((cpath 不是有效文件路径 $\vee$ 非 clang) $\rightarrow$ 返回 None $\wedge$ sidecar_unchanged(cpath)) $\wedge$ (valid(cpath) $\wedge$ clang $\rightarrow$ let changed = ($\exists i \in 0..n-1$. success_i) in return = (cpath if changed else None) $\wedge$ (changed $\rightarrow$ j = max{ i | success_i } . sidecar_content(cpath) = serialize(_normalize_info_dict(c_new_info_j))) $\wedge$ ($\neg$ changed $\rightarrow$ sidecar_unchanged(cpath))) 其中 success_i $\leftrightarrow$ (no_exception_until_write(i) $\wedge$ cresult_i $\neq$ None $\wedge$ cresult_i.info_updated $\wedge$ typeof(cresult_i.new_info) = dict)

- **代码证据:** Line 35: with open(f'{cpath}.info.json', "w", encoding="utf-8") as f: Line 36: json.dump(_normalize_info_dict(c_new_info), f, indent=2, ensure_ascii=False)
- **触发条件:** 代码用 LLM 的输出逐字替换整个辅助文件，但 LLM 的后置条件并不保证其保留所有现有的 callee 条目。这可能导致根据规约应保持不变 callee 信息的丢失。
- **验证器触发:** 当 _llm_check_caller_info_update 返回的 new_info 字典省略了不相关的 callee 条目时，_reconcile_caller 会用 LLM 的输出逐字覆盖调用方的 .info.json，永久丢失规约要求保留的 callee 信息。
- **Probe 标准输出:** [TopdownLayers] Phase 1 (test): 2 functions, 2 layers -> spec_prompts/phase_01_topdown_layers.json CONFIRMED — callee info lost: expected {'callee_func',

'other_func'}, got {'callee_func'}. 调用方的 .info.json 被仅包含已协调 callee 条目的内容覆盖, 丢失了所有其他 callee 条目。

► SPEC sidecar (完整)

```
{
  "signature": "_reconcile_caller(caller_fqn, updates, base_idx) -> str | None",
  "pre_condition": "caller_fqn 是一个非空字符串, 标识一个已提取的函数。其对应的源文件存在且有一个可读的 .info.json 辅助文件。updates 是一个非空的 (callee_name: str, callee_new_spec: dict) 对列表, 其中每个 callee_new_spec 字典包含字符串键 'signature'、'pre_condition' 和 'post_condition'。base_idx 是一个非负整数。",
  "post_condition": "updates 中的每个 (callee_name, callee_new_spec) 对被依次处理。对每个对, 判断调用方的现有 .info.json 中的 callee 条目是否需要修订以与 callee_new_spec 保持一致—该判断考虑调用方的源代码、当前 .info.json 以及被调用方的新規約。当需要修订时, 调用方的整个 .info.json 将被替换为一个协调后的版本, 其中 callee 条目与 callee_new_spec 以及所有之前已协调的 callee 条目一致。当任何对都不需要修订时, .info.json 保持不变。如果至少发生了一次 .info.json 替换, 则返回调用方绝对源文件路径的字符串; 当调用方源文件不存在、无法确定其编程语言、其 .info.json 不可读或未应用任何替换时, 返回 None。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_llm_check_caller_info_update",
      "signature": "_llm_check_caller_info_update(proj_dir, work_dir, artifact_idx, caller_fqn, callee_name, callee_lang, caller_path, callee_new_spec, caller_info, caller_source) -> dict | None",
      "pre_condition": "artifact_idx 是一个非负整数。caller_fqn 和 callee_name 是非空字符串。callee_lang 是一个有效的语言标识符字符串。callee_new_spec 是一个包含 'signature'、'pre_condition' 和 'post_condition' 字符串字段的字典。caller_info 是一个符合 .info.json 模式的字典。caller_source 是调用方原始源代码的字符串。",
      "post_condition": "返回一个包含键 'info_updated' (布尔值) 的字典。当 info_updated 为真时, 还返回键 'new_info', 其值为一个符合 .info.json 模式的字典, 其中 callee 条目与 callee_new_spec 一致。当评估无法产生有效判定时, 返回 None。"
    },
    {
      "name": "_normalize_info_dict",
      "signature": "_normalize_info_dict(info: dict) -> dict",
      "pre_condition": "info 是一个可能包含 'callees' 键的字典, 该键的值是一个字典列表, 每个字典包含字段 'name'、'signature'、'pre_condition'、'post_condition' 的任意子集。",
      "post_condition": "返回一个仅包含 'callees' 键的字典, 其值是一个字典列表, 每个字典按顺序包含 'name'、'signature'、'pre_condition'、'post_condition' 键。输入 callee 条目中缺失的字段用空字符串填充。输入 callee 条目中的多余键被移除。"
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_update_specs_for_intent::_reconcile_caller.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "updates 中的每个 (callee_name, callee_new_spec) 对被依次处理。对每个对, 判断调用方的现有 .info.json 中的 callee 条目是否需要修订以与 callee_new_spec 保持一致—该判断考虑调用方的源代码、当前 .info.json 以及被调用方的新規約。当需要修订时, 调用方的整个 .info.json 将被替换为一个协调后的版本, 其中 callee 条目与 callee_new_spec 以及所有之前已协调的 callee 条目一致。当任何对都不需要修订时, .info.json 保持不变。如果至少发生了一次 .info.json 替换, 则返回调用方绝对源文件路径的字符串; 当调用方源文件不存在、无法确定其编程语言、其 .info.json 不可读或未应用任何替换时, 返回 None。",
    "actual_behavior": "函数正常完成 (即没有引发异常) 后, 以下情况成立:\n\n设 cpath = file_map.get(caller_fqn)。如果 cpath 不是指向现有文件的非空字符串, 或语言无法确定 (clang 为假值), 则函数返回 None, 且辅助文件 cpath + '.info.json' 未被修改。\n\n否则, 设 n = len(updates)。对于 i 从 0 到 n-1, 令 (cname_i, cspec_i) = updates[i]。在第 i 次迭代期间:\n  - 从 cpath 读取源代码。如果发生 OSError, 函数终止并抛出该异常。\n  - 加载辅助信息。如果发生 OSError 或 JSONDecodeError, 迭代继续到 i+1。\n  - cresult_i = _llm_check_caller_info_update(...)。如果 cresult_i 为 None 或 cresult_i.get('info_updated') 不为真, 继续。\n  - c_new_info_i = cresult_i.get('new_info'); 如果不是
```

```
isinstance(..., dict), 继续。\\n - 辅助文件被 _normalize_info_dict(c_new_info_i) 的输出覆盖。如果发生 OSError, 函数终止并抛出该异常。\\n - changed 被设为 True。\\n\\n定义 success_i 当且仅当在第 i 次迭代写入之前没有引发异常且 cresult_i 不是 None 且 cresult_i.info_updated 为真且 cresult_i.new_info 是一个 dict。如果存在任意 i 满足 success_i, 则 changed = True; 否则 changed = False。\\n\\n正常返回值 = 若 changed 为真则为 cpath, 否则为 None。\\n\\n正常返回后辅助文件的状态:\\n - 如果 changed = False: 文件未更改 (与调用前状态相同, 除了临时文件系统元数据)。\\n - 如果 changed = True: 令 j = max{ i | success_i }。文件包含 _normalize_info_dict(c_new_info_j) 的 JSON 序列化输出。其结构是一个字典, 具有单个键 'callees', 其值为一个字典列表, 每个字典按顺序包含键 'name', 'signature', 'pre_condition', 'post_condition', 缺失字段用空字符串填充, 多余键被移除。\\n\\n如果引发任何异常 (例如 OSError, 或来自 _llm_check_caller_info_update), 函数不会正常返回, 且辅助文件可能处于任意状态 (损坏、部分写入或未更改)。\\n\\n正式后置条件 (对于正常终止):\\n exists cpath = file_map.get(caller_fqn), cext, clang .\\n ( cpath 不是有效文件路径 v 非 clang ) → 返回 None ∧ sidecar_unchanged(cpath) )\\n ∧ ( valid(cpath) ∧ clang →\\n let changed = ( ∃ i ∈ 0..n-1 . success_i ) in\\n return = ( cpath if changed else None ) ∧\\n ( changed → j = max{ i | success_i } .\\n sidecar_content(cpath) = serialize(_normalize_info_dict(c_new_info_j)) ) ∧\\n ( ¬changed → sidecar_unchanged(cpath) ) )\\n 其中 success_i ↔ (no_exception_until_write(i) ∧ cresult_i ≠ None ∧ cresult_i.info_updated ∧ typeof(cresult_i.new_info) = dict)", "code_evidence": "Line 35: with open(f\\\"{cpath}.info.json\\\", \\\"w\\\", encoding=\\\"utf-8\\\") as f:\\nLine 36: json.dump(_normalize_info_dict(c_new_info), f, indent=2, ensure_ascii=False)", "trigger_condition": "代码用 LLM 的输出逐字替换整个辅助文件, 但 LLM 的后置条件并不保证其保留所有现有的 callee 条目。这可能导致根据规约应保持不变 callee 信息的丢失。"} }
```

- Bug Validator result (完整)
- Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌, 使用每条事件后的载荷链接打开。

编号	目的	状态	尝试	区块	开始时间	令牌数
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:59:08.100204Z	5120
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:00:08.981524Z	8070

事件 1 — [llm_a4c863bf222a4f98803c03c2703493ca](#)

- parsed_json (完整)

```
{ "post_condition": "当函数正常完成时 (即未引发异常), 以下条件成立:\\n\\n设 cpath = file_map.get(caller_fqn)。若 cpath 不是指代已存在文件的非空字符串, 或无法确定语言 (clang 为假值), 则函数返回 None, 且侧车文件 cpath + '.info.json' 保持不变。\\n\\n否则, 令 n = len(updates)。对于 i 从 0 到 n-1, 令 (cname_i, cspec_i) = updates[i]。在迭代 i 期间:\\n - 从 cpath 读取源代码。若发生 OSError, 则函数以该异常终止。\\n - 加载侧车信息。若发生 OSError 或 JSONDecodeError, 则迭代继续至 i+1。\\n - cresult_i = _llm_check_caller_info_update(...)。若 cresult_i 为 None 或 cresult_i.get('info_updated') 不为 true, 则继续。\\n - c_new_info_i = cresult_i.get('new_info'); 若非 isinstance(..., dict), 则继续。\\n - 侧车文件被 _normalize_info_dict(c_new_info_i) 覆写。若发生 OSError, 则函数以该异常终止。\\n - changed 设为 True。\\n\\n定义 success_i, 当且仅当在迭代 i 中, 在写入前未引发异常, 且 cresult_i 不为 None, 且 cresult_i.info_updated 为 true, 且 cresult_i.new_info 为 dict。若存在任一 i 满足 success_i, 则 changed = True; 否则 changed = False。\\n\\n正常返回值 = 若 changed 为 true, 则返回 cpath, 否则返回 None。\\n\\n正常返回后侧车文件的状态:\\n - 若 changed = False: 文件未更改 (与调用前状态相同, 除临时文件系统元数据外)。\\n - 若 changed = True: 令 j = max{ i | success_i }。文件包含 _normalize_info_dict(c_new_info_j) 的 JSON 序列化输出。其结构为包含单个键 'callees' 的字典, 该键的值是一个字典列表, 每个字典恰好包含以下键: 'name', 'signature', 'pre_condition', 'post_condition', 按此顺序排列, 缺失字段填充空字符串, 多余键移除。\\n\\n若引发任何异常 (如 OSError, 或来自 _llm_check_caller_info_update), 则函数不会正常返回, 且侧车文件可能处于任意状态 (损坏、部分写入或未更改)。\\n\\n形式化后置条件 (正常终止时):\\n exists cpath = file_map.get(caller_fqn), cext, clang .\\n ( ( cpath 不是有效文件路径 v 非 clang ) → return = None ∧ sidecar_unchanged(cpath) )\\n ∧ ( valid(cpath) ∧ clang →\\n let changed = ( ∃ i ∈ 0..n-1 . success_i ) in\\n return = ( 若 changed 则为 cpath 否则为 None ) ∧\\n ( changed → ∃ j = max{ i | success_i } .\\n sidecar_content(cpath) = serialize(_normalize_info_dict(c_new_info_j)) ) )\\n ∧ ( ¬changed → sidecar_unchanged(cpath) ) )\\n 其中 success_i ≡ (直到写入时未发生异常 ∧ cresult_i ≠ None ∧ cresult_i.info_updated ∧ typeof(cresult_i.new_info) = dict)" }
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_a4c863bf222a4f98803c03c2703493ca_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_a4c863bf222a4f98803c03c2703493ca_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_a4c863bf222a4f98803c03c2703493ca_response.txt)

事件 2 — llm_df656f0149554c69a20442be0089199d

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "假设 file_map.get('caller1') 返回 '/tmp/caller.py', 该文件存在且语言已确定。侧车文件 '/tmp/caller.py.info.json' 初始内容为 {\"callees\": [{\"name\": \"foo\", \"signature\": \"...\"}, {\"name\": \"bar\", \"signature\": \"...\"}], \"pre_condition\": \"...\", \"post_condition\": \"...\"}}。updates 列表为 [{\"bar\", {\"signature\": \"...\", \"pre_condition\": \"...\", \"post_condition\": \"...\"}}]。为 bar 调用的 LLM 返回 {\"info_updated\": true, \"new_info\": {\"callees\": [{\"name\": \"bar\", ...}]}} (即, 丢弃了已存在的 'foo' 条目)。规范要求生成的 .info.json 应包含与 bar 及所有先前已调和条目一致的被调用方条目, 其中包括已存在的 'foo'; 但代码仅使用 bar 进行覆写, 违反了规范。",
  "offending_statements": "第 35 行: with open(f\"{cpath}.info.json\", \"w\", encoding=\"utf-8\") as f:\n第 36 行:     json.dump(_normalize_info_dict(c_new_info), f, indent=2, ensure_ascii=False)",
  "reason": "代码将 LLM 的输出原封不动地替换整个侧车文件, 但 LLM 的后置条件并不保证保留所有现有的被调用方条目。根据规范, 这可能导致那些本应保持不变的被调用方信息丢失。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_df656f0149554c69a20442be0089199d_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_df656f0149554c69a20442be0089199d_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_df656f0149554c69a20442be0089199d_response.txt) **复核动作**

- 先由维护者确认预期契约, 再决定修改实现或重写 SPEC; 当前不应直接登记为真实 Bug。

模块: orchestration

FMA-MISMATCH-270 — main-py--_clean_previous_run

- 四类结论: 契约不确定
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块: Phase 6 — Reasoning & Verification Engine / orchestration; 原源码 main.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** 由 work_dir 表示的文件系统路径不再指向已存在的目录。如果在调用前该路径指向一个目录, 则该目录及其中的所有文件和子目录已从文件系统中递归删除。
- **Actual behavior:** 如果函数在未引发异常的情况下完成, 则以下断言成立: 如果调用前 `os.path.isdir(work_dir)` 为真, 则调用后 `os.path.exists(work_dir)` 为假 (目录及其内容已被删除); 如果调用前 `work_dir` 不是一个目录 (或不存在), 则 `work_dir` 处的文件系统状态保持不变。如果函数引发异常, 则 `work_dir` 的状态未指定 (可能被部分删除或保持不变)。形式化表述为: $(\text{is_dir}(\text{work_dir}) \wedge \text{pre}) \rightarrow \neg \text{exists}(\text{work_dir}) \wedge \text{post} \vee (\neg \text{is_dir}(\text{work_dir}) \wedge \text{pre}) \rightarrow \text{unchanged}(\text{work_dir}) \wedge \text{post} \vee (\text{exceptional_return} \rightarrow \text{top})$
- **Code evidence:** Line 3: `if os.path.isdir(work_dir):` Line 4: `shutil.rmtree(work_dir)`

- **Trigger condition:** `os.path.isdir` 跟踪符号链接，因此指向目录的符号链接被视为目录，条件为真。但是，`shutil.rmtree` 删除的是符号链接本身，而不是目标目录。目标目录及其内容保持不变，这违反了要求递归删除目录及其所有内容的规定。
- **Validator trigger:** 当 `work_dir` 是指向目录的符号链接时，`os.path.isdir` 返回 `True`，但 `shutil.rmtree` 引发 `OSError`（Python 3.12+）或仅删除符号链接，使目标目录保持不变。
- **Probe stdout:** CONFIRMED \u2014 `shutil.rmtree` 在符号链接上引发 `OSError`（无法在符号链接上调用 `rmtree`）；目标目录及内容未改动，位于：'/tmp/tmpj6g9l18x/real_target'\n 标记文件完好：'/tmp/tmpj6g9l18x/real_target/important_data.txt'

► SPEC sidecar（完整）

```
{
  "signature": "_clean_previous_run(work_dir)",
  "pre_condition": "work_dir 是一个表示文件系统路径的字符串。",
  "post_condition": "由 work_dir 表示的文件系统路径不再指向已存在的目录。如果在调用前该路径指向一个目录，则该目录及其中的所有文件和子目录已从文件系统中递归删除。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/main-py/_clean_previous_run.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "由 work_dir 表示的文件系统路径不再指向已存在的目录。如果在调用前该路径指向一个目录，则该目录及其中的所有文件和子目录已从文件系统中递归删除。",
    "actual_behavior": "如果函数在未引发异常的情况下完成，则以下断言成立：如果调用前 `os.path.isdir(work_dir)` 为真，则调用后 `os.path.exists(work_dir)` 为假（目录及其内容已被删除）；如果调用前 `work_dir` 不是一个目录（或不存在），则 `work_dir` 处的文件系统状态保持不变。如果函数引发异常，则 `work_dir` 的状态未指定（可能被部分删除或保持不变）。形式化表述为：\\( (\\text{normal}\\_return \\rightarrow (\\text{is\\_dir}(work\_dir)_{\\text{pre}} \\rightarrow \\neg \\text{exists}(work\_dir)_{\\text{post}})) \\wedge (\\neg \\text{is\\_dir}(work\_dir)_{\\text{pre}} \\rightarrow \\text{unchanged}(work\_dir)) ) \\wedge (\\text{exceptional}\\_return \\rightarrow \\text{top}) \\)",
    "code_evidence": "Line 3:     if os.path.isdir(work_dir):\nLine 4:         shutil.rmtree(work_dir)",
    "trigger_condition": "os.path.isdir 跟踪符号链接，因此指向目录的符号链接被视为目录，条件为真。但是，shutil.rmtree 删除的是符号链接本身，而不是目标目录。目标目录及其内容保持不变，这违反了要求递归删除目录及其所有内容的规定。"
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "main-py--_clean_previous_run",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/main-py/_clean_previous_run.py",
  "function_name": "_clean_previous_run",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_main-py--_clean_previous_run.py",
  "detail_file": "fm_agent/bug_validation/main-py--_clean_previous_run.md",
}
```

```
"probe_stdout": "CONFIRMED \\u2014 shutil.rmtree 在符号链接上引发 OSError (无法在符号链接上调用 rmtree) ; 目标目录及内容未改动, 位于: '/tmp/tmpj6g9l18x/real_target'\\n 标记文件完好: '/tmp/tmpj6g9l18x/real_target/important_data.txt'",
  "trigger_summary": "当 work_dir 是指向目录的符号链接时, os.path.isdir 返回 True, 但 shutil.rmtree 引发 OSError (Python 3.12+) 或仅删除符号链接, 使目标目录保持不变。"
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌, 使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:44:35.074310Z	1812
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:45:05.654801Z	2775

事件 1 — llm_91080a4292e24638b63bbc8701746743

► parsed_json (完整)

```
{
  "post_condition": "如果函数在未引发异常的情况下完成, 则以下断言成立: 如果调用前 `os.path.isdir(work_dir)` 为真, 则调用后 `os.path.exists(work_dir)` 为假 (目录及其内容已被删除); 如果调用前 `work_dir` 不是一个目录 (或不存在), 则 `work_dir` 处的文件系统状态保持不变。如果函数引发异常, 则 `work_dir` 的状态未指定 (可能被部分删除或保持不变)。形式化表述为: \\( (\\text{normal}\\_return \\rightarrow (\\text{is\\_dir}(work\_dir)\_{pre}) \\rightarrow \\neg \\text{exists}(work\_dir)\_{post}) \\wedge (\\neg \\text{is\\_dir}(work\_dir)\_{pre}) \\rightarrow \\text{unchanged}(work\_dir) ) \\wedge (\\text{exceptional}\\_return \\rightarrow \\top ) \\)"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_91080a4292e24638b63bbc8701746743_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_91080a4292e24638b63bbc8701746743_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_91080a4292e24638b63bbc8701746743_response.txt)

事件 2 — llm_5e35e7b909674ee58d9a26d0cfc21bf5

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "work_dir 是一个指向现有目录的符号链接 (例如 /tmp/link -> /tmp/readdir)。",
  "offending_statements": "Line 3:   if os.path.isdir(work_dir):\\nLine 4:  shutil.rmtree(work_dir)",
  "reason": "os.path.isdir 跟踪符号链接, 因此指向目录的符号链接被视为目录, 条件为真。但是, shutil.rmtree 删除的是符号链接本身, 而不是目标目录。目标目录及其内容保持不变, 这违反了要求递归删除目录及其所有内容的规定。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_5e35e7b909674ee58d9a26d0cfc21bf5_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_5e35e7b909674ee58d9a26d0cfc21bf5_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_5e35e7b909674ee58d9a26d0cfc21bf5_response.txt) **复核动作**

- 先由维护者确认预期契约, 再决定修改实现或重写 SPEC; 当前不应直接登记为真实 Bug。

- 四类结论： 契约不确定
- 分类依据： 沿用旧 `fm_agent/bug_list.md` 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator： `confirmed`；attempts 1
- Phase / 模块： Phase 6 — Reasoning & Verification Engine / `scope_analysis`；原源码 `src/scope.py`
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**： 返回一个非负浮点数。得分为以下加权分量的总和：(a) 对类名各组成部分中每一个与 `signals['backtick_idents']` 中的 token 匹配的 token，加上固定的反引号名称重叠权重；(b) 对类名各组成部分中每一个与 `signals['plain_idents']` 中的 token 匹配的 token，加上固定的类名重叠权重；(c) 对类名各组成部分中每一个与 `signals['all_words']` 中的 token 匹配的 token，加上相同的固定类名重叠权重；(d) 对类名各组成部分中每一个与 `signals['dotted_classes']` 中的 token 匹配的 token，加上固定的点号引用重叠权重；(e) 当 `cls['docstring']` 非空且为真值时：对文档字符串中每一个由四个或更多字符组成、不是停用词，且与 `signals['all_words']` 中的 token 匹配的字母词，加上固定的类文档重叠权重。当 `cls['docstring']` 为空或假值时，分量 (e) 贡献为零。当没有任何名称部分 token 及符合条件的文档字符串词 token 与任何指定的信号集重叠时，返回值恰好为零。
- **Actual behavior**： 该函数返回一个非负浮点得分，计算方式为：令 `name = cls['name']`；`doc = cls['docstring']`；`name_parts = _name_parts(name)`。得分 = `|name_parts signals['backtick_idents']| * W_BACKTICK_NAME + |name_parts signals['plain_idents']| * W_CLASS_NAME_MATCH + |name_parts signals['all_words']| * W_CLASS_NAME_MATCH + |name_parts signals['dotted_classes']| * W_DOTTED_REF + (如果 doc " 则 $\sum_{w \in \text{re.findall}(\text{r}\backslash\text{b}([a-zA-Z]\{4,\})\backslash\text{b}', \text{doc.lower()})} \text{signals}[\text{'all_words'}][w] * W_CLASS_DOC_MATCH}$ 否则 0)`。输入 `cls` 和 `signals` 未被修改；不会抛出异常。
- **Code evidence**： 第 15 行：`re.findall(r'\b([a-zA-Z]{4,})\b', cls['docstring'].lower())`
- **Trigger condition**： 该正则表达式将词提取限制为 ASCII 字母 `[a-zA-Z]`，但规范要求提取所有字母词（包括非 ASCII 字母）。对于 `docstring 'nave'`，代码未找到匹配词并得分为 0，而规范将会给予 `W_CLASS_DOC_MATCH` 得分，因为 `'nave'` 是一个五字符的字母词且出现在 `signals['all_words']` 中。
- **Validator trigger**： 仅限于 ASCII 的正则表达式 `[a-zA-Z]` 无法匹配类文档字符串中的非 ASCII 字母词，导致 `_score_class` 遗漏符合条件的文档词与 `signals['all_words']` 的重叠。
- **Probe stdout**： CONFIRMED — actual: 0.0 | expected: 2.0

► SPEC sidecar（完整）

```
{
  "signature": "_score_class(cls: dict, signals: dict[str, set[str]]) -> float",
  "pre_condition": "cls 是一个字典，至少包含键 'name'（字符串，类标识符）和 'docstring'（字符串，类的文档字符串或空字符串）。signals 是一个字典，键为信号类别字符串，每个值是包含小写字符串 token 的集合。该字典至少包含键 'backtick_idents'、'plain_idents'、'all_words' 和 'dotted_classes'。",
  "post_condition": "返回一个非负浮点数。得分为以下加权分量的总和：(a) 对类名各组成部分中每一个与 signals['backtick_idents'] 中的 token 匹配的 token，加上固定的反引号名称重叠权重；(b) 对类名各组成部分中每一个与 signals['plain_idents'] 中的 token 匹配的 token，加上固定的类名重叠权重；(c) 对类名各组成部分中每一个与 signals['all_words'] 中的 token 匹配的 token，加上相同的固定类名重叠权重；(d) 对类名各组成部分中每一个与 signals['dotted_classes'] 中的 token 匹配的 token，加上固定的点号引用重叠权重；(e) 当 cls['docstring'] 非空且为真值时：对文档字符串中每一个由四个或更多字符组成、不是停用词，且与 signals['all_words'] 中的 token 匹配的字母词，加上固定的类文档重叠权重。当 cls['docstring'] 为空或假值时，分量 (e) 贡献为零。当没有任何名称部分 token 及符合条件的文档字符串词 token 与任何指定的信号集重叠时，返回值恰好为零。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "_name_parts",
      "signature": "_name_parts(name: str) -> set[str]",
      "pre_condition": "name 是一个表示类标识符的非空字符串。",
      "post_condition": "返回一个小写字符串 token 的集合，这些 token 是通过在单词边界——包括下划线分隔符以及小写字母到大写字母的转换处——分割类名获得的，使得每个返回的 token 都是原始标识符的一个语义组成部分。"
    }
  ]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/scope-
py/_score_class.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个非负浮点数。得分为以下加权分量的总和：(a) 对类名各组成部分中每一个与
signals['backtick_idsents'] 中的 token 匹配的 token，加上固定的反引号名称重叠权重；(b) 对类名各组成部分中每一个与
signals['plain_idsents'] 中的 token 匹配的 token，加上固定的类名重叠权重；(c) 对类名各组成部分中每一个与
signals['all_words'] 中的 token 匹配的 token，加上相同的固定类名重叠权重；(d) 对类名各组成部分中每一个与
signals['dotted_classes'] 中的 token 匹配的 token，加上固定的点号引用重叠权重；(e) 当 cls['docstring'] 非空且为真值
时：对文档字符串中每一个由四个或更多字符组成、不是停用词，且与 signals['all_words'] 中的 token 匹配的字母词，加上固定的类
文档重叠权重。当 cls['docstring'] 为空或假值时，分量 (e) 贡献为零。当没有任何名称部分 token 及符合条件的文档字符串词
token 与任何指定的信号集重叠时，返回值恰好为零。",
    "actual_behavior": "该函数返回一个非负浮点得分，计算方式为：令 name = cls['name']; doc = cls['docstring'];
name_parts = _name_parts(name)。得分 = |name_parts signals['backtick_idsents']| * W_BACKTICK_NAME +
|name_parts signals['plain_idsents']| * W_CLASS_NAME_MATCH + |name_parts signals['all_words']| *
W_CLASS_NAME_MATCH + |name_parts signals['dotted_classes']| * W_DOTTED_REF + ( 如果 doc 非空 则 |{w | w
re.findall(r'\b([a-zA-Z]{4,})\b', doc.lower()) |} w _STOP} signals['all_words']| * W_CLASS_DOC_MATCH 否
则 0 )。输入 cls 和 signals 未被修改；不会抛出异常。",
    "code_evidence": "Line 15: re.findall(r'\b([a-zA-Z]{4,})\b', cls['docstring'].lower())",
    "trigger_condition": "该正则表达式将词提取限制为 ASCII 字母 [a-zA-Z]，但规范要求提取所有字母词（包括非 ASCII 字
母）。对于 docstring 'nave'，代码未找到匹配词并得分为 0，而规范将会给予 W_CLASS_DOC_MATCH 得分，因为 'nave' 是一个五字
符的字母词且出现在 signals['all_words'] 中。"
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--scope-py--_score_class",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/scope-
py/_score_class.py",
  "function_name": "_score_class",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--scope-py--_score_class.py",
  "detail_file": "fm_agent/bug_validation/src--scope-py--_score_class.md",
  "probe_stdout": "CONFIRMED - actual: 0.0 | expected: 2.0\n",
  "trigger_summary": "仅限于 ASCII 的正则表达式 [a-zA-Z] 无法匹配类文档字符串中的非 ASCII 字母词，导致 _score_class
遗漏符合条件的文档词与 signals['all_words'] 的重叠。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T05:13:15.432429Z	2193
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T05:13:37.303103Z	7998

事件 1 — llm_0d94d1ab4b01478d9143a0cf9738e02f

► parsed_json (完整)

```
{
  "post_condition": "该函数返回一个非负浮点得分，计算方式为：令 name = cls['name']; doc = cls['docstring']; name_parts = _name_parts(name)。得分 = |name_parts ∩ signals['backtick_idents']| * W_BACKTICK_NAME + |name_parts ∩ signals['plain_idents']| * W_CLASS_NAME_MATCH + |name_parts ∩ signals['all_words']| * W_CLASS_NAME_MATCH + |name_parts ∩ signals['dotted_classes']| * W_DOTTED_REF + ( 如果 doc ≠ '' 则 |{w | w ∈ re.findall(r'\\b([a-zA-Z]{4,})\\b', doc.lower()) ∧ w ∉ _STOP} ∩ signals['all_words']| * W_CLASS_DOC_MATCH 否则 0 )。输入 cls 和 signals 未被修改；不会抛出异常。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_0d94d1ab4b01478d9143a0cf9738e02f_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_0d94d1ab4b01478d9143a0cf9738e02f_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_0d94d1ab4b01478d9143a0cf9738e02f_response.txt)

事件 2 — llm_bb457c53155d4cdeb07208b28d856343

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "cls = {'name': 'MyClass', 'docstring': 'naïve'}; signals = {'backtick_idents': set(), 'plain_idents': set(), 'all_words': {'naïve'}, 'dotted_classes': set()}; 所有权重为正，例如 W_CLASS_DOC_MATCH = 1.0。",
  "offending_statements": "第 15 行: re.findall(r'\\b([a-zA-Z]{4,})\\b', cls['docstring'].lower())",
  "reason": "该正则表达式将词提取限制为 ASCII 字母 [a-zA-Z]，但规范要求提取所有字母词（包括非 ASCII 字母）。对于 docstring 'naïve'，代码未找到匹配词并得分为 0，而规范将会给予 W_CLASS_DOC_MATCH 得分，因为 'naïve' 是一个五字符的字母词且出现在 signals['all_words'] 中。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_bb457c53155d4cdeb07208b28d856343_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_bb457c53155d4cdeb07208b28d856343_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_bb457c53155d4cdeb07208b28d856343_response.txt) **复核动作**

- 先由维护者确认预期契约，再决定修改实现或重写 SPEC；当前不应直接登记为真实 Bug。

可能有 Bug（40 项）

Phase 1 — Initialization & Configuration

模块: configuration

- 四类结论：可能有 Bug
- 分类依据：probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- Validator: `confirmed`; attempts 1
- Phase / 模块: Phase 1 — Initialization & Configuration / `configuration`; 原源码 `src/configure_llm.py`
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** 当 `updates` 为空时引发 `ConfigWizardError`。当 `updates` 中的任意键值对验证失败时引发 `ConfigWizardError` (键不是可识别的 LLM 配置字段名, 或值不符合该字段的预期格式)。当 `text` 无法被标准库解析器解析为有效 TOML, 或 `text` 解码为非表的 TOML 值时引发 `ConfigWizardError`。否则返回一个包含有效 TOML 的字符串, 其中: 对于 `updates` 中的每个键, `[llm]` 部分下的对应字段设置为 `updates[key]`; 所有其他 TOML 部分、更新键以外的所有字段、所有注释, 以及更新字段值以外的所有空白, 均原样保留自 `text`。当 `text` 不包含 `[llm]` 部分时, 追加一个新的 `[llm]` 部分, 包含 `updates` 中的所有字段。
- **Actual behavior:** 函数 `update_llm_settings_toml_text` 要么返回一个修改后的 TOML 字符串, 该字符串将 `updates` 中所有期望的 LLM 设置集成到输入 `[llm]` 的 `text` 部分, 要么在前置条件被违反时引发错误。更精确地说:
 - 如果 `updates` 为假值 (例如空字典), 则引发 `ConfigWizardError('Provide at least one LLM setting to update.')`。
 - 如果 `updates` 中的任意 (key, value) 对在 `validate_llm_setting(key, value)` 中失败, 则该函数引发的错误会传播 (此错误表明 `key` 不可识别, 或 `value` 不满足字段的类型/格式约束)。
 - 如果 `text` 因任何原因无法解析为 TOML (即 `tomllib.loads(text)` 引发 `tomllib.TOMLDecodeError`), 则引发 `ConfigWizardError('Existing fm-agent.toml is invalid TOML; refusing to overwrite it.')`。
 - 如果 `text` 成功解码, 但结果是一个真值且不是字典 (例如字符串、整数、数组), 则引发 `ConfigWizardError('Existing fm-agent.toml must decode to a table/object.')`。

否则 (所有检查通过), 函数返回一个新字符串 `r`, 这是一个有效的 TOML 文档, 满足:

1. `r` 包含一个 `[llm]` 部分 (顶层表)。
2. 在该 `[llm]` 部分中, 对于 `k` 中的每个键 `updates`, 有且只有一行形如 `{k:<9>} = {_quote_toml_string(updates[k])}` 的键值行 (其中 `_quote_toml_string` 产生一个 TOML 有效的带引号字符串)。原始 `[llm]` 部分中该键的任何先前值被替换; 如果键原本不存在, 则追加到该部分的末尾。
3. `text` 的所有其他内容 (其他部分、注释、格式化、`[llm]` 以外的空行) 在 `r` 中保留, 除非原始 `text` 为空, 则 `r` 仅由 `[llm]\n` 后跟必要的键值行 (每行以换行符结束) 组成。
4. 当原始 `text` 包含一个 `[llm]` 部分时, `updates` 中未提及的任何现有 LLM 键在 `r` 中保持不变。

象征性地, 给定前置条件 $P(\text{text}, \text{updates})$: (`updates` 为空) 函数引发 `ConfigWizardError` 返回值未定义 (存在 $(k,v) \in \text{updates}$ 使得 `validate_llm_setting(k,v)` 失败) 函数引发错误 返回值未定义 (`tomllib.loads(text)` 失败或返回真值非字典) 函数引发 `ConfigWizardError` 返回值未定义 否则 函数返回 `r`, 其中 `r` 是满足上述 1-4 属性的字符串。

- **Code evidence:** Line 37: `kv_match = _KV_RE.match(line)` Line 38: `if kv_match and kv_match.group(2) in target:`
- **Trigger condition:** 代码使用正则表达式 (`_KV_RE`) 来识别键值行。当键被引用时 (例如 `"api_key"`), 正则表达式可能不匹配, 导致原始行被当作非键行处理。键无法被识别, 因此原始行被保留不变, 新的键值对随后被追加 (第 44-47 行)。输出包含重复键, 这是无效的 TOML, 并且未按规范要求替换字段。

- **Validator trigger:** 当输入的 TOML 使用带引号的键（例如 "name"）时，_KV_RE 正则表达式匹配失败，导致旧的键值行被保留，新的裸键行被追加，产生重复键。
- **Probe stdout:** CONFIRMED — duplicate "name" keys found in output (2 occurrences) Actual output:

```
'[llm]\n"name" = "old-model"\nprovider = "old-provider"\nname = "new-model"\n'
```

 Expected: single "name" key with value "new-model"

► SPEC sidecar（完整）

```
{
  "signature": "update_llm_settings_toml_text(text: str, updates: dict[str, str]) -> str",
  "pre_condition": "text 是一个字符串；updates 是一个字典，映射以点号分隔的 LLM 部分字段路径字符串到它们期望的字符串值。",
  "post_condition": "当 updates 为空时引发 ConfigWizardError。当 updates 中的任意键值对验证失败时引发 ConfigWizardError（键不是可识别的 LLM 配置字段名，或值不符合该字段的预期格式）。当 text 无法被标准库解析器解析为有效 TOML，或 text 解码为非表的 TOML 值时引发 ConfigWizardError。否则返回一个包含有效 TOML 的字符串，其中：对于 updates 中的每个键，[llm] 部分下的对应字段设置为 updates[key]；所有其他 TOML 部分、更新键以外的所有字段、所有注释，以及更新字段值以外的所有空白，均原样保留自 text。当 text 不包含 [llm] 部分时，追加一个新的 [llm] 部分，包含 updates 中的所有字段。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "validate_llm_setting",
      "signature": "validate_llm_setting(key: str, value: str) -> None",
      "pre_condition": "key 是一个字符串，标识一个 LLM 配置字段名；value 是一个字符串，表示该字段期望的新值。",
      "post_condition": "当 key 是可识别的 LLM 配置字段名，且 value 符合该字段预期的类型和格式时，无错误完成。当 key 不是可识别的字段名，或 value 不满足该字段的类型和格式约束时引发错误。"
    }
  ]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/configure_llm-py/update_llm_settings_toml_text.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "当 updates 为空时引发 ConfigWizardError。当 updates 中的任意键值对验证失败时引发 ConfigWizardError（键不是可识别的 LLM 配置字段名，或值不符合该字段的预期格式）。当 text 无法被标准库解析器解析为有效 TOML，或 text 解码为非表的 TOML 值时引发 ConfigWizardError。否则返回一个包含有效 TOML 的字符串，其中：对于 updates 中的每个键，[llm] 部分下的对应字段设置为 updates[key]；所有其他 TOML 部分、更新键以外的所有字段、所有注释，以及更新字段值以外的所有空白，均原样保留自 text。当 text 不包含 [llm] 部分时，追加一个新的 [llm] 部分，包含 updates 中的所有字段。",
    "actual_behavior": "函数 update_llm_settings_toml_text 要么返回一个修改后的 TOML 字符串，该字符串将`updates`中所有期望的 LLM 设置集成到输入`text`的`[llm]`部分，要么在前置条件被违反时引发错误。更精确地说：\n\n- 如果`updates`为假值（例如空字典），则引发 ConfigWizardError('Provide at least one LLM setting to update.').\n- 如果`updates`中的任意 (key, value) 对在`validate_llm_setting(key, value)`中失败，则该函数引发的错误会传播（此错误表明`key`不可识别，或`value`不满足字段的类型/格式约束）。\n- 如果`text`因任何原因无法解析为 TOML（即`tomllib.loads(text)`引发`tomllib.TOMLDecodeError`），则引发 ConfigWizardError('Existing fm-agent.toml is invalid TOML; refusing to overwrite it.').\n- 如果`text`成功解码，但结果是一个真值且不是字典（例如字符串、整数、数组），则引发 ConfigWizardError('Existing fm-agent.toml must decode to a table/object.').\n\n否则（所有检查通过），函数返回一个新字符串`r`，这是一个有效的 TOML 文档，满足：\n1. `r` 包含一个`[llm]`部分（顶层表）。\n2. 在该`[llm]`部分中，对于`updates`中的每个键`k`，有且只有一行形如`{k:<9} = {_quote_toml_string(updates[k])}`的键值行（其中`_quote_toml_string`产生一个 TOML 有效的带引号字符串）。原始`[llm]`部分中该键的任何先前值被替换；如果键原本不存在，则追加到该部分的末尾。\n3. `text`的所有其他内容（其他部分、注释、格式化、`[llm]`以外的空行）在`r`中保留，除非原始`text`为空，则`r`仅由`[llm]\n`后跟必要的键值行（每行以换行符结束）组成。\n4. 当原始`text`包含一个`[llm]`部分时，`updates`中未提及的任何现有 LLM 键在`r`中保持不变。
\n\n象征性地，给定前置条件 P(text, updates):\n（updates 为空） 函数引发 ConfigWizardError 返回值未定义\n（存在 (k,v) ∈ updates 使得 validate_llm_setting(k,v) 失败） 函数引发错误 返回未定义\n（tomllib.loads(text) 失败或返回真值非字典） 函数引发 ConfigWizardError 返回未定义\n否则 函数返回 r，其中 r 是满足上述 1-4 属性的字符串。",
  }
}
```

```
"code_evidence": "Line 37: kv_match = _KV_RE.match(line)\nLine 38: if kv_match and kv_match.group(2)\nin target:",\n  "trigger_condition": "代码使用正则表达式（_KV_RE）来识别键值行。当键被引用时（例如 \"api_key\"），正则表达式可能\n不匹配，导致原始行被当作非键行处理。键无法被识别，因此原始行被保留不变，新的键值对随后被追加（第 44-47 行）。输出包含重复键，\n这是无效的 TOML，并且未按规范要求替换字段。\"\n}\n}
```

► Bug Validator result（完整）

```
{\n  "id": "src--configure_llm-py--update_llm_settings_toml_text",\n  "source_file": "src/configure_llm.py",\n  "function_name": "update_llm_settings_toml_text",\n  "confirmation_status": "confirmed",\n  "attempts": 1,\n  "probe_script": "fm_agent/bug_validation/probe_src--configure_llm-py--update_llm_settings_toml_text.py",\n  "detail_file": "fm_agent/bug_validation/src--configure_llm-py--update_llm_settings_toml_text.md",\n  "probe_stdout": "CONFIRMED - duplicate \"name\" keys found in output (2 occurrences)\n  Actual output:\n '[llm]\\n\\n\"name\" = \"old-model\"\\n\\nprovider = \"old-provider\"\\n\\nname          = \"new-model\"\\n\\n'\n  Expected: single \"name\" key with value \"new-model\"\\n\\n",\n  "trigger_summary": "当输入的 TOML 使用带引号的键（例如 \"name\"）时，_KV_RE 正则表达式匹配失败，导致旧的键值行被保\n留，新的裸键行被追加，产生重复键。\"\n}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:09:27.922585Z	4158
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:10:14.358311Z	5243

事件 1 — [llm_0069fc6457f94443a0677b8f44de88e3](#)

► parsed_json（完整）

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_0069fc6457f94443a0677b8f44de88e3_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_0069fc6457f94443a0677b8f44de88e3_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_0069fc6457f94443a0677b8f44de88e3_response.txt)

事件 2 — [llm_52b8a00569bf4cc8a311e54afd8e485c](#)

► parsed_json（完整）

```
{\n  "verdict": "MISMATCH",\n  "counterexample": "text = '[llm]\\n\\n\"api_key\" = \"old_value\"\\n\\n', updates = {'api_key': 'new_value'}",\n  "offending_statements": "Line 37: kv_match = _KV_RE.match(line)\nLine 38: if kv_match and\nkv_match.group(2) in target:",\n  "reason": "代码使用正则表达式（_KV_RE）来识别键值行。当键被引用时（例如 \"api_key\"），正则可能无法匹配，导致原始行被\n当作非键行处理。该键未被识别，因此原始行保持不变，而新的键值对稍后（第 44-47 行）被追加。输出包含重复键，这是无效的 TOML，且\n未按规范要求替换字段。\"\n}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_52b8a00569bf4cc8a311e54afd8e485c_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_52b8a00569bf4cc8a311e54afd8e485c_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_52b8a00569bf4cc8a311e54afd8e485c_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块: environment

FMA-MISMATCH-038 — src--env_check-py--_check_oh_my_openagent

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块: Phase 1 — Initialization & Configuration / environment; 原源码 src/env_check.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** 当 oh-my-openagent 可通过 bunx 访问并在 10 秒内响应其 version 标志时, 返回 (True, None), 表示该工具已安装并可执行。当 oh-my-openagent 因任何原因无法通过 bunx 调用时 (包括 bunx 本身不可用、工具未安装或调用超过 10 秒), 返回 (False, "oh-my-openagent is not installed (bunx unavailable or timed out)")。该函数不产生任何副作用, 不会创建、修改或删除任何文件。
- **Actual behavior:** 函数调用后, 以下情形之一成立:
 1. 如果 subprocess 的导入失败, 则引发 ImportError (或其子类), 函数未返回。未产生任何元组。
 2. 否则, 函数返回一个 2 元组 (result, message), 其中:
 - 如果子进程运行完成未引发任何异常, 则 result 为 True, message 为 None。
 - 如果在 subprocess.run(...) 期间发生任何异常 (包括 CalledProcessError、FileNotFoundError、TimeoutExpired 等), 则 result 为 False, message 等于字符串 "oh-my-openagent is not installed (bunx unavailable or timed out)"。

形式化描述: 令 S 为调用后的状态, R 为返回值, E 为任何未捕获的异常。

- (E = none) (import subprocess succeeded)
- (E = none) (R is a 2-tuple) (R[0] {True, False})
- (E = none R[0] = True) (R[1] is None) (subprocess.run completed without exception)
- (E = none R[0] = False) (R[1] = "oh-my-openagent is not installed (bunx unavailable or timed out)") (subprocess.run raised an exception)
- 若 import subprocess 引发了异常, 则 E 为该异常, 且不存在返回值。
- **Code evidence:** Line 4: subprocess.run(...); Line 8: return True, None
- **Trigger condition:** 代码未检查子进程返回码或输出。非零退出状态 (例如 oh-my-openagent 缺失时) 导致函数返回 True, 违反了规范要求 (在调用工具失败时应返回 False)。
- **Validator trigger:** subprocess.run() 未使用 check=True, 在非零退出码时不会引发异常; 当 bunx 以非零退出时, 函数返回 (True, None) 而非 (False, error_message)。
- **Probe stdout:** CONFIRMED — Bug 已复现: 即使子进程以代码 1 退出, 函数仍然返回 (True, None)。规范要求无法调用工具时返回 (False, 'oh-my-openagent is not installed (bunx unavailable or timed out)').

► SPEC sidecar (完整)

```
{
  "signature": "_check_oh_my_openagent() -> tuple[bool, str | None]",
  "pre_condition": "不需要前置条件—该函数不接受任何参数，也没有外部状态依赖。",
  "post_condition": "当 oh-my-openagent 可通过 bunx 访问并在 10 秒内响应其 version 标志时，返回 (True, None)，表示该工具已安装并可执行。当 oh-my-openagent 因任何原因无法通过 bunx 调用时（包括 bunx 本身不可用、工具未安装或调用超过 10 秒），返回 (False, \"oh-my-openagent is not installed (bunx unavailable or timed out)\")。该函数不产生任何副作用—不会创建、修改或删除任何文件。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/env_check-py/_check_oh_my_openagent.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "当 oh-my-openagent 可通过 bunx 访问并在 10 秒内响应其 version 标志时，返回 (True, None)，表示该工具已安装并可执行。当 oh-my-openagent 因任何原因无法通过 bunx 调用时（包括 bunx 本身不可用、工具未安装或调用超过 10 秒），返回 (False, \"oh-my-openagent is not installed (bunx unavailable or timed out)\")。该函数不产生任何副作用，不会创建、修改或删除任何文件。",
    "actual_behavior": "函数调用后，以下情形之一成立：\n1. 如果 `subprocess` 的导入失败，则引发 `ImportError`（或其子类），函数未返回。未产生任何元组。\n2. 否则，函数返回一个 2 元组 `(result, message)`，其中：\n   - 如果子进程运行完成未引发任何异常，则 `result` 为 `True`，`message` 为 `None`。\n   - 如果在 `subprocess.run(...)` 期间发生任何异常（包括 `CalledProcessError`、`FileNotFoundError`、`TimeoutExpired` 等），则 `result` 为 `False`，`message` 等于字符串 `\"oh-my-openagent is not installed (bunx unavailable or timed out)\"`。\\n\\n形式化描述：令 `S` 为调用后的状态，`R` 为返回值，`E` 为任何未捕获的异常。\\n- `(E = none) (import subprocess succeeded)`\\n- `(E = none) (R is a 2-tuple) (R[0] {True, False})`\\n- `(E = none R[0] = True) (R[1] is None) (subprocess.run completed without exception)`\\n- `(E = none R[0] = False) (R[1] = \"oh-my-openagent is not installed (bunx unavailable or timed out)\") (subprocess.run raised an exception)`\\n- 若 `import subprocess` 引发了异常，则 `E` 为该异常，且不存在返回值。",
    "code_evidence": "Line 4: subprocess.run(...); Line 8: return True, None",
    "trigger_condition": "代码未检查子进程返回码或输出。非零退出状态（例如 oh-my-openagent 缺失时）导致函数返回 True，违反了规范要求（在调用工具失败时应返回 False）。"
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--env_check-py--_check_oh_my_openagent",
  "source_file": "src/env_check.py",
  "function_name": "_check_oh_my_openagent",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--env_check-py--_check_oh_my_openagent.py",
  "detail_file": "fm_agent/bug_validation/src--env_check-py--_check_oh_my_openagent.md",
  "probe_stdout": "CONFIRMED — Bug 已复现：即使子进程以代码 1 退出，函数仍然返回 (True, None)。规范要求无法调用工具时返回 (False, 'oh-my-openagent is not installed (bunx unavailable or timed out)').\\n",
  "trigger_summary": "subprocess.run() 未使用 check=True，在非零退出码时不会引发异常；当 bunx 以非零退出时，函数返回 (True, None) 而非 (False, error_message)。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T15:37:00.390795Z	1347
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T15:37:15.579456Z	3518

事件 1 — llm_a61c737b8b544a3fb490f9e744b6c302

► parsed_json (完整)

```
{
  "post_condition": "函数调用后，以下情形之一成立：\n1. 如果 `subprocess` 的导入失败，则引发 `ImportError`（或其子类），函数未返回。未产生任何元组。\n2. 否则，函数返回一个 2 元组 `(result, message)`，其中：\n    - 如果子进程运行完成未引发任何异常，则 `result` 为 `True`，`message` 为 `None`。\n    - 如果在 `subprocess.run(...)` 期间发生任何异常（包括 `CalledProcessError`、`FileNotFoundError`、`TimeoutExpired` 等），则 `result` 为 `False`，`message` 等于字符串 `\"oh-my-openagent is not installed (bunx unavailable or timed out)\"`。\\n\\n形式化描述：令 `S` 为调用后的状态，`R` 为返回值，`E` 为任何未捕获的异常。\\n- `(E = none) ↔ (import subprocess succeeded)`\\n- `(E = none) → (R is a 2-tuple) ∧ (R[0] ∈ {True, False})`\\n- `(E = none ∧ R[0] = True) → (R[1] is None) ∧ (subprocess.run completed without exception)`\\n- `(E = none ∧ R[0] = False) → (R[1] = \"oh-my-openagent is not installed (bunx unavailable or timed out)\") ∧ (subprocess.run raised an exception)`\\n- 若 `import subprocess` 引发了异常，则 `E` 为该异常，且不存在返回值。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_a61c737b8b544a3fb490f9e744b6c302_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_a61c737b8b544a3fb490f9e744b6c302_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_a61c737b8b544a3fb490f9e744b6c302_response.txt)

事件 2 — llm_ab7199e380844587bfbd9b2478895214

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "系统安装了 bunx，但未安装 oh-my-openagent。运行 'bunx oh-my-openagent --version' 退出码为 1 并输出错误信息，但未引发 Python 异常。函数返回 (True, None) 而非 (False, \"oh-my-openagent is not installed (bunx unavailable or timed out)\")。",
  "offending_statements": "Line 4: subprocess.run(...); Line 8: return True, None",
  "reason": "代码未检查子进程返回码或输出。非零退出状态（例如 oh-my-openagent 缺失时）导致函数返回 True，违反了规范要求（在调用工具失败时应返回 False）。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_ab7199e380844587bfbd9b2478895214_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_ab7199e380844587bfbd9b2478895214_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_ab7199e380844587bfbd9b2478895214_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

Phase 2 — Language Backend & Function Extraction

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 `fm_agent/bug_list.md` 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: `confirmed`; attempts 1
- Phase / 模块: Phase 2 — Language Backend & Function Extraction / `call_graph_edges`; 原源码 `src/call_graph_edges.py`
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** 当 `label` 包含至少一个 `:::`, 且最后一个 `:::` 之前的子串由类似 POSIX 路径的组件组成时, 返回一个规范的 FQN, 其中: (1) 路径部分中任何前导的 `./` 前缀被移除; (2) 路径最终文件名组件中最后一个 `.` 被替换为 `-`; (3) 所有父目录路径组件中, 排除 `.` 和空字符串后, 用 `:::` 连接形成 FQN 前缀; (4) 函数名附加以最后一个 `:::` 分隔的段。当 `label` 不匹配此模式时, 返回清洗后未作变更的 `label`。
- **Actual behavior:** 该函数返回一个新字符串, 且无副作用。设 `cl = _clean_label(label)` (去除首尾空白后的 `label`)。如果 `_is_path_function_label(cl)` 返回 `True`, 则令 `(path_str, func) = cl.rsplitt(':::', 1)`; 令 `stripped_path = path_str.lstrip('./')` (移除所有前导的 `.` 和 `/`); 令 `pp = PurePosixPath(stripped_path)`; 令 `base = pp.name`; 令 `last_dot = base.rfind('.')`; 如果 `last_dot > 0` 则令 `func_dir = (base[:last_dot] + '-' + base[last_dot+1:])`, 否则为 `base`; 令 `parts = [p for p in pp.parent.parts if p not in {', '.'}]`; 然后返回值为 `:::'.join(parts + [func_dir, func])`。否则, 返回值为 `cl`。形式上为: `result = ((lambda cl: (lambda path_str, func: (lambda stripped_path: (lambda pp: (lambda base: (lambda last_dot: (lambda func_dir: (lambda parts: ':::'.join(parts + [func_dir, func]))([p for p in pp.parent.parts if p not in {', '.'}]))((base[:last_dot] + '-' + base[last_dot+1:]) if last_dot > 0 else base)) (base.rfind('.')))(pp.name))(PurePosixPath(stripped_path))(path_str.lstrip('./')))(*cl.rsplitt(':::', 1)) if _is_path_function_label(cl) else cl)(_clean_label(label))`
- **Code evidence:** 第 5 行: `path = path.lstrip('./')`
- **Trigger condition:** 规范要求只移除前导的 `./` 前缀, 但 `lstrip('./')` 会移除所有前导的 `.` 和 `/` 字符, 因此 `../foo::func` 错误地变为 `foo::func`, 而非 `...:foo::func`。
- **Validator trigger:** `lstrip('./')` 会剥离所有前导的 `.` 和 `/` 字符, 因此 `../foo-c::func` 丢失了 `../` 父组件, 变为 `foo-c::func`, 而非 `...:foo-c::func`。
- **Probe stdout:** `CONFIRMED — actual: 'foo-c::func' | expected: '...:foo-c::func'`

► SPEC sidecar (完整)

```
{
  "signature": "_normalize_endpoint_label(label: str) -> str",
  "pre_condition": "label 是非空字符串。如果 label 表示一个文件路径限定的函数名, 路径分隔符已被规范化为 FQN 分隔符 ':::' (即 label 中不含 '/' 字符)。",
  "post_condition": "当 label 包含至少一个 ':::', 且最后一个 ':::' 之前的子串由类似 POSIX 路径的组件组成时, 返回一个规范的 FQN, 其中: (1) 路径部分中任何前导的 './' 前缀被移除; (2) 路径最终文件名组件中最后一个 '.' 被替换为 '-'; (3) 所有父目录路径组件中, 排除 '.' 和空字符串后, 用 ':::' 连接形成 FQN 前缀; (4) 函数名附加以最后一个 ':::' 分隔的段。当 label 不匹配此模式时, 返回清洗后未作变更的 label。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_clean_label",
      "signature": "_clean_label(label: str) -> str",

```

```

    "pre_condition": "label 是字符串。",
    "post_condition": "返回去除首尾空白后的 label。 "
  },
  {
    "name": "_is_path_function_label",
    "signature": "_is_path_function_label(label: str) -> bool",
    "pre_condition": "label 是已清洗的字符串。",
    "post_condition": "当 label 包含 '::' 且最后一个 '::' 之前的子串由一个或多个类似 POSIX 路径的组件组成时，返回 True；否则返回 False。 "
  }
]
}

```

► Logic verification result（完整）

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/call_graph_edges-
py/_normalize_endpoint_label.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "当 label 包含至少一个 '::', 且最后一个 '::' 之前的子串由类似 POSIX 路径的组件组成时，返回一个规范的 FQN，其中：(1) 路径部分中任何前导的 './' 前缀被移除；(2) 路径最终文件名组件中最后一个 '.' 被替换为 '-'；(3) 所有父目录路径组件中，排除 '.' 和空字符串后，用 '::' 连接形成 FQN 前缀；(4) 函数名附加以最后一个 '::' 分隔的段。当 label 不匹配此模式时，返回清洗后未作变更的 label。",
    "actual_behavior": "该函数返回一个新字符串，且无副作用。设 cl = _clean_label(label)（去除首尾空白后的 label）。如果 _is_path_function_label(cl) 返回 True，则令 (path_str, func) = cl.rsplitlet(':', 1)；令 stripped_path = path_str.lstrip('./')（移除所有前导的 '.' 和 '/'）；令 pp = PurePosixPath(stripped_path)；令 base = pp.name；令 last_dot = base.rfind('.')；如果 last_dot > 0 则令 func_dir = (base[:last_dot] + '-' + base[last_dot+1:])，否则为 base；令 parts = [p for p in pp.parent.parts if p not in {'', '.'}]; 然后返回值为 '::'.join(parts + [func_dir, func])。否则，返回值为 cl。形式上为：\nresult = ( (lambda cl: (lambda path_str, func: (lambda stripped_path: (lambda pp: (lambda base: (lambda last_dot: (lambda func_dir: (lambda parts: '::'.join(parts + [func_dir, func]))([p for p in pp.parent.parts if p not in {'', '.'}]))((base[:last_dot] + '-' + base[last_dot+1:]) if last_dot > 0 else base))(base.rfind('.')))(pp.name))(PurePosixPath(stripped_path))(path_str.lstrip('./')))(*cl.rsplitlet(':', 1)) if _is_path_function_label(cl) else cl )(_clean_label(label))),
    "code_evidence": "Line 5: path = path.lstrip(\"./\")",
    "trigger_condition": "规范要求只移除前导的 './' 前缀，但 lstrip('./') 会移除所有前导的 '.' 和 '/' 字符，因此 '../foo::func' 错误地变为 'foo::func'，而非 '...:foo::func'。"
  }
}

```

► Bug Validator result（完整）

```

{
  "id": "src--call_graph_edges-py--_normalize_endpoint_label",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/call_graph_edges-
py/_normalize_endpoint_label.py",
  "function_name": "_normalize_endpoint_label",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--call_graph_edges-py--_normalize_endpoint_label.py",
  "detail_file": "fm_agent/bug_validation/src--call_graph_edges-py--_normalize_endpoint_label.md",
  "probe_stdout": "CONFIRMED - actual: 'foo-c::func' | expected: '...:foo-c::func'",
  "trigger_summary": "lstrip(\"./\") 会剥离所有前导的 '.' 和 '/' 字符，因此 '../foo.c::func' 丢失了 '..' 父组件，变为 'foo-c::func'，而非 '...:foo-c::func'。"
}

```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:48:57.381680Z	3754
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:49:47.872550Z	4778

事件 1 — llm_4a39c0868104484da399a60a3113a519

► parsed_json (完整)

```
{
  "post_condition": "该函数返回一个新字符串，且无副作用。设 cl = _clean_label(label) (去除首尾空白后的 label)。如果 _is_path_function_label(cl) 返回 True，则令 (path_str, func) = cl.rsplit(':', 1); 令 stripped_path = path_str.lstrip('./') (移除所有前导的 '.' 和 '/')；令 pp = PurePosixPath(stripped_path); 令 base = pp.name; 令 last_dot = base.rfind('.'); 如果 last_dot > 0 则令 func_dir = (base[:last_dot] + '-' + base[last_dot+1:]), 否则为 base; 令 parts = [p for p in pp.parent.parts if p not in {'', '.'}]; 然后返回值为 ':'.join(parts + [func_dir, func])。否则，返回值为 cl。形式上为：\nresult = ( (lambda cl: (lambda path_str, func: (lambda stripped_path: (lambda pp: (lambda base: (lambda last_dot: (lambda func_dir: (lambda parts: ':'.join(parts + [func_dir, func]))([p for p in pp.parent.parts if p not in {'', '.'}]))((base[:last_dot] + '-' + base[last_dot+1:]) if last_dot > 0 else base))(base.rfind('.'))(pp.name))(PurePosixPath(stripped_path)))(path_str.lstrip('./')))(*cl.rsplit(':', 1)) if _is_path_function_label(cl) else cl )(_clean_label(label))"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_4a39c0868104484da399a60a3113a519_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_4a39c0868104484da399a60a3113a519_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_4a39c0868104484da399a60a3113a519_response.txt)

事件 2 — llm_4ccf150d496a4cc3b9477004fddae6f7

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "../foo::func",
  "offending_statements": "第 5 行: path = path.lstrip(\"./\")",
  "reason": "规范要求只移除前导的 './' 前缀，但 lstrip('./') 会移除所有前导的 '.' 和 '/' 字符，因此 '../foo::func' 错误地变为 'foo::func'，而非 '...:foo::func'。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_4ccf150d496a4cc3b9477004fddae6f7_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_4ccf150d496a4cc3b9477004fddae6f7_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_4ccf150d496a4cc3b9477004fddae6f7_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-058 — src--call_graph_edges-py--load_call_edges

- 四类结论：可能有 Bug
- 分类依据：probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- Validator: confirmed; attempts 1

- **Phase / 模块:** Phase 2 — Language Backend & Function Extraction / `call_graph_edges`; 原源码 `src/call_graph_edges.py`
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC 声明:** 当 `path` 为 `None` 时, 返回空列表。当 `path` 标识单个文件时, 返回从该文件解析出的 `CallEdge` 对象, 且所有重复边已被移除。当 `path` 标识一个目录时, 返回在该目录下递归找到的每一个符合条件的边文件的 `CallEdge` 对象, 合并为一个列表, 且所有重复边均被移除。在所有非 `None` 的情况下, 返回的每个元素都是 `CallEdge`, 且对于任意不同的 `caller` 和 `callee` 规范对, 最多出现一个 `CallEdge`。基于目录遍历得到的结果以与文件系统遍历顺序一致的稳定顺序返回。
- **实际行为:** 自然语言: 如果 `path` 为 `None`, 返回值为空列表。否则, 令 `p = Path(path)`。如果 `p` 是一个目录, 函数通过 `p.rglob('*')` 递归收集文件, 选出那些 `is_file()` 和 `_is_edge_file()` 返回 `True` 的文件, 按字典序排序, 依次通过 `CallEdge` 加载每个文件的 `_load_call_edge_file` 列表, 按该顺序拼接, 按 `caller/callee` 规范去重并保留首次出现, 最后返回结果列表。如果 `p` 是一个文件, 函数从该文件通过 `CallEdge` 加载 `_load_call_edge_file` 列表, 去重并返回结果。如果上述任一步骤抛出异常 (例如 `Path` 构造、文件系统遍历、文件读取或 JSON 解析), 该异常会传播出去, 且不返回值。

形式化逻辑: 设 `ret` 表示成功执行后的返回值。

- 若 `path is None`: `ret = []`。
- 否则定义 `p = Path(path)`。
 - 若 `p.is_dir()`: `files = [f for f in sorted(p.rglob('*')) if f.is_file() and _is_edge_file(f)]` `edges = concatenation of [(_load_call_edge_file(f)) for f in files]` `ret = dedupe(edges)`, 其中 `dedupe` 确保 `i < j`, (`ret[i].caller == ret[j].caller` `ret[i].callee == ret[j].callee`) 且每个元素出现在其在 `edges` 中最早出现的位置。
 - 若 `p.is_file()`: `ret = dedupe(_load_call_edge_file(p))`。若上述任一操作抛出异常 `E`, 函数以异常 `E` 退出, 且不产生 `ret`。
- **代码证据:** Line 8: `for file_path in sorted(edge_path.rglob("*"))`:
- **触发条件:** 规范要求结果顺序“与文件系统遍历顺序一致”, 即 `rglob` 产生条目的顺序。代码对 `rglob` 结果调用了 `sorted()`, 这强制施加了字母顺序, 可能与遍历顺序不同, 从而违反了规范。
- **Validator 触发器:** 代码对 `rglob` 结果调用 `sorted()`, 强制施加字母顺序, 而非规范要求的文件系统遍历顺序; 当两个拥有相同边键的文件以错误顺序处理时可观察到。
- **Probe 标准输出:** `rglob` 遍历顺序: `['z.json', 'a.json']` `sorted(rglob)` 顺序: `['a.json', 'z.json']` `load_call_edges` 合并边的来源: `'SOURCE_FROM_a'` `CONFIRMED` — `load_call_edges` 使用了排序后的顺序 (来源 `'=SOURCE_FROM_a'`) 而非遍历顺序 (期望 `'=SOURCE_FROM_z'`)

► SPEC sidecar (完整)

```
{
  "signature": "load_call_edges(path: str | os.PathLike | None) -> list[CallEdge]",
  "pre_condition": "path 为 None, 或标识一个包含 JSON 格式的补充调用边信息的文件, 或标识一个其递归内容中可能包含 JSON 调用边文件的目录。",
  "post_condition": "当 path 为 None 时, 返回空列表。当 path 标识单个文件时, 返回从该文件解析出的 CallEdge 对象, 且所有重复边已被移除。当 path 标识一个目录时, 返回在该目录下递归找到的每一个符合条件的边文件的 CallEdge 对象, 合并为一个列表, 且所有重复边均被移除。在所有非 None 的情况下, 返回的每个元素都是 CallEdge, 且对于任意不同的 caller 和 callee 规范对, 最多出现一个 CallEdge。基于目录遍历得到的结果以与文件系统遍历顺序一致的稳定顺序返回。"
}
```

► INFO / callees sidecar (完整)


```
{
  "callees": [
    {
      "name": "_is_edge_file",
      "signature": "_is_edge_file(path: Path) -> bool",
      "pre_condition": "path 指向磁盘上的一个文件。",
      "post_condition": "当文件被识别为包含补充调用边数据时返回 True；否则返回 False。"
    },
    {
      "name": "_load_call_edge_file",
      "signature": "_load_call_edge_file(path: Path) -> list[CallEdge]",
      "pre_condition": "path 指向一个包含有效补充调用边数据且符合预期 JSON schema 的文件。",
      "post_condition": "返回从文件中解析出的 CallEdge 对象列表，顺序与文件中出现的顺序一致。"
    },
    {
      "name": "_dedupe_edges",
      "signature": "_dedupe_edges(edges: list[CallEdge]) -> list[CallEdge]",
      "pre_condition": "edges 是 CallEdge 对象的列表，可能包含具有相同 caller 和 callee 规范的条目。",
      "post_condition": "返回一个 CallEdge 对象列表，其中任意两个条目不共享相同的 caller 规范和 callee 规范。首次出现的相对顺序得以保留。"
    }
  ]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/call_graph_edges-py/load_call_edges.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "当 path 为 None 时，返回空列表。当 path 标识单个文件时，返回从该文件解析出的 CallEdge 对象，且所有重复边已被移除。当 path 标识一个目录时，返回在该目录下递归找到的每一个符合条件的边文件的 CallEdge 对象，合并为一个列表，且所有重复边均被移除。在所有非 None 的情况下，返回的每个元素都是 CallEdge，且对于任意不同的 caller 和 callee 规范对，最多出现一个 CallEdge。基于目录遍历得到的结果以与文件系统遍历顺序一致的稳定顺序返回。",
    "actual_behavior": "自然语言：如果 `path` 为 `None`，返回值为空列表。否则，令 `p = Path(path)`。如果 `p` 是一个目录，函数通过 `p.rglob('*')` 递归收集文件，选出那些 `is_file()` 和 `_is_edge_file()` 返回 `True` 的文件，按字典序排序，依次通过 `_load_call_edge_file` 加载每个文件的 `CallEdge` 列表，按该顺序拼接，按 caller/callee 规范去重并保留首次出现，最后返回结果列表。如果 `p` 是一个文件，函数从该文件通过 `_load_call_edge_file` 加载 `CallEdge` 列表，去重并返回结果。如果上述任一步骤抛出异常（例如 `Path` 构造、文件系统遍历、文件读取或 JSON 解析），该异常会传播出去，且不返回值。\\n\\n形式化逻辑：\\n设 `ret` 表示成功执行后的返回值。\\n- 若 `path is None`：`ret = []`。\\n- 否则定义 `p = Path(path)`。\\n- 若 `p.is_dir()`：\\n    `files = [f for f in sorted(p.rglob('*')) if f.is_file() and _is_edge_file(f)]`\\n    `edges = 对 [(_load_call_edge_file(f)) for f in files] 的串联`\\n    `ret = dedupe(edges)`，其中 `dedupe` 确保 `i < j, (ret[i].caller == ret[j].caller and ret[i].callee == ret[j].callee)` 且每个元素出现在其在 `edges` 中最早出现的位置。\\n- 若 `p.is_file()`：\\n    `ret = dedupe(_load_call_edge_file(p))`。\\n若上述任一操作抛出异常 `E`，函数以异常 `E` 退出，且不产生 `ret`。",
    "code_evidence": "Line 8:         for file_path in sorted(edge_path.rglob(\"*\")): ",
    "trigger_condition": "规范要求结果顺序“与文件系统遍历顺序一致”，即 rglob 产生条目的顺序。代码对 rglob 结果调用了 sorted()，这强制施加了字母顺序，可能与遍历顺序不同，从而违反了规范。"
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--call_graph_edges-py--load_call_edges",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/call_graph_edges-py/load_call_edges.py",
  "function_name": "load_call_edges",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--call_graph_edges-py--load_call_edges.py",
  "detail_file": "fm_agent/bug_validation/src--call_graph_edges-py--load_call_edges.md",
  "probe_stdout": "rglob 遍历顺序： ['z.json', 'a.json']\\nsorted(rglob) 顺序： ['a.json', 'z.json']\\nload_call_edges 合并边的来源： 'SOURCE_FROM_a'\\nCONFIRMED - load_call_edges 使用了排序后的顺序（来源
```

```
= 'SOURCE_FROM_a') 而非遍历顺序 (期望='SOURCE_FROM_z') \n",
"trigger_summary": "代码对 rglob 结果调用 sorted(), 强制施加字母顺序, 而非规范要求的文件系统遍历顺序; 当两个拥有相同边键的文件以错误顺序处理时可观察到。"
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌, 使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:35:00.362325Z	1880
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:35:21.966251Z	4865

事件 1 — [llm_1a2626db48f7472281e7578a242bfc8b](#)

► parsed_json (完整)

```
{
  "post_condition": "自然语言: 如果 `path` 为 `None`, 返回值为空列表。否则, 令 `p = Path(path)`。如果 `p` 是一个目录, 函数通过 `p.rglob('*')` 递归收集文件, 选出那些 `is_file()` 和 `_is_edge_file()` 返回 `True` 的文件, 按字典序排序, 依次通过 `_load_call_edge_file` 加载每个文件的 `CallEdge` 列表, 按该顺序拼接, 按 caller/callee 规范去重并保留首次出现, 最后返回结果列表。如果 `p` 是一个文件, 函数从该文件通过 `_load_call_edge_file` 加载 `CallEdge` 列表, 去重并返回结果。如果上述任一步骤抛出异常 (例如 `Path` 构造、文件系统遍历、文件读取或 JSON 解析), 该异常会传播出去, 且不返回值。\\n\\n形式化逻辑: \\n设 `ret` 表示成功执行后的返回值。\\n- 若 `path is None`: `ret = []`。\\n- 否则定义 `p = Path(path)`。\\n- 若 `p.is_dir()`: \\n    `files = [f for f in sorted(p.rglob('*')) if f.is_file() and _is_edge_file(f)]`\\n    `edges = 对 [(_load_call_edge_file(f)) for f in files] 的串联`\\n    `ret = dedupe(edges)`, 其中 `dedupe` 确保 `\\forall i<j, \\neg(ret[i].caller == ret[j].caller \\wedge ret[i].callee == ret[j].callee)` 且每个元素出现在其在 `edges` 中最早出现的位置。\\n- 若 `p.is_file()`: \\n    `ret = dedupe(_load_call_edge_file(p))`。\\n若上述任一操作抛出异常 `E`, 函数以异常 `E` 退出, 且不产生 `ret`。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_1a2626db48f7472281e7578a242bfc8b_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_1a2626db48f7472281e7578a242bfc8b_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_1a2626db48f7472281e7578a242bfc8b_response.txt)

事件 2 — [llm_b9b8e8906e5a4e5fb67f20d4f1e8fbbb](#)

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "一个包含两个边文件的目录: 'b_edges.json' 和 'a_edges.json', 其中自然的文件系统顺序 (由 rglob 返回) 先将 'b_edges.json' 放在第一位 (例如因为它先被创建)。而代码中的排序将它们重排为 'a_edges.json' 然后 'b_edges.json', 违反了要求的顺序。",
  "offending_statements": "Line 8:         for file_path in sorted(edge_path.rglob(\"*\")): ",
  "reason": "规范要求结果顺序“与文件系统遍历顺序一致”, 即 rglob 产生条目的顺序。代码对 rglob 结果调用了 sorted(), 这强制施加了字母顺序, 可能与遍历顺序不同, 从而违反了规范。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_b9b8e8906e5a4e5fb67f20d4f1e8fbbb_message_00_system.txt) · [user_prompt]

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块：codegraph

FMA-MISMATCH-063 — [src--languages--codegraph-py--_bare_function_name](#)

- 四类结论： 可能有 Bug
- 分类依据： 沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator： [confirmed](#)；attempts 2
- Phase / 模块： Phase 2 — Language Backend & Function Extraction / [codegraph](#)；原源码 [src/languages/codegraph.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**： 返回去除了所有 tree-sitter 装饰的、未经限定的裸函数或方法名。结果为空值当且仅当输入为空字符串或纯空白。对于包含 'operator' 关键字的名称，结果为完整的 operator 说明符：关键字 'operator' 后跟运算符符号或关键字后缀，中间多余的空白被压缩。对于所有其他名称，结果是去除前缀装饰后、在任何尾随参数列表、模板体或类型注解之前提取的第一个字母或字母数字标识符令牌。返回的字符串不包含 '::' 或 '.' 限定符分隔符。如果无法从非空输入中提取任何标识符令牌，则原样返回去除空白后的输入。
- **Actual behavior**： 函数返回一个字符串 r。令 s = name.strip()。如果 s 为空，则 r = ""。否则，定义 tail = (如果 s 中包含 '::'，则取 s.rsplit(':', 1)[1].lstrip()，否则，如果 s 中包含 '.', 则取 s.rsplit('.', 1)[1].lstrip())，如果不包含 '.', 则 tail = s。如果 tail 以 'operator' 开头：令 rest = tail[8:].lstrip()；如果 rest 以 '[' 开头：r = 'operator['；否则如果 rest 以 ')' 开头：r = 'operator()'；否则如果 rest 恰为 'new' 或由 'new' 后跟可选空白、再跟 '['、可选空白、']' 且无其他内容构成，则 r = 'operator new['（如果 rest 中包含 '['）否则为 'operator new'；否则如果 rest 恰为 'delete' 或由 'delete' 后跟可选空白、再跟 '['、可选空白、']' 且无其他内容构成，则 r = 'operator delete['（如果 rest 中包含 '['）否则为 'operator delete'；否则：设 prefix 为 rest 中最长的初始子串，仅包含集合 {+, -, , /, %, &, |, ^, ~, !, =, <, >, ,} 中的字符；如果 prefix 非空，r = 'operator' + prefix。如果 r 仍未赋值，则检查 s：如果 s 以一个由单词字符（字母数字或下划线）构成的结尾序列，且该序列前紧邻字符串开头、 '::' 或 '.'，则 r 为该单词序列；否则如果 s 匹配模式 '(' 后跟可选空白、"、可选空白、一个单词、可选空白、')'，则 r 为该单词；否则如果 s 匹配模式 '*' 后跟可选空白、一个单词，则 r 为该单词；否则如果 s 以一个单词开头，则 r 为该单词；否则 r = s。
- **Code evidence**： Line 43: m = re.search(r'(?![.:])(w+)\$', name) Line 45: return m.group(1)
- **Trigger condition**： 对于输入 'func -> int'，代码返回 'int'（最后一个单词），但规范要求返回在尾随类型注解之前的第一个标识符令牌，即 'func'。
- **Validator trigger**： 输入 'foo::bar -> int'：限定符 '::' 被剥离得到 'bar'，但第 49 行的正则表达式在原始名称上搜索并失败，因此回退逻辑错误地返回 'foo'。
- **Probe stdout**： CONFIRMED — actual: 'foo' | expected: 'bar'

► SPEC sidecar（完整）

```
{
  "signature": "_bare_function_name(name: str) -> str",
  "pre_condition": "name 是一个可能包含 tree-sitter 装饰的字符串 — 包括限定名前缀（用 '::' 或 '.' 分隔）、指针/接收器语法、类型注解、参数列表、模板体以及完整的函数签名。",
  "post_condition": "返回去除了所有 tree-sitter 装饰的、未经限定的裸函数或方法名。结果为空值当且仅当输入为空字符串或纯空白。对于包含 'operator' 关键字的名称，结果为完整的 operator 说明符：关键字 'operator' 后跟运算符符号或关键字后缀，中间多余的空白被压缩。对于所有其他名称，结果是去除前缀装饰后、在任何尾随参数列表、模板体或类型注解之前提取的第一个字母或字母数字标识符令牌。返回的字符串不包含 '::' 或 '.' 限定符分隔符。如果无法从非空输入中提取任何标识符令牌，则原样返回去除空白后的输入。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-py/_bare_function_name.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回去除了所有 tree-sitter 装饰的、未经限定的裸函数或方法名。结果为空值当且仅当输入为空字符串或纯空白。对于包含 'operator' 关键字的名称，结果为完整的 operator 说明符：关键字 'operator' 后跟运算符符号或关键字后缀，中间多余的空白被压缩。对于所有其他名称，结果是去除前缀装饰后、在任何尾随参数列表、模板体或类型注解之前提取的第一个字母或字母数字标识符令牌。返回的字符串不包含 ':' 或 '.' 限定符分隔符。如果无法从非空输入中提取任何标识符令牌，则原样返回去除空白后的输入。",
    "actual_behavior": "函数返回一个字符串 r。令 s = name.strip()。如果 s 为空，则 r = ''。否则，定义 tail = (如果 s 中包含 '::'，则取 s.rsplit(':', 1)[1].lstrip()，否则，如果 s 中包含 '.'，则取 s.rsplit('.', 1)[1].lstrip())，如果不包含 '.'，则 tail = s。如果 tail 以 'operator' 开头：令 rest = tail[8:].lstrip()；如果 rest 以 '[' 开头：r = 'operator['；否则如果 rest 以 '(' 开头：r = 'operator('；否则如果 rest 恰为 'new' 或由 'new' 后跟可选空白、再跟 '['、可选空白、']' 且无其他内容构成，则 r = 'operator new[' (如果 rest 中包含 '[') 否则为 'operator new'；否则如果 rest 恰为 'delete' 或由 'delete' 后跟可选空白、再跟 '['、可选空白、']' 且无其他内容构成，则 r = 'operator delete[' (如果 rest 中包含 '[') 否则为 'operator delete'；否则：设 prefix 为 rest 中最长的初始子串，仅包含集合 {+, -, *, /, %, &, |, ^, ~, !, =, <, >, ,} 中的字符；如果 prefix 非空，r = 'operator' + prefix。如果 r 仍未赋值，则检查 s：如果 s 以一个由单词字符（字母数字或下划线）构成的结尾序列，且该序列前紧邻字符串开头、':' 或 '.'，则 r 为该单词序列；否则如果 s 匹配模式 '(' 后跟可选空白、'*'、可选空白、一个单词、可选空白、')'，则 r 为该单词；否则如果 s 匹配模式 '*' 后跟可选空白、一个单词，则 r 为该单词；否则如果 s 以一个单词开头，则 r 为该单词；否则 r = s。",
    "code_evidence": "Line 43: m = re.search(r'(?^[:|.|\\.|\\w+)$', name)\nLine 45: return m.group(1)",
    "trigger_condition": "对于输入 'func -> int'，代码返回 'int'（最后一个单词），但规范要求返回在尾随类型注解之前的第一个标识符令牌，即 'func'。"
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--languages--codegraph-py--_bare_function_name",
  "source_file": "src/languages/codegraph.py",
  "function_name": "_bare_function_name",
  "confirmation_status": "confirmed",
  "attempts": 2,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--codegraph-py--_bare_function_name.py",
  "detail_file": "fm_agent/bug_validation/src--languages--codegraph-py--_bare_function_name.md",
  "probe_stdout": "CONFIRMED - actual: 'foo' | expected: 'bar'\n",
  "trigger_summary": "输入 'foo::bar -> int'：限定符 '::' 被剥离得到 'bar'，但第 49 行的正则表达式在原始名称上搜索并失败，因此回退逻辑错误地返回 'foo'。"
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:40:14.002946Z	8126
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:41:59.630850Z	5440

事件 1 — `llm_75ba04ec2f57465d95353dc0da67795e`

► parsed_json (完整)

```
{
  "post_condition": "函数返回一个字符串 r。令 s = name.strip()。如果 s 为空，则 r = ''。否则，定义 tail = (如果 s 中包含 ':::', 则取 s.rsplit(':::', 1)[1].lstrip(), 否则，如果 s 中包含 '.', 则取 s.rsplit('.', 1)[1].lstrip()), 如果不包含 '.', 则 tail = s。如果 tail 以 'operator' 开头: 令 rest = tail[8:].lstrip(); 如果 rest 以 '[' 开头: r = 'operator['; 否则如果 rest 以 '()' 开头: r = 'operator()'; 否则如果 rest 恰为 'new' 或由 'new' 后跟可选空白、再跟 '['、可选空白、']' 且无其他内容构成, 则 r = 'operator new[' (如果 rest 中包含 '[') 否则为 'operator new'; 否则如果 rest 恰为 'delete' 或由 'delete' 后跟可选空白、再跟 '['、可选空白、']' 且无其他内容构成, 则 r = 'operator delete[' (如果 rest 中包含 '[') 否则为 'operator delete'; 否则: 设 prefix 为 rest 中最长的初始子串, 仅包含集合 {+, -, *, /, %, &, |, ^, ~, !, =, <, >, ,} 中的字符; 如果 prefix 非空, r = 'operator' + prefix。如果 r 仍未赋值, 则检查 s: 如果 s 以一个由单词字符 (字母数字或下划线) 构成的结尾序列, 且该序列前紧邻字符串开头、':' 或 '.', 则 r 为该单词序列; 否则如果 s 匹配模式 '(' 后跟可选空白、'*'、可选空白、一个单词、可选空白、')', 则 r 为该单词; 否则如果 s 匹配模式 '*' 后跟可选空白、一个单词, 则 r 为该单词; 否则如果 s 以一个单词开头, 则 r 为该单词; 否则 r = s。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_75ba04ec2f57465d95353dc0da67795e_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_75ba04ec2f57465d95353dc0da67795e_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_75ba04ec2f57465d95353dc0da67795e_response.txt)

事件 2 — `llm_60d15a7ae6ac4798b5037da4886789fb`

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "func -> int",
  "offending_statements": "Line 43: m = re.search(r'(?^[:|\\.|\\w+)$', name)\nLine 45: return m.group(1)",
  "reason": "对于输入 'func -> int', 代码返回 'int' (最后一个单词), 但规范要求返回在尾随类型注解之前的第一个标识符令牌, 即 'func'。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_60d15a7ae6ac4798b5037da4886789fb_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_60d15a7ae6ac4798b5037da4886789fb_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_60d15a7ae6ac4798b5037da4886789fb_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足; 若可满足, 将其转为回归测试并修实现。

FMA-MISMATCH-066 — `src--languages--codegraph-py--_fqn_for`

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 `fm_agent/bug_list.md` 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: `confirmed`; attempts 1
- Phase / 模块: Phase 2 — Language Backend & Function Extraction / `codegraph`; 原源码 `src/languages/codegraph.py`
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** 返回一个字符串, 由以下组成: (1) `file_path` 目录按 '/' 分割后得到的非空段, (2) `file_path` 的基名将其最后一个 '.' 替换为 '.' (当基名不包含 '.' 时保持不变), 以及 (3) `name`。所有部分由 ':::' 连接。返回的字符串至少

包含两个 ':' 分隔符，且其中没有任何部分为空。

- **实际行为：** 该函数返回一个字符串 FQN，通过规范化 `file_path`（将 `os.sep` 替换为 `'/'`），使用 `os.path.dirname` 和 `os.path.basename` 提取其目录和基名，然后将基名中的最后一个 '.'（如果有）替换为 '-' 以产生带横线的基名。最终结果为 `'::'.join([comp for comp in dirname(normalized).split('/') if comp] + [dashed_basename, name])`，其中 `normalized = file_path.replace(os.sep, '/')`、`dirname = os.path.dirname(normalized)`、`base = os.path.basename(normalized)`、`dashed_basename = base[:last_dot] + '-' + base[last_dot+1:] if (last_dot := base.rfind('.')) > 0 else base`。在给定的有效前置条件下，不会产生副作用，也不会引发异常。
- **代码证据：** Line 14: `dashed = base[:last_dot] + '-' + base[last_dot + 1:] if last_dot > 0 else base`
- **触发条件：** 规约要求在基名中将最后一个 '.' 替换为 '-'，无论该点的位置如何。而代码仅在 `last_dot > 0` 时才执行替换，排除了点位于第一个字符的情况（例如基名 `'hidden'`）。对于输入 `file_path='dir/.hidden'`，`name='func'`，代码返回 `'dir::hidden::func'`，而规约要求 `'dir::-hidden::func'`，这违反了规约。
- **Validator 触发：** 当基名以 '.' 开头时（例如 `'hidden'`），`last_dot=0` 未能通过 `> 0` 的守卫条件，因此点号未被替换为 '-'，未满足规约要求。
- **Probe stdout：** CONFIRMED — actual: `'dir::hidden::func'` | expected: `'dir::-hidden::func'`

► SPEC sidecar（完整）

```
{
  "signature": "_fqn_for(file_path: str, name: str) -> str",
  "pre_condition": "file_path 是一个非空相对源文件路径字符串，使用 '/' 分隔目录和基名部分。name 是一个非空的规范化函数标识符字符串。",
  "post_condition": "返回一个字符串，由以下组成：(1) file_path 目录按 '/' 分割后得到的非空段，(2) file_path 的基名将其最后一个 '.' 替换为 '-'（当基名不包含 '.' 时保持不变），以及 (3) name。所有部分由 '::' 连接。返回的字符串至少包含两个 '::' 分隔符，且其中没有任何部分为空。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-py/_fqn_for.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个字符串，由以下组成：(1) file_path 目录按 '/' 分割后得到的非空段，(2) file_path 的基名将其最后一个 '.' 替换为 '-'（当基名不包含 '.' 时保持不变），以及 (3) name。所有部分由 '::' 连接。返回的字符串至少包含两个 '::' 分隔符，且其中没有任何部分为空。",
    "actual_behavior": "该函数返回一个字符串 FQN，通过规范化 `file_path`（将 `os.sep` 替换为 '/'），使用 `os.path.dirname` 和 `os.path.basename` 提取其目录和基名，然后将基名中的最后一个 '.'（如果有）替换为 '-' 以产生带横线的基名。最终结果为 ``::'.join([comp for comp in dirname(normalized).split('/') if comp] + [dashed_basename, name])`，其中 `normalized = file_path.replace(os.sep, '/')`、`dirname = os.path.dirname(normalized)`、`base = os.path.basename(normalized)`、`dashed_basename = base[:last_dot] + '-' + base[last_dot+1:] if (last_dot := base.rfind('.')) > 0 else base`。在给定的有效前置条件下，不会产生副作用，也不会引发异常。",
    "code_evidence": "Line 14: dashed = base[:last_dot] + \"-\" + base[last_dot + 1:] if last_dot > 0 else base",
    "trigger_condition": "规约要求在基名中将最后一个 '.' 替换为 '-'，无论该点的位置如何。而代码仅在 last_dot > 0 时才执行替换，排除了点位于第一个字符的情况（例如基名 'hidden'）。对于输入 file_path='dir/.hidden'，name='func'，代码返回 'dir::hidden::func'，而规约要求 'dir::-hidden::func'，这违反了规约。"
  }
}
```

```
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--codegraph-py--_fqn_for",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/codegraph-py/_fqn_for.py",
  "function_name": "_fqn_for",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--codegraph-py--_fqn_for.py",
  "detail_file": "fm_agent/bug_validation/src--languages--codegraph-py--_fqn_for.md",
  "probe_stdout": "CONFIRMED - actual: 'dir::-hidden::func' | expected: 'dir::-hidden::func'",
  "trigger_summary": "当基名以 '.' 开头时（例如 '.hidden'），last_dot=0 未能通过 > 0 的守卫条件，因此点号未被替换为 '-', 未满足规约要求。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	目的	状态	尝试次数	块	开始时间	令牌数
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:27:42.645827Z	1924
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:28:05.721293Z	4705

事件 1 — [llm_393516c9e10e427a9f1c40794339f324](#)

► parsed_json（完整）

```
{
  "post_condition": "该函数返回一个字符串 FQN，通过规范化 `file_path`（将 `os.sep` 替换为 '/'），使用 `os.path.dirname` 和 `os.path.basename` 提取其目录和基名，然后将基名中的最后一个 '.'（如果有）替换为 '-' 以产生带横线的基名。最终结果为 `::'.join([comp for comp in dirname(normalized).split('/') if comp] + [dashed_basename, name])`，其中 `normalized = file_path.replace(os.sep, '/')`、`dirname = os.path.dirname(normalized)`、`base = os.path.basename(normalized)`、`dashed_basename = base[:last_dot] + '-' + base[last_dot+1:]` if `(last_dot := base.rfind('.')) > 0` else `base`。在给定的有效前置条件下，不会产生副作用，也不会引发异常。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_393516c9e10e427a9f1c40794339f324_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_393516c9e10e427a9f1c40794339f324_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_393516c9e10e427a9f1c40794339f324_response.txt)

事件 2 — [llm_4ab1d5ae30ee4617839830ca7e25c73c](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "file_path = 'dir/.hidden', name = 'func'",
  "offending_statements": "Line 14: dashed = base[:last_dot] + \"-\" + base[last_dot + 1:] if last_dot > 0 else base",
  "reason": "规约要求在基名中将最后一个 '.' 替换为 '-', 无论该点的位置如何。而代码仅在 last_dot > 0 时才执行替换，排除了点位于第一个字符的情况（例如基名 '.hidden'）。对于输入 file_path='dir/.hidden', name='func'，代码返回
```

```
'dir:::hidden::func', 而规约要求 'dir::-hidden::func', 这违反了规约。"  
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_4ab1d5ae30ee4617839830ca7e25c73c_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_4ab1d5ae30ee4617839830ca7e25c73c_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_4ab1d5ae30ee4617839830ca7e25c73c_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块: extraction

FMA-MISMATCH-071 — [src--extract-py--_extract_func_name_brace](#)

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: [confirmed](#); attempts 1
- Phase / 模块: Phase 2 — Language Backend & Function Extraction / [extraction](#); 原源码 [src/extract.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** 当 signature_text 包含可识别的函数声明时，返回裸函数名字字符串；否则返回 None。对于匹配运算符重载模式 ('operator' 后跟运算符标记或关键字 (例如 '[]'、'()'、'new[]'、算术/位运算符)，且紧邻 '(' 前) 的声明，返回完整的运算符说明符字符串。对于所有其他声明，在从 signature_text 中移除模板/泛型尖括号区域后，返回紧邻 '(' 前的第一个单词标记，且其文本不属于 lang_cfg[keywords]。当经过关键字过滤后不存在此类标记时返回 None。
- **Actual behavior:** 该函数返回表示提取出的函数名的字符串，若无法提取到名称则返回 None。具体而言：
 - 若 signature_text 包含与正则表达式 `\b(operator\s*(?:\[ \]| \(\)| \[+ \- \* / % & | ^ ~ ! = < > ] + | new(?: \s* \[ \s* \])? | delete(?: \s* \[ \s* \])?) ) \s* \((` 匹配的子串，则返回捕获组 (运算符关键字后跟其重载说明，例如 'operator+'、'operator()'、'operator new[]')。
 - 否则，令 cleaned_text 为 `_strip_angle_brackets(signature_text)` 的结果。对于 cleaned_text 中该正则表达式 `\b(\w+) \s* \((` 的每一个匹配 (按从左到右的顺序)，若捕获组 (括号前的标识符) 不属于 lang_cfg[keywords]，则返回该标识符。
 - 若找不到此类标识符，则返回 None。

形式化表示: 令 `op_pattern = \b(operator\s*(?:\[ \]| \(\)| \[+ \- \* / % & | ^ ~ ! = < > ] + | new(?: \s* \[ \s* \])? | delete(?: \s* \[ \s* \])?) ) \s* \((` 令 `id_pattern = \b(\w+) \s* \((` 令 `keywords = lang_cfg[keywords]` 令 `remove_brackets(s) = _strip_angle_brackets(s)`

若匹配 `m = search(op_pattern, signature_text)`，则返回 `m.group(1)`。否则令 `cleaned = remove_brackets(signature_text)`。若匹配 `m` 在 `finditer(id_pattern, cleaned)` 中，且 `m.group(1) ∉ keywords`，则返回第一个这样的 `m.group(1)`。否则返回 None。

- **Code evidence:** 第 4 行: `m = re.search(...)` 第 9 行: `if m:` 第 10 行: `return m.group(1)`
- **Trigger condition:** 代码在剥离尖括号之前，在整个 signature_text 中搜索运算符模式。这导致对模板参数列表中的 'operator+' 产生错误匹配，尽管整个声明并不是运算符重载。规范要求运算符重载模式仅适用于运算符声明；对所有其他声明，应在移除模板/泛型尖括号区域后再提取名称，在此示例中应返回 'bar'。
- **Validator trigger:** 括号内的 angle-bracket 模板参数中的 `operator()`，在尖括号被剥离之前就被 `re.search` 匹配到，导致错误的运算符重载检测。
- **Probe stdout:** CONFIRMED — actual: 'operator()' | expected: 'bar'

► SPEC sidecar（完整）

```
{
  "signature": "_extract_func_name_brace(signature_text: str, lang_cfg: dict) -> str | None",
  "pre_condition": "signature_text 是一个字符串，包含来自括号分隔语言的候选函数签名，可能跨越多行的连接行。lang_cfg 是一个字典，其 'keywords' 键映射到一个语言保留字的集合，必须从返回的函数名中排除这些保留字。",
  "post_condition": "当 signature_text 包含可识别的函数声明时，返回裸函数名字符串；否则返回 None。对于匹配运算符重载模式（'operator' 后跟运算符标记或关键字（例如 '['、'()'、'new[]'、算术/位运算符），且紧邻 '(' 前）的声明，返回完整的运算符说明符字符串。对于所有其他声明，在从 signature_text 中移除模板/泛型尖括号区域后，返回紧邻 '(' 前的第一个单词标记，且其文本不属于 lang_cfg['keywords']。当经过关键字过滤后不存在此类标记时返回 None。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "_strip_angle_brackets",
      "signature": "_strip_angle_brackets(text: str) -> str",
      "pre_condition": "text 是一个字符串，可能包含由尖括号界定的区域，如模板或泛型参数列表。",
      "post_condition": "返回移除了每个最外层平衡的 '<...>' 区域的文本：界定尖括号及其间的所有内容均被剥离。嵌套会被解析——在外部 '<' 与其匹配的 '>' 之间出现的 '<' 属于内部区域，不会终止外部区域。任何平衡 '<...>' 区域之外的所有字符按其原始相对顺序保留。"
    }
  ]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/extract-py/_extract_func_name_brace.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "当 signature_text 包含可识别的函数声明时，返回裸函数名字符串；否则返回 None。对于匹配运算符重载模式（'operator' 后跟运算符标记或关键字（例如 '['、'()'、'new[]'、算术/位运算符），且紧邻 '(' 前）的声明，返回完整的运算符说明符字符串。对于所有其他声明，在从 signature_text 中移除模板/泛型尖括号区域后，返回紧邻 '(' 前的第一个单词标记，且其文本不属于 lang_cfg['keywords']。当经过关键字过滤后不存在此类标记时返回 None。",
    "actual_behavior": "该函数返回表示提取出的函数名的字符串，若无法提取到名称则返回 None。具体而言：\n- 若 signature_text 包含与正则表达式 `\\b(operator\\s*(?:\\[[\\]\\|\\(\\)|[+\\-*/%&|^!=<>]+|new(?:\\s*\\[[\\s*\\])?|delete(?:\\s*\\[[\\s*\\])?))\\s*\\(\\` 匹配的子串，则返回捕获组（运算符关键字后跟其重载说明，例如 'operator+'、'operator()'、'operator new[]'）。\n- 否则，令 cleaned_text 为 _strip_angle_brackets(signature_text) 的结果。对于 cleaned_text 中该正则表达式 `\\b(\\w+)\\s*\\(\\` 的每一个匹配（按从左到右的顺序），若捕获组（括号前的标识符）不属于 lang_cfg['keywords']，则返回该标识符。\n- 若找不到此类标识符，则返回 None。\\n\\n形式化表示：\n令 op_pattern = /\\b(operator\\s*(?:\\[[\\]\\|\\(\\)|[+\\-*/%&|^!=<>]+|new(?:\\s*\\[[\\s*\\])?|delete(?:\\s*\\[[\\s*\\])?))\\s*\\(\\n令 id_pattern = /\\b(\\w+)\\s*\\(\\n令 keywords = lang_cfg['keywords']\n令 remove_brackets(s) = _strip_angle_brackets(s)\n\\n\\n若 匹配 m = search(op_pattern, signature_text)，则返回 m.group(1)。\n否则令 cleaned = remove_brackets(signature_text)。\n若 匹配 m 在 finditer(id_pattern, cleaned) 中，且 m.group(1) ∉ keywords，则返回第一个这样的 m.group(1)。\n否则返回 None。",
    "code_evidence": "Line 4: m = re.search(...)\nLine 9: if m:\nLine 10: return m.group(1)",
    "trigger_condition": "代码在剥离尖括号之前，在整个 signature_text 中搜索运算符模式。这导致对模板参数列表中的 'operator+' 产生错误匹配，尽管整个声明并不是运算符重载。规范要求运算符重载模式仅适用于运算符声明；对所有其他声明，应在移除模板/泛型尖括号区域后再提取名称，在此示例中应返回 'bar'。"
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--extract-py--_extract_func_name_brace",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/extract-
```

```
py/_extract_func_name_brace.py",
"function_name": "_extract_func_name_brace",
"confirmation_status": "confirmed",
"attempts": "1",
"probe_script": "fm_agent/bug_validation/probe_src--extract-py--_extract_func_name_brace.py",
"detail_file": "fm_agent/bug_validation/src--extract-py--_extract_func_name_brace.md",
"probe_stdout": "CONFIRMED - actual: 'operator()' | expected: 'bar'\n",
"trigger_summary": "括号内的 angle-bracket 模板参数中的 operator(), 在尖括号被剥离之前就被 re.search 匹配到, 导致错误的运算符重载检测。"
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌, 使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:27:40.136546Z	2345
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:28:06.608460Z	9948

事件 1 — [llm_f63868ecf26f4c5c86a7d24faebfce0e](#)

► parsed_json (完整)

```
{
  "post_condition": "该函数返回表示提取出的函数名的字符串, 若无法提取到名称则返回 None。具体而言:\n- 若 signature_text 包含与正则表达式 '\\b(operator\\s*(?:\\[\\]|\\(\\)|[+\\-*/%&|^!=<>]+|new(?:\\s*\\[\\s*\\])?|delete(?:\\s*\\[\\s*\\])?)\\s*\\(\\` 匹配的子串, 则返回捕获组 (运算符关键字后跟其重载说明, 例如 'operator+', 'operator()', 'operator new[]')。\\n- 否则, 令 cleaned_text 为 _strip_angle_brackets(signature_text) 的结果。对于 cleaned_text 中该正则表达式 '\\b(\\w+)\\s*\\(\\` 的每一个匹配 (按从左到右的顺序), 若捕获组 (括号前的标识符) 不属于 lang_cfg['keywords'], 则返回该标识符。\\n- 若找不到此类标识符, 则返回 None。\\n\\n形式化表示:\n令 op_pattern = /\\b(operator\\s*(?:\\[\\]|\\(\\)|[+\\-*/%&|^!=<>]+|new(?:\\s*\\[\\s*\\])?|delete(?:\\s*\\[\\s*\\])?)\\s*\\(\\n令 id_pattern = /\\b(\\w+)\\s*\\(\\n令 keywords = lang_cfg['keywords']\n令 remove_brackets(s) = _strip_angle_brackets(s)\n\\n\\n若 存在 匹配 m = search(op_pattern, signature_text), 则返回 m.group(1)。\\n否则令 cleaned = remove_brackets(signature_text)。\\n若 存在 匹配 m 在 finditer(id_pattern, cleaned) 中, 且 m.group(1) \u2208 keywords, 则返回第一个这样的 m.group(1)。\\n否则返回 None。"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_f63868ecf26f4c5c86a7d24faebfce0e_message_00_system.txt) · [user_prompt](fm_agent/trace/payloads/llm_f63868ecf26f4c5c86a7d24faebfce0e_message_01_user.txt) · [assistant_output](fm_agent/trace/payloads/llm_f63868ecf26f4c5c86a7d24faebfce0e_response.txt)

事件 2 — [llm_008aa78483d5445f9caf4c250f580b2d](#)

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "输入: signature_text = \"template<class T, T operator+(T,T)> void bar(int)\\n\", lang_cfg = {\"keywords\": [\"void\\n\", \"int\\n\", \"template\\n\"]}。实际输出: \"operator+\\n\"。根据规范预期输出: \"bar\\n\"。",
  "offending_statements": "第 4 行: m = re.search(...)\\n第 9 行: if m:\\n第 10 行: return m.group(1)",
  "reason": "代码在剥离尖括号之前, 在整个 signature_text 中搜索运算符模式。这导致对模板参数列表中的 'operator+' 产生错误匹配, 尽管整个声明并不是运算符重载。规范要求运算符重载模式仅适用于运算符声明; 对所有其他声明, 应在移除模板/泛型尖括号区域后再提取名称, 在此示例中应返回 'bar'。"
}
```


- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_008aa78483d5445f9caf4c250f580b2d_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_008aa78483d5445f9caf4c250f580b2d_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_008aa78483d5445f9caf4c250f580b2d_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-073 — src--extract-py--_extract_functions_indent

- 四类结论：可能有 Bug
- 分类依据：沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块：Phase 2 — Language Backend & Function Extraction / extraction; 原源码 src/extract.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim**: 返回一个由 (name, start_idx, end_idx) 组成的元组列表，每个元组对应 lines 中发现的一个顶层函数定义。start_idx 和 end_idx 是从 0 开始的包含性的行索引。从 start_idx 到 end_idx 的区间覆盖整个函数：包括任何前置装饰器行、函数定义头（可能跨越多行）以及函数体，但不包括紧跟在最后一个非空函数体行之后的尾部空行。name 是定义行上紧跟在函数定义关键字之后的标识符 token。函数按照在 lines 中出现的顺序返回。当未找到任何函数定义时返回空列表。
- **Actual behavior**: 该函数返回一个由 functions 元组 (name, start, end) 组成的列表，其中每个元组对应 lines 中发现的一个顶层函数定义。函数定义是匹配模式 def name(的行（允许任意缩进）。start 是函数的第一行的索引，包括任何前置装饰器行（以 '@' 开头的行）。end 是函数体的最后一个非空行的索引。函数体由定义行之后缩进大于该定义行缩进的行、空行以及恰好与定义行缩进相同但以右括号开头并后跟 ':' 或 '->' 的行（多行签名的续行）组成。函数体刚好在第一个非空、缩进小于或等于定义行缩进且不是签名续行的行之前结束。函数按照其 def 行出现的顺序提取，跳过任何位于先前提取的函数体内的 def 行（即仅提取最外层函数）。输入 lines 和 lang_cfg 不会被修改。

正式地，令 $n = \text{len}(\text{lines})$ 并定义：

- `is_blank(l)` `l.strip() == ''`
- `is_decorator(l)` `l.strip().startswith('@')`
- `is_func_def(l)` `re.match(r'^(\s*)def\s+(\w+)\s*(\(', l)` is not None; 若为真，则令 `name(l)` 为捕获的名称，`indent(l) = len(leading_whitespace)`。
- `is_sig_cont(l, ind)` `len(l) - len(l.lstrip()) == ind` 和 `re.match(r'\)\s*(:|->)', l.lstrip())` is not None。

那么 $\text{functions} = [(N_0, S_0, E_0), \dots, (N_{m-1}, S_{m-1}, E_{m-1})]$ 对于某些 $m \geq 0$ ，其 def 位置 $D_0 < D_1 < \dots < D_{m-1}$ 严格递增，且满足：

1. `is_func_def(lines[D_k])` 成立， $N_k = \text{name}(\text{lines}[D_k])$ ， $I_k = \text{indent}(\text{lines}[D_k])$ 。
2. S_k 是满足 `s  $\geq D_k$` 的最小索引，其中 `[s, D_k-1]` 中的每一行都是装饰器，且 `s=0` 或 `lines[s-1]` 不是装饰器。
3. 令 B_k 为满足 `b  $\geq D_{k+1}$` 或 (`b == n` 且 `not is_blank(lines[b])` 且 `indent(lines[b])  $\geq I_k$`) 的最小索引 `not is_sig_cont(lines[b], I_k)`。则 E_k 是满足 `e  $\geq D_k$` 且 `e < B_k` 或 `e == D_k` 的最大索引 `not is_blank(lines[e])`。
4. 对于每个 k ， $D_{k+1} > E_k$ （若 $k < m-1$ ），并且对于任何不在 d 中的索引 `is_func_def(lines[d])` 且 $\{D_k\}$ ，则 d 位于某个区间 $[S_k, E_k]$ 中，或者 d 会产生一个 B_k 但该情况因扫描顺序被排除。
5. `functions` 中不存在其他元组。

- **Code evidence:** Line 28: `if line_indent == indent and re.match(r'\s*(:|->)', l.lstrip()):`
- **Trigger condition:** 该代码仅在缩进与定义行相同且以 ')' 开头后跟 ':' 或 '->' 的行时识别多行函数签名。当位于同一缩进的续行不以 ')' 开头时，例如参数行或不带 ')' 前缀的闭合行，代码会错误地将其视为函数结束，从而截断区间。在反例中，行 'b):' 的缩进为 0，与 `def` 的缩进相同，但不匹配签名续行的正则表达式，因此内部循环中断，导致区间为 [0,0] 而不是覆盖整个函数。
- **Validator trigger:** 不以 ')' 开头的多行函数签名续行会导致在 `src/extract.py` 的第 562 行处过早截断函数的区间。
- **Probe stdout:** CONFIRMED — function body truncated. Extracted source: 'def foo(a,\n'

► SPEC sidecar (完整)

```
{
  "signature": "_extract_functions_indent(lines: list[str], lang_cfg: dict) -> list[tuple[str, int, int]]",
  "pre_condition": "lines 是一个字符串列表，表示一个以缩进定界的语言的源代码行，每行具有规范化的行尾。lang_cfg 是一个语言配置字典，适用于 body 类型为 'indent' 的语言。",
  "post_condition": "返回一个由 (name, start_idx, end_idx) 组成的元组列表，每个元组对应 lines 中发现的一个顶层函数定义。start_idx 和 end_idx 是从 0 开始的包含性的行索引。从 start_idx 到 end_idx 的区间覆盖整个函数：包括任何前置装饰器行、函数定义头（可能跨越多行）以及函数体——但不包括紧跟在最后一个非空函数体行之后的尾部空行。name 是定义行上紧跟在函数定义关键字之后的标识符 token。函数按照在 lines 中出现的顺序返回。当未找到任何函数定义时返回空列表。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/extract-py/_extract_functions_indent.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个由 (name, start_idx, end_idx) 组成的元组列表，每个元组对应 lines 中发现的一个顶层函数定义。start_idx 和 end_idx 是从 0 开始的包含性的行索引。从 start_idx 到 end_idx 的区间覆盖整个函数：包括任何前置装饰器行、函数定义头（可能跨越多行）以及函数体，但不包括紧跟在最后一个非空函数体行之后的尾部空行。name 是定义行上紧跟在函数定义关键字之后的标识符 token。函数按照在 lines 中出现的顺序返回。当未找到任何函数定义时返回空列表。",
    "actual_behavior": "该函数返回一个由 `functions` 元组 `(name, start, end)` 组成的列表，其中每个元组对应 `lines` 中发现的一个顶层函数定义。函数定义是匹配模式 `def name(` 的行（允许任意缩进）。`start` 是函数的第一行的索引，包括任何前置装饰器行（以 `@` 开头的行）。`end` 是函数体的最后一个非空行的索引。函数体由定义行之后缩进大于该定义行缩进的行、空行以及恰好与定义行缩进相同但以右括号开头并后跟 `:` 或 `->` 的行（多行签名的续行）组成。函数体刚好在第一个非空、缩进小于或等于定义行缩进且不是签名续行的行之前结束。函数按照其 `def` 行出现的顺序提取，跳过任何位于先前提取的函数体内的 `def` 行（即仅提取最外层函数）。输入 `lines` 和 `lang_cfg` 不会被修改。\\n\\n正式地，令 `n = len(lines)` 并定义：\\n- `is_blank(l) l.strip() == ''`\\n- `is_decorator(l) l.strip().startswith('@')`\\n- `is_func_def(l) re.match(r'^((\\s*)def\\s+(\\w+))\\s*\\(', l) is not None`；若为真，则令 `name(l)` 为捕获的名称，`indent(l) = len(leading_whitespace)`。\\n- `is_sig_cont(l, ind) len(l) - len(l.strip()) == ind` 和 `re.match(r'\\s*(:|->)', l.lstrip()) is not None`。\\n\\n那么 `functions` = `[(N_0, S_0, E_0), ..., (N_{m-1}, S_{m-1}, E_{m-1})]` 对于某些 `m >= 0`，其 `def` 位置 `D_0 < D_1 < ... < D_{m-1}` 严格递增，且满足：\\n1. `is_func_def(lines[D_k])` 成立，`N_k = name(lines[D_k])`，`I_k = indent(lines[D_k])`。\\n2. `S_k` 是满足 `s <= D_k` 的最小索引，其中 `[s, D_k-1]` 中的每一行都是装饰器，且 `s=0` 或 `lines[s-1]` 不是装饰器。\\n3. 令 `B_k` 为满足 `b == n` 或 `(not is_blank(lines[b]) & I_k)` 且 `not is_sig_cont(lines[b], I_k)` 的最小索引 `b < D_{k+1}`。则 `E_k` 是满足 `D_k <= b < B_k` 且 `b == D_k` 或 `not is_blank(lines[b])` 的最大索引 `b`。\\n4. 对于每个 `k`，`D_{k+1} > E_k`（若 `k < m-1`），并且对于任何不在 `[D_k]` 中的索引 `d` 且 `is_func_def(lines[d])`，则 `d` 位于某个区间 `[S_k, E_k]` 中，或者 `d` 会产生一个 `B_k < d` 但该情况因扫描顺序被排除。\\n5. `functions` 中不存在其他元组。",
    "code_evidence": "Line 28: if line_indent == indent and re.match(r'\\s*(:|->)', l.lstrip()):",
    "trigger_condition": "该代码仅在缩进与定义行相同且以 ')' 开头后跟 ':' 或 '->' 的行时识别多行函数签名。当位于同一缩进的续行不以 ')' 开头时，例如参数行或不带 ')' 前缀的闭合行，代码会错误地将其视为函数结束，从而截断区间。在反例中，行 'b):' 的缩进为 0，与 def 的缩进相同，但不匹配签名续行的正则表达式，因此内部循环中断，导致区间为 [0,0] 而不是覆盖整个函数。"
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--extract-py--_extract_functions_indent",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/src/extract-py/_extract_functions_indent.py",
  "function_name": "_extract_functions_indent",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--extract-py--_extract_functions_indent.py",
  "detail_file": "fm_agent/bug_validation/src--extract-py--_extract_functions_indent.md",
  "probe_stdout": "CONFIRMED - function body truncated. Extracted source: 'def foo(a,\n",
  "trigger_summary": "不以 ')' 开头的多行函数签名续行会导致在 src/extract.py 的第 562 行处过早截断函数的区间。"
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:10:45.335775Z	4892
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:11:46.016526Z	6502

事件 1 — [llm_3e27da8d4e9d46a9a93e0233410593d8](#)

► parsed_json (完整)

- 载荷: [system_prompt]

```
(fm_agent/trace/payloads/llm_3e27da8d4e9d46a9a93e0233410593d8_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_3e27da8d4e9d46a9a93e0233410593d8_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_3e27da8d4e9d46a9a93e0233410593d8_response.txt)
```

事件 2 — llm_376d274c0a68424795c32186864702ef

► parsed json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "lines = [\"def foo(a,\\\", \\\"b):\\\", \\\"    return a + b\\\"]\",
  "offending_statements": "Line 28: if line_indent == indent and re.match(r'\\\\\\\\\\\\s*(:|->)', l.lstrip()):",
  "reason": "代码仅在缩进与定义相同的行以 ')' 开头后跟 ':' 或 '->' 时,才将其识别为多行函数签名。当具有相同缩进的续行不以 ')' 开头时,例如参数行或不带 ')' 前缀的闭括号行,代码会错误地将其视为函数结束,从而截断了函数的范围。在此反例中,行 'b):' 的缩进为 0,与 def 的缩进相同,但不匹配签名续行的正则表达式,因此内层循环终止,得到范围 [0,0],而非覆盖整个函数。"
}
```

- 载荷: [system_prompt]

```
(fm_agent/trace/payloads/llm_376d274c0a68424795c32186864702ef_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_376d274c0a68424795c32186864702ef_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_376d274c0a68424795c32186864702ef_response.txt) **复核动作**
```

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

- 四类结论：可能有 Bug
- 分类依据：沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论。
- Validator： [confirmed](#)；attempts [1](#)
- Phase / 模块： 第 2 阶段 — 语言后端与函数提取 / [extraction](#)；原源码 [src/extract.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**： 返回一个由 `text` 形成的字符串，该字符串移除了每一个最外层的、平衡的 `<...>` 区域：分隔符 `<` 和 `>` 字符及其之间的所有内容均被剥离。嵌套解析方式为——在活动的 `<...>` 区域内遇到的 `<` 属于内部区域，不会终止外部区域。不属于任何平衡 `<...>` 区域的字符将按其原始相对顺序保留。未闭合任何活动 `<...>` 区域的 `>` 字符将从结果中省略。
- **Actual behavior**： 该函数返回一个新的字符串，该字符串省略了所有出现的 `<` 和 `>`，并且也省略了出现在平衡尖括号对内的所有字符（遵循标准嵌套规则）。形式上，为索引 `i` ($0 \leq i \leq \text{len}(\text{text})$) 定义字符前深度 `d(i)`：`d(0)=0`，且对于 $i \geq 0$ ：若 `text[i]='<'`，则 `d(i+1) = d(i)+1`；若 `text[i]='>'`，则 `d(i+1) = max(0, d(i)-1)`；否则 `d(i+1) = d(i)`。然后输出字符串是所有满足 `text[i] ∈ {'<','>'}` 且 `d(i)=0` 的字符 `text[i]` 的连接。未匹配的 `<` 导致该字符之后直到字符串末尾（或直到遇到匹配的 `>`）的所有字符被视为处于区域内，从而被移除；深度为零时的任何未匹配 `>` 也会被移除。
- **Code evidence**： Line 6: `if ch == '<':` Line 7: `depth += 1`
- **Trigger condition**： 该代码无条件移除 `<` 字符，即使它们未匹配且不属于任何平衡区域。规约（Condition B）要求保留未匹配的 `<`，因为只应移除最外层的平衡 `<...>` 区域（以及未匹配的 `>`）。对于输入 `<`，代码返回 `"`，但规约要求返回 `<`。
- **Validator trigger**： 输入 `<`（未匹配的尖括号）返回空字符串而非 `<` — 该 `<` 字符在未平衡时被无条件丢弃，而非保留。
- **Probe stdout**： `CONFIRMED — actual: " | expected: '<'`

► SPEC sidecar（完整）

```
{
  "signature": "_strip_angle_brackets(text) -> str",
  "pre_condition": "text 是一个字符串，可能包含尖括号分隔的区域，例如模板或泛型参数列表。",
  "post_condition": "返回一个由 text 形成的字符串，该字符串移除了每一个最外层的、平衡的 '<...>' 区域：分隔符 '<' 和 '>' 字符及其之间的所有内容均被剥离。嵌套解析方式为——在活动的 '<...>' 区域内遇到的 '<' 属于内部区域，不会终止外部区域。不属于任何平衡 '<...>' 区域的字符将按其原始相对顺序保留。未闭合任何活动 '<...>' 区域的 '>' 字符将从结果中省略。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► 逻辑验证结果（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/extract-py/_strip_angle_brackets.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个由 text 形成的字符串，该字符串移除了每一个最外层的、平衡的 '<...>' 区域：分隔符 '<' 和 '>' 字符及其之间的所有内容均被剥离。嵌套解析方式为——在活动的 '<...>' 区域内遇到的 '<' 属于内部区域，不会终止外部区域。不属于任何平衡 '<...>' 区域的字符将按其原始相对顺序保留。未闭合任何活动 '<...>' 区域的 '>' 字符将从结果中省略。",
```

```
"actual_behavior": "该函数返回一个新的字符串，该字符串省略了所有出现的 '<' 和 '>', 并且也省略了出现在平衡尖括号对内的所有字符（遵循标准嵌套规则）。形式上，为索引 i (0 ≤ i ≤ len(text)) 定义字符前深度 d(i): d(0)=0, 且对于 i≥0: 若 text[i]='<', 则 d(i+1) = d(i)+1; 若 text[i]='>', 则 d(i+1) = max(0, d(i)-1); 否则 d(i+1) = d(i)。然后输出字符串是所有满足 text[i] ∉ {'<', '>'} 且 d(i)=0 的字符 text[i] 的连接。未匹配的 '<' 导致该字符之后直到字符串末尾（或直到遇到匹配的 '>') 的所有字符被视为处于区域内，从而被移除；深度为零时的任何未匹配 '>' 也会被移除。",
"code_evidence": "Line 6: if ch == '<':\nLine 7: depth += 1",
"trigger_condition": "该代码无条件移除 '<' 字符，即使它们未匹配且不属于任何平衡区域。规约 (Condition B) 要求保留未匹配的 '<', 因为只应移除最外层的平衡 '<...>' 区域（以及未匹配的 '>')。对于输入 '<', 代码返回 '', 但规约要求返回 '<'。"
}
```

► Bug Validator result（完整）

```
{
  "id": "src--extract-py--_strip_angle_brackets",
  "source_file": "fm_agent/extracted_functions/src/extract-py/_strip_angle_brackets.py",
  "function_name": "_strip_angle_brackets",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--extract-py--_strip_angle_brackets.py",
  "detail_file": "fm_agent/bug_validation/src--extract-py--_strip_angle_brackets.md",
  "probe_stdout": "CONFIRMED - actual: '' | expected: '<'\n",
  "trigger_summary": "输入 '<'（未匹配的尖括号）返回空字符串而非 '<' - 该 '<' 字符在未平衡时被无条件丢弃，而非保留。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:40:12.173233Z	2954
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:41:01.106134Z	3650

事件 1 — [llm_b5e785664e2d4205b6a4416d453edff9](#)

► parsed_json（完整）

```
{
  "post_condition": "该函数返回一个新的字符串，该字符串省略了所有出现的 '<' 和 '>', 并且也省略了出现在平衡尖括号对内的所有字符（遵循标准嵌套规则）。形式上，为索引 i (0 ≤ i ≤ len(text)) 定义字符前深度 d(i): d(0)=0, 且对于 i≥0: 若 text[i]='<', 则 d(i+1) = d(i)+1; 若 text[i]='>', 则 d(i+1) = max(0, d(i)-1); 否则 d(i+1) = d(i)。然后输出字符串是所有满足 text[i] ∉ {'<', '>'} 且 d(i)=0 的字符 text[i] 的连接。未匹配的 '<' 导致该字符之后直到字符串末尾（或直到遇到匹配的 '>') 的所有字符被视为处于区域内，从而被移除；深度为零时的任何未匹配 '>' 也会被移除。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_b5e785664e2d4205b6a4416d453edff9_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_b5e785664e2d4205b6a4416d453edff9_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_b5e785664e2d4205b6a4416d453edff9_response.txt)

事件 2 — [llm_fd5bafb6bd20c47eb879e553e1c1046e8](#)

► parsed_json（完整）


```
{
  "verdict": "MISMATCH",
  "counterexample": "<",
  "offending_statements": "Line 6: if ch == '<':\nLine 7: depth += 1",
  "reason": "该代码无条件移除 '<' 字符，即使它们未匹配且不属于任何平衡区域。规约 (Condition B) 要求保留未匹配的 '<'，因为只应移除最外层的平衡 '<...>' 区域（以及未匹配的 '>')。对于输入 '<'，代码返回 ''，但规约要求返回 '<'。"
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_fd5bafbbd20c47eb879e553e1c1046e8_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_fd5bafbbd20c47eb879e553e1c1046e8_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_fd5bafbbd20c47eb879e553e1c1046e8_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-077 — [src--extract-py--extract_functions_from_file](#)

- 四类结论：可能有 Bug
- 分类依据：probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- Validator: [confirmed](#); attempts 1
- Phase / 模块：Phase 2 — Language Backend & Function Extraction / [extraction](#); 原源码 [src/extract.py](#)
- 证据：[抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: 返回一个 (func_name, source_text) 元组列表，每个元组对应文件中通过特定语言提取规则找到的一个函数。每个 func_name 都经过规范化：不安全字符被翻译，文件中出现的重名通过按首次出现顺序追加数字后缀的方式进行去重。source_text 包含完整的函数体，从开始到结束（包含两端），行结尾统一为 '\n'，且函数体以尾随换行符结束。函数按源代码顺序出现。当语言没有文件局部正则提取能力时，返回空列表。
- **Actual behavior**: 在正常执行时，文件被读取并关闭，无副作用；返回值是一个 (deduped_name, source_text) 元组列表。如果 lang_cfg["body"] 既不是 "brace" 也不是 "indent"，则列表为空。否则，对于通过特定语言提取（大括号匹配或缩进）检测到的每个顶层函数，包含一个元组。deduped_name 是函数原始名称经过规范化的形式（将 '/' 翻译为 ')，若同一规范化名称出现多次，则追加数字后缀 '{count}'，从第二次出现开始计数（从 1 开始）。source_text 包含该函数的拼接源代码行（从开始索引到结束索引，包含两端，行间用换行符分隔，并带有一个尾随换行符）。元组的顺序与文件中函数的顺序一致。形式化地，令 lines = [l.rstrip('\n').rstrip('\r') for l in readlines(filepath)]; 令 lang_cfg = LANG_CONFIG[lang_key]。若 lang_cfg["body"] $\notin$ {"brace", "indent"}: result = []。否则令 P = （若 lang_cfg["body"] = "brace" 则为 _extract_functions_brace(lines, lang_key, lang_cfg)，否则为 extract_functions_indent(lines, lang_cfg)，其中 P 是一个长度为 m 的 (name_i, start_i, end_i) 列表 (i = 0..m-1)。对于每个 i，令 cname_i = canonicalize(name_i)，count_i = |{j < i : canonicalize(P[j][0]) = cname_i}|。则 deduped_name_i = cname_i （若 count_i = 0 则为此，否则为 cname_i + " + count_i"）；source_i = '\n'.join(lines[start_i .. end_i]) + '\n'; result = [(deduped_name_i, source_i) | i = 0..m-1]。
- **Code evidence**: 第 9 行: lines = [l.rstrip('\n').rstrip('\r') for l in lines]
- **Trigger condition**: 代码在移除换行符之后，再去除每一行的尾随回车符，这会移除源内容中字面上的 \r（例如字符串内部）。这破坏了函数体，违反了规格要求 source_text 应包含完整函数体且仅将行结尾归一化的规定。
- **Validator trigger**: 源内容中字面 CR 字节 (0x0D) 被当作行结尾并去除，破坏了提取的函数体。
- **Probe stdout**: CONFIRMED — CR 字节已从函数体中去除。actual: 'def func_with_cr():\n return "prefix\n'

► SPEC sidecar (完整)

```
{
  "signature": "extract_functions_from_file(filepath, lang_key) -> list[tuple[str, str]]",
```

```

"pre_condition": "filepath 指向一个存在且可读的文件。lang_key 是 LANG_CONFIG 中存在的语言键。",
"post_condition": "返回一个 (func_name, source_text) 元组列表, 每个元组对应文件中通过特定语言提取规则找到的一个函数。每个 func_name 都经过规范化: 不安全字符被翻译, 文件中出现的重名通过按首次出现顺序追加数字后缀的方式进行去重。source_text 包含完整的函数体, 从开始到结束 (包含两端), 行结尾统一为 '\\n', 且函数体以尾随换行符结束。函数按源代码顺序出现。当语言没有文件局部正则提取能力时, 返回空列表。"
}

```

► INFO / callees sidecar (完整)

```

{
  "callees": [
    {
      "name": "_extract_functions_brace",
      "signature": "_extract_functions_brace(lines: list[str], lang_key: str, lang_cfg: dict) -> list[tuple[str, int, int]]",
      "pre_condition": "lines 是一个非空的、行结尾已统一的字符串列表。lang_key 是已识别的语言键。lang_cfg 是对应语言且 body 类型为 'brace' 的 LANG_CONFIG 条目。",
      "post_condition": "返回一个 (func_name, start_index, end_index) 元组列表, 通过大括号匹配识别文件中的每个顶层函数, 并使用 lang_cfg 中特定语言的关键字和前缀过滤器。索引从 0 开始, 且为闭合区间。"
    },
    {
      "name": "_extract_functions_indent",
      "signature": "_extract_functions_indent(lines: list[str], lang_cfg: dict) -> list[tuple[str, int, int]]",
      "pre_condition": "lines 是一个非空的、行结尾已统一的字符串列表。lang_cfg 是对应 body 类型为 'indent' 的语言的 LANG_CONFIG 条目。",
      "post_condition": "返回一个 (func_name, start_index, end_index) 元组列表, 通过缩进边界识别文件中的每个顶层函数, 并使用 lang_cfg 中特定语言的关键字和前缀过滤器。索引从 0 开始, 且为闭合区间。"
    },
    {
      "name": "canonicalize",
      "signature": "canonicalize(func_name: str) -> str",
      "pre_condition": "func_name 是一个原始函数名字符串, 可能包含对文件系统路径或完全限定名分隔符不安全的字符。",
      "post_condition": "返回一个文件系统安全、FQN 兼容的字符串, 其中 '/' 字符被翻译为 '_'。该结果适用于文件路径和 FQN 组件。"
    }
  ]
}

```

► Logic verification result (完整)

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/extract-py/extract_functions_from_file.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个 (func_name, source_text) 元组列表, 每个元组对应文件中通过特定语言提取规则找到的一个函数。每个 func_name 都经过规范化: 不安全字符被翻译, 文件中出现的重名通过按首次出现顺序追加数字后缀的方式进行去重。source_text 包含完整的函数体, 从开始到结束 (包含两端), 行结尾统一为 '\\n', 且函数体以尾随换行符结束。函数按源代码顺序出现。当语言没有文件局部正则提取能力时, 返回空列表。",
    "actual_behavior": "在正常执行时, 文件被读取并关闭, 无副作用; 返回值是一个 (deduped_name, source_text) 元组列表。如果 lang_cfg[\"body\"] 既不是 \"brace\" 也不是 \"indent\", 则列表为空。否则, 对于通过特定语言提取 (大括号匹配或缩进) 检测到的每个顶层函数, 包含一个元组。deduped_name 是函数原始名称经过规范化的形式 (将 '/' 翻译为 '_'), 若同一规范化名称出现多次, 则追加数字后缀 '_{count}', 从第二次出现开始计数 (从 1 开始)。source_text 包含该函数的拼接源代码行 (从开始索引到结束索引, 包含两端, 行间用换行符分隔, 并带有一个尾随换行符)。元组的顺序与文件中函数的顺序一致。形式化地, 令 lines = [l.rstrip('\\n').rstrip('\\r') for l in readlines(filepath)]; 令 lang_cfg = LANG_CONFIG[lang_key]。若 lang_cfg[\"body\"] ∉ {\"brace\", \"indent\"}: result = []。否则令 P = (若 lang_cfg[\"body\"] = \"brace\" 则为 _extract_functions_brace(lines, lang_key, lang_cfg), 否则为 _extract_functions_indent(lines, lang_cfg)), 其中 P 是一个长度为 m 的 (name_i, start_i, end_i) 列表 (i = 0..m-1)。对于每个 i, 令 cname_i = canonicalize(name_i), count_i = |{j < i : canonicalize(P[j][0]) = cname_i}|。则 deduped_name_i = cname_i (若 count_i = 0 则为此, 否则为 cname_i + '_' + count_i); source_i = '\\n'.join(lines[start_i .. end_i]) + '\\n'; result = [(deduped_name_i, source_i) | i = 0..m-1]。",
    "code_evidence": "Line 9: lines = [l.rstrip('\\n').rstrip('\\r') for l in lines]",
    "trigger_condition": "代码在移除换行符之后, 再去除每一行的尾随回车符, 这会移除源内容中字面上的 \\r (例如字符串内部)。这破坏了函数体, 违反了规格要求 source_text 应包含完整函数体且仅将行结尾归一化的规定。"
  }
}

```

```
}
}
```

► Bug Validator result（完整）

```
{
  "id": "src--extract-py--extract_functions_from_file",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/extract-
py/extract_functions_from_file.py",
  "function_name": "extract_functions_from_file",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--extract-py--extract_functions_from_file.py",
  "detail_file": "fm_agent/bug_validation/src--extract-py--extract_functions_from_file.md",
  "probe_stdout": "CONFIRMED - 函数体中的 CR 字节已被去除。actual: 'def func_with_cr():\n    return
\nprefix\n'",
  "trigger_summary": "源内容中字面 CR 字节 (0x0D) 被当作行结尾并去除，破坏了提取的函数体。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:04:41.014808Z	3494
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:05:18.751119Z	7255

事件 1 — [llm_c05ba7db55c640aba1890d9b4e83b81a](#)

► parsed_json（完整）

```
{
  "post_condition": "在正常执行时，文件被读取并关闭，无副作用；返回值是一个 (deduped_name, source_text) 元组列表。如
果 lang_cfg[\"body\"] 既不是 \"brace\" 也不是 \"indent\"，则列表为空。否则，对于通过特定语言提取（大括号匹配或缩进）检
测到的每个顶层函数，包含一个元组。deduped_name 是函数原始名称经过规范化的形式（将 '/' 翻译为 '_'），若同一规范化名称出现多
次，则追加数字后缀 '_{count}'，从第二次出现开始计数（从 1 开始）。source_text 包含该函数的拼接源代码行（从开始索引到结束索
引，包含两端，行间用换行符分隔，并带有一个尾随换行符）。元组的顺序与文件中函数的顺序一致。形式化地，令 lines =
[l.rstrip('\\n').rstrip('\\r') for l in readlines(filepath)]；令 lang_cfg = LANG_CONFIG[lang_key]。若
lang_cfg[\"body\"] \notin {\"brace\", \"indent\"}：result = []。否则令 P = （若 lang_cfg[\"body\"] = \"brace\" 则
为 _extract_functions_brace(lines, lang_key, lang_cfg)，否则为 _extract_functions_indent(lines,
lang_cfg)），其中 P 是一个长度为 m 的 (name_i, start_i, end_i) 列表 (i = 0..m-1)。对于每个 i，令 cname_i =
canonicalize(name_i)，count_i = |{j < i : canonicalize(P[j][0]) = cname_i}|。则 deduped_name_i = cname_i
（若 count_i = 0 则为此，否则为 cname_i + '_' + count_i）；source_i = '\\n'.join(lines[start_i .. end_i]) +
'\\n'；result = [(deduped_name_i, source_i) | i = 0..m-1]。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_c05ba7db55c640aba1890d9b4e83b81a_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_c05ba7db55c640aba1890d9b4e83b81a_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_c05ba7db55c640aba1890d9b4e83b81a_response.txt)

事件 2 — [llm_d0f6a08847f94dd8b45a00f52b4d2975](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "一个 Python 源文件，其内容中包含某行在字符串字面量内带有尾随回车符，例如：\n\ndef foo():\nprint(\"hello\\r\")\n\n(该行在换行符之前确实以 \\r 结尾)。语言键对应使用缩进提取的语言（例如 'python'）。",
  "offending_statements": "第 9 行: lines = [l.rstrip('\\n').rstrip('\\r') for l in lines]",
  "reason": "代码在移除换行符之后，再去除每一行的尾随回车符，这会移除源内容中字面上的 \\r（例如字符串内部）。这破坏了函数体，违反了规格要求 source_text 应包含完整函数体且仅将行结尾归一化的规定。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_d0f6a08847f94dd8b45a00f52b4d2975_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_d0f6a08847f94dd8b45a00f52b4d2975_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_d0f6a08847f94dd8b45a00f52b4d2975_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修复实现。

模块: language_handlers

FMA-MISMATCH-097 — src -- languages -- erlang-py -- _elp_argv

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块: Phase 2 — Language Backend & Function Extraction / language_handlers; 原源码 src/languages/erlang.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** 返回一个非空字符串列表，其最后一个元素是“server”，适用于将 ELP 作为一个从标准输入读取 JSON-RPC 请求并向标准输出写入 JSON-RPC 响应的子进程来调用。前面的元素构成了对已配置的 ELP 二进制文件的一个平台合适的调用形式。返回值是稳定的：如果 ELP 后端配置没有改变，两次调用返回相等的列表；如果 ELP 后端发生了可能对相同输入项目产生不同分析输出的改变，则返回不相等的列表。
- **Actual behavior:** 该函数返回一个表示用于启动 ELP 服务器的参数向量的字符串列表。令 cmd = settings.erlang.command.strip() 和 tokens = shlex.split(cmd, posix=(os.name != 'nt'))。如果 tokens 为空，令 base = ['elp']; 否则 base = tokens。返回的列表是 base + ['server']。因此，结果永远非空，始终将 'server' 作为其最后一个元素，并且其第一个元素要么是 settings.erlang.command 去除空白后的非空内容按 token 分割的结果，要么是 'elp'（如果没有配置有效的命令）。
- **Code evidence:** 第 6 行: return [*argv, "server"]
- **Trigger condition:** 返回的列表变为 ['elp', 'server', 'server']，这并没有形成一个平台合适的调用形式，因为前面的元素现在包含一个重复的“server”，可能会使 ELP 感到困惑。规范要求在这样的“server”之前的参数构成一个正确的调用；重复的“server”违反了这一要求。
- **Validator trigger:** 当 settings.erlang.command 是 'elp server'（或任何已经包含 'server' 的命令）时，无条件的追加会在参数向量中产生一个重复的“server”。
- **Probe stdout:** CONFIRMED — _elp_argv() 返回一个包含重复“server”的列表：实际值: ['elp', 'server', 'server'] | 预期值（无重复）: ['elp', 'server']

► SPEC sidecar（完整）

```
{
  "signature": "_elp_argv() -> list[str]",
  "pre_condition": "ELP 后端二进制文件存在于配置的位置或可通过系统路径发现。",
  "post_condition": "返回一个非空字符串列表，其最后一个元素是“server”，适用于将 ELP 作为一个从标准输入读取 JSON-RPC 请求并向标准输出写入 JSON-RPC 响应的子进程来调用。前面的元素构成了对已配置的 ELP 二进制文件的一个平台合适的调用形式。返回值是稳定的：如果 ELP 后端配置没有改变，两次调用返回相等的列表；如果 ELP 后端发生了可能对相同输入项目产生不同分析输出的改变，则返回不相等的列表。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_elp_argv.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个非空字符串列表，其最后一个元素是“server”，适用于将 ELP 作为一个从标准输入读取 JSON-RPC 请求并向标准输出写入 JSON-RPC 响应的子进程来调用。前面的元素构成了对已配置的 ELP 二进制文件的一个平台合适的调用形式。返回值是稳定的：如果 ELP 后端配置没有改变，两次调用返回相等的列表；如果 ELP 后端发生了可能对相同输入项目产生不同分析输出的改变，则返回不相等的列表。",
    "actual_behavior": "该函数返回一个表示用于启动 ELP 服务器的参数向量的字符串列表。令 `cmd = settings.erlang.command.strip()` 和 `tokens = shlex.split(cmd, posix=(os.name != 'nt'))`。如果 `tokens` 为空，令 `base = ['elp']`；否则 `base = tokens`。返回的列表是 `base + ['server']`。因此，结果永远非空，始终将 `server` 作为其最后一个元素，并且其第一个元素要么是 `settings.erlang.command` 去除空白后的非空内容按 token 分割的结果，要么是 `elp`（如果没有配置有效的命令）。",
    "code_evidence": "Line 6: return [*argv, \"server\"]",
    "trigger_condition": "返回的列表变为 ['elp', 'server', 'server']，这并没有形成一个平台合适的调用形式，因为前面的元素现在包含一个重复的“server”，可能会使 ELP 感到困惑。规范要求在这样的“server”之前的参数构成一个正确的调用；重复的“server”违反了这一要求。"
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--erlang-py--_elp_argv",
  "source_file": "src/languages/erlang.py",
  "function_name": "_elp_argv",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--_elp_argv.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--_elp_argv.md",
  "probe_stdout": "CONFIRMED - _elp_argv() 返回一个包含重复“server”的列表：实际值：['elp', 'server', 'server'] | 预期值（无重复）：['elp', 'server']",
  "trigger_summary": "当 settings.erlang.command 是 'elp server'（或任何已经包含 'server' 的命令）时，无条件的追加会在参数向量中产生一个重复的“server”。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:27:04.054694Z	1261
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:27:20.774869Z	4907

事件 1 — llm_72c4938f0da547428ef5f0631328abe6

► parsed_json (完整)

```
{
  "post_condition": "该函数返回一个表示用于启动 ELP 服务器的参数向量的字符串列表。令 `cmd = settings.erlang.command.strip()` 和 `tokens = shlex.split(cmd, posix=(os.name != 'nt'))`。如果 `tokens` 为空, 令 `base = ['elp']`; 否则 `base = tokens`。返回的列表是 `base + ['server']`。因此, 结果永远非空, 始终将 `server` 作为其最后一个元素, 并且其第一个元素要么是 `settings.erlang.command` 去除空白后的非空内容按 token 分割的结果, 要么是 `elp` (如果没有配置有效的命令)。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_72c4938f0da547428ef5f0631328abe6_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_72c4938f0da547428ef5f0631328abe6_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_72c4938f0da547428ef5f0631328abe6_response.txt)

事件 2 — llm_d1505f4ad70848ab9bc33f0fe6408870

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "当配置字符串中包含作为独立参数的单词`server`时, 例如, 在 POSIX 系统上的 `settings.erlang.command = 'elp server'`。",
  "offending_statements": "第 6 行: return [*argv, \"server\"]",
  "reason": "返回的列表变为 ['elp', 'server', 'server'], 这并没有形成一个平台合适的调用形式, 因为前面的元素现在包含一个重复的`server`, 可能会使 ELP 感到困惑。规范要求最后的`server`之前的参数构成一个正确的调用; 重复的`server`违反了这一要求。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_d1505f4ad70848ab9bc33f0fe6408870_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_d1505f4ad70848ab9bc33f0fe6408870_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_d1505f4ad70848ab9bc33f0fe6408870_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足; 若可满足, 将其转为回归测试并修实现。

FMA-MISMATCH-098 — src--languages--erlang-py--_escape_component

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: confirmed; 尝试次数 1
- Phase / 模块: Phase 2 — 语言后端与函数提取 / language_handlers; 原源码 src/languages/erlang.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** 返回一个字符串，其中 `value` 中每个字符如果是 ASCII 字母数字 (`a-z, A-Z, 0-9`) 或下划线，则保留在其原始相对位置，其他每个字符则替换为一个下划线后跟其 Unicode 码点，以零填充的两位十六进制数表示。输出是确定性的：相同的输入字符串总是产生相同的输出字符串。输出中的每个字符都属于集合 `[a-zA-Z0-9_]`。
- **Actual behavior:** 该函数通过按顺序遍历 `c` 中的每个字符 `value`，并追加 `c` 本身（如果 `c` 是 ASCII 字符并且 `c.isalnum()` 是 `True` 或者 `c` 等于 `'_'`），或者追加字符串 `_` 后跟 `ord(c)` 的小写零填充（最小宽度 2）十六进制表示，从而返回一个字符串。形式化地说：如果 `process(c) = c` 则令 `c.isascii()` and `(c.isalnum() or c == '_')`，否则令 `'_' + f'{ord(c):02x}'`。然后返回的字符串等于 `''.join(process(c) for c in value)`。输出始终非空，因为至少有一个字符贡献至少一个字符。
- **Code evidence:** 第 7 行: `result.append(f"_{ord(char):02x}")`
- **Trigger condition:** 对于输入字符串 `" (U+03C0)`，代码返回 `'_3c0'`，它使用了三个十六进制数字。规范要求每个非字母数字下划线字符后跟一个下划线后跟零填充的两位十六进制数字。由于 `0x3C0` 需要三个数字，输出违反了“两位数字”要求。
- **Validator trigger:** 输入 `'π' (U+03C0, 序号 960=0x3C0)` 产生 `'_3c0'`（包含 3 个十六进制数字），但规范要求恰好 2 个零填充的十六进制数字。
- **Probe stdout:** 已确认 — 实际: `'_3c0'` | 预期（规范要求两位数）: `'_c0'` | 输出中的十六进制位数: 3（规范要求恰好 2）

► SPEC 附属文件（完整）

```
{
  "signature": "_escape_component(value: str) -> str",
  "pre_condition": "value 是一个非空字符串。",
  "post_condition": "返回一个字符串，其中 value 中每个字符如果是 ASCII 字母数字（a-z, A-Z, 0-9）或下划线，则保留在其原始相对位置，其他每个字符则替换为一个下划线后跟其 Unicode 码点，以零填充的两位十六进制数表示。输出是确定性的：相同的输入字符串总是产生相同的输出字符串。输出中的每个字符都属于集合 [a-zA-Z0-9_] 。"
}
```

► INFO / 被调用者附属文件（完整）

```
{
  "callees": []
}
```

► 逻辑验证结果（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_escape_component.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个字符串，其中 value 中每个字符如果是 ASCII 字母数字（a-z, A-Z, 0-9）或下划线，则保留在其原始相对位置，其他每个字符则替换为一个下划线后跟其 Unicode 码点，以零填充的两位十六进制数表示。输出是确定性的：相同的输入字符串总是产生相同的输出字符串。输出中的每个字符都属于集合 [a-zA-Z0-9_] 。",
    "actual_behavior": "该函数通过按顺序遍历 `value` 中的每个字符 `c`，并追加 `c` 本身（如果 `c` 是 ASCII 字符并且 `c.isalnum()` 是 `True` 或者 `c` 等于 `_`），或者追加字符串 `_` 后跟 `ord(c)` 的小写零填充（最小宽度 2）十六进制表示，从而返回一个字符串。形式化地说：如果 `c.isascii() and (c.isalnum() or c == '_')` 则令 `process(c) = c`，否则令 `_` + `f'{ord(c):02x}'`。然后返回的字符串等于 `''.join(process(c) for c in value)`。输出始终非空，因为至少有一个字符贡献至少一个字符。",
    "code_evidence": "Line 7: result.append(f\"_{ord(char):02x}\")",
    "trigger_condition": "对于输入字符串 `'' (U+03C0)`，代码返回 `\\'_3c0\\'`，它使用了三个十六进制数字。规范要求每个非字母数字下划线字符后跟一个下划线后跟零填充的两位十六进制数字。由于 `0x3C0` 需要三个数字，输出违反了“两位数字”要求。"
  }
}
```

► 缺陷验证器结果（完整）

```
{
  "id": "src--languages--erlang-py--_escape_component",
  "source_file": "src/languages/erlang.py",
  "function_name": "_escape_component",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--_escape_component.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--_escape_component.md",
  "probe_stdout": "已确认 — 实际: '_3c0' | 预期（规范要求两位数）: '_c0' | 输出中的十六进制位数: 3（规范要求恰好 2）\n",
  "trigger_summary": "输入 'π' (U+03C0, 序号 960=0x3C0) 产生 '_3c0'（包含 3 个十六进制数字），但规范要求恰好 2 个零填充的十六进制数字。"
}
```

► 推理器流程（2 条事件）

共 2 条直接推理器事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	用途	状态	尝试	块	开始时间	令牌数
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:39:08.871476Z	1566
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:39:34.308838Z	3201

事件 1 — [llm_bef5cf913de2469cb7140325cc4821d6](#)

► 解析后的 JSON（完整）

```
{
  "post_condition": "该函数通过按顺序遍历 `value` 中的每个字符 `c`，并追加 `c` 本身（如果 `c` 是 ASCII 字符并且 `c.isalnum()` 是 `True` 或者 `c` 等于 `'_`'），或者追加字符串 `"_` 后跟 `ord(c)` 的小写零填充（最小宽度 2）十六进制表示，从而返回一个字符串。形式化地说：如果 `c.isascii() and (c.isalnum() or c == '_`' 则令 `process(c) = c`，否则令 `'_`' + f'{ord(c):02x}`'。然后返回的字符串等于 `''.join(process(c) for c in value)`。输出始终非空，因为至少有一个字符贡献至少一个字符。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_bef5cf913de2469cb7140325cc4821d6_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_bef5cf913de2469cb7140325cc4821d6_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_bef5cf913de2469cb7140325cc4821d6_response.txt)

事件 2 — [llm_36d7fee00d1b42ff86689555a63107d4](#)

► 解析后的 JSON（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "π",
  "offending_statements": "第 7 行: result.append(f\"_{ord(char):02x}\")",
  "reason": "对于输入字符串 'π' (U+03C0)，代码返回 `\\'_3c0\\`，它使用了三个十六进制数字。规范要求每个非字母数字下划线字符后跟一个下划线后跟零填充的两位十六进制数字。由于 0x3C0 需要三个数字，输出违反了“两位数字”要求。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_36d7fee00d1b42ff86689555a63107d4_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_36d7fee00d1b42ff86689555a63107d4_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_36d7fee00d1b42ff86689555a63107d4_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-099 — [src--languages--erlang-py--_function_id](#)

- 四类结论：可能有 Bug
- 分类依据：沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator： [confirmed](#)；attempts 1
- Phase / 模块： Phase 2 — Language Backend & Function Extraction / [language_handlers](#)；原源码 [src/languages/erlang.py](#)
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

SPEC claim： 返回一个规范函数标识字符串，该字符串由从 uri 派生的模块标识符、从 label 中提取的非限定函数名以及从 label 中提取的 arity 组成，以双下划线连接，格式为 `__module__name__arity`。每个组分（`__`）中任何与双下划线分隔符冲突的字符都会被确定性替换。非限定函数名是 label 中最后一个正斜杠之前的部分；当该部分包含冒号且不以单引号字符开头时，仅使用最后一个冒号之后的子串作为非限定名称。当 label 不是字符串、label 不包含正斜杠分隔符，或 label 中最后一个正斜杠之后的部分无法解析为非负整数时，抛出 `ValueError`。

- **Actual behavior：** 如果 label 字符串包含斜杠 `'/'` 且最后一个斜杠之后的子串是一个有效的非负整数，则函数返回一个由以下步骤形成的字符串：(1) 通过 `_module_from_uri(uri)` 从 uri 计算模块标识符，(2) 从最后一个斜杠之前的部分提取函数名：如果该部分包含冒号 `':'` 且不以单引号开头，则函数名取最后一个冒号之后的子串；否则为斜杠之前的整个部分，(3) 对模块标识符和函数名应用 `_escape_component`，(4) 将它们与双下划线和整数 arity 串联： `f"{_escape_component(_module_from_uri(uri)), _escape_component(name_without_module), arity}"`。如果 label 不包含斜杠或斜杠之后的子串无法转换为非负整数，则抛出 `ValueError`，消息为 `"ELP function label has no valid arity: {label!r}"`，由原始异常链接而来。假设函数 `_module_from_uri` 和 `_escape_component` 在给定前置条件下始终产生确定性结果；它们抛出的任何异常会直接传播。形式上，对于所有满足前置条件的 `uri`、`label`： `( name, arity_s : label = name + '/' + arity_s arity_s ) result = concat( _escape_component( _module_from_uri(uri)), "", _escape_component(name_without_module), "", arity_s)`，其中 `name_without_module = 如果 ( ':' name name[0] )` 则 `rsplit(name, ':', 1)[1]` 否则 `name`；否则函数抛出带有指定消息的 `ValueError`。
 - **Code evidence：** Line 4: `int(arity)`
 - **Trigger condition：** 代码未验证 arity 子串表示一个非负整数。`int('-1')` 会成功，因此 `label='foo/-1'` 返回包含 `'-1'` 的字符串，而不是按要求抛出 `ValueError`。此外，代码没有处理 label 不是字符串的情况（例如 `label=42`），会抛出 `AttributeError` 而非 `ValueError`。
 - **Validator trigger：** 负数 arity 未经检查地通过 `int()` 检查；非字符串 label 抛出 `AttributeError` 而非 `ValueError`
 - **Probe stdout：** `[negative_arity] CONFIRMED` — 负数 arity 未被拒绝。`label='foo/-1'` 返回了 `'module__foo__-1'`，而不是抛出 `ValueError\n[non_string_label] CONFIRMED` — 非字符串 label 抛出 `AttributeError` 而非 `ValueError`（规格要求 `ValueError`） `\nCONFIRMED` — 所有 Bug 方面已重现\n
- SPEC sidecar（完整）

```
{
  "signature": "_function_id(uri: str, label: str) -> str",
  "pre_condition": "uri 是一个非空字符串，表示 Erlang 源文件的绝对文件路径。label 是一个来自 ELP 符号的非空字符串，编码了一个函数名和一个 arity，其中 arity 位于最后一个正斜杠之后，并可解析为非负整数。",
  "post_condition": "返回一个规范函数标识字符串，该字符串由从 uri 派生的模块标识符、从 label 中提取的非限定函数名以及从 label 中提取的 arity 组成，以双下划线连接，格式为 <module>__<name>__<arity>。每个组分 (<module>、<name>) 中任何与双下划线分隔符冲突的字符都会被确定性替换。非限定函数名是 label 中最后一个正斜杠之前的部分；当该部分包含冒号且不以单引号字符开头时，仅使用最后一个冒号之后的子串作为非限定名称。当 label 不是字符串、label 不包含正斜杠分隔符，或 label 中最后一个正斜杠之后的部分无法解析为非负整数时，抛出 ValueError。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_module_from_uri",
      "signature": "_module_from_uri(uri: str) -> str",
      "pre_condition": "uri 是一个非空字符串，表示 Erlang 源文件的绝对文件路径。",
      "post_condition": "返回一个确定性字符串标识符，表示位于 uri 的源文件所属的模块。相同的 uri 始终产生相同的模块标识符。"
    },
    {
      "name": "_escape_component",
      "signature": "_escape_component(component: str) -> str",
      "pre_condition": "component 是一个非空字符串（模块标识符或函数名）。",
      "post_condition": "返回 component 的转换版本，其中任何与函数标识符格式中的双下划线分隔符冲突的字符被确定性替换。相同的 component 值始终产生相同的结果。"
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_function_id.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个规范函数标识字符串，该字符串由从 uri 派生的模块标识符、从 label 中提取的非限定函数名以及从 label 中提取的 arity 组成，以双下划线连接，格式为 <module>__<name>__<arity>。每个组分 (<module>、<name>) 中任何与双下划线分隔符冲突的字符都会被确定性替换。非限定函数名是 label 中最后一个正斜杠之前的部分；当该部分包含冒号且不以单引号字符开头时，仅使用最后一个冒号之后的子串作为非限定名称。当 label 不是字符串、label 不包含正斜杠分隔符，或 label 中最后一个正斜杠之后的部分无法解析为非负整数时，抛出 ValueError。",
    "actual_behavior": "如果 label 字符串包含斜杠 '/' 且最后一个斜杠之后的子串是一个有效的非负整数，则函数返回一个由以下步骤形成的字符串：(1) 通过 `_module_from_uri(uri)` 从 uri 计算模块标识符，(2) 从最后一个斜杠之前的部分提取函数名：如果该部分包含冒号 ':' 且不以单引号开头，则函数名取最后一个冒号之后的子串；否则为斜杠之前的整个部分，(3) 对模块标识符和函数名应用 `_escape_component`，(4) 将它们与双下划线和整数 arity 串联：`f\"{escaped_module}__{escaped_name}__{arity}\"`。如果 label 不包含斜杠或斜杠之后的子串无法转换为非负整数，则抛出 `ValueError`，消息为 `\"ELP function label has no valid arity: {label!r}\"`，由原始异常链接而来。假设函数 `_module_from_uri` 和 `_escape_component` 在给定前置条件下始终产生确定性结果；它们抛出的任何异常会直接传播。形式上，对于所有满足前置条件的 `uri`、`label`：( name, arity_s : label = name + '/' + arity_s  arity_s )  result = concat(_escape_component(_module_from_uri(uri)), \"__\", _escape_component(name_without_module), \"__\", arity_s), 其中 name_without_module = 如果 (':' name name[0]  '') 则 rsplit(name, ':', 1)[1] 否则 name；否则函数抛出带有指定消息的 ValueError。",
    "code_evidence": "Line 4: int(arity)",
    "trigger_condition": "代码未验证 arity 子串表示一个非负整数。int('-1') 会成功，因此 label='foo/-1' 返回包含 '__-1__' 的字符串，而不是按要求抛出 ValueError。此外，代码没有处理 label 不是字符串的情况（例如 label=42），会抛出 AttributeError 而非 ValueError。"
  }
}
```

► Bug Validator result (完整)


```
{
  "id": "src--languages--erlang-py--_function_id",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/src/languages/erlang.py",
  "function_name": "_function_id",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--_function_id.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--_function_id.md",
  "probe_stdout": "[negative_arity] CONFIRMED - 负数 arity 未被拒绝。label='foo/-1' 返回了 'module__foo__-1', 而不是抛出 ValueError\\n[non_string_label] CONFIRMED - 非字符串 label 抛出 AttributeError 而非 ValueError (规格要求 ValueError)\\nCONFIRMED - 所有 Bug 方面已重现\\n",
  "trigger_summary": "负数 arity 未经检查地通过 int() 检查;非字符串 label 抛出 AttributeError 而非 ValueError"
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:27:22.086677Z	2215
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:27:52.051626Z	3916

事件 1 — llm_570efaec3d5548ff9503cc01167504de

► parsed_json (完整)

```
{
  "post_condition": "如果 label 字符串包含斜杠 '/' 且最后一个斜杠之后的子串是一个有效的非负整数，则函数返回一个由以下步骤形成的字符串：(1) 通过 `_module_from_uri(uri)` 从 uri 计算模块标识符，(2) 从最后一个斜杠之前的部分提取函数名：如果该部分包含冒号 ':' 且不以单引号开头，则函数名取最后一个冒号之后的子串；否则为斜杠之前的整个部分，(3) 对模块标识符和函数名应用 `_escape_component`，(4) 将它们与双下划线和整数 arity 串联：`f\"{escaped_module}__{escaped_name}__{arity}\"`。如果 label 不包含斜杠或斜杠之后的子串无法转换为非负整数，则抛出 `ValueError`，消息为 `\"ELP function label has no valid arity: {label!r}\"`，由原始异常链接而来。假设函数 `_module_from_uri` 和 `_escape_component` 在给定前置条件下始终产生确定性结果；它们抛出的任何异常会直接传播。形式上，对于所有满足前置条件的 `uri`、`label`：(∃ name, arity_s : label = name + '/' + arity_s ∧ arity_s ∈ ℕ) ⇒ result = concat(_escape_component(_module_from_uri(uri)), \"__\", _escape_component(name_without_module), \"__\", arity_s)，其中 name_without_module = 如果 (':' ∈ name ∧ name[0] ≠ '') 则 rsplit(name, ':', 1)[1] 否则 name；否则函数抛出带有指定消息的 ValueError。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_570efaec3d5548ff9503cc01167504de_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_570efaec3d5548ff9503cc01167504de_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_570efaec3d5548ff9503cc01167504de_response.txt)

事件 2 — llm_9299b3156a8f44dfa01dcd2e085ef876

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "uri='dummy.erl', label='foo/-1'",
  "offending_statements": "Line 4: int(arity)",
  "reason": "代码未验证 arity 子串表示一个非负整数。int('-1') 会成功，因此 label='foo/-1' 返回包含 '__-1__' 的字符串，而不是按要求抛出 ValueError。此外，代码没有处理 label 不是字符串的情况（例如 label=42），会抛出 AttributeError 而"
}
```

```
非 ValueError。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_9299b3156a8f44dfa01dcd2e085ef876_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_9299b3156a8f44dfa01dcd2e085ef876_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_9299b3156a8f44dfa01dcd2e085ef876_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-102 — src--languages--erlang-py--_project_fingerprint

- 四类结论：可能有 Bug
- 分类依据：沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块：Phase 2 — Language Backend & Function Extraction / language_handlers; 原源码 src/languages/erlang.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim**: 返回一个元组，该元组确定性地、唯一地标识位于 proj_dir 的 Erlang 项目的当前状态。对相同的 proj_dir 两次调用 _project_fingerprint 返回相等元组，当且仅当在两次调用之间，proj_dir 下与 Erlang 分析相关的任何文件均未发生更改。文件更改是指对文件内容、字节大小或最后修改时间戳的任何修改。返回的元组不依赖于文件系统枚举顺序或其他非确定性因素。
- **Actual behavior**: 如果函数正常终止，返回值是一个元组 (backend, records)，满足：
- backend = _elp_argv(), 标识当前 ELP 后端版本与配置。
- records = tuple(sorted(set((os.path.relpath(p, root), os.stat(p).st_size, os.stat(p).st_mtime_ns) for p in paths))) 其中 root = os.path.abspath(proj_dir), paths = [p for p in _iter_project_files(root, {'.erl', '.hrl'})] + [os.path.join(root, name) for name in _PROJECT_CONFIG_FILES if os.path.isfile(os.path.join(root, name))], 且 sorted() 按绝对路径字典序排序。每条记录是一个 3 元组 (rel_path, size, mtime_ns)。记录已排序、去重，并包含 root 下递归找到的每一个 .erl/.hrl 源文件，以及位于 root 直属（非递归）且在 _PROJECT_CONFIG_FILES 中列出且存在的每一个项目配置文件。文件元数据（大小、修改时间）反映调用 os.stat() 那一刻的状态。在给定前置条件下，records 非空。如果任何 os.stat()、os.path.isfile()、文件系统迭代或 os.path.abspath() 调用引发 OSError（例如 FileNotFoundError、PermissionError），异常将向上传播，函数不产生返回值；所有局部状态被丢弃。
- **Code evidence**: Line 12
- **Trigger condition**: 指纹仅由文件大小和修改时间戳构建，忽略文件内容。因此，保持大小和 mtime_ns 不变的内容修改不会被检测到，这违反了要求：只要任何文件内容发生更改，指纹就必须改变（文件更改定义为对内容、大小或 mtime 的任何修改）。
- **Validator trigger**: 指纹仅使用 (size, mtime_ns) 并忽略文件内容；相同长度且保留 mtime 的内容更改未被检测到。
- **Probe stdout**: CONFIRMED — 尽管内容不同，指纹仍然相等: fp1fp2, size=4949, mtime_ns=1785172396729580385==1785172396729580385, but content differs

► SPEC sidecar（完整）

```
{
  "signature": "_project_fingerprint(proj_dir: str) -> tuple",
  "pre_condition": "proj_dir 是一个非空字符串，解析为绝对路径后，指向一个包含 Erlang 项目结构的现有目录，且其中至少存在一个源文件。",
  "post_condition": "返回一个元组，该元组确定性地、唯一地标识位于 proj_dir 的 Erlang 项目的当前状态。对相同的
```

```
proj_dir 两次调用 _project_fingerprint 返回相等元组，当且仅当在两次调用之间，proj_dir 下与 Erlang 分析相关的任何文件均未发生更改。文件更改是指对文件内容、字节大小或最后修改时间戳的任何修改。返回的元组不依赖于文件系统枚举顺序或其他非确定性因素。"
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_iter_project_files",
      "signature": "_iter_project_files(root: str, extensions: set[str]) -> Iterator[str]",
      "pre_condition": "root 是指向现有目录的绝对路径；extensions 是一个非空的文件扩展名字符串集合，每个扩展名以句点开头，且仅包含 ASCII 字母数字字符。",
      "post_condition": "返回一个迭代器，产出扩展名在 extensions 中的每一个普通文件的绝对路径，这些文件位于 root 或其任意子目录下，采用递归遍历。每个匹配文件恰好产出一。目录、符号链接以及扩展名不在 extensions 中的文件从产出中排除。"
    },
    {
      "name": "_elp_argv",
      "signature": "_elp_argv() -> tuple",
      "pre_condition": "ELP 后端二进制文件已安装，并可通过配置的命令路径访问。",
      "post_condition": "返回一个标识当前 ELP 后端版本与配置的值。当 ELP 后端未发生更改时，连续调用返回的值保持相等；当 ELP 后端以可能为相同输入项目产生不同分析输出的方式发生变化时，返回的值不同。"
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/_project_fingerprint.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个元组，该元组确定性地、唯一地标识位于 proj_dir 的 Erlang 项目的当前状态。对相同的 proj_dir 两次调用 _project_fingerprint 返回相等元组，当且仅当在两次调用之间，proj_dir 下与 Erlang 分析相关的任何文件均未发生更改。文件更改是指对文件内容、字节大小或最后修改时间戳的任何修改。返回的元组不依赖于文件系统枚举顺序或其他非确定性因素。",
    "actual_behavior": "如果函数正常终止，返回值是一个元组 (backend, records)，满足：\n- backend = _elp_argv(), 标识当前 ELP 后端版本与配置。
\n- records = tuple(sorted(set( (os.path.relpath(p, root), os.stat(p).st_size, os.stat(p).st_mtime_ns) for p in paths )))\n 其中 root = os.path.abspath(proj_dir),\n  paths = [p for p in _iter_project_files(root, {'.erl', '.hrl'})] + [os.path.join(root, name) for name in _PROJECT_CONFIG_FILES if os.path.isfile(os.path.join(root, name))],\n 且 sorted() 按绝对路径字典序排序。
\n每条记录是一个 3 元组 (rel_path, size, mtime_ns)。记录已排序、去重，并包含 root 下递归找到的每一个 .erl/.hrl 源文件，以及位于 root 直属（非递归）且在 _PROJECT_CONFIG_FILES 中列出且存在的每一个项目配置文件。文件元数据（大小、修改时间）反映调用 os.stat() 那一刻的状态。在给定前置条件下，records 非空。
\n如果任何 os.stat()、os.path.isfile()、文件系统迭代或 os.path.abspath() 调用引发 OSError（例如 FileNotFoundError、PermissionError），异常将向上传播，函数不产生返回值；所有局部状态被丢弃。",
    "code_evidence": "Line 12",
    "trigger_condition": "指纹仅由文件大小和修改时间戳构建，忽略文件内容。因此，保持大小和 mtime_ns 不变的内容修改不会被检测到，这违反了要求：只要任何文件内容发生更改，指纹就必须改变（文件更改定义为对内容、大小或 mtime 的任何修改）。"
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--languages--erlang-py--_project_fingerprint",
  "source_file": "src/languages/erlang-py/_project_fingerprint.py",
  "function_name": "_project_fingerprint",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--_project_fingerprint.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--_project_fingerprint.md",
}
```

```
"probe_stdout": "CONFIRMED - 尽管内容不同，指纹仍然相等：fp1==fp2, size=49==49,
mtime_ns=1785172396729580385==1785172396729580385, but content differs\n",
  "trigger_summary": "指纹仅使用 (size, mtime_ns) 并忽略文件内容；相同长度且保留 mtime 的内容更改未被检测到。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T17:10:35.326233Z	3410
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T17:11:21.359453Z	3324

事件 1 — [llm_bbab17cb593b4a00a2529943d04743f9](#)

► parsed_json（完整）

```
{
  "post_condition": "如果函数正常终止，返回值是一个元组 (backend, records)，满足：\n- backend = _elp_argv(), 标识当前 ELP 后端版本与配置。
\n- records = tuple(sorted(set( (os.path.relpath(p, root), os.stat(p).st_size, os.stat(p).st_mtime_ns) for p in paths )))\n  其中 root = os.path.abspath(proj_dir),\n  paths = [p for p in _iter_project_files(root, ['.erl', '.hrl'])] + [os.path.join(root, name) for name in _PROJECT_CONFIG_FILES if os.path.isfile(os.path.join(root, name))],\n  且 sorted() 按绝对路径字典序排序。
\n每条记录是一个 3 元组 (rel_path, size, mtime_ns)。记录已排序、去重，并包含 root 下递归找到的每一个 .erl/.hrl 源文件，以及位于 root 直属（非递归）且在 _PROJECT_CONFIG_FILES 中列出且存在的每一个项目配置文件。文件元数据（大小、修改时间）反映调用 os.stat() 那一刻的状态。在给定前置条件下，records 非空。
\n如果任何 os.stat()、os.path.isfile()、文件系统迭代或 os.path.abspath() 调用引发 OSError（例如 FileNotFoundError、PermissionError），异常将向上传播，函数不产生返回值；所有局部状态被丢弃。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_bbab17cb593b4a00a2529943d04743f9_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_bbab17cb593b4a00a2529943d04743f9_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_bbab17cb593b4a00a2529943d04743f9_response.txt)

事件 2 — [llm_e16e55040a9c493db14bb421df1e888f](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir = '/tmp/erl' 包含一个文件 'src/mod.erl'，内容为 'a' (size=1)。令 mtime_ns = 1000000000。第一次调用后，将文件内容修改为 'b' (size=1)，并将 mtime_ns 精确设置为 1000000000（例如通过 os.utime）。当再次调用 _project_fingerprint 时，它返回相同的元组，因为 size 和 mtime_ns 相同，但文件内容发生了更改。",
  "offending_statements": "Line 12",
  "reason": "指纹仅由文件大小和修改时间戳构建，忽略文件内容。因此，保持大小和 mtime_ns 不变的内容修改不会被检测到，这违反了要求：只要任何文件内容发生更改，指纹就必须改变（文件更改定义为对内容、大小或 mtime 的任何修改）。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_e16e55040a9c493db14bb421df1e888f_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_e16e55040a9c493db14bb421df1e888f_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_e16e55040a9c493db14bb421df1e888f_response.txt) **复核动作**

- 核对 `probe` 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-106 — `src--languages--erlang-py--batch_extract`

- 四类结论： 可能有 Bug
- 分类依据： 沿用旧 `fm_agent/bug_list.md` 的逐条审计结论： 沿用旧清单的逐条审计结论。
- Validator： `not_confirmed`； attempts 1
- Phase / 模块： Phase 2 — Language Backend & Function Extraction / `language_handlers`； 原源码 `src/languages/erlang.py`
- 证据： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**： 返回一个字典，其键为 Erlang 源文件的绝对文件路径（字符串），值为（`function_id`: str, `body`: str）对的列表。每个 `function_id` 是一个规范化的、模块限定的名称，可用作完全限定名（FQN）。每个 `body` 是从源文件中提取的该函数的原始源代码文本。当 Erlang Language Platform 后端不可用或 `proj_dir` 中不包含任何可提取的 Erlang 函数时，返回一个空字典。
- **Actual behavior**： 如果函数正常返回，返回值为一个字典，其中每个键是 `proj_dir` 指定的目录内一个 Erlang 源文件的绝对文件路径字符串，每个值是由分析提取的元组（`function_id`, `body`）列表。如果发生异常（例如，由于 `proj_dir` 无效、I/O 错误或 `_analysis_or_empty` 的结果缺少 `functions` 属性而从 `_analysis_or_empty` 抛出），异常会传播，函数不返回值。
- **Code evidence**： 第 3 行： `return _analysis_or_empty(proj_dir).functions`
- **Trigger condition**： 规范要求当 Erlang Language Platform 后端不可用时，该函数返回一个空字典。然而，代码直接返回 `_analysis_or_empty` 的 `.functions` 属性，没有任何异常处理。如果后端不可用，`_analysis_or_empty` 可能抛出一个异常（如条件 A 中的示例所示），该异常会传播并阻止函数返回空字典。这违反了规范对该场景的要求。
- **Validator trigger**： 在未安装 ELP 的情况下对包含 `.erl` 文件的目录调用 `batch_extract`； `_analysis_or_empty` 捕获后端不可用异常，并返回一个空的 `ErlangAnalysis(functions={}, edges={})`，因此 `batch_extract` 按规范返回 `{}`。
- **Probe stdout**： `WARNING:root:ELP Erlang analysis unavailable for /tmp/erlang_probe_ov_1w_vt: [Errno 2] No such file or directory: 'elp' NOT CONFIRMED — batch_extract(...) returned: {} (no exception raised)`

► SPEC sidecar（完整）

```
{
  "signature": "batch_extract(proj_dir: str) -> dict",
  "pre_condition": "proj_dir 是一个字符串，指向可能包含 Erlang 源文件的文件系统目录",
  "post_condition": "返回一个字典，其键为 Erlang 源文件的绝对文件路径（字符串），值为（function_id: str, body: str）对的列表。每个 function_id 是一个规范化的、模块限定的名称，可用作完全限定名（FQN）。每个 body 是从源文件中提取的该函数的原始源代码文本。当 Erlang Language Platform 后端不可用或 proj_dir 中不包含任何可提取的 Erlang 函数时，返回一个空字典。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic 验证结果（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-
```



```
py/batch_extract.py",
"verdict": "MISMATCH",
"gaps": {
  "spec_claim": "返回一个字典，其键为 Erlang 源文件的绝对文件路径（字符串），值为（function_id: str, body: str）对
的列表。每个 function_id 是一个规范化的、模块限定的名称，可用作完全限定名（FQN）。每个 body 是从源文件中提取的该函数的原始
源代码文本。当 Erlang Language Platform 后端不可用或 proj_dir 中不包含任何可提取的 Erlang 函数时，返回一个空字典。",
  "actual_behavior": "如果函数正常返回，返回值为一个字典，其中每个键是 proj_dir 指定的目录内一个 Erlang 源文件的绝对
文件路径字符串，每个值是由分析提取的元组（function_id, body）列表。如果发生异常（例如，由于 proj_dir 无效、I/O 错误或
_analysis_or_empty 的结果缺少 `functions` 属性而从 _analysis_or_empty 抛出），异常会传播，函数不返回值。",
  "code_evidence": "Line 3: return _analysis_or_empty(proj_dir).functions",
  "trigger_condition": "规范要求当 Erlang Language Platform 后端不可用时，该函数返回一个空字典。然而，代码直接返回
_analysis_or_empty 的 .functions 属性，没有任何异常处理。如果后端不可用，_analysis_or_empty 可能抛出一个异常（如条件
A 中的示例所示），该异常会传播并阻止函数返回空字典。这违反了规范对该场景的要求。"
}
```

► Bug Validator 结果（完整）

```
{
  "id": "src--languages--erlang-py--batch_extract",
  "source_file": "src/languages/erlang.py",
  "function_name": "batch_extract",
  "confirmation_status": "not_confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--batch_extract.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--batch_extract.md",
  "probe_stdout": "WARNING:root:ELP Erlang analysis unavailable for /tmp/erlang_probe_ov_1w_vt: [Errno 2]
No such file or directory: 'elp'\nNOT CONFIRMED - batch_extract(...) returned: {} (no exception raised)",
  "trigger_summary": "在未安装 ELP 的情况下对包含 .erl 文件的目录调用 batch_extract; _analysis_or_empty 捕获后端不
可用异常，并返回一个空的 ErlangAnalysis(functions={}, edges={}), 因此 batch_extract 按规范返回 {}。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:33:55.023783Z	1251
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:34:10.627515Z	3951

事件 1 — [llm_f22fbf4007b94f2da81b378a9a33557c](#)

► parsed_json（完整）

```
{
  "post_condition": "如果函数正常返回，返回值为一个字典，其中每个键是 proj_dir 指定的目录内一个 Erlang 源文件的绝对文件
路径字符串，每个值是由分析提取的元组（function_id, body）列表。如果发生异常（例如，由于 proj_dir 无效、I/O 错误或
_analysis_or_empty 的结果缺少 `functions` 属性而从 _analysis_or_empty 抛出），异常会传播，函数不返回值。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_f22fbf4007b94f2da81b378a9a33557c_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_f22fbf4007b94f2da81b378a9a33557c_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_f22fbf4007b94f2da81b378a9a33557c_response.txt)

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir = \"\\/valid/dir\\\" (一个存在且包含 .erl 文件的目录), 而 Erlang Language Platform 后端不可用。代码调用 _analysis_or_empty, 它抛出一个异常 (例如, BackendUnavailableError), 异常传播, 因此函数未按规范要求返回一个空字典。",
  "offending_statements": "第 3 行: return _analysis_or_empty(proj_dir).functions",
  "reason": "规范要求当 Erlang Language Platform 后端不可用时, 该函数返回一个空字典。然而, 代码直接返回 _analysis_or_empty 的 .functions 属性, 没有任何异常处理。如果后端不可用, _analysis_or_empty 可能抛出一个异常 (如条件 A 中的示例所示), 该异常会传播并阻止函数返回空字典。这违反了规范对该场景的要求。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_d1a4b0e9886444e59add6e8247b3fb57_message_00_system.txt) · [user_prompt]
 (fm_agent/trace/payloads/llm_d1a4b0e9886444e59add6e8247b3fb57_message_01_user.txt) · [assistant_output]
 (fm_agent/trace/payloads/llm_d1a4b0e9886444e59add6e8247b3fb57_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足; 若可满足, 将其转为回归测试并修复实现。

FMA-MISMATCH-107 — src -- languages -- erlang -- py -- call_edges

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- 阶段 / 模块: Phase 2 — 语言后端与函数提取 / language_handlers; 原源码 src/languages/erlang.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** 返回一个字典, 将项目中所有 Erlang 函数的每个调用者 FQN 字符串映射到一组被调用者 FQN 字符串。调用者和被调用者 FQN 都遵循整个流程中使用的相同规范化、模块限定的命名约定。当 Erlang Language Platform 后端不可用或无法从项目中提取调用边时, 返回一个空字典。
- **Actual behavior:** 该函数返回一个字典, 包含从项目目录 proj_dir 导出的模块限定的 Erlang 调用边, 格式为注册表格式。如果 proj_dir 不存在、不包含可分析的 Erlang 源文件或发生内部错误, 则返回空字典。不传播任何异常, 也不修改任何可变的全局状态。形式化表示为: proj_dir str, let result = call_edges(proj_dir); then isinstance(result, dict) (filesystem_contains_analyzable_erlang(proj_dir) result = edges(analysis_or_empty(callgraph_project_root(proj_dir)))) (filesystem_contains_analyzable_erlang(proj_dir) result = {}) 执行正常终止。
- **Code evidence:** Line 3
- **Trigger condition:** 该代码在任何内部错误发生时 (通过 _analysis_or_empty) 无条件返回空字典, 但规范将返回空字典限制为仅在后端不可用或无法提取调用边的特定情况下。对于具有可调用函数且后端可用的有效项目, 内部错误会导致代码违反规范, 返回 {} 而不是预期的调用边字典。
- **Validator trigger:** _analysis_or_empty() 通过裸的 'except Exception' 捕获所有异常, 导致 call_edges() 对任何内部错误都返回 {} —— 而不仅仅是规范要求的后端不可用或没有调用边的情况。
- **Probe stdout:** WARNING:root:ELP Erlang analysis unavailable for /tmp/probe_erlang_2yqvwx7y: [Errno 2] No such file or directory: 'elp' WARNING:root:ELP Erlang analysis unavailable for /tmp/probe_erlang_2yqvwx7y: Simulated mid-analysis ELP internal error CONFIRMED — call_edges() silently returned {} when an internal error (RuntimeError) was raised during analysis. The spec allows {} only for backend-unavailable or no-call-edges cases, but _analysis_or_empty swallows ALL exceptions. result_normal={} result_bug={}

► SPEC sidecar (完整)

```
{
  "signature": "call_edges(proj_dir: str) -> dict",
  "pre_condition": "proj_dir 是一个字符串，指向可能包含具有静态可分析调用关系的 Erlang 源文件的文件系统目录",
  "post_condition": "返回一个字典，将项目中所有 Erlang 函数的每个调用者 FQN 字符串映射到一组被调用者 FQN 字符串。调用者和被调用者 FQN 都遵循整个流程中使用的相同规范化、模块限定的命名约定。当 Erlang Language Platform 后端不可用或无法从项目中提取调用边时，返回一个空字典。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/call_edges.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个字典，将项目中所有 Erlang 函数的每个调用者 FQN 字符串映射到一组被调用者 FQN 字符串。调用者和被调用者 FQN 都遵循整个流程中使用的相同规范化、模块限定的命名约定。当 Erlang Language Platform 后端不可用或无法从项目中提取调用边时，返回一个空字典。",
    "actual_behavior": "该函数返回一个字典，包含从项目目录 `proj_dir` 导出的模块限定的 Erlang 调用边，格式为注册表格式。如果 `proj_dir` 不存在、不包含可分析的 Erlang 源文件或发生内部错误，则返回空字典。不传播任何异常，也不修改任何可变的全局状态。形式化表示为： proj_dir str, let result = call_edges(proj_dir); then isinstance(result, dict) (filesystem_contains_analyzable_erlang(proj_dir) result = edges(analysis_or_empty(callgraph_project_root(proj_dir)))) (filesystem_contains_analyzable_erlang(proj_dir) result = {}) execution terminates normally.",
    "code_evidence": "Line 3",
    "trigger_condition": "该代码在任何内部错误发生时（通过 _analysis_or_empty）无条件返回空字典，但规范将返回空字典限制为仅在后端不可用或无法提取调用边的特定情况下。对于具有可调用函数且后端可用的有效项目，内部错误会导致代码违反规范，返回 {} 而不是预期的调用边字典。"
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--languages--erlang-py--call_edges",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/erlang-py/call_edges.py",
  "function_name": "call_edges",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--erlang-py--call_edges.py",
  "detail_file": "fm_agent/bug_validation/src--languages--erlang-py--call_edges.md",
  "probe_stdout": "WARNING:root:ELP Erlang analysis unavailable for /tmp/probe_erlang_2yqvwx7y: [Errno 2] No such file or directory: 'elp'\nWARNING:root:ELP Erlang analysis unavailable for /tmp/probe_erlang_2yqvwx7y: Simulated mid-analysis ELP internal error\n\nCONFIRMED – call_edges() silently returned {} when an internal error (RuntimeError) was raised during analysis. The spec allows {} only for backend-unavailable or no-call-edges cases, but _analysis_or_empty swallows ALL exceptions.\n\nresult_normal={} result_bug={}",
  "trigger_summary": "_analysis_or_empty() 通过裸露的 except Exception 捕获所有异常，导致 call_edges() 对于任何内部错误都返回 {} ——而不仅仅是规范要求的后端不可用或无调用边的情况。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:34:15.248595Z	1457
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:34:34.369771Z	4930

事件 1 — llm_ea6ec95ca2f04996aa6c9031e46b57c3

► parsed_json（完整）

```
{
  "post_condition": "该函数返回一个字典，包含从项目目录 `proj_dir` 导出的模块限定的 Erlang 调用边，格式为注册表。如果 `proj_dir` 不存在、不包含可分析的 Erlang 源文件或发生内部错误，则返回空字典。不传播任何异常，也不修改任何可变的全局状态。形式化表示为： $\forall \text{proj\_dir} \in \text{str}, \text{令 } \text{result} = \text{call\_edges}(\text{proj\_dir}); \text{则 } \text{isinstance}(\text{result}, \text{dict}) \wedge (\text{filesystem\_contains\_analyzable\_erlang}(\text{proj\_dir}) \Rightarrow \text{result} = \text{edges}(\text{analysis\_or\_empty}(\text{callgraph\_project\_root}(\text{proj\_dir})))) \wedge (\neg \text{filesystem\_contains\_analyzable\_erlang}(\text{proj\_dir}) \Rightarrow \text{result} = \{\}) \wedge \text{执行正常终止。}"$ 
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_ea6ec95ca2f04996aa6c9031e46b57c3_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_ea6ec95ca2f04996aa6c9031e46b57c3_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_ea6ec95ca2f04996aa6c9031e46b57c3_response.txt)

事件 2 — llm_4acab196eb104490b017828277346985

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir = '/tmp/valid_erlang_project', 包含一个简单的 Erlang 模块及其函数调用（例如，module1 调用 module2），其中 Erlang Language Platform 后端可用，但发生了瞬态内部错误（例如，底层分析库中的意外异常）。",
  "offending_statements": "Line 3",
  "reason": "该代码在任何内部错误发生时（通过 _analysis_or_empty）无条件返回空字典，但规范将返回空字典限制为仅在后端不可用或无法提取调用边的特定情况下。对于具有可调用函数且后端可用的有效项目，内部错误会导致代码违反规范，返回 {} 而不是预期的调用边字典。"
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_4acab196eb104490b017828277346985_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_4acab196eb104490b017828277346985_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_4acab196eb104490b017828277346985_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修复实现。

FMA-MISMATCH-114 — src--languages--python-py--batch_extract

- 四类结论：可能有 Bug
- 分类依据：probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。

- **Validator:** [confirmed](#); attempts [1](#)
- **Phase / 模块:** Phase 2 — Language Backend & Function Extraction / [language_handlers](#); 原源码 [src/languages/python.py](#)
- **证据:** [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** 返回一个字典，其键是 `proj_dir` 中发现的每个 Python 源文件的绝对文件路径 (`str`)。每个键的值是一个 `(func_name: str, func_body: str)` 2 元组的列表，该文件中定义的每个函数对应一个元组。不包含任何函数定义的文件对应一个空列表。当静态分析后端无法初始化时，返回一个空字典。
- **Actual behavior:** 执行后，若未发生异常，函数返回一个字典。设 `cg = CodeGraphExtractor.from_proj_dir(proj_dir)`。若 `cg` 不为 `None`，返回的字典 `R` 等于 `cg.get_functions_by_file('python', proj_dir)`；即，对于 `proj_dir` 下每个 Python 源文件的绝对文件路径 `p`，`R[p]` 是该文件中所有函数的 `(func_name, func_body)` 元组的列表。若 `cg` 为 `None`，则 `R = {}`。若在 `from_proj_dir` 或 `get_functions_by_file` 期间引发异常，异常将传播，`batch_extract` 不返回任何值。

形式逻辑： 正常终止： `( cg {CodeGraphExtractor, None}. cg = CodeGraphExtractor.from_proj_dir(proj_dir) ( (cg None R = cg.get_functions_by_file('python', proj_dir)) (cg = None R = {}) ) )`。 异常终止： 引发异常 `E`，函数栈展开。

- **Code evidence:** 第 4 行： `return cg.get_functions_by_file("python", proj_dir) if cg else {}`
- **Trigger condition:** 规约要求项目中的每个 Python 文件都是一个键，若没有函数则为空列表。实现依赖于 `get_functions_by_file`，它可能只包含至少包含一个函数的文件，从而遗漏没有函数的文件。
- **Validator trigger:** `batch_extract` 依赖于 `get_functions_by_file`，该函数查询 `codegraph` 以获取函数/方法节点；没有函数的文件不会产生任何行，并被静默地从结果字典中忽略，违反了规约中每个 Python 文件都必须是一个键的要求（如果没有函数，则为空列表）。
- **Probe stdout:** `CONFIRMED` — 1 个 Python 文件缺失于结果字典中： `['no_funcs.py']`。存在的有： `['has_funcs.py']`。规约要求每个 Python 文件都是一个键。

► SPEC sidecar (完整)

```
{
  "signature": "batch_extract(proj_dir: str) -> dict",
  "pre_condition": "proj_dir 是一个指向现有目录的路径",
  "post_condition": "返回一个字典，其键是 proj_dir 中发现的每个 Python 源文件的绝对文件路径 (str)。每个键的值是一个 (func_name: str, func_body: str) 2 元组的列表，该文件中定义的每个函数对应一个元组。不包含任何函数定义的文件对应一个空列表。当静态分析后端无法初始化时，返回一个空字典。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "CodeGraphExtractor.from_proj_dir",
      "signature": "from_proj_dir(proj_dir: str) -> CodeGraphExtractor | None",
      "pre_condition": "proj_dir 是一个指向现有目录的路径",
      "post_condition": "返回一个使用项目源索引初始化的 CodeGraphExtractor 实例，如果初始化失败则返回 None"
    },
    {
      "name": "get_functions_by_file",
      "signature": "get_functions_by_file(language: str, proj_dir: str) -> dict",
      "pre_condition": "language 是一个识别的语言键；proj_dir 是一个指向现有目录的路径",
      "post_condition": "返回一个字典，其键是匹配该语言的源文件的绝对文件路径 (str)，其值是该文件中每个函数的 (func_name: str, func_body: str) 元组列表"
    }
  ]
}
```



```
]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/python-py/batch_extract.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个字典，其键是 proj_dir 中发现的每个 Python 源文件的绝对文件路径（str）。每个键的值是一个（func_name: str, func_body: str）2 元组的列表，该文件中定义的每个函数对应一个元组。不包含任何函数定义的文件对应一个空列表。当静态分析后端无法初始化时，返回一个空字典。",
    "actual_behavior": "执行后，若未发生异常，函数返回一个字典。设 cg = CodeGraphExtractor.from_proj_dir(proj_dir)。若 cg 不为 None，返回的字典 R 等于 cg.get_functions_by_file('python', proj_dir)；即，对于 proj_dir 下每个 Python 源文件的绝对文件路径 p，R[p] 是该文件中所有函数的（func_name, func_body）元组的列表。若 cg 为 None，则 R = {}。若在 from_proj_dir 或 get_functions_by_file 期间引发异常，异常将传播，batch_extract 不返回任何值。\\n\\n形式逻辑：\\n正常终止：( cg {CodeGraphExtractor, None}. cg = CodeGraphExtractor.from_proj_dir(proj_dir) ( (cg None R = cg.get_functions_by_file('python', proj_dir)) (cg = None R = {}) ) )。\\n异常终止：引发异常 E，函数栈展开。",
    "code_evidence": "Line 4: return cg.get_functions_by_file(\"python\\", proj_dir) if cg else {}",
    "trigger_condition": "规约要求项目中的每个 Python 文件都是一个键，若没有函数则为空列表。实现依赖于 get_functions_by_file，它可能只包含至少包含一个函数的文件，从而遗漏没有函数的文件。"
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--languages--python-py--batch_extract",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/languages/python-py/batch_extract.py",
  "function_name": "batch_extract",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--languages--python-py--batch_extract.py",
  "detail_file": "fm_agent/bug_validation/src--languages--python-py--batch_extract.md",
  "probe_stdout": "CONFIRMED - 1 个 Python 文件缺失于结果字典中：['no_funcs.py']。存在的有：['has_funcs.py']。规约要求每个 Python 文件都是一个键。",
  "trigger_summary": "batch_extract 依赖于 get_functions_by_file，该函数查询 codegraph 以获取函数/方法节点；没有函数的文件不会产生任何行，并被静默地从结果字典中忽略，违反了规约中每个 Python 文件都必须是一个键的要求（如果没有函数，则为空列表）。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T16:38:18.348307Z	2609
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T16:38:48.209212Z	2710

事件 1 — [llm_db8fba5fb8a745eba9efe6c8ea52c338](#)

► parsed_json（完整）

```
{
  "post_condition": "执行后，若未发生异常，函数返回一个字典。设 cg = CodeGraphExtractor.from_proj_dir(proj_dir)。若 cg 不为 None，返回的字典 R 等于 cg.get_functions_by_file('python', proj_dir)；即，对于 proj_dir 下每个 Python 源文件的绝对文件路径 p，R[p] 是该文件中所有函数的 (func_name, func_body) 元组的列表。若 cg 为 None，则 R = {}。若在 from_proj_dir 或 get_functions_by_file 期间引发异常，异常将传播，batch_extract 不返回任何值。\\n\\n形式逻辑：\\n正常终止：(\\exists cg \\in \\{CodeGraphExtractor, None\\}. cg = CodeGraphExtractor.from_proj_dir(proj_dir) \\wedge ( (cg \\neq None \\wedge R = cg.get_functions_by_file('python', proj_dir)) \\vee (cg = None \\wedge R = \\{\\}) ) )。\\n异常终止：引发异常 E，函数栈展开。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_db8fba5fb8a745eba9efe6c8ea52c338_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_db8fba5fb8a745eba9efe6c8ea52c338_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_db8fba5fb8a745eba9efe6c8ea52c338_response.txt)

事件 2 — llm_310362618f694a21a98333ad2ed7f083

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir='/tmp/test_project'，其中包含一个单独的 Python 文件 'empty.py'，内容为 '# just a comment'（没有函数定义）。若 from_proj_dir 成功并返回一个非 None 的 CodeGraphExtractor，则 cg.get_functions_by_file('python', proj_dir) 返回 {}（一个没有键的空字典），因为没有找到任何函数。代码返回 {}。然而，规约要求字典包含键 '/tmp/test_project/empty.py'，值为 []。",
  "offending_statements": "第 4 行: return cg.get_functions_by_file(\"python\", proj_dir) if cg else {}",
  "reason": "规约要求项目中的每个 Python 文件都是一个键，若没有函数则为空列表。实现依赖于 get_functions_by_file，它可能只包含至少包含一个函数的文件，从而遗漏没有函数的文件。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_310362618f694a21a98333ad2ed7f083_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_310362618f694a21a98333ad2ed7f083_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_310362618f694a21a98333ad2ed7f083_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

Phase 3 — Pipeline Planning & Domain Knowledge

模块: pipeline_setup

FMA-MISMATCH-128 — src--pipeline_setup-py--collapse_phases_to_one

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块: Phase 3 — Pipeline Planning & Domain Knowledge / pipeline_setup; 原源码 src/pipeline_setup.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** 如果 phases.json 包含至少一个阶段，则 work_dir 下的文件将被重写，使得: (1) 'phases' 数组包含且仅包含一个阶段元素; (2) 单个阶段的 'modules' 数组是按 'phase' 升序排列的所有原始阶段中每个 'modules'

数组的串联，保持每个原始阶段的 'modules' 数组内模块的相对顺序；(3) 单个阶段的 'description' 是一个字符串，通过将一个空行分隔符连接所有原始阶段按 'phase' 升序排列的去除了首尾空白后的非空描述来形成；(4) 单个阶段的 'depends_on_phases' 是一个空列表。此外，如果在 '<work_dir>/spec_prompts/domain_context/' 下存在一个或多个 'phase_types.txt' 文件，则它们去除首尾空白后的内容按阶段升序（用空行分隔）连接到同一目录下的 'phase_01_types.txt' 文件中，然后该目录中所有其他 'phase_types.txt' 文件被删除。如果 phases.json 包含零个阶段，则不会发生文件修改。

- **Actual behavior:** 如果 'phases' 列表为空，文件系统保持不变，函数立即返回。否则，在正常执行（无异常）之后：

1. 位于 phases_path (work_dir/phases.json) 的文件被覆写为一个 JSON 对象，其 'phases' 键包含一个单元素列表，该列表包含一个合并后的阶段对象 M。M 拥有与原始排序列表中第一个阶段相同的键（包括其原始 'phase' 编号，该编号不一定为 1），并具有以下修改：
 - 'name' 被设置为 "Unified Analysis Phase"。
 - 'description' 是所有原始阶段中每个非空且去除首尾空白后的 'description' 字符串按排序顺序以 "\n\n" 分隔的串联。如果没有非空的，此字段存在但为空。
 - 'modules' 是一个列表，包含来自每个原始阶段的每个模块字典，保留阶段的顺序以及每个阶段内模块的顺序。
 - 'depends_on_phases' 被设置为空列表 []。
2. 在目录 work_dir/spec_prompts/domain_context 中：
 - a. 令 T 为每个原始阶段（使用其 'phase' 编号，补零至两位）的路径 phase_{02d}types.txt 的集合。
 - b. 如果至少存在一个这样的文件，则会创建（或覆写）一个文件 phase_01_types.txt，其中包含 T 中所有现有文件去除首尾空白后的内容，按顺序以 "\n\n" 分隔符和一个尾随换行符串联。该目录中匹配通配模式 "phase*_types.txt" 的所有其他文件（包括那些不在 T 中的）均被删除，但新写入的 phase_01_types.txt 除外。在此步骤之后，匹配该模式的唯一文件是包含合并内容的 phase_01_types.txt。
 - c. 如果 T 中没有文件存在，则不创建新文件且不删除任何文件；目录保持原样。

所有操作假定前置条件所要求的现有文件和目录都是可读/可写的。如果发生 I/O 错误（例如，无法写入 phases.json 或创建合并后的类型文件），函数会引发异常，且文件系统可能处于部分修改的状态。

- **Code evidence:** 第 10 行: f'Phase {phase['phase']} ({phase['name']}): {phase_description}'
- **Trigger condition:** 规范要求合并后的描述仅为所有阶段去除首尾空白后的非空描述，并用空行连接。代码错误地在每个描述前插入了一个前缀（例如 'Phase 1 (Test): '），改变了内容。
- **Validator trigger:** 代码在每个描述前添加了 'Phase N (Name): ' 前缀，而不是仅用空行连接去除首尾空白后的非空描述。
- **Probe stdout:** CONFIRMED — 描述包含多余的阶段前缀: actual: 'Phase 1 (Analysis Phase): First phase handles initial setup\n\nPhase 2 (Verification Phase): Second phase verifies the results\n\nPhase 3 (Cleanup Phase): Final phase does cleanup' | expected: 'First phase handles initial setup\n\nSecond phase verifies the results\n\nFinal phase does cleanup'

► SPEC sidecar（完整）

```
{
  "signature": "_collapse_phases_to_one(work_dir)",
  "pre_condition": "work_dir 是一个字符串路径，指向一个包含可读的 phases.json 文件的目录，该文件内容符合 phases.json 模式：顶层对象有一个 'phases' 键，其值是一个阶段对象列表，每个对象至少包含 'phase'（int，从 1 开始索引），并可选择包含 'name'（str）、'description'（str）、'modules'（模块字典列表）和 'depends_on_phases'（整数列表），
  "post_condition": "如果 phases.json 包含至少一个阶段，则 work_dir 下的文件将被重写，使得：(1) 'phases' 数组包含且仅包含一个阶段元素；(2) 单个阶段的 'modules' 数组是按 'phase' 升序排列的所有原始阶段中每个 'modules' 数组的串联，保持每个原始阶段的 'modules' 数组内模块的相对顺序；(3) 单个阶段的 'description' 是一个字符串，通过将一个空行分隔符连接所有原始阶段按 'phase' 升序排列的去除了首尾空白后的非空描述来形成；(4) 单个阶段的 'depends_on_phases' 是一个空列表。此外，如果在 '<work_dir>/spec_prompts/domain_context/' 下存在一个或多个 'phase*_types.txt' 文件，则它们去除首尾空白后的内容按阶段升序（用空行分隔）连接到同一目录下的 'phase_01_types.txt' 文件中，然后该目录中所有其他 'phase*_types.txt' 文件被
```

```
删除。如果 phases.json 包含零个阶段，则不会发生文件修改。"
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/pipeline_setup-
py/_collapse_phases_to_one.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "如果 phases.json 包含至少一个阶段，则 work_dir 下的文件将被重写，使得：(1) 'phases' 数组包含且仅
包含一个阶段元素；(2) 单个阶段的 'modules' 数组是按 'phase' 升序排列的所有原始阶段中每个 'modules' 数组的串联，保持每个
原始阶段的 'modules' 数组内模块的相对顺序；(3) 单个阶段的 'description' 是一个字符串，通过将一个空行分隔符连接所有原始阶
段按 'phase' 升序排列的去除了首尾空白后的非空描述来形成；(4) 单个阶段的 'depends_on_phases' 是一个空列表。此外，如果在
'<work_dir>/spec_prompts/domain_context/' 下存在一个或多个 'phase_*_types.txt' 文件，则它们去除首尾空白后的内容按
阶段升序（用空行分隔）连接到同一目录下的 'phase_01_types.txt' 文件中，然后该目录中所有其他 'phase_*_types.txt' 文件被
删除。如果 phases.json 包含零个阶段，则不会发生文件修改。",
    "actual_behavior": "如果 'phases' 列表为空，文件系统保持不变，函数立即返回。否则，在正常执行（无异常）之后：\n\n1.
位于 phases_path(work_dir/phases.json) 的文件被覆写为一个 JSON 对象，其 'phases' 键包含一个单元列表，该列表包含一个
合并后的阶段对象 M。M 拥有与原始排序列表中第一个阶段相同的键（包括其原始 'phase' 编号，该编号不一定为 1），并具有以下修
改：\n  - 'name' 被设置为 \"Unified Analysis Phase\"。
  - 'description' 是所有原始阶段中每个非空且去除首尾空白
后的 'description' 字符串按排序顺序以 \"\\n\\n\\n\" 分隔的串联。如果没有非空的，此字段存在但为空。
  - 'modules' 是一个
列表，包含来自每个原始阶段的每个模块字典，保留阶段的顺序以及每个阶段内模块的顺序。
  - 'depends_on_phases' 被设置为空列
表 []。
\n\n2. 在目录 work_dir/spec_prompts/domain_context 中：
  a. 令 T 为每个原始阶段（使用其 'phase' 编号，
补零至两位）的路径 phase_{:02d}_types.txt 的集合。
  b. 如果至少存在一个这样的文件，则会创建（或覆写）一个文件
phase_01_types.txt，其中包含 T 中所有现有文件去除首尾空白后的内容，按顺序以 \"\\n\\n\\n\" 分隔符和一个尾随换行符串联。该目
录中匹配通配模式 \"phase_*_types.txt\" 的所有其他文件（包括那些不在 T 中的）均被删除，但新写入的 phase_01_types.txt 除
外。在此步骤之后，匹配该模式的唯一文件是包含合并内容的 phase_01_types.txt。
  c. 如果 T 中没有文件存在，则不创建新文件
且不删除任何文件；目录保持原样。
\n\n所有操作假定前置条件所要求的现有文件和目录都是可读/可写的。如果发生 I/O 错误（例如，无法
写入 phases.json 或创建合并后的类型文件），函数会引发异常，且文件系统可能处于部分修改的状态。",
    "code_evidence": "Line 10: f\"Phase {phase['phase']} ({phase['name']}): {phase['description']}\"",
    "trigger_condition": "规范要求合并后的描述仅为所有阶段去除首尾空白后的非空描述，并用空行连接。代码错误地在每个描述前插
入了一个前缀（例如 'Phase 1 (Test): '），改变了内容。"
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--pipeline_setup-py--_collapse_phases_to_one",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/pipeline_setup-
py/_collapse_phases_to_one.py",
  "function_name": "_collapse_phases_to_one",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--pipeline_setup-py--_collapse_phases_to_one.py",
  "detail_file": "fm_agent/bug_validation/src--pipeline_setup-py--_collapse_phases_to_one.md",
  "probe_stdout": "CONFIRMED - 描述包含多余的阶段前缀: actual: 'Phase 1 (Analysis Phase): First phase handles
initial setup\\n\\n\\nPhase 2 (Verification Phase): Second phase verifies the results\\n\\n\\nPhase 3 (Cleanup
Phase): Final phase does cleanup' | expected: 'First phase handles initial setup\\n\\n\\nSecond phase
verifies the results\\n\\n\\nFinal phase does cleanup'",
  "trigger_summary": "代码在每个描述前添加了 'Phase N (Name): ' 前缀，而不是仅用空行连接去除首尾空白后的非空描述。"
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:31:37.357893Z	5332
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:32:42.560868Z	4702

事件 1 — [llm_30322ce94dc34ad8b2b58a76cafb1c65](#)

► [parsed_json](#) (完整)

- 载荷: [\[system_prompt\]](#)

([fm_agent/trace/payloads/llm_30322ce94dc34ad8b2b58a76cafb1c65_message_00_system.txt](#)) · [\[user_prompt\]](#)

([fm_agent/trace/payloads/llm_30322ce94dc34ad8b2b58a76cafb1c65_message_01_user.txt](#)) · [\[assistant_output\]](#)

([fm_agent/trace/payloads/llm_30322ce94dc34ad8b2b58a76cafb1c65_response.txt](#))

事件 2 — [llm_7277412f28514647ae25196e276739fb](#)

► [parsed_json](#) (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "phases.json 包含一个具有非空描述的单一阶段：{\\\"phases\\\": [{\\\"phase\\\": 1, \\\"name\\\": \\\"Test\\\", \\\"description\\\": \\\"A description.\\\", \\\"modules\\\": []}]}。",
  "offending_statements": "Line 10: f\\\"Phase {phase['phase']} ({phase['name']}): {phase_description}\\\"",
  "reason": "规范要求合并后的描述应仅为所有阶段经过去空格处理的非空描述，并用空行连接。代码错误地在每个描述前插入了前缀（例如 'Phase 1 (Test): '），从而改变了内容。"
}
```

- 载荷: [\[system_prompt\]](#)

([fm_agent/trace/payloads/llm_7277412f28514647ae25196e276739fb_message_00_system.txt](#)) · [\[user_prompt\]](#)

([fm_agent/trace/payloads/llm_7277412f28514647ae25196e276739fb_message_01_user.txt](#)) · [\[assistant_output\]](#)

([fm_agent/trace/payloads/llm_7277412f28514647ae25196e276739fb_response.txt](#)) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-131 — [src--pipeline_setup.py--_domain_context_complete](#)

- 四类结论：可能有 Bug
- 分类依据：沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator：confirmed；尝试次数 1
- Phase / 模块：Phase 3 — 管道规划与领域知识 / [pipeline_setup](#)；原源码 [src/pipeline_setup.py](#)
- 证据：[抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- SPEC 声明：当且仅当满足以下所有条件时返回 True：(1) work_dir 包含一个有效的 phases.json 文件——一个可读、可解析的 JSON 文件，其顶层对象有一个 'phases' 数组，其中每个元素都有一个非 None 整数 'phase' 键；(2) 文件 work_dir/spec_prompts/domain_context/engine_overview.txt 存在；(3) 对于 'phases' 数组中存在的每个阶段编号 P，文件 work_dir/spec_prompts/domain_context/phase_P_types.txt 存在，其中 P 以零填充的两位整数格式表示。如果上述任一条件不满足，包括 phases.json 缺失、不可读、非有效 JSON，或其 'phases' 数组包含一个 'phase' 键缺失或为 None 的元素，则返回 False。该函数不会修改文件系统上的任何文件。

- **实际行为：** 文件系统状态与调用之前完全一致；没有文件被创建、修改或删除，也没有外部资源被改变。该函数要么返回 True 要么返回 False，且不引发任何异常（所有内部异常均被捕获）。该函数当且仅当满足以下所有条件时返回 True，否则返回 False：(1) 位于路径 `os.path.join(work_dir, 'phases.json')` 的文件存在、可读，并包含语法有效的 JSON（由 `json_file_is_valid` 判定）。(2) 位于路径 `os.path.join(work_dir, 'spec_prompts', 'domain_context', 'engine_overview.txt')` 的文件存在。(3) 在成功打开和解析 `phases.json` 文件后，得到的 JSON 对象包含一个键 `'phases'`，其值是可迭代的，并且对于该序列中的每个元素，该元素在键 `'phase'` 下具有非 None 值（通常为整数），并且位于路径 `os.path.join(work_dir, 'spec_prompts', 'domain_context', f'phase{phase:02d}_types.txt')` 的文件存在。形式化地：(返回值为 True) $\leftrightarrow (\text{json_file_is_valid}(\text{phases_path}) \wedge \text{os.path.exists}(\text{engine_overview_path}) \wedge \forall \text{phase} \in \text{parse}(\text{json.load}(\text{open}(\text{phases_path}))).\text{get}('phases', []), \text{phase.get}('phase') \neq \text{None} \wedge \text{os.path.exists}(\text{phase_types_path}(\text{phase.get}('phase')))))$ ，其中 `phases_path = os.path.join(work_dir, 'phases.json')`，`engine_overview_path = os.path.join(domain_dir, 'engine_overview.txt')`，`domain_dir = os.path.join(work_dir, 'spec_prompts', 'domain_context')`，`phase_types_path(n) = os.path.join(domain_dir, f'phase{n:02d}_types.txt')`。如果不满足上述任一条件，函数返回 False；例如，缺失文件、无效 JSON、缺失 `'phases'` 键、阶段元素缺失 `'phase'` 键或缺失阶段类型文件，均返回 False。只有当所有检查都通过时，第 21 行的 `return` 语句才会被到达。
- **代码证据：** 第 14 行：`for phase in phases_data.get("phases", []):`
- **触发条件：** 如果顶层 JSON 值不是字典（例如是数组），`_json_file_is_valid` 返回 True，但 `json.load` 返回一个列表，在该列表上调用 `.get` 会引发一个未捕获的 `AttributeError`，导致函数传播异常而不是返回 False，违反了规范。
- **Validator 触发：** 当 `phases.json` 包含一个顶层 JSON 数组时，`_json_file_is_valid` 返回 True，但 `phases_data.get()` 失败并引发 `AttributeError`，传播异常而不是返回 False。
- **Probe stdout：** CONFIRMED — 实际：引发 `AttributeError` | 预期：`return False`

► SPEC sidecar（完整）

```
{
  "signature": "_domain_context_complete(work_dir)",
  "pre_condition": "work_dir 是一个目录路径",
  "post_condition": "当且仅当满足以下所有条件时返回 True：(1) work_dir 包含一个有效的 phases.json 文件——一个可读、可解析的 JSON 文件，其顶层对象有一个 'phases' 数组，其中每个元素都有一个非 None 整数 'phase' 键；(2) 文件 work_dir/spec_prompts/domain_context/engine_overview.txt 存在；(3) 对于 'phases' 数组中存在的每个阶段编号 P，文件 work_dir/spec_prompts/domain_context/phase_P_types.txt 存在，其中 P 以零填充的两位整数格式表示。如果上述任一条件不满足，包括 phases.json 缺失、不可读、非有效 JSON，或其 'phases' 数组包含一个 'phase' 键缺失或为 None 的元素，则返回 False。该函数不会修改文件系统上的任何文件。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "_json_file_is_valid",
      "signature": "_json_file_is_valid(path)",
      "pre_condition": "path 是一个表示文件系统路径的字符串，该路径可能对应或不对应一个存在的、可读的文件",
      "post_condition": "如果位于 path 的文件存在、可读且包含语法有效的 JSON，则返回 True。如果文件不存在、无法读取或其内容不是有效的 JSON，则返回 False。不会修改文件或文件系统。"
    }
  ]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/pipeline_setup-  
py/_domain_context_complete.py",
```

```
"verdict": "MISMATCH",
"gaps": {
  "spec_claim": "当且仅当满足以下所有条件时返回 True：(1) work_dir 包含一个有效的 phases.json 文件——一个可读、可解析的 JSON 文件，其顶层对象有一个 'phases' 数组，其中每个元素都有一个非 None 整数 'phase' 键；(2) 文件 work_dir/spec_prompts/domain_context/engine_overview.txt 存在；(3) 对于 'phases' 数组中存在的每个阶段编号 P，文件 work_dir/spec_prompts/domain_context/phase_P_types.txt 存在，其中 P 以零填充的两位整数格式表示。如果上述任一条件不满足，包括 phases.json 缺失、不可读、非有效 JSON，或其 'phases' 数组包含一个 'phase' 键缺失或为 None 的元素，则返回 False。该函数不会修改文件系统上的任何文件。",
  "actual_behavior": "文件系统状态与调用之前完全一致；没有文件被创建、修改或删除，也没有外部资源被改变。该函数要么返回 True 要么返回 False，且不引发任何异常（所有内部异常均被捕获）。该函数当且仅当满足以下所有条件时返回 True，否则返回 False：(1) 位于路径 os.path.join(work_dir, 'phases.json') 的文件存在、可读，并包含语法有效的 JSON（由 _json_file_is_valid 判定）。(2) 位于路径 os.path.join(work_dir, 'spec_prompts', 'domain_context', 'engine_overview.txt') 的文件存在。(3) 在成功打开和解析 phases.json 文件后，得到的 JSON 对象包含一个键 'phases'，其值是可迭代的，并且对于该序列中的每个元素，该元素在键 'phase' 下具有非 None 值（通常为整数），并且位于路径 os.path.join(work_dir, 'spec_prompts', 'domain_context', f'phase_{phase:02d}_types.txt') 的文件存在。形式化地：(返回值为 True) ⇔ ( _json_file_is_valid(phases_path) ∧ os.path.exists(engine_overview_path) ∧ ∀ phase ∈ parse(json.load(open(phases_path))).get('phases', []), phase.get('phase') ≠ None ∧ os.path.exists(phase_types_path(phase.get('phase')))) ), 其中 phases_path = os.path.join(work_dir, 'phases.json'), engine_overview_path = os.path.join(domain_dir, 'engine_overview.txt'), domain_dir = os.path.join(work_dir, 'spec_prompts', 'domain_context'), phase_types_path(n) = os.path.join(domain_dir, f'phase_{n:02d}_types.txt')。如果不满足上述任一条件，函数返回 False；例如，缺失文件、无效 JSON、缺失 'phases' 键、阶段元素缺失 'phase' 键或缺失阶段类型文件，均返回 False。只有当所有检查都通过时，第 21 行的 return 语句才会被到达。",
  "code_evidence": "Line 14: for phase in phases_data.get(\"phases\", []):",
  "trigger_condition": "如果顶层 JSON 值不是字典（例如是数组），_json_file_is_valid 返回 True，但 json.load 返回一个列表，在该列表上调用 .get 会引发一个未捕获的 AttributeError，导致函数传播异常而不是返回 False，违反了规范。"
}
```

► Bug Validator result（完整）

```
{
  "id": "src--pipeline_setup-py--_domain_context_complete",
  "source_file": "src/pipeline_setup-py/_domain_context_complete.py",
  "function_name": "_domain_context_complete",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--pipeline_setup-py--_domain_context_complete.py",
  "detail_file": "fm_agent/bug_validation/src--pipeline_setup-py--_domain_context_complete.md",
  "probe_stdout": "CONFIRMED - 实际: 引发 AttributeError | 预期: return False",
  "trigger_summary": "当 phases.json 包含一个顶层 JSON 数组时，_json_file_is_valid 返回 True，但 phases_data.get() 失败并引发 AttributeError，传播异常而不是返回 False。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:31:35.352913Z	2354
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:31:57.581934Z	3884

事件 1 — [llm_a63d6a6abf9d4b96972d9ccfc81e2d68](#)

► parsed_json（完整）

```
{
  "post_condition": "文件系统状态与调用之前完全一致；没有文件被创建、修改或删除，也没有外部资源被改变。该函数要么返回 True 要么返回 False，且不引发任何异常（所有内部异常均被捕获）。该函数当且仅当满足以下所有条件时返回 True，否则返回 False：(1) 位于路径 os.path.join(work_dir, 'phases.json') 的文件存在、可读，并包含语法有效的 JSON（由 _json_file_is_valid 判
```

定)。(2) 位于路径 `os.path.join(work_dir, 'spec_prompts', 'domain_context', 'engine_overview.txt')` 的文件存在。(3) 在成功打开和解析 `phases.json` 文件后,得到的 JSON 对象包含一个键 `'phases'`,其值是可迭代的,并且对于该序列中的每个元素,该元素在键 `'phase'` 下具有非 `None` 值(通常为整数),并且位于路径 `os.path.join(work_dir, 'spec_prompts', 'domain_context', f'phase_{phase:02d}_types.txt')` 的文件存在。形式化地:(返回值为 `True`) $\leftrightarrow$ (`_json_file_is_valid(phases_path) \wedge os.path.exists(engine_overview_path) \wedge \forall phase \in parse(json.load(open(phases_path))).get('phases', [])`, `phase.get('phase') \neq None \wedge os.path.exists(phase_types_path(phase.get('phase')))`),其中 `phases_path = os.path.join(work_dir, 'phases.json')`, `engine_overview_path = os.path.join(domain_dir, 'engine_overview.txt')`, `domain_dir = os.path.join(work_dir, 'spec_prompts', 'domain_context')`, `phase_types_path(n) = os.path.join(domain_dir, f'phase_{n:02d}_types.txt')`。如果不满足上述任一条件,函数返回 `False`;例如,缺失文件、无效 JSON、缺失 `'phases'` 键、阶段元素缺失 `'phase'` 键或缺失阶段类型文件,均返回 `False`。只有当所有检查都通过时,第 21 行的 `return` 语句才会被到达。"

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_a63d6a6abf9d4b96972d9ccfc81e2d68_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_a63d6a6abf9d4b96972d9ccfc81e2d68_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_a63d6a6abf9d4b96972d9ccfc81e2d68_response.txt)

事件 2 — `llm_805360204eed43559bea09193928ca25`

► `parsed_json` (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "work_dir 中 phases.json 包含有效的 JSON 数组 `[]`, 且 `work_dir/spec_prompts/domain_context/engine_overview.txt` 存在。",
  "offending_statements": "第 14 行: for phase in phases_data.get(\"phases\", []):",
  "reason": "如果顶层 JSON 值不是字典 (例如是数组), _json_file_is_valid 返回 True, 但 json.load 返回一个列表, 在该列表上调用 .get 会引发一个未捕获的 AttributeError, 导致函数传播异常而不是返回 False, 违反了规范。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_805360204eed43559bea09193928ca25_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_805360204eed43559bea09193928ca25_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_805360204eed43559bea09193928ca25_response.txt) **复核动作**

- 核对 `probe` 前置条件是否可由真实调用路径满足; 若可满足, 将其转为回归测试并修实现。

FMA-MISMATCH-132 — `src--pipeline_setup.py--ensure_source_files_in_phases`

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 `fm_agent/bug_list.md` 的逐条审计结论: 沿用旧清单的逐条审计结论。
- **Validator:** `confirmed`; attempts 1
- **Phase / 模块:** Phase 3 — Pipeline Planning & Domain Knowledge / `pipeline_setup`; 原源码 `src/pipeline_setup.py`
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** 当 `required_source_files` 为 `None` 或空时, 返回 `{'forced': [], 'augmented': {}, 'augmented_modules': []}` 而不读取或修改 `phases_json`。否则, 保证返回后 `required_source_files` 中的每个路径 (反斜杠目录分隔符规范化为正斜杠) 都会出现在 `phases.json` 文件中恰好一个模块的 `source_files` 列表中。返回一个字典, 其中: `'forced'` 是在调用前尚未出现在任何阶段模块的 `source_files` 中的所需路径列表 (当所有路径都已存在时为空); `'augmented'` 将接收阶段 (阶段编号最小的阶段) 的数字阶段标识符映射到添加到它的路径列表; `'augmented_modules'` 列出每个接收到新文件的模块, 每个条目包含模块的阶段编号、模块名称和添加的文件

路径。如果 `phases.json` 文件最初不包含任何阶段，则创建一个编号为 1 的单一阶段，其中包含一个模块，其 `source_files` 列表包含 `required_source_files` 中的所有路径。接收模块的 `description` 字段会被扩展以包含一条注释，引用添加的源文件（不会重复任何已存在的描述内容）。不会删除任何现有阶段，不会丢失任何现有模块的源文件，并且预先存在的阶段编号保持不变。

- **Actual behavior:** 如果函数在不引发异常的情况下完成，则以下成立：令 `F_before` 为调用前 `phases_json` 处文件的 JSON 内容。
- 如果 `required_source_files` 为假值 (`None` 或空序列)：函数返回 `{'forced': [], 'augmented': {}, 'augmented_modules': []}` 并且文件不变 (`F_after = F_before`)。
- 否则，令 `required_norm = {` 将 `p` 中的反斜杠替换为正斜杠，其中 `p` 来自 `required_source_files` `}` 且 `existing_norm = {` 将 `p` 中的反斜杠替换为正斜杠，其中 `p` 来自 `F_before` 中任何阶段任何模块所列的所有 `source_files` `}`。
 - 如果 `required_norm ⊆ existing_norm`，函数返回相同的空字典且文件不变。
 - 否则，`missing = [ sf for sf in required_source_files if sf with backslashes replaced not in existing_norm ]`（保持顺序）。文件被转换：选择 `F_before` 中最早的阶段（按数字 `'phase'` 键）；如果不存在阶段，则创建一个新阶段，其 `'phase': 1`，名称 `'Entry Points'`，描述 `'Entry-point source files.'`，空 `modules`，空 `depends_on_phases`。在该阶段的 `'modules'` 列表中，如果为空，则附加一个新模块，名称为 `'entry_points'`，空 `description`，空 `source_files`。该列表中的第一个模块的 `'source_files'` 通过追加 `missing` 中的所有路径（保持原样，不进行标准化）来扩展，其 `'description'` 更新为 `_merge_descriptions(old_description, 'Includes required entry-point source file(s): ' + ', '.join(missing) + '.')`。不会修改其他阶段或模块。`phases_json` 处的文件被覆盖为生成的 JSON 对象 (`F_after`)。函数返回：`{'forced': missing, 'augmented': {earliest_phase_number: list(missing)}, 'augmented_modules': [{phase: earliest_phase_number, 'module': name_of_first_module, 'added_files': list(missing)}]}` 其中 `earliest_phase_number` 是所选最早阶段的 `'phase'` 值。如果发生异常（例如，文件 I/O 错误、无效 JSON），函数引发异常，并且不对返回值或文件状态做出任何保证。

正式地，对于所有正常执行： $\exists r$ 使得 $\text{ReturnValue} = r \wedge ((\text{required_source_files} = \text{None} \vee \text{required_source_files} = []) \Rightarrow (r = \{\text{forced}: [], \text{augmented}: \{\}, \text{augmented_modules}: []\} \wedge \text{file_unchanged}))$

$\wedge ((\text{required_source_files} \neq \text{None} \wedge \text{required_source_files} \neq []) \Rightarrow \text{let req_norm} = \{ \text{normalize}(p) \mid p \in \text{required_source_files} \} \text{ in let exist_norm} = \{ \text{normalize}(q) \mid q \in \text{all_file_source_files}(\text{F_before}) \} \text{ in } (\text{req_norm} \subseteq \text{exist_norm} \Rightarrow r = \{\text{forced}: [], \text{augmented}: \{\}, \text{augmented_modules}: []\} \wedge \text{file_unchanged}))$

```
 $\wedge ((\text{req\_norm} \not\subseteq \text{exist\_norm}$ 
 $\Rightarrow \text{let missing} = [ p \in \text{required\_source\_files} \mid \text{normalize}(p) \not\in \text{exist\_norm} ] \text{ in}$ 
 $\text{let phases\_before} = \text{F\_before.phases}$  (or [] if absent) in
 $\text{let earliest\_phase} = (\text{if phases\_before} = [] \text{ then new\_phase}(1) \text{ else argmin}_{\{p \in \text{phases\_before}\}} p.\text{phase})$ 
in
 $\text{let modules\_after} = (\text{if earliest\_phase.modules} = [] \text{ then } [\text{new\_module}(\text{'entry\_points'}, '', [])] \text{ else}$ 
 $\text{earliest\_phase.modules}) \text{ in}$ 
 $\text{let mod} = \text{modules\_after}[0] \text{ in}$ 
 $\text{let mod\_desc\_before} = \text{mod.description}$  (or '' if absent) in
 $\text{let mod\_files\_before} = \text{mod.source\_files}$  (or [] if absent) in
 $\text{file\_after} = \text{file\_before}$  with earliest\_phase replaced by:
 $\text{earliest\_phase}[\text{modules} \rightarrow \text{modules\_after}[0 \rightarrow (\text{mod\_files\_before} ++ \text{missing}, \text{description} \rightarrow$ 
 $\text{\_merge\_descriptions}(\text{mod\_desc\_before}, \text{note}))]]$ 
 $\text{where note} = \text{'Includes required entry-point source file(s): ' + join(missing, ', ') + '.'}$ 
 $\wedge r = \{\text{forced}: \text{missing}, \text{augmented}: \{\text{earliest\_phase.phase}: \text{missing}\}, \text{augmented\_modules}: [\{\text{phase}: \text{earliest\_phase.phase}, \text{module}: \text{mod.name}, \text{added\_files}: \text{missing}\}]\}$ 
 $\wedge \text{file}(\text{phases\_json}) = \text{file\_after}$ 
 $)$ 
```

) 其中 `normalize(s)` = 将 `s` 中的所有反斜杠字符替换为正斜杠。

- **Code evidence:** Line 27: missing = [sf for sf in required_source_files if sf.replace("\", "/") not in listed] Line 28: if not missing: Line 29: return {"forced": [], "augmented": {}, "augmented_modules": []}
- **Trigger condition:** 规范保证返回后每个所需源文件恰好出现在一个模块中。当所需文件已存在于多个模块中时，代码不会检测或修复此重复；它只是不加修改地提前返回。这使文件处于违反后置条件的状态。
- **Validator trigger:** 当所需源文件已存在于多个模块中时，代码不加去重地提前返回，使文件处于违反“恰好一个模块”后置条件的状态。
- **Probe stdout:** CONFIRMED — actual: forced=[], foo.py appears in 2 modules after call | expected (spec): exactly 1 module

► SPEC sidecar (完整)

```
{
  "signature": "_ensure_source_files_in_phases(phases_json, required_source_files) -> dict",
  "pre_condition": "phases_json 是一个指向有效 phases.json 文件的路径，其 schema 符合 phase_03_types 规范，其中 phases 是一个有序的对象列表，每个对象都包含一个数字 'phase' 键和一个 'modules' 列表；required_source_files 是 None 或一个文件路径字符串序列",
  "post_condition": "当 required_source_files 为 None 或空时，返回 {'forced': [], 'augmented': {}}, 'augmented_modules': []} 而不读取或修改 phases_json。否则，保证返回后 required_source_files 中的每个路径（反斜杠目录分隔符规范化为正斜杠）都会出现在 phases.json 文件中恰好一个模块的 source_files 列表中。返回一个字典，其中：'forced' 是在调用前尚未出现在任何阶段模块的 source_files 中的所需路径列表（当所有路径都已存在时空）；'augmented' 将接收阶段（阶段编号最小的阶段）的数字阶段标识符映射到添加到它的路径列表；'augmented_modules' 列出每个接收到新文件的模块，每个条目包含模块的阶段编号、模块名称和添加的文件路径。如果 phases.json 文件最初不包含任何阶段，则创建一个编号为 1 的单一阶段，其中包含一个模块，其 source_files 列表包含 required_source_files 中的所有路径。接收模块的 description 字段会被扩展以包含一条注释，引用添加的源文件（不会重复任何已存在的描述内容）。不会删除任何现有阶段，不会丢失任何现有模块的源文件，并且预先存在的阶段编号保持不变。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_merge_descriptions",
      "signature": "_merge_descriptions(target_desc, source_desc) -> str",
      "pre_condition": "target_desc 是一个字符串（现有描述）；source_desc 是一个非空字符串（要附加的描述文本）",
      "post_condition": "返回一个字符串，通过将 source_desc 附加到 target_desc 形成。如果两者均非空，则用一个空格分隔。如果 target_desc 已经包含 source_desc 的完整文本，则不附加 source_desc，并原样返回 target_desc。"
    }
  ]
}
```

► Logic verification result (完整)

► Bug Validator 结果 (完整)

```
{
  "id": "src--pipeline_setup-py--_ensure_source_files_in_phases",
  "source_file": "src/pipeline_setup-py/_ensure_source_files_in_phases.py",
  "function_name": "_ensure_source_files_in_phases",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--pipeline_setup-py--_ensure_source_files_in_phases.py",
  "detail_file": "fm_agent/bug_validation/src--pipeline_setup-py--_ensure_source_files_in_phases.md",
  "probe_stdout": "已确认 — 实际行为：forced=[], foo.py 在调用后出现在 2 个模块中 | 预期（规范）：恰好 1 个模块",
  "trigger_summary": "当必需的源文件已存在于多个模块中时，代码会提前返回而不进行去重，使文件处于违反“恰好一个模块”后置条件的状态。"
}
```


► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:31:29.334546Z	5440
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:32:22.833094Z	7181

事件 1 — [llm_5acae9f41b23475181f8f7ce3856979a](#)

► parsed_json（完整）

```
{
  "post_condition": "如果函数在不引发异常的情况下完成，则如下所述成立：\n令 F_before 为调用前 phases_json 处文件的 JSON 内容。\\n- 如果 required_source_files 为假值（None 或空序列）：函数返回 {'forced': [], 'augmented': {}, 'augmented_modules': []}，且文件不变（F_after = F_before）。\\n- 否则，令 required_norm = { 将 p 中的反斜杠替换为正斜杠，其中 p 来自 required_source_files }，且 existing_norm = { 将 q 中的反斜杠替换为正斜杠，其中 q 来自 F_before 中任何阶段任何模块列出的所有 source_files }。\\n  - 如果 required_norm ⊆ existing_norm，函数返回相同的空字典，且文件不变。\\n  - 否则，missing = [ sf for sf in required_source_files if 将 sf 中的反斜杠替换后不在 existing_norm 中 ]（保持顺序）。文件被转换：选择 F_before 中最早的阶段（按数字 'phase' 键排序）；如果不存在任何阶段，则创建一个新阶段，其 'phase' 为 1，name 为 'Entry Points'，description 为 'Entry-point source files.'，modules 为空，depends_on_phases 为空。在该阶段的 'modules' 列表中，如果为空，则追加一个 name 为 'entry_points'、空 description 和空 source_files 的新模块。该列表中的第一个模块，其 'source_files' 通过追加 missing 中的所有路径（按给定形式，不经规范化）来扩展，其 'description' 更新为 _merge_descriptions(old_description, 'Includes required entry-point source file(s): ' + ', '.join(missing) + '.'). 不修改其他阶段或模块。phases_json 处的文件被结果 JSON 对象（F_after）覆盖。函数返回：\\n {'forced': missing, \\n 'augmented': {earliest_phase_number: list(missing)}, \\n 'augmented_modules': [{ 'phase': earliest_phase_number, 'module': name_of_first_module, 'added_files': list(missing)}]}\\n 其中 earliest_phase_number 是所选最早阶段的 'phase' 值。\\n如果发生异常（例如，文件 I/O 错误、无效 JSON），函数会引发异常，且不对返回值或文件状态作出任何保证。\\n\\n形式化地，对于所有正常执行：\\n∃ r 使得 返回值 = r ∧\\n ( (required_source_files = None ∨ required_source_files = []) →\\n      (r = {forced: [], augmented: {}, augmented_modules: []} ∧ 文件不变) )\\n ∧\\n ( required_source_files ≠ None ∧ required_source_files ≠ [] →\\n      令 req_norm = { normalize(p) | p ∈ required_source_files } in\\n      令 exist_norm = { normalize(q) | q ∈ all_file_source_files(F_before) } in\\n      (req_norm ⊆ exist_norm → r = {forced: [], augmented: {}, augmented_modules: []} ∧ 文件不变)\\n      ∧\\n      (req_norm ⊄ exist_norm →\\n      令 missing = [ p ∈ required_source_files | normalize(p) ∉ exist_norm ] in\\n      令 phases_before = F_before.phases (如果不存在则为 []) in\\n      令 earliest_phase = (如果 phases_before = [] 则 new_phase(1) 否则 argmin{p∈phases_before} p.phase) in\\n      令 modules_after = (如果 earliest_phase.modules = [] 则 [new_module('entry_points', '', [])] 否则 earliest_phase.modules) in\\n      令 mod = modules_after[0] in\\n      令 mod_desc_before = mod.description (如果不存在则为 '') in\\n      令 mod_files_before = mod.source_files (如果不存在则为 []) in\\n      file_after = 将 file_before 中的 earliest_phase 替换为：\\n      earliest_phase[modules → modules_after[0 → (mod_files_before ++ missing, description → _merge_descriptions(mod_desc_before, note))]]\\n      其中 note = 'Includes required entry-point source file(s): ' + join(missing, ', ') + '.\\n      ∧ r = {forced: missing, augmented: {earliest_phase.phase: missing}, augmented_modules: [{phase: earliest_phase.phase, module: mod.name, added_files: missing}]}\\n      ∧ file(phases_json) = file_after\\n      )\\n )\\n其中 normalize(s) = 将 s 中的所有反斜杠字符替换为正斜杠。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_5acae9f41b23475181f8f7ce3856979a_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_5acae9f41b23475181f8f7ce3856979a_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_5acae9f41b23475181f8f7ce3856979a_response.txt)

事件 2 — [llm_d43f66766f9349fbbb1078f9455ad60c](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "输入：phases_json 文件包含一个 JSON 对象，其中有两个阶段，每个阶段都包含一个模块，这些模块在其 source_files 数组中列出了同一个源文件 'main.py'。示例内容：{'phases': [{'phase': 1, 'modules': [{'name': 'mod1', 'source_files': ['main.py']}]}, {'phase': 2, 'modules': [{'name': 'mod2', 'source_files': ['main.py']}]}]。required_source_files 为 ['main.py']。该函数会发现 'main.py'（规范化后）已经列出，将 missing 计算为空，返回 {'forced': [], 'augmented': {}, 'augmented_modules': []}，并保持文件不变。调用后，'main.py' 仍然出现在两个不同的模块中，这违反了规范中要求的每个 required_source_files 中的路径都必须恰好出现在一个模块中的规定。",
  "offending_statements": "第 27 行:missing = [sf for sf in required_source_files if sf.replace(\"\\\\\\\\\", \"/\") not in listed]\n第 28 行:if not missing:\n第 29 行:    return {\"forced\": [], \"augmented\": {}, \"augmented_modules\": []}",
  "reason": "规范保证在返回后每个必需的源文件都恰好出现在一个模块中。当必需的源文件已存在于多个模块中时，代码不会检测或修复这种重复；它只是直接提前返回而不做任何修改。这会使文件处于违反后置条件的状态。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_d43f66766f9349fbbb1078f9455ad60c_message_00_system.txt) · [user_prompt]
 (fm_agent/trace/payloads/llm_d43f66766f9349fbbb1078f9455ad60c_message_01_user.txt) · [assistant_output]
 (fm_agent/trace/payloads/llm_d43f66766f9349fbbb1078f9455ad60c_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修改实现。

FMA-MISMATCH-139 — [src--pipeline_setup.py--_run_generate_phases](#)

- 四类结论：可能有 Bug
- 分类依据：沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: [confirmed](#); attempts 1
- Phase / 模块：Phase 3 — Pipeline Planning & Domain Knowledge / [pipeline_setup](#)；原源码 [src/pipeline_setup.py](#)
- 证据：[抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**：如果函数正常返回（不退出进程）：要么 `plugin_stage.type` 是 'pass' 或 'replace'（插件完全负责 `phases.json` 的输出），要么 `phases.json` 存在于 `work_dir` 下的预期路径，其中包含一个阶段列表，每个阶段有 phase number（整数）、name（字符串）、description（字符串）、modules（一个对象列表，每个对象包含 name、description 和 source_files），以及 depends_on_phases（一个整数列表，引用其他阶段的编号）。在增量模式下，现有的 `phases.json` 会被更新以反映当前源代码状态：添加新出现的模块和源文件，移除文件已不存在的条目，并根据需要调整阶段分配，同时保留仍然准确的条目。在子模块过滤模式下，`source_files` 条目仅引用指定子目录路径内的文件。如果经过 `OPENCODE_MAX_RETRIES` 次尝试后，`phases.json` 缺失、存在模式验证错误，或者在子模块模式下未能覆盖指定子目录下的当前源文件，进程会以代码 1 退出，并输出描述失败原因的诊断信息。
- **实际行为**：代码块执行后，以下情况之一成立：
 1. (终止) 如果 `phase_plan_ready`（在第 121130 行赋值）是 `False` 并且 `attempt >= OPENCODE_MAX_RETRIES`，程序调用 `sys.exit(1)` 并终止；不再执行后续语句。
 2. (异常) 如果 `phase_plan_ready` 是 `True` 且执行了 `break` 分支，随后条件 `plugin_stage is not None and plugin_stage.type == "modify" and plugin_stage.output_process` 成立，并且由 `run_plugin_command` 执行的命令以非零代码退出，则抛出 `CalledProcessError`；代码块未正常完成。
 3. (正常完成) 否则，代码块正常结束，既不终止程序也不抛出异常。在这种情况下：
 - 如果 `phase_plan_ready` 是 `True`：则已退出外层循环（通过 `break`）。之后，如果 `plugin_stage is not None and plugin_stage.type == "modify" and plugin_stage.output_process` 为真，

则通过 `plugin_stage.output_process` 执行了 `shell` 命令 `run_plugin_command`, 其中 `cwd=plugin_root`, 环境变量 `FM_AGENT_PLUGIN_ROOT` 设置为 `plugin_root`, 标签为 `"generate_phase_plan post-process"`, 并成功完成 (退出代码 0)。如果该条件为假, 则未运行任何插件后处理。

- 如果 `phase_plan_ready` 是 `False` 且 `attempt < OPENCODE_MAX_RETRIES`: 则通过 `missing` 记录并输出到 `stdout` 一条警告信息, 其中包含计算得到的 `logging.warning` 原因; `time.sleep(10)` 已完成。循环未被中断, 因此控制将返回到外层循环的顶部进行下一次迭代 (在此代码块之外)。
- 代码块中引入的所有其他程序状态保持不变: `prompt`、`command`、`attempt`、`proj_dir`、`submodules` (如果存在)、`is_incremental`、`prev_mtime`、`phases_json`、`phase_plan_errors`、`OPENCODE_MAX_RETRIES`、`plugin_root`、`plugin_stage`。跟踪目录仍包含阶段 `'generate_phases_json'` 的事件, 其元数据为 `{'attempt': attempt}`。

正式地, 设 `ready_before = False`, 并定义 `ready_after` 为:

- `submodules` and `_phases_cover_current_sources(phases_json, proj_dir, submodules)` 或
- `(not submodules)` and `is_incremental` and `(os.path.getmtime(phases_json) != prev_mtime or _phases_cover_current_sources(phases_json, proj_dir))` 或
- `(not submodules)` and `(not is_incremental)`。定义 `exit_attempt = not ready_after and attempt >= OPENCODE_MAX_RETRIES`。定义 `plugin_condition = (plugin_stage is not None and plugin_stage.type == "modify" and plugin_stage.output_process)`。那么正常完成的后置条件为: `exit_attempt (ready_after (loop_broken (plugin_condition (run_plugin_command_succeeded no_exception))) ) (ready_after (attempt < OPENCODE_MAX_RETRIES logged_warning slept_10s loop_broken))` 所有其他变量保持不变, 如上所述。
- **代码证据:** 第 125 行: `phase_plan_ready =` (第 126 行: `os.path.getmtime(phases_json) != prev_mtime` 第 127 行: `or _phases_cover_current_sources(phases_json, proj_dir)` 第 128 行:)
- **触发条件:** 在增量模式下, 规范要求更新后的 `phases.json` 能反映当前的源代码状态。代码使用基于 `mtime` 的检查作为覆盖率检查的替代方案, 因此即使覆盖率不完整, 只要重写了文件就会被接受为 `ready`, 这违反了所需的保证。
- **Validator 触发:** 在增量模式下, 第 1034-1037 行基于 `mtime` 的 OR 检查在覆盖率不完整时仍接受 `phases.json` 的重写, 绕过了所需的 `_phases_cover_current_sources` 验证。
- **Probe stdout:** `CONFIRMED` — `mtime_changed=True`, `coverage_ok=False`, `buggy_phase_plan_ready=True`, `expected_phase_plan_ready=False`

► SPEC sidecar (完整)

```
{
  "signature": "_run_generate_phases(proj_dir, work_dir, script_dir, is_incremental=False, resume=False,
submodules=None, plugin_stage=None, plugin_root=None)",
  "pre_condition": "proj_dir 是一个存在的目录, 包含项目源代码树; work_dir 是一个可写目录, 用于管道输出; script_dir 是一个存在的目录, 包含 FM-Agent 脚本; is_incremental 和 resume 是布尔值; submodules 如果不是 None, 则是 proj_dir 内子目录路径的序列; plugin_stage 是 PluginStageConfig 或 None; plugin_root 是 Path 或 None",
  "post_condition": "如果函数正常返回 (不退出进程): 要么 plugin_stage.type 是 'pass' 或 'replace' (插件完全负责 phases.json 的输出), 要么 phases.json 存在于 work_dir 下的预期路径, 其中包含一个阶段列表, 每个阶段有 phase number (整数)、name (字符串)、description (字符串)、modules (一个对象列表, 每个对象包含 name、description 和 source_files), 以及 depends_on_phases (一个整数列表, 引用其他阶段的编号)。在增量模式下, 现有的 phases.json 会被更新以反映当前源代码状态: 添加新出现的模块和源文件, 移除文件已不存在的条目, 并根据需要调整阶段分配, 同时保留仍然准确的条目。在子模块过滤模式下, source_files 条目仅引用指定子目录路径内的文件。如果经过 OPENCODE_MAX_RETRIES 次尝试后, phases.json 缺失、存在模式验证错误, 或者在子模块模式下未能覆盖指定子目录下的当前源文件, 进程会以代码 1 退出, 并输出描述失败原因的诊断信息。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "build_llm_cli_command",
      "signature": "build_llm_cli_command(model, prompt, cwd, files) -> str",
      "pre_condition": "model 是标识要使用的 LLM 模型的字符串; prompt 是包含自然语言指令的字符串; cwd 是一个有效的目录路径; files 是存在且可读的绝对文件路径的列表",
      "post_condition": "返回一个 CLI 命令字符串, 当在环境中执行时, 该命令会使用给定的模型、提示和文件附件调用配置好的 LLM 后端"
    },
    {
      "name": "run_opencode_traced",
      "signature": "run_opencode_traced(proj_dir, work_dir, command, stage, input_files, output_files, summary, metadata)",
      "pre_condition": "proj_dir 是一个存在的项目目录; work_dir 是一个可写目录; command 是一个有效的 CLI 命令字符串; stage 是标识管道阶段的字符串; input_files 是相对于 proj_dir 的文件路径列表, 这些文件存在且可读; output_files 是相对于 proj_dir 的文件路径列表, 调用的进程预期会产生这些文件",
      "post_condition": "当命令执行时正常返回。返回后, 如果基于 LLM 的流程成功完成, output_files 预期会存在于 proj_dir 下的指定路径; 结构化的跟踪事件会写入跟踪目录"
    },
    {
      "name": "run_plugin_command",
      "signature": "run_plugin_command(cmd, plugin_root, proj_dir, label)",
      "pre_condition": "cmd 是一个非空字符串, 指定一个 shell 命令; plugin_root 是一个有效的目录路径; proj_dir 是一个存在的目录; label 是一个标识操作的字符串, 用于日志消息",
      "post_condition": "执行命令, 工作目录设置为 plugin_root, 并在子进程环境中将 FM_AGENT_PLUGIN_ROOT 设置为 plugin_root。如果命令以非零代码退出, 则抛出 CalledProcessError"
    }
  ]
}
```

- Logic verification result (完整)
- Bug Validator 结果 (完整)

```
{
  "id": "src--pipeline_setup-py--_run_generate_phases",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/pipeline_setup-py/_run_generate_phases.py",
  "function_name": "_run_generate_phases",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--pipeline_setup-py--_run_generate_phases.py",
  "detail_file": "fm_agent/bug_validation/src--pipeline_setup-py--_run_generate_phases.md",
  "probe_stdout": "CONFIRMED - mtime_changed=True, coverage_ok=False, buggy_phase_plan_ready=True, expected_phase_plan_ready=False",
  "trigger_summary": "在增量模式下, 第 1034-1037 行的基于 mtime 的 OR 检查接受 phases.json 重写, 即使覆盖不完整, 绕过了所需的 _phases_cover_current_sources 验证。"
}
```

► Reasoner 流程 (6 条事件)

共 6 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌, 使用每条事件后的载荷链接打开。

#	目的	状态	尝试	块	开始	令牌数
1	generate_block_post_condition	成功	1	0/4	2026-07-27T18:19:56.763043Z	6956
2	check_post_implies_spec	成功	1	0/4	2026-07-27T18:21:17.019249Z	7616
3	generate_block_post_condition	成功	1	1/4	2026-07-27T18:22:22.407902Z	12298
4	generate_block_post_condition	成功	1	2/4	2026-07-27T18:24:41.293104Z	5357
5	generate_block_post_condition	成功	1	3/4	2026-07-27T18:25:32.782452Z	8269

#	目的	状态	尝试	块	开始	令牌数
6	check_post_implies_spec	mismatch	1	3/4	2026-07-27T18:26:56.143836Z	6232

事件 1 — llm_cc1db75bfff17408da24e55fbae7555ea

► parsed_json (完整)

```
{
  "post_condition": "如果块中的任何操作引发异常（例如，导入错误、文件 I/O 错误、来自 run_plugin_command 的 CalledProcessError），则函数未正常完成，异常会传播。否则，该块执行时没有异常，完成后（返回或块结束）会出现以下情况之一：\n\n1. 提前返回（pass）：如果 plugin_stage ≠ None ∧ plugin_stage.type = 'pass'，则打印一条消息，并立即返回。不绑定变量 phases_json、prev_mtime、phase_plan_errors、_resume_skip；不发生文件修改。\n\n2. 提前返回（replace）：如果 plugin_stage ≠ None ∧ plugin_stage.type = 'replace'，则打印一条消息，然后调用 run_plugin_command(plugin_stage.replace_cmd, plugin_root, proj_dir, label='generate_phase_plan')。只要命令以代码 0 退出，函数就返回。如果命令失败，CalledProcessError 传播。\n\n3. 流程继续（所有其他情况）：函数不在块内返回；执行到达第 40 行并继续，具有以下本地状态：\n    - phases_json = os.path.join(work_dir, 'phases.json')\n    - prev_mtime = phases_json 的 mtime（如果存在）否则为 None\n    - phase_plan_errors = _phase_plan_schema_errors(phases_json)（如果 phases_json 存在）否则为 []\n    - _resume_skip = (resume ∧ _phase_plan_complete(work_dir))\n    - 如果 _resume_skip 为真，则打印一条恢复消息。\n    - 如果 plugin_stage ≠ None ∧ plugin_stage.type = 'modify' ∧ bool(plugin_stage.input_md)：\n        workflow_src = str(plugin_root / plugin_stage.input_md)\n        workflow_dst = os.path.join(work_dir, 'workflow_generate_phases.md')\n        shutil.copy2(workflow_src, workflow_dst)（成功复制）\n    user_knowledge_paths = list_staged_domain_knowledge_relpaths(work_dir)\n    如果 user_knowledge_paths ≠ []：\n        文件 workflow_dst 被追加一个部分，包含：\n        - 分隔符 "\\n---\\n\\n\\n\\n"\n        - 标题 "\\n## User-Provided Domain Knowledge\\n\\n\\n\\n"\n        - 说明性文本\n        - 格式化后的 user_knowledge_paths 项目符号列表（通过 format_domain_knowledge_bullets）。\n    - 否则（plugin_stage 为 None，类型不是 'modify'，或者 input_md 为假值），不会发生复制或追加。\n    - 以上所有局部变量均已定义，并且没有产生其他文件系统副作用。\n\n形式上：\n\npost-condition ≡\n  (no exception) →\n    ( (plugin_stage ≠ None ∧ plugin_stage.type = 'pass' ∧ printed('...skip...') ∧ return())\n      ∨ (plugin_stage ≠ None ∧ plugin_stage.type = 'replace' ∧ printed('...running...') ∧ run_plugin_command_success(plugin_stage.replace_cmd, plugin_root, proj_dir) ∧ return())\n      ∨ (¬(plugin_stage ≠ None ∧ plugin_stage.type = 'pass') ∧ ¬(plugin_stage ≠ None ∧ plugin_stage.type = 'replace') ∧\n        phases_json = join(work_dir, 'phases.json') ∧\n        prev_mtime = (exist(phases_json) ? mtime(phases_json) : None) ∧\n        phase_plan_errors = (exist(phases_json) ? schema_errors(phases_json) : []) ∧\n        _resume_skip = (resume ∧ phase_plan_complete(work_dir)) ∧\n        (_resume_skip → printed('RESUME ...')) ∧\n        [ (plugin_stage ≠ None ∧ plugin_stage.type = 'modify' ∧ truthy(plugin_stage.input_md)) →\n          workflow_src = str(plugin_root / plugin_stage.input_md) ∧\n          workflow_dst = join(work_dir, 'workflow_generate_phases.md') ∧\n          copy2_success(workflow_src, workflow_dst) ∧\n          user_knowledge_paths = list_staged_domain_knowledge_relpaths(work_dir) ∧\n          (user_knowledge_paths ≠ [] → file_appended(workflow_dst, knowledge_markdown(user_knowledge_paths)) )\n        ; else True\n        ]\n        )\n        )\n        )\n        )\n\n(其中 truthy(x) 表示 x 不为 None 且不是空字符串等。)"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_cc1db75bfff17408da24e55fbae7555ea_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_cc1db75bfff17408da24e55fbae7555ea_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_cc1db75bfff17408da24e55fbae7555ea_response.txt)

事件 2 — llm_6da3a5fa7b6a424f9aed0aeaa1e7d8ac

► parsed_json (完整)

```
{
  "verdict": "MATCH",
  "counterexample": null,
  "offending_statements": null,
  "reason": ""
}
```


- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_6da3a5fa7b6a424f9aed0aeaa1e7d8ac_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_6da3a5fa7b6a424f9aed0aeaa1e7d8ac_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_6da3a5fa7b6a424f9aed0aeaa1e7d8ac_response.txt)

事件 3 — llm_f7c282e3e0b84015bdab97be24ef1541

► parsed_json (完整)

```
{
  "post_condition": "代码块执行时不引发异常。else 分支处于活动状态。_prepare_workflow_file(proj_dir, work_dir, script_dir, \"workflow_generate_phases.md\") 被调用并成功完成, 将 workflow 文件放置或更新在 work_dir 中。绑定了三个字符串变量: fm_reminder = \"IMPORTANT: The fm_agent/ directory is NOT part of the project source code. It is a workspace for storing your output files only. Do NOT include fm_agent/ paths in phases.json. Do NOT modify any existing project files.\"; incremental_reminder = \"IMPORTANT: An existing fm_agent/phases.json from a previous run is already present. Do NOT regenerate it from scratch. Instead, inspect the current state of the source code and UPDATE the existing fm_agent/phases.json so it reflects the current version of the code: add modules and source files that are new, remove entries whose files no longer exist, and adjust phases as needed. Preserve entries that are still accurate.\"; submodule_reminder 如果 submodules 为假值则为 \"\", 否则根据第 55-60 行包含允许的子目录列表。一个 for 循环从 attempt = 1 到 OPENCODE_MAX_RETRIES 开始。如果 _resume_skip 为真, 则立即执行 break; 控制权传递到循环后的下一条语句。否则, 循环运行; 每次迭代中, 根据以下规则构建提示: 当 resume 为 false 的第一次尝试使用特定的初始提示, 否则使用重试提示; is_incremental 用 incremental_reminder 扩展它; 如果 phase_plan_errors 非空, 则包含模式修复指令 (schema_repair_prompt 的构建从第 79 行开始, 并假定在完整循环体中完成)。随后, 从该提示构建 CLI 命令 (通过 build_llm_cli_command) 并通过 run_opencode_traced 执行, 这会调用 LLM 来生成或更新 phases.json 以及 work_dir 中可能的其他文件。循环后 (通过 break 或尝试次数耗尽), 函数返回或继续其下一条语句。在此块内不会发生其他持久性副作用。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_f7c282e3e0b84015bdab97be24ef1541_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_f7c282e3e0b84015bdab97be24ef1541_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_f7c282e3e0b84015bdab97be24ef1541_response.txt)

事件 4 — llm_4ecf75fd81f745bc96fa5d6c8a123536

► parsed_json (完整)

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_4ecf75fd81f745bc96fa5d6c8a123536_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_4ecf75fd81f745bc96fa5d6c8a123536_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_4ecf75fd81f745bc96fa5d6c8a123536_response.txt)

事件 5 — llm_9fc18aae2bbc4c33ac4798500f5ae073

► parsed_json (完整)

```
{
  "post_condition": "代码块执行后, 以下情形之一成立: \n1. (终止) 如果 `phase_plan_ready` (在第 121-130 行赋值) 为 `False` 且 `attempt >= OPENCODE_MAX_RETRIES`, 程序调用 `sys.exit(1)` 并终止; 不执行任何后续语句。 \n2. (异常) 如果 `phase_plan_ready` 为 `True` 并执行 `break`, 且条件 `plugin_stage is not None and plugin_stage.type == \"modify\"` and `plugin_stage.output_process` 成立, 并且由 `run_plugin_command` 执行的命令以非零代码退出, 则引发一个 `CalledProcessError`; 该代码块未正常完成。 \n3. (正常完成) 否则该代码块正常结束, 未终止程序也未引发异常。在这种情况下: \n- 如果 `phase_plan_ready` 为 `True`: 外层循环已通过 `break` 退出。之后, 如果 `plugin_stage is not None and plugin_stage.type == \"modify\"` and `plugin_stage.output_process` 为真, 则 shell 命令 `plugin_stage.output_process` 已由 `run_plugin_command` 执行, 参数 `cwd=plugin_root`, 环境变量 `FM_AGENT_PLUGIN_ROOT` 设置为 `plugin_root`, 标签为 `\"generate_phase_plan post-process\"`, 并成功完成 (退出码 0)。如果该条件为假, 则未运行插件后处理。 \n- 如果 `phase_plan_ready` 为 `False` 且 `attempt < OPENCODE_MAX_RETRIES`: 包含计算得出的 `missing` 原因的警告消息已通过 `logging.warning` 记录并输出到 stdout; `time.sleep(10)` 已完成。循环未被中断, 因此控制权将返回到外层循环的顶部, 进入下一次迭代 (在该代码块之外)。 \n- 所有带入该代码块的其他程序状态保持不变: `prompt`、`command`、`attempt`、`proj_dir`、`submodules` (若存在)、`is_incremental`、`prev_mtime`、`phases_json`、`phase_plan_errors`、`OPENCODE_MAX_RETRIES`、`plugin_root`、
```

```
plugin_stage`. trace 目录仍然包含一个阶段为 `generate_phases_json`、元数据为 `{'attempt': attempt}` 的事件。
\n\n形式化地，令 `ready_before = False`，并定义 `ready_after` 为：\n- `submodules and
_phases_cover_current_sources(phases_json, proj_dir, submodules)` 或\n- `(not submodules) and
is_incremental and (os.path.getmtime(phases_json) != prev_mtime or
_phases_cover_current_sources(phases_json, proj_dir))` 或\n- `(not submodules) and (not
is_incremental)`。
\n定义 `exit_attempt = not ready_after and attempt >= OPENCODE_MAX_RETRIES`。
\n定义 `plugin_condition = (plugin_stage is not None and plugin_stage.type == "modify" and
plugin_stage.output_process)`。
\n则正常完成的后置条件为：\n-`exit_attempt ^ (ready_after ^ (loop_broken ^
(plugin_condition ^ (run_plugin_command_succeeded ^ no_exception))) ) ^ (~ready_after ^ (attempt <
OPENCODE_MAX_RETRIES ^ logged_warning ^ slept_10s ^ ~loop_broken))`
\n所有其他变量如上所述保持不变。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_9fc18aae2bbc4c33ac4798500f5ae073_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_9fc18aae2bbc4c33ac4798500f5ae073_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_9fc18aae2bbc4c33ac4798500f5ae073_response.txt)

事件 6 — llm_006a771e44824bb1984db30e0954f619

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "设置 is_incremental=True, submodules=False, prev_mtime 为一个较旧的时间戳，并提供一个刚被
LLM 重写的 phases.json 文件（因此其 mtime 与 prev_mtime 不同），但其内容未能通过 _phases_cover_current_sources 检
查（例如，出现了一个未被引用的新源文件）。确保 phase_plan_errors 为空，以便能够到达就绪判定代码块。然后代码将把
phase_plan_ready 设为 True，因为 os.path.getmtime(phases_json) != prev_mtime，即使
_phases_cover_current_sources 返回 False。循环中断，阶段结束，而未要求一个最新的 phases.json。",
  "offending_statements": "第 125 行：                                phase_plan_ready = (\n第 126 行：
os.path.getmtime(phases_json) != prev_mtime\n第 127 行：                                or
_phases_cover_current_sources(phases_json, proj_dir)\n第 128 行：                                )",
  "reason": "在增量模式下，规约要求更新后的 phases.json 反映当前的源代码状态。代码使用基于 mtime 的检查作为覆盖率检查的
替代方案，因此仅仅重写文件（即使覆盖率不完整）就被接受为就绪，这违反了所需保证。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_006a771e44824bb1984db30e0954f619_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_006a771e44824bb1984db30e0954f619_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_006a771e44824bb1984db30e0954f619_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块: plugins

FMA-MISMATCH-146 — src--plugin-py--run_plugin_command

- 四类结论: 可能有 Bug
- 分类依据: probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- Validator: **confirmed**; attempts 1
- Phase / 模块: Phase 3 — Pipeline Planning & Domain Knowledge / **plugins**; 原源码 **src/plugin.py**
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: 将 cmd 作为一个 shell 子进程执行。cmd 中的相对文件路径相对于 plugin_root 解释。子进程运行时的工作目录设置为 plugin_root，环境变量 FM_AGENT_PLUGIN_ROOT 设置为 plugin_root 的字符串表示。如

果命令以非零代码退出，会向 `stdout` 打印一条包含 `label`、命令字符串和退出代码的诊断信息，然后引发 `subprocess.CalledProcessError`。如果命令以零代码退出，函数正常返回，不报错。

- **Actual behavior:** 该函数要么在已解析的 `shell` 命令（相对路径在 `plugin_root` 下解析）在目录 `proj_dir` 中成功运行（退出代码为 0），且 `FM_AGENT_PLUGIN_ROOT` 设置为 `plugin_root`，未打印错误信息后返回 `None`；要么在向 `stdout` 打印包含 `label`、返回码和已解析命令的错误信息后引发 `subprocess.CalledProcessError` 异常。这两种情况下均未修改其他程序状态。形式化表述：`(result = None resolved_cmd = _resolve_command(cmd, plugin_root) subprocess_success(resolved, proj_dir, plugin_root) printed_error) (exception = CalledProcessError resolved_cmd = _resolve_command(cmd, plugin_root) subprocess_failed(resolved, proj_dir, plugin_root, exit_code) printed_error(resolved, exit_code, label) exit_code 0)`
- **Code evidence:** Line 11: `subprocess.run(resolved, shell=True, check=True, cwd=proj_dir, env=env)`
- **Trigger condition:** 规范要求子进程运行时工作目录设置为 `plugin_root`，但代码中设置了 `cwd=proj_dir`。对于任何 `proj_dir != plugin_root` 的输入，工作目录会存在差异，违反规范。例如，对于给定输入，命令 `'pwd'` 的输出会是 `'/home/user'` 而非 `'/tmp/plugin'`。
- **Validator trigger:** 当 `proj_dir != plugin_root` 时触发该缺陷；`subprocess.run()` 接收到的 `cwd=proj_dir`，而非规范要求的 `cwd=plugin_root`。
- **Probe stdout:** `hello CONFIRMED — actual cwd: '/tmp/probe_p41_nvxx/proj_dir' | expected cwd: '/tmp/probe_p41_nvxx/plugin_root'`

► SPEC sidecar（完整）

```
{
  "signature": "run_plugin_command(cmd: str, plugin_root: Path, proj_dir: str, label: str = '') -> None",
  "pre_condition": "cmd 是一个非空字符串，指定一个 shell 命令；plugin_root 是一个有效的目录路径；proj_dir 是一个存在的目录；label 是一个用于日志消息中标识操作的字符串。",
  "post_condition": "将 cmd 作为一个 shell 子进程执行。cmd 中的相对文件路径相对于 plugin_root 解释。子进程运行时的工作目录设置为 plugin_root，环境变量 FM_AGENT_PLUGIN_ROOT 设置为 plugin_root 的字符串表示。如果命令以非零代码退出，会向 stdout 打印一条包含 label、命令字符串和退出代码的诊断信息，然后引发 subprocess.CalledProcessError。如果命令以零代码退出，函数正常返回，不报错。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/plugin-py/run_plugin_command.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "将 cmd 作为一个 shell 子进程执行。cmd 中的相对文件路径相对于 plugin_root 解释。子进程运行时的工作目录设置为 plugin_root，环境变量 FM_AGENT_PLUGIN_ROOT 设置为 plugin_root 的字符串表示。如果命令以非零代码退出，会向 stdout 打印一条包含 label、命令字符串和退出代码的诊断信息，然后引发 subprocess.CalledProcessError。如果命令以零代码退出，函数正常返回，不报错。",
    "actual_behavior": "该函数要么在已解析的 shell 命令（相对路径在 plugin_root 下解析）在目录 proj_dir 中成功运行（退出代码为 0），且 FM_AGENT_PLUGIN_ROOT 设置为 plugin_root，未打印错误信息后返回 None；要么在向 stdout 打印包含 label、返回码和已解析命令的错误信息后引发 subprocess.CalledProcessError 异常。这两种情况下均未修改其他程序状态。形式化表述：(result = None resolved_cmd = _resolve_command(cmd, plugin_root) subprocess_success(resolved, proj_dir, plugin_root) printed_error) (exception = CalledProcessError resolved_cmd = _resolve_command(cmd, plugin_root) subprocess_failed(resolved, proj_dir, plugin_root, exit_code) printed_error(resolved, exit_code, label) exit_code 0)",
    "code_evidence": "Line 11: subprocess.run(resolved, shell=True, check=True, cwd=proj_dir, env=env)",
    "trigger_condition": "规范要求子进程运行时工作目录设置为 plugin_root，但代码中设置了 cwd=proj_dir。对于任何"
  }
}
```

```
proj_dir != plugin_root 的输入，工作目录会存在差异，违反规范。例如，对于给定输入，命令 'pwd' 的输出会是 '/home/user' 而非 '/tmp/plugin'。"
```

► Bug Validator result（完整）

```
{
  "id": "src--plugin-py--run_plugin_command",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/plugin-
py/run_plugin_command.py",
  "function_name": "run_plugin_command",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--plugin-py--run_plugin_command.py",
  "detail_file": "fm_agent/bug_validation/src--plugin-py--run_plugin_command.md",
  "probe_stdout": "hello\nCONFIRMED - actual cwd: '/tmp/probe_p41_nvvx/proj_dir' | expected cwd:
'/tmp/probe_p41_nvvx/plugin_root'",
  "trigger_summary": "当 proj_dir != plugin_root 时触发该缺陷；subprocess.run() 接收到的 cwd=proj_dir，而非规范
要求的 cwd=plugin_root。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:32:29.839334Z	2121
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:32:53.053572Z	2546

事件 1 — [llm_814565bf127348fea85a2f95ff326b31](#)

► parsed_json（完整）

```
{
  "post_condition": "该函数要么在已解析的 shell 命令（相对路径在 plugin_root 下解析）在目录 proj_dir 中成功运行（退出
代码为 0），且 FM_AGENT_PLUGIN_ROOT 设置为 plugin_root，未打印错误信息后返回 None；要么在向 stdout 打印包含 label、
返回码和已解析命令的错误信息后引发 subprocess.CallledProcessError 异常。这两种情况下均未修改其他程序状态。形式化表述：
(result = None ^ resolved_cmd = _resolve_command(cmd, plugin_root) ^ subprocess_success(resolved,
proj_dir, plugin_root) ^ ¬printed_error) v (exception = CalledProcessError ^ resolved_cmd =
_resolve_command(cmd, plugin_root) ^ subprocess_failed(resolved, proj_dir, plugin_root, exit_code) ^
printed_error(resolved, exit_code, label) ^ exit_code ≠ 0)"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_814565bf127348fea85a2f95ff326b31_message_00_system.txt)
· [user_prompt](fm_agent/trace/payloads/llm_814565bf127348fea85a2f95ff326b31_message_01_user.txt) ·
[assistant_output](fm_agent/trace/payloads/llm_814565bf127348fea85a2f95ff326b31_response.txt)

事件 2 — [llm_687f0f8731bc45e583ba66828a409bc6](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
```



```
"counterexample": "cmd='pwd', plugin_root=Path('/tmp/plugin'), proj_dir='/home/user', label='',
"offending_statements": "Line 11: subprocess.run(resolved, shell=True, check=True, cwd=proj_dir,
env=env)",
"reason": "规范要求子进程运行时工作目录设置为 plugin_root, 但代码中设置了 cwd=proj_dir。对于任何 proj_dir !=
plugin_root 的输入, 工作目录会存在差异, 违反规范。例如, 对于给定输入, 命令 'pwd' 的输出会是 '/home/user' 而非
'/tmp/plugin'。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_687f0f8731bc45e583ba66828a409bc6_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_687f0f8731bc45e583ba66828a409bc6_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_687f0f8731bc45e583ba66828a409bc6_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足; 若可满足, 将其转为回归测试并修实现。

Phase 4 — LLM Communication & Tracing

模块: llm_client

FMA-MISMATCH-159 — [src--llm_client-py--_parse_json_response](#)

- 四类结论: 可能有 Bug
- 分类依据: probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险, 列为实现缺陷候选。
- Validator: [confirmed](#); attempts 1
- Phase / 模块: Phase 4 — LLM Communication & Tracing / [llm_client](#); 原源码 [src/llm_client.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** 当 response 恰好包含一个 JSON 对象或数组时, 通过结构解析 (识别任意嵌套深度的平衡 JSON 括号对) 确定, 分别以 dict 或 list 形式返回该值。在以下情况下抛出 ValueError: (a) response 不是字符串; (b) 文本中不存在任何 JSON 对象或数组; (c) 文本中存在多个 JSON 对象或数组; (d) 直接全字符串解析成功, 但生成的是非结构化 JSON 标量 (字符串、数字、布尔值或 null), 而非对象或数组。提取过程从左到右扫描整个文本; 结构解析意味着包含 '{' 或 '[' 字符但不具备平衡 JSON 结构的非 JSON 文本不会产生匹配。
- **Actual behavior:** 执行后, 如果 response 不是字符串, 则抛出带有消息 'LLM 响应必须是 JSON 字符串' 的 ValueError。否则, 令 text = response.strip()。如果 json.loads(text) 成功并返回一个 data 或 dict 类型的值 list, 函数返回 data。如果 json.loads(text) 成功但 data 不是 dict 或 list, 抛出带有消息 'LLM 响应必须包含 JSON 对象或数组' 的 ValueError。如果 json.loads(text) 抛出 JSONDecodeError (记为 exc), 函数使用 JSON 解码器扫描 text, 按顺序查找所有有效的 JSON 对象和数组。令 values 为这些找到的值的列表。如果 len(values) 恰好为 1, 返回 values[0]。如果 len(values) > 1, 抛出带有消息 'LLM 响应包含多个 JSON 值' 的 ValueError。如果 len(values) == 0, 抛出带有消息 ValueError 并从 'LLM response is not valid JSON: {exc}' 链出的 exc。

形式化地, 若 is_str(x) 表示 x 为字符串, full_parse(s) 在 json.loads(s) 成功时返回解析值, 否则错误, is_dict_or_list(v) 表示 v 是 dict 或 list, 且 scan(s) 返回按扫描算法提取的 dict/list 值的有序列表:

```
post_condition ( is_str(response) Raise(ValueError("LLM 响应必须是 JSON 字符串")) ) ( is_str(response) let t =
strip(response) in ( (full_parse(t) = v is_dict_or_list(v)) Return(v) ) (full_parse(t) = v is_dict_or_list(v))
Raise(ValueError("LLM 响应必须包含 JSON 对象或数组")) ) (full_parse(t) = (error) let exc = error in ( |scan(t)| = 1
Return(scan(t)[0]) ) ( |scan(t)| > 1 Raise(ValueError("LLM 响应包含多个 JSON 值")) ) ( |scan(t)| = 0
Raise(ValueError(f"LLM response is not valid JSON: {exc}")) from exc ) ) )
```


- **Code evidence:** 第 17 行: `object_start = text.find("{", index)` 第 18 行: `array_start = text.find("[", index)` 第 22 行: `start = min(starts)` 第 24 行: `data, end = decoder.raw_decode(text, start)` 第 28 行: `if isinstance(data, (dict, list)): values.append(data)`
- **Trigger condition:** 扫描逻辑查找任何 '{' 或 '[' 字符, 而不考虑其是否位于 JSON 字符串内部, 从而导致提取出现在带引号字符串中的有效 JSON 对象。规范要求结构解析需遵守字符串边界, 因此嵌入字符串中的 JSON 对象不应视为匹配。在此反例中, 代码返回 `{'a': 1}`, 而规范将抛出 `ValueError`, 因为不存在独立的 JSON 对象/数组。
- **Validator trigger:** 扫描逻辑查找 '{' 或 '[' 而未检查是否位于 JSON 字符串内, 因此嵌入在带引号字符串中的 JSON 对象被错误地提取为独立值。
- **Probe stdout:** CONFIRMED — bug reproduced: `actual={'a': 1} | expected='ValueError (no standalone JSON object/array)'`

► SPEC sidecar (完整)

```
{
  "signature": "_parse_json_response(response) -> dict | list",
  "pre_condition": "response 是一个字符串, 可能包含零个、一个或多个 JSON 对象或数组, 可能嵌入在 markdown 代码围栏中或穿插说明性文字。",
  "post_condition": "当 response 恰好包含一个 JSON 对象或数组时—通过结构解析 (识别任意嵌套深度的平衡 JSON 括号对) 确定—分别以 dict 或 list 形式返回该值。在以下情况下抛出 ValueError: (a) response 不是字符串; (b) 文本中不存在任何 JSON 对象或数组; (c) 文本中存在多个 JSON 对象或数组; (d) 直接全字符串解析成功, 但生成的是非结构化 JSON 标量 (字符串、数字、布尔值或 null), 而非对象或数组。提取过程从左到右扫描整个文本; 结构解析意味着包含 '{' 或 '[' 字符但不具备平衡 JSON 结构的非 JSON 文本不会产生匹配。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/llm_client-py/_parse_json_response.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "当 response 恰好包含一个 JSON 对象或数组时 通过结构解析 (识别任意嵌套深度的平衡 JSON 括号对) 确定分别以 dict 或 list 形式返回该值。在以下情况下抛出 ValueError: (a) response 不是字符串; (b) 文本中不存在任何 JSON 对象或数组; (c) 文本中存在多个 JSON 对象或数组; (d) 直接全字符串解析成功, 但生成的是非结构化 JSON 标量 (字符串、数字、布尔值或 null), 而非对象或数组。提取过程从左到右扫描整个文本; 结构解析意味着包含 '{' 或 '[' 字符但不具备平衡 JSON 结构的非 JSON 文本不会产生匹配。",
    "actual_behavior": "执行后, 如果 `response` 不是字符串, 则抛出带有消息 'LLM 响应必须是 JSON 字符串' 的 `ValueError`。否则, 令 `text = response.strip()`。如果 `json.loads(text)` 成功并返回一个 `dict` 或 `list` 类型的值 `data`, 函数返回 `data`。如果 `json.loads(text)` 成功但 `data` 不是 `dict` 或 `list`, 抛出带有消息 'LLM 响应必须包含 JSON 对象或数组' 的 `ValueError`。如果 `json.loads(text)` 抛出 `JSONDecodeError` (记为 `exc`), 函数使用 JSON 解码器扫描 `text`, 按顺序查找所有有效的 JSON 对象和数组。令 `values` 为这些找到的值的列表。如果 `len(values)` 恰好为 1, 返回 `values[0]`。如果 `len(values) > 1`, 抛出带有消息 'LLM 响应包含多个 JSON 值' 的 `ValueError`。如果 `len(values) == 0`, 抛出带有消息 'LLM response is not valid JSON: {exc}' 并从 `exc` 链出的 `ValueError`。
    \n\n形式化地, 若 `is_str(x)` 表示 `x` 为字符串, `full_parse(s)` 在 `json.loads(s)` 成功时返回解析值, 否则错误, `is_dict_or_list(v)` 表示 `v` 是 dict 或 list, 且 `scan(s)` 返回按扫描算法提取的 dict/list 值的有序列表:
    \n\npost-condition \n ( is_str(response) Raise(ValueError("\nLLM response must be a JSON string\n")) )\n ( is_str(response) let t = strip(response) in\n ( (full_parse(t) = v is_dict_or_list(v)) Return(v) )\n (full_parse(t) = v is_dict_or_list(v)) Raise(ValueError("\nLLM response must contain a JSON object or array\n")) )\n (full_parse(t) = (error) let exc = error in\n ( |scan(t)| = 1 Return(scan(t)[0]) )\n ( |scan(t)| > 1 Raise(ValueError("\nLLM response contains multiple JSON values\n")) )\n ( |scan(t)| = 0 Raise(ValueError(f"\nLLM response is not valid JSON: {exc}\n")) from exc )\n )\n ),
    "code_evidence": "Line 17: object_start = text.find(\"{\", index)\nLine 18: array_start = text.find(\"[\"
```

```
[\", index)\\nLine 22: start = min(starts)\\nLine 24: data, end = decoder.raw_decode(text, start)\\nLine 28:
if isinstance(data, (dict, list)): values.append(data)",
    "trigger_condition": "扫描逻辑查找任何 '{' 或 '[' 字符，而不考虑其是否位于 JSON 字符串内部，从而导致提取出现在带
引号字符串中的有效 JSON 对象。规范要求结构解析需遵守字符串边界，因此嵌入字符串中的 JSON 对象不应视为匹配。在此反例中，代码返
回 {'a': 1}，而规范将抛出 ValueError，因为不存在独立的 JSON 对象/数组。"
}
```

► Bug Validator result（完整）

```
{
  "id": "src--llm_client-py--_parse_json_response",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/llm_client-
py/_parse_json_response.py",
  "function_name": "_parse_json_response",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--llm_client-py--_parse_json_response.py",
  "detail_file": "fm_agent/bug_validation/src--llm_client-py--_parse_json_response.md",
  "probe_stdout": "CONFIRMED - bug reproduced: actual={'a': 1} | expected='ValueError (no standalone JSON
object/array)\\n'",
  "trigger_summary": "扫描逻辑查找 '{' 或 '[' 而未检查是否位于 JSON 字符串内，因此嵌入在带引号字符串中的 JSON 对象被错
误地提取为独立值。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T19:15:23.822349Z	4319
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T19:16:11.024595Z	9049

事件 1 — [llm_d2bb1195cc514cec88c68b5bd4a5e032](#)

► parsed_json（完整）

```
{
  "post_condition": "执行后，如果 `response` 不是字符串，则抛出带有消息 'LLM 响应必须是 JSON 字符串' 的
`ValueError`。否则，令 `text = response.strip()`。如果 `json.loads(text)` 成功并返回一个 `dict` 或 `list` 类型的
值 `data`，函数返回 `data`。如果 `json.loads(text)` 成功但 `data` 不是 `dict` 或 `list`，抛出带有消息 'LLM 响应必
须包含 JSON 对象或数组' 的 `ValueError`。如果 `json.loads(text)` 抛出 `JSONDecodeError`（记为 `exc`），函数使用
JSON 解码器扫描 `text`，按顺序查找所有有效的 JSON 对象和数组。令 `values` 为这些找到的值的列表。如果 `len(values)` 恰
好为 1，返回 `values[0]`。如果 `len(values) > 1`，抛出带有消息 'LLM 响应包含多个 JSON 值' 的 `ValueError`。如果
`len(values) == 0`，抛出带有消息 'LLM response is not valid JSON: {exc}' 并从 `exc` 链出的 `ValueError`。
\\n\\n形式化地，若 `is_str(x)` 表示 `x` 为字符串，`full_parse(s)` 在 `json.loads(s)` 成功时返回解析值，否则错误，
`is_dict_or_list(v)` 表示 `v` 是 dict 或 list，且 `scan(s)` 返回按扫描算法提取的 dict/list 值的有序列表：
\\n\\npost_condition ≡\\n ( ¬is_str(response) → Raise(ValueError("\\LLM response must be a JSON string\\n"))
)\\n ∧ ( is_str(response) → let t = strip(response) in\\n      ( (full_parse(t) = v ∧ is_dict_or_list(v)) →
Return(v) )\\n      ∧ (full_parse(t) = v ∧ ¬is_dict_or_list(v)) → Raise(ValueError("\\LLM response must
contain a JSON object or array\\n")) )\\n      ∧ (full_parse(t) = ⊥ (error) → let exc = error in\\n      (
|scan(t)| = 1 → Return(scan(t)[0]) )\\n      ∧ ( |scan(t)| > 1 → Raise(ValueError("\\LLM response
contains multiple JSON values\\n")) )\\n      ∧ ( |scan(t)| = 0 → Raise(ValueError(f\\"LLM response is not
valid JSON: {exc}\\n")) from exc )\\n      )\\n )"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_d2bb1195cc514cec88c68b5bd4a5e032_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_d2bb1195cc514cec88c68b5bd4a5e032_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_d2bb1195cc514cec88c68b5bd4a5e032_response.txt)

事件 2 — [llm_1c8e778079634522a7544e8fd7c44a2f](#)

► parsed_json (完整)

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_1c8e778079634522a7544e8fd7c44a2f_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_1c8e778079634522a7544e8fd7c44a2f_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_1c8e778079634522a7544e8fd7c44a2f_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块: [opencode_trace](#)

FMA-MISMATCH-164 — [src--opencode_trace.py--_copy_opencode_output](#)

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: [confirmed](#); attempts 1
- Phase / 模块: Phase 4 — LLM Communication & Tracing / [opencode_trace](#); 原源码 [src/opencode_trace.py](#)
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** 流已关闭；若 `trace_log_path` 不为 `None`，则在 `trace_log_path` 处存在一个文件，其中包含在流耗尽之前从流中读取的每个字节的拼接，该文件已关闭，其内容采用 UTF-8 编码，不可编码字符被替换；若 `trace_log_path` 为 `None`，则不创建文件；在所有执行路径下，流已关闭并且（如果已打开）跟踪日志文件已关闭，包括在读取或写入期间引发异常的情况。
- **Actual behavior:** 函数完成后（无论是正常完成还是通过未处理的异常完成），以下成立：(1) `stream` 对象已关闭。(2) 如果 `trace_log_path` 不为 `None` 并且打开文件的尝试成功，则在 `trace_log_path` 处存在一个文件，该文件已关闭，其内容等于 `stream.read(4096)` 返回的字符串的有限前缀的拼接——具体来说，是在流耗尽或发生错误之前成功读取的所有块。(3) 如果 `trace_log_path` 不为 `None` 但文件无法打开（`open()` 调用引发了异常），则该路径处的文件保持不变，并且未写入任何内容。(4) 如果 `trace_log_path` 为 `None`，则没有打开或写入任何文件。形式化：

```
stream.closed = True
```

```
( (trace_log_path None)
```

```
( ( prefix P of stream_chunks : file_content(trace_log_path) = concat(P) file_closed(trace_log_path) ) (
```

```
file_unchanged(trace_log_path) opened(trace_log_path) ) ) )
```

其中 `stream_chunks` 是通过重复调用 `stream.read(4096)` 直到耗尽或发生异常而获得的字符串序列。

- **Code evidence:** Line 10: `trace_log.write(chunk)`
- **Trigger condition:** 当流是二进制流时，`stream.read(4096)` 返回字节（例如 `b'\xff'`）。在第 10 行，代码试图将这些字节直接写入以 `encoding='utf-8'` 打开的文本文件。这会引发 `TypeError`，因为 `write()` 期望的是字符串而不是字节。规范要求将从流中读取的每个字节作为 UTF-8 编码文本写入文件，并替换不可编码的字符（即用 'replace' 解码字节并编码为 UTF-8）。代码未执行此转换，导致无法生成所需的文件内容，而是引发异常。

- **Validator trigger:** 将二进制流 (BytesIO) 传递给 `_copy_opencode_output` 会导致 `trace_log.write()` 引发 `TypeError`, 因为文本模式文件期望 `str` 但接收到 `bytes`。
 - **Probe stdout:** CONFIRMED — 向文本模式跟踪日志写入字节时引发 `TypeError: write() 参数必须是 str, 而不是 bytes`
- SPEC sidecar (完整)

```
{
  "signature": "_copy_opencode_output(stream, trace_log_path=None)",
  "pre_condition": "stream 是一个可读类文件对象, 其重复的 read() 调用返回数据块直到耗尽; trace_log_path 要么为 None, 要么是一个文件系统路径, 可以在该路径处以 UTF-8 编码打开文件进行写入",
  "post_condition": "流已关闭; 若 trace_log_path 不为 None, 则在 trace_log_path 处存在一个文件, 其中包含在流耗尽之前从流中读取的每个字节的拼接, 该文件已关闭, 其内容采用 UTF-8 编码, 不可编码字符被替换; 若 trace_log_path 为 None, 则不创建文件; 在所有执行路径下, 流已关闭并且 (如果已打开) 跟踪日志文件已关闭, 包括在读取或写入期间引发异常的情况"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/_copy_opencode_output.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "流已关闭; 若 trace_log_path 不为 None, 则在 trace_log_path 处存在一个文件, 其中包含在流耗尽之前从流中读取的每个字节的拼接, 该文件已关闭, 其内容采用 UTF-8 编码, 不可编码字符被替换; 若 trace_log_path 为 None, 则不创建文件; 在所有执行路径下, 流已关闭并且 (如果已打开) 跟踪日志文件已关闭, 包括在读取或写入期间引发异常的情况",
    "actual_behavior": "函数完成后 (无论是正常完成还是通过未处理的异常完成), 以下成立: (1) `stream` 对象已关闭。 (2) 如果 `trace_log_path` 不为 `None` 并且打开文件的尝试成功, 则在 `trace_log_path` 处存在一个文件, 该文件已关闭, 其内容等于 `stream.read(4096)` 返回的字符串的有限前缀的拼接—具体来说, 是在流耗尽或发生错误之前成功读取的所有块。 (3) 如果 `trace_log_path` 不为 `None` 但文件无法打开 (`open()` 调用引发了异常), 则该路径处的文件保持不变, 并且未写入任何内容。 (4) 如果 `trace_log_path` 为 `None`, 则没有打开或写入任何文件。形式化: \n\nstream.closed = True \n( (trace_log_path None) \n ( ( prefix P of stream_chunks : file_content(trace_log_path) = concat(P) file_closed(trace_log_path) )\n ( file_unchanged(trace_log_path) opened(trace_log_path) )\n )\n)\n\n其中 `stream_chunks` 是通过重复调用 `stream.read(4096)` 直到耗尽或发生异常而获得的字符串序列。",
    "code_evidence": "Line 10: trace_log.write(chunk)",
    "trigger_condition": "当流是二进制流时, stream.read(4096) 返回字节 (例如 b'\\xff')。在第 10 行, 代码试图将这些字节直接写入以 encoding='utf-8' 打开的文本文件。这会引发 TypeError, 因为 write() 期望的是字符串而不是字节。规范要求将从流中读取的每个字节作为 UTF-8 编码文本写入文件, 并替换不可编码的字符 (即用 'replace' 解码字节并编码为 UTF-8)。代码未执行此转换, 导致无法生成所需的文件内容, 而是引发异常。"
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--opencode_trace-py--_copy_opencode_output",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/_copy_opencode_output.py",
  "function_name": "_copy_opencode_output",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--opencode_trace-py--_copy_opencode_output.py",
  "detail_file": "fm_agent/bug_validation/src--opencode_trace-py--_copy_opencode_output.md",
}
```

```
"probe_stdout": "CONFIRMED - 向文本模式跟踪日志写入字节时引发 TypeError: write() 参数必须是 str, 而不是 bytes",
"trigger_summary": "将二进制流 (BytesIO) 传递给 _copy_opencode_output 会导致 trace_log.write() 引发
TypeError, 因为文本模式文件期望 str 但接收到 bytes。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:46:54.280072Z	6264
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:48:34.068676Z	8842

事件 1 — [llm_63e1cc5ad01240cea0e6c394ec244bc9](#)

► parsed_json（完整）

```
{
  "post_condition": "函数完成后（无论是正常完成还是通过未处理的异常完成），以下成立：(1) `stream` 对象已关闭。(2) 如果 `trace_log_path` 不为 `None` 并且打开文件的尝试成功，则在 `trace_log_path` 处存在一个文件，该文件已关闭，其内容等于 `stream.read(4096)` 返回的字符串的有限前缀的拼接—具体来说，是在流耗尽或发生错误之前成功读取的所有块。(3) 如果 `trace_log_path` 不为 `None` 但文件无法打开（`open()` 调用引发了异常），则该路径处的文件保持不变，并且未写入任何内容。(4) 如果 `trace_log_path` 为 `None`，则没有打开或写入任何文件。形式化：\n\nstream.closed = True  ^\n((trace_log_path != None) ->\n  ( ( \exists prefix P of stream_chunks : file_content(trace_log_path) = concat(P) ^\n    file_closed(trace_log_path) )\n    v ( file_unchanged(trace_log_path) ^ -> opened(trace_log_path) )\n  )\n)\n\n其中 `stream_chunks` 是通过重复调用 `stream.read(4096)` 直到耗尽或发生异常而获得的字符串序列。"
}
```

- 载荷：[system_prompt]

(fm_agent/trace/payloads/llm_63e1cc5ad01240cea0e6c394ec244bc9_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_63e1cc5ad01240cea0e6c394ec244bc9_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_63e1cc5ad01240cea0e6c394ec244bc9_response.txt)

事件 2 — [llm_4f1e6634533b4e20b9bf6caa6d6e481f](#)

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "stream = io.BytesIO(b'\\xff'), trace_log_path = 'output.log'",
  "offending_statements": "Line 10: trace_log.write(chunk)",
  "reason": "当流是二进制流时，stream.read(4096) 返回字节（例如 b'\\xff'）。在第 10 行，代码试图将这些字节直接写入以 encoding='utf-8' 打开的文本文件。这会引发 TypeError，因为 write() 期望的是字符串而不是字节。规范要求将从流中读取的每个字节作为 UTF-8 编码文本写入文件，并替换不可编码的字符（即用 'replace' 解码字节并编码为 UTF-8）。代码未执行此转换，导致无法生成所需的文件内容，而是引发异常。"
}
```

- 载荷：[system_prompt]

(fm_agent/trace/payloads/llm_4f1e6634533b4e20b9bf6caa6d6e481f_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_4f1e6634533b4e20b9bf6caa6d6e481f_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_4f1e6634533b4e20b9bf6caa6d6e481f_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

- 四类结论：可能有 Bug
- 分类依据：沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论：沿用旧清单的逐条审计结论。
- **Validator**： [confirmed](#)；尝试次数 1
- **Phase / 模块**：Phase 4 — LLM Communication & Tracing / [opcode_trace](#)；原源码 [src/opcode_trace.py](#)
- **证据**： [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**： 返回一个文件系统路径字符串，该路径位于由 `work_dir` 衍生的 `trace` 目录结构内，唯一标识给定 `event_id` 的日志文件；返回的路径仅从 `work_dir` 和 `event_id` 可推导出，适用于写入子进程 `stdout` 日志内容
- **Actual behavior**： 该函数返回一个表示文件路径的字符串。令 `trace_dir = _trace_dir(work_dir)` 且 `payload_dir = _payload_dir(trace_dir)`。返回值 `ret` 满足：`ret == os.path.join(payload_dir, f'{event_id}_opcode.log')`。返回的字符串是将 `payload` 目录路径与由 `event_id` 和后缀 `'_opcode.log'` 组成的文件名拼接而成，使用操作系统特定的路径分隔符。
- **Code evidence**： 第 2 行：`return os.path.join(_payload_dir(_trace_dir(work_dir)), f'{event_id}_opcode.log')`
- **Trigger condition**： 规范要求返回的路径必须在 `trace` 目录结构内。当 `event_id='.././malicious'` 时，返回的路径会逃逸出 `trace` 目录，违反该要求。
- **Validator trigger**： `event_id='.././malicious'` 导致 `os.path.join` 遍历出 `trace` 目录，将日志写到预期的沙箱之外。
- **Probe stdout**： CONFIRMED — 路径遍历成功 `work_dir: /tmp/tmpbeyck_mv/run` `trace_dir: /tmp/tmpbeyck_mv/run/trace` `expected_under: /tmp/tmpbeyck_mv/run/trace` `actual_path: /tmp/tmpbeyck_mv/run/malicious_opcode.log` `event_id: '.././malicious'` escaped trace directory

► SPEC sidecar（完整）

```
{
  "signature": "_opcode_log_path(work_dir, event_id) -> str",
  "pre_condition": "work_dir 是一个有效的目录路径;event_id 是一个非空字符串",
  "post_condition": "返回一个文件系统路径字符串，该路径位于由 work_dir 衍生的 trace 目录结构内，唯一标识给定 event_id 的日志文件；返回的路径仅从 work_dir 和 event_id 可推导出，适用于写入子进程 stdout 日志内容"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "_trace_dir",
      "signature": "_trace_dir(work_dir) -> str",
      "pre_condition": "work_dir 是一个有效的目录路径",
      "post_condition": "返回 work_dir 内的 trace 子目录路径"
    },
    {
      "name": "_payload_dir",
      "signature": "_payload_dir(trace_dir) -> str",
      "pre_condition": "trace_dir 是一个有效的目录路径",
      "post_condition": "返回 trace_dir 内的 payload 子目录路径"
    }
  ]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-
py/_opencode_log_path.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个文件系统路径字符串，该路径位于由 work_dir 衍生的 trace 目录结构内，唯一标识给定 event_id 的
日志文件；返回的路径仅从 work_dir 和 event_id 可推导出，适用于写入子进程 stdout 日志内容",
    "actual_behavior": "该函数返回一个表示文件路径的字符串。令 trace_dir = _trace_dir(work_dir) 且 payload_dir =
_payload_dir(trace_dir)。返回值 ret 满足：ret == os.path.join(payload_dir, f\"{event_id}_opencode.log\"). 返
回的字符串是将 payload 目录路径与由 event_id 和后缀 '_opencode.log' 组成的文件名拼接而成，使用操作系统特定的路径分隔
符。",
    "code_evidence": "Line 2: return os.path.join(_payload_dir(_trace_dir(work_dir)), f\"
{event_id}_opencode.log\")",
    "trigger_condition": "规范要求返回的路径必须在 trace 目录结构内。当 event_id='.././malicious' 时，返回的路径会
逃逸出 trace 目录，违反该要求。"
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--opencode_trace-py--_opencode_log_path",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-
py/_opencode_log_path.py",
  "function_name": "_opencode_log_path",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--opencode_trace-py--_opencode_log_path.py",
  "detail_file": "fm_agent/bug_validation/src--opencode_trace-py--_opencode_log_path.md",
  "probe_stdout": "CONFIRMED — 路径遍历成功\n work_dir:      /tmp/tmpbeyck_mv/run\n trace_dir:
/tmp/tmpbeyck_mv/run/trace\n expected_under: /tmp/tmpbeyck_mv/run/trace\n actual_path:
/tmp/tmpbeyck_mv/run/malicious_opencode.log\n event_id:      '.././malicious'\n escaped trace
directory",
  "trigger_summary": "event_id='.././malicious' 导致 os.path.join 遍历出 trace 目录，将日志写到预期的沙箱之外。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T19:01:41.018641Z	875
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T19:01:50.483045Z	2027

事件 1 — [llm_445e5183a4094482811a532d1c0367cb](#)

► parsed_json（完整）

```
{
  "post_condition": "该函数返回一个表示文件路径的字符串。令 trace_dir = _trace_dir(work_dir) 且 payload_dir =
_payload_dir(trace_dir)。返回值 ret 满足：ret == os.path.join(payload_dir, f\"{event_id}_opencode.log\"). 返
回的字符串是将 payload 目录路径与由 event_id 和后缀 '_opencode.log' 组成的文件名拼接而成，使用操作系统特定的路径分隔
符。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_445e5183a4094482811a532d1c0367cb_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_445e5183a4094482811a532d1c0367cb_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_445e5183a4094482811a532d1c0367cb_response.txt)

事件 2 — llm_4fe094cbd5c0465197983bd23d6d652f

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "work_dir='/tmp', event_id='../../malicious'",
  "offending_statements": "第 2 行: return os.path.join(_payload_dir(_trace_dir(work_dir)), f\"
{event_id}_opcode.log\")",
  "reason": "规范要求返回的路径必须在 trace 目录结构内。当 event_id='../../malicious' 时, 返回的路径会逃逸出 trace
目录, 违反该要求。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_4fe094cbd5c0465197983bd23d6d652f_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_4fe094cbd5c0465197983bd23d6d652f_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_4fe094cbd5c0465197983bd23d6d652f_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足; 若可满足, 将其转为回归测试并修实现。

FMA-MISMATCH-167 — src--opcode_trace.py--_opcode_trace_path

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块: Phase 4 — LLM Communication & Tracing / opcode_trace; 原源码
src/opcode_trace.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** 返回一个文件系统路径字符串, 在由 work_dir 派生的跟踪目录层次结构中, 为给定的 event_id 唯一标识一个跟踪文件。该返回路径适合写入原始 LLM 请求/响应跟踪数据。
- **Actual behavior:** 该函数返回一个字符串, 该字符串等于将 work_dir 的跟踪目录路径 (由 _trace_dir(work_dir) 获得)、字面文件夹名 'opcode' 和文件名 {event_id}.jsonl 拼接而成的文件系统路径。形式上: 设 r 为返回值; 则 `r = os.path.join(_trace_dir(work_dir), 'opcode', event_id + '.jsonl')`。
- **Code evidence:** Line 2: `return os.path.join(_trace_dir(work_dir), "opcode", f"{event_id}.jsonl")`
- **Trigger condition:** 代码未对 event_id 进行清理, 允许路径遍历字符。例如, 若 event_id 为 '../other', 返回的路径会解析到 'opcode' 子目录之外, 且不同的 event_ids 可能映射到同一文件 (如 '../other' 和 'opcode/../other' 均得出 `trace_dir/other.jsonl`)。这违反了规范中返回路径应在预期层次结构内为给定 event_id 唯一标识跟踪文件的要求。
- **Validator trigger:** 代码未对 event_id 进行清理, 允许路径遍历字符 (如 '../other') 生成逃逸预期 opcode/ 子目录的路径, 违反了规范所要求的唯一性和包含性保证。
- **Probe stdout:** CONFIRMED — 路径遍历: `event_id='../other' → /tmp/tmpkvvgwabw/trace/other.jsonl` 逃逸了 `/tmp/tmpkvvgwabw/trace/opcode`

► SPEC sidecar (完整)

```
{
  "signature": "_opcode_trace_path(work_dir, event_id) -> str",
  "pre_condition": "work_dir 是一个存在的目录路径且 event_id 是一个非空字符串",
  "post_condition": "返回一个文件系统路径字符串, 在由 work_dir 派生的跟踪目录层次结构中, 为给定的 event_id 唯一标识一个跟踪文件。该返回路径适合写入原始 LLM 请求/响应跟踪数据。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_trace_dir",
      "signature": "_trace_dir(work_dir) -> str",
      "pre_condition": "work_dir 是一个有效的目录路径",
      "post_condition": "返回一个指向 work_dir 下跟踪目录的文件系统路径字符串"
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/_opcode_trace_path.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个文件系统路径字符串, 在由 work_dir 派生的跟踪目录层次结构中, 为给定的 event_id 唯一标识一个跟踪文件。该返回路径适合写入原始 LLM 请求/响应跟踪数据。",
    "actual_behavior": "该函数返回一个字符串, 该字符串等于将 work_dir 的跟踪目录路径 (由 _trace_dir(work_dir) 获得)、字面文件夹名 'opencode' 和文件名 {event_id}.jsonl 拼接而成的文件系统路径。形式上: 设 r 为返回值; 则 r = os.path.join(_trace_dir(work_dir), 'opencode', event_id + '.jsonl')。",
    "code_evidence": "Line 2: return os.path.join(_trace_dir(work_dir), \"opencode\", f\"{event_id}.jsonl\")",
    "trigger_condition": "代码未对 event_id 进行清理, 允许路径遍历字符。例如, 若 event_id 为 '../other', 返回的路径会解析到 'opencode' 子目录之外, 且不同的 event_ids 可能映射到同一文件 (如 '../other' 和 'opencode/../other' 均得出 trace_dir/other.jsonl)。这违反了规范中返回路径应在预期层次结构内为给定 event_id 唯一标识跟踪文件的要求。"
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--opencode_trace-py--_opcode_trace_path",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/_opcode_trace_path.py",
  "function_name": "_opcode_trace_path",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--opencode_trace-py--_opcode_trace_path.py",
  "detail_file": "fm_agent/bug_validation/src--opencode_trace-py--_opcode_trace_path.md",
  "probe_stdout": "CONFIRMED - 路径遍历: event_id='../other' -> /tmp/tmpkvvgwabw/trace/other.jsonl 逃逸了 /tmp/tmpkvvgwabw/trace/opencode",
  "trigger_summary": "代码未对 event_id 进行清理, 允许路径遍历字符 (如 '../other') 生成逃逸预期 opencode/ 子目录的路径, 违反了规范所要求的唯一性和包含性保证。"
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	目的	状态	尝试	块	开始时间	token
1	generate_block_post_condition	success	1	0/1	2026-07-27T19:01:00.985390Z	1071
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T19:01:11.789548Z	3214

事件 1 — llm_f9acc373be164d69950d6dbe5d3577d1

► parsed_json (完整)

```
{
  "post_condition": "该函数返回一个字符串，该字符串等于将 work_dir 的跟踪目录路径（由 _trace_dir(work_dir) 获得）、字面文件夹名 'opencode' 和文件名 {event_id}.jsonl 拼接而成的文件系统路径。形式上：设 r 为返回值；则 r = os.path.join(_trace_dir(work_dir), 'opencode', event_id + '.jsonl')。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_f9acc373be164d69950d6dbe5d3577d1_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_f9acc373be164d69950d6dbe5d3577d1_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_f9acc373be164d69950d6dbe5d3577d1_response.txt)

事件 2 — llm_15c0c88d711840e8b1963e8879d3aa3a

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "event_id = '../other'",
  "offending_statements": "Line 2: return os.path.join(_trace_dir(work_dir), \"opencode\\", f\"{event_id}.jsonl\")",
  "reason": "代码未对 event_id 进行清理，允许路径遍历字符。例如，若 event_id 为 '../other'，返回的路径会解析到 'opencode' 子目录之外，且不同的 event_ids 可能映射到同一文件（如 '../other' 和 'opencode/../other' 均得出 trace_dir/other.jsonl）。这违反了规范中返回路径应在预期层次结构内为给定 event_id 唯一标识跟踪文件的要求。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_15c0c88d711840e8b1963e8879d3aa3a_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_15c0c88d711840e8b1963e8879d3aa3a_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_15c0c88d711840e8b1963e8879d3aa3a_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-171 — src--opencode_trace-py--_wait_opencode_process

- 四类结论: 可能的 Bug
- 分类依据: probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- Validator: confirmed; attempts 1
- Phase / 模块: Phase 4 — LLM Communication & Tracing / opencode_trace; 原源码 src/opencode_trace.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

- **SPEC 声明：** 阻塞直到 `proc` 中的进程完成或超时到期。返回一个二元组 (`exit_code`, `error`)。当进程在 `timeout_seconds` 内退出时：`exit_code` 为整数的进程退出码，`error` 为 `None`。当进程未在 `timeout_seconds` 内退出时：`proc` 被终止 (`SIGTERM`)，若在 10 秒宽限期内终止未完成，则强制杀死 (`SIGKILL`)；`exit_code` 为强制终止后的进程退出码，当该退出码为假值 (0 或 `None`) 时映射为 -15；`error` 为一个指示超时状况的描述性字符串。
- **实际行为：** 正常返回时（无未处理的异常），函数返回二元组 (`ec`, `em`)，其中 $ec \in \mathbb{Z}$ ， $em \in \{\text{None}\} \cup \{s \mid s == f'\text{timeout after } \{\text{timeout_seconds}\}s'\}$ 。子进程 `subprocess.Popen` 实例 `proc` 已回收：`proc.wait()` 被调用（可能在 `terminate/kill` 之后），因此 `proc.returncode` $\in \mathbb{Z}$ 且子进程已不在运行。若初始的 `proc.wait(timeout=timeout_seconds)` 正常完成而未引发 `TimeoutExpired`（无超时情况），则 `ec` = `proc.returncode`，`em` = `None`。若引发 `TimeoutExpired`（超时情况），则 `em` = `f'timeout after {timeout_seconds}s'`。在超时情况下，调用了 `proc.terminate()`；若随后的 `proc.wait(timeout=10)` 引发 `TimeoutExpired`，则调用 `proc.kill()`，然后不带超时地调用 `proc.wait()`。最终，除非 `proc.returncode == 0`，否则 `ec` = `proc.returncode`，当 `proc.returncode == 0` 时 `ec` = -15（如此，超时后被杀死而发出的成功退出码永远不会被报告为成功）。属性 `proc.returncode` 等于最后一次成功 `wait` 的实际退出状态，即使 `ec` 为 -15 时其值也可能为 0。
- **代码证据：** 第 18 行: `if not exit_code:` 第 19 行: `exit_code = -15 # 超时被杀死绝不记录为成功`
- **触发条件：** 规范仅将强制终止 (`SIGKILL`) 之后的退出码映射为 -15。而代码在超时路径中无条件地将任何假值退出码映射为 -15，包括进程通过 `SIGTERM` 正常退出且退出码为 0 的情况。这导致了不匹配。
- **Validator 触发：** 超时并在 10 秒宽限期内通过 `SIGTERM` 正常退出（退出码 0）的子进程，其退出码被错误地重映射为 -15；-15 映射应仅在 `SIGKILL` 之后适用。
- **Probe stdout：** `WARNING:root:opencode test timed out after 1s, killing: echo hello CONFIRMED — actual: -15 | expected: 0`

► SPEC sidecar（完整）

```
{
  "signature": "_wait_opencode_process(proc, command, stage, timeout_seconds) -> (int, str | None)",
  "pre_condition": "proc 是一个正在运行的 subprocess.Popen 实例，其进程尚未被等待。command 是非空的字符串序列。stage 是非空字符串。timeout_seconds 是正整数。",
  "post_condition": "阻塞直到 proc 中的进程完成或超时到期。返回一个二元组 (exit_code, error)。当进程在 timeout_seconds 内退出时：exit_code 为整数的进程退出码，error 为 None。当进程未在 timeout_seconds 内退出时：proc 被终止 (SIGTERM)，若在 10 秒宽限期内终止未完成，则强制杀死 (SIGKILL)；exit_code 为强制终止后的进程退出码，当该退出码为假值 (0 或 None) 时映射为 -15；error 为一个指示超时状况的描述性字符串。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/_wait_opencode_process.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "阻塞直到 proc 中的进程完成或超时到期。返回一个二元组 (exit_code, error)。当进程在 timeout_seconds 内退出时：exit_code 为整数的进程退出码，error 为 None。当进程未在 timeout_seconds 内退出时：proc 被终止 (SIGTERM)，若在 10 秒宽限期内终止未完成，则强制杀死 (SIGKILL)；exit_code 为强制终止后的进程退出码，当该退出码为假值 (0 或 None) 时映射为 -15；error 为一个指示超时状况的描述性字符串。",
    "actual_behavior": "正常返回时（无未处理的异常），函数返回二元组 (ec, em)，其中  $ec \in \mathbb{Z}$ ， $em \in \{\text{None}\} \cup \{s \mid s == f'\text{timeout after } \{\text{timeout\_seconds}\}s'\}$ 。子进程 subprocess.Popen 实例 proc 已回收：proc.wait() 被调用（可能在 terminate/kill 之后），因此 proc.returncode  $\in \mathbb{Z}$  且子进程已不在运行。若初始的 proc.wait(timeout=timeout_seconds) 正常完成而未引发 TimeoutExpired（无超时情况），则 ec = proc.returncode，em = None。若引发 TimeoutExpired（超时情
```

```
况), 则 em = f'timeout after {timeout_seconds}s'。在超时情况下, 调用了 proc.terminate(); 若随后的
proc.wait(timeout=10) 引发 TimeoutExpired, 则调用 proc.kill(), 然后不带超时地调用 proc.wait()。最终, 除非
proc.returncode == 0, 否则 ec = proc.returncode, 当 proc.returncode == 0 时 ec = -15 (如此, 超时后被杀死而发出的
成功退出码永远不会被报告为成功)。属性 proc.returncode 等于最后一次成功 wait 的实际退出状态, 即使 ec 为 -15 时其值也可能
为 0。",
    "code_evidence": "Line 18: if not exit_code:\nLine 19:         exit_code = -15 # killed-on-timeout must
never record as success",
    "trigger_condition": "规范仅将强制终止 (SIGKILL) 之后的退出码映射为 -15。而代码在超时路径中无条件地将任何假值退出码
映射为 -15, 包括进程通过 SIGTERM 正常退出且退出码为 0 的情况。这导致了不匹配。"
}
}
```

► Bug Validator result (完整)

```
{
  "id": "src--opcode_trace-py--_wait_opcode_process",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/src/opcode_trace.py",
  "function_name": "_wait_opcode_process",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--opcode_trace-py--_wait_opcode_process.py",
  "detail_file": "fm_agent/bug_validation/src--opcode_trace-py--_wait_opcode_process.md",
  "probe_stdout": "WARNING:root:opcode test timed out after 1s, killing: echo hello\n\nCONFIRMED - actual:
-15 | expected: 0\n",
  "trigger_summary": "超时并在 10 秒宽限期内通过 SIGTERM 正常退出 (退出码 0) 的子进程, 其退出码被错误地重映射为
-15; -15 映射应仅在 SIGKILL 之后适用。"
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌, 使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T19:01:47.026720Z	2995
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T19:02:25.328498Z	4445

事件 1 — [llm_2accfe7642f14f8b99eaeac544099338](#)

► parsed_json (完整)

```
{
  "post_condition": "正常返回时 (无未处理的异常), 函数返回二元组 (ec, em), 其中 ec ∈ ℤ, em ∈ {None} ∪ {s | s ==
f'timeout after {timeout_seconds}s'}. 子进程 subprocess.Popen 实例 proc 已回收: proc.wait() 被调用 (可能在
terminate/kill 之后), 因此 proc.returncode ∈ ℤ 且子进程已不在运行。若初始的 proc.wait(timeout=timeout_seconds)
正常完成而未引发 TimeoutExpired (无超时情况), 则 ec = proc.returncode, em = None。若引发 TimeoutExpired (超时情
况), 则 em = f'timeout after {timeout_seconds}s'。在超时情况下, 调用了 proc.terminate(); 若随后的
proc.wait(timeout=10) 引发 TimeoutExpired, 则调用 proc.kill(), 然后不带超时地调用 proc.wait()。最终, 除非
proc.returncode == 0, 否则 ec = proc.returncode, 当 proc.returncode == 0 时 ec = -15 (如此, 超时后被杀死而发出的
成功退出码永远不会被报告为成功)。属性 proc.returncode 等于最后一次成功 wait 的实际退出状态, 即使 ec 为 -15 时其值也可能
为 0。"
}
```

(fm_agent/trace/payloads/llm_2accfe7642f14f8b99eaeac544099338_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_2accfe7642f14f8b99eaeac544099338_response.txt)

事件 2 — llm_9dfeb9e0abb14f9ba9f23a2a2f4d6ad7

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "一个子进程，在收到 SIGTERM 后，在 10 秒宽限期内以退出码 0 退出。例如，一个 Python 脚本：\nimport signal\nimport sys\nsignal.signal(signal.SIGTERM, lambda s, f: sys.exit(0))\nwhile True: pass\n当以 timeout_seconds=1 调用 _wait_opencode_process 时，初始 wait 超时；terminate() 发送 SIGTERM；进程捕获该信号并以 0 退出。代码将 exit_code 设为 -15，但规范要求 exit_code 为 0，因为没有发生强制杀死。",
  "offending_statements": "第 18 行:if not exit_code:\n第 19 行:         exit_code = -15 # 超时被杀死绝不记录为成功",
  "reason": "规范仅将强制终止（SIGKILL）之后的退出码映射为 -15。而代码在超时路径中无条件地将任何假值退出码映射为 -15，包括进程通过 SIGTERM 正常退出且退出码为 0 的情况。这导致了不匹配。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_9dfeb9e0abb14f9ba9f23a2a2f4d6ad7_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_9dfeb9e0abb14f9ba9f23a2a2f4d6ad7_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_9dfeb9e0abb14f9ba9f23a2a2f4d6ad7_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-177 — src--opencode_trace-py--run_opencode_traced

- 四类结论：可能有 Bug
- 分类依据：沿用旧 fm_agent/bug_list.md 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块：Phase 4 — LLM Communication & Tracing / opencode_trace; 原源码 src/opencode_trace.py
- 证据：抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** 当子进程以代码 0 退出且无错误时：返回一个 returncode=0 的 subprocess.CompletedProcess。当子进程以非零代码退出或发生错误时：引发 subprocess.CallProcessError，其 returncode 等于子进程退出码。在每一条执行路径（成功、失败或异常）中，恰好一条结构化事件记录被原子性地追加到从 work_dir 派生的 trace 目录下的 events.jsonl 文件中。记录的事件包含：唯一的 event_id、stage 标签、覆盖整个执行窗口的 ISO 8601 开始和结束时间戳、完整的命令列表、进程退出码、状态为 "success"（exit_code=0 且无错误）或 "error"（exit_code≠0 或存在错误）、提供的 function_ids、input_files、output_files、summary、error 和 metadata 值，以及对应的 opencode 日志和跟踪文件路径。所有后台线程（日志流和标准输入馈送）在此函数正常返回或传播异常之前均被 join 并终止。
- **Actual behavior:** run_opencode_traced 执行后，无论它是正常返回还是引发异常，以下均成立。形如 "opencode_<hex_uuid>" 的事件标识符 event_id 已生成，两个 ISO 8601 UTC 时间戳 started 和 ended 已捕获，且 ended >= started。调用 record_opencode_call(work_dir=work_dir, event_id=event_id, stage=stage, status='success' if exit_code == 0 else 'error', started=started, ended=ended, command=command, function_ids=function_ids, input_files=input_files, output_files=output_files, exit_code=exit_code, summary=summary, error=error, metadata=metadata, opencode_log_path=_opencode_log_path(work_dir, event_id), opencode_trace_path=_opencode_trace_path(work_dir, event_id)) 精确执行一次，将对应的 JSON 事件记录原子性地追加到从 work_dir 派生的 trace 目录下的 events.jsonl 中。exit_code 和 error 的值反映最终结果：

如果 `_wait_opencode_process` 设置了 `exit_code` 和 `error`，则 `exit_code` 是整数进程退出码，`error` 是字符串或 `None`；如果引发了 `subprocess.CalledProcessError`，则 `exit_code` 是 `exc.returncode`，`error` 是 `error` 或 `str(exc)`；否则 `exit_code` 保持 0，`error` 为 `None`。任何非 `None` 的 `log_thread` 或 `stdin_thread` 都已被 `join`（如果在 `finally` 块中仍存活，则等待其完成），确保位于 `opencode_log_path` 的日志文件被完全写入并关闭。如果 `_start_opencode_process` 未引发异常而完成，子进程 `proc` 已终止，其退出码得到反映。该函数要么正常返回一个 `subprocess.CompletedProcess` 实例，其 `args` 为 `command_argv(command)`，`returncode` 为 `exit_code`（在此情况下始终为 0），要么传播一个异常，该异常在检测到 `error` 或非零 `exit_code` 时始终为 `subprocess.CalledProcessError`，或是子进程管理产生的任何其他异常。在所有情况下，除了显式创建的文件（日志、跟踪和事件记录）外，`work_dir` 和 `proj_dir` 不被修改。

- **Code evidence:** Line 45: `status="success" if exit_code == 0 else "error"`,
- **Trigger condition:** 代码仅通过与 0 比较 `exit_code` 来决定事件状态。规格要求当 `exit_code≠0` 或存在错误时状态为 'error'。此处进程以代码 0 退出，但 `_wait_opencode_process` 报告一个错误字符串，因此状态应为 'error'，而非 'success'。
- **Validator trigger:** 当子进程以代码 0 退出但 `_wait_opencode_process` 报告错误字符串时，事件状态被记录为 'success' 而非 'error'，因为状态逻辑仅检查 `exit_code==0`，完全忽略了 `error` 参数。
- **Probe stdout:** CONFIRMED — `record_opencode_call` 收到 `status='success'`，期望为 'error'。Bug: 尽管 `error='mock error: something went wrong during execution'` 且 `exit_code=0`，`status` 为 'success'。状态逻辑仅检查 `exit_code==0`，忽略了 `error` 参数。

► SPEC sidecar（完整）

```
{
  "signature": "run_opencode_traced(*, proj_dir, work_dir, command, stage, function_ids=None,
input_files=None, output_files=None, summary=None, metadata=None) -> CompletedProcess",
  "pre_condition": "提供 proj_dir、work_dir、command 和 stage。command 是非空字符串序列。work_dir 是可写的目录路径。",
  "post_condition": "当子进程以代码 0 退出且无错误时：返回一个 returncode=0 的 subprocess.CompletedProcess。当子进程以非零代码退出或发生错误时：引发 subprocess.CalledProcessError，其 returncode 等于子进程退出码。在每一条执行路径（成功、失败或异常）中，恰好一条结构化事件记录被原子性地追加到从 work_dir 派生的 trace 目录下的 events.jsonl 文件中。记录的事件包含：唯一的 event_id、stage 标签、覆盖整个执行窗口的 ISO 8601 开始和结束时间戳、完整的命令列表、进程退出码、状态为 \"success\" (exit_code=0 且无错误) 或 \"error\" (exit_code≠0 或存在错误)、提供的 function_ids、input_files、output_files、summary、error 和 metadata 值，以及对应的 opencode 日志和跟踪文件路径。所有后台线程（日志流和标准输入馈送）在此函数正常返回或传播异常之前均被 join 并终止。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "new_event_id",
      "signature": "new_event_id(prefix) -> str",
      "pre_condition": "prefix 是字符串 \"opencode\"",
      "post_condition": "返回格式为 \"<prefix>_<hex_uuid>\" 的唯一字符串标识符，该标识符与先前调用返回的所有标识符均不同"
    },
    {
      "name": "utc_now_iso",
      "signature": "utc_now_iso() -> str",
      "pre_condition": "无",
      "post_condition": "以 ISO 8601 格式字符串返回当前的 UTC 日期和时间"
    },
    {
      "name": "_opencode_log_path",
      "signature": "_opencode_log_path(work_dir, event_id) -> str",
      "pre_condition": "work_dir 为现有目录路径，event_id 为非空字符串",
      "post_condition": "返回 work_dir 内对 event_id 唯一的文件路径，适用于写入子进程标准输出日志"
    }
  ],
}
```



```

"name": "_opencode_trace_path",
"signature": "_opencode_trace_path(work_dir, event_id) -> str",
"pre_condition": "work_dir 为现有目录路径, event_id 为非空字符串",
"post_condition": "返回 work_dir 内对 event_id 唯一的文件路径, 适用于写入原始 LLM 请求/响应跟踪数据"
},
{
"name": "_start_opencode_process",
"signature": "_start_opencode_process(proj_dir, work_dir, event_id, command, opencode_log_path) -> (Popen, Thread | None, Thread | None)",
"pre_condition": "proj_dir 和 work_dir 是有效的目录路径; command 是非空字符串列表; opencode_log_path 是可写的文件路径",
"post_condition": "返回一个三元组 (一个正在运行的 subprocess.Popen 实例, 一个将子进程标准输出流式写入 opencode_log_path 的可选线程, 一个向子进程馈送标准输入的可选线程)。子进程环境包含 OPENCODE_CONFIG_CONTENT、TRACE_DIR 和 LLM_API_KEY 注入。"
},
{
"name": "_wait_opencode_process",
"signature": "_wait_opencode_process(proc, command, stage) -> (int, str | None)",
"pre_condition": "proc 是由 _start_opencode_process 返回的正在运行的 subprocess.Popen",
"post_condition": "阻塞直到子进程终止, 然后返回 (exit_code, error), 其中 exit_code 是整数进程退出码, 若进程异常终止则 error 为错误描述字符串, 若正常退出则为 None"
},
{
"name": "command_argv",
"signature": "command_argv(command) -> list[str]",
"pre_condition": "command 是非空字符串序列",
"post_condition": "返回以可执行文件为首元素的字符串列表, 适用于错误消息和事件记录中的显示"
},
{
"name": "record_opencode_call",
"signature": "record_opencode_call(*, work_dir, event_id, stage, status, started, ended, command, function_ids, input_files, output_files, exit_code, summary, error, metadata, opencode_log_path, opencode_trace_path) -> None",
"pre_condition": "started 和 ended 为 ISO 8601 UTC 时间戳字符串, 且 ended 不早于 started; status 为 \"success\" 或 \"error\" 之一; exit_code 为整数",
"post_condition": "原子性地将一条结构化 JSON 事件记录追加到从 work_dir 派生的 trace 目录中的 events.jsonl 文件。记录包含 event_id、stage、status、started、ended、command、exit_code 以及所有可选元数据字段。调用成功时, 调用方可观察到 events.jsonl 中恰好新增一行包含完整事件。"
}
]
}

```

► Logic verification result (完整)

```

{
"function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/opencode_trace-py/run_opencode_traced.py",
"verdict": "MISMATCH",
"gaps": {
"spec_claim": "当子进程以代码 0 退出且无错误时: 返回一个 returncode=0 的 subprocess.CompletedProcess。当子进程以非零代码退出或发生错误时: 引发 subprocess.CallledProcessError, 其 returncode 等于子进程退出码。在每一条执行路径 (成功、失败或异常) 中, 恰好一条结构化事件记录被原子性地追加到从 work_dir 派生的 trace 目录下的 events.jsonl 文件中。记录的事件包含: 唯一的 event_id、stage 标签、覆盖整个执行窗口的 ISO 8601 开始和结束时间戳、完整的命令列表、进程退出码、状态为 \"success\" (exit_code=0 且无错误) 或 \"error\" (exit_code≠0 或存在错误)、提供的 function_ids、input_files、output_files、summary、error 和 metadata 值, 以及对应的 opencode 日志和跟踪文件路径。所有后台线程 (日志流和标准输入馈送) 在此函数正常返回或传播异常之前均被 join 并终止。",
"actual_behavior": "run_opencode_traced 执行后, 无论它是正常返回还是引发异常, 以下均成立。形如 \"opencode_<hex_uuid>\" 的事件标识符 event_id 已生成, 两个 ISO 8601 UTC 时间戳 started 和 ended 已捕获, 且 ended >= started。调用 record_opencode_call(work_dir=work_dir, event_id=event_id, stage=stage, status='success' if exit_code == 0 else 'error', started=started, ended=ended, command=command, function_ids=function_ids, input_files=input_files, output_files=output_files, exit_code=exit_code, summary=summary, error=error, metadata=metadata, opencode_log_path=opencode_log_path(work_dir, event_id), opencode_trace_path=opencode_trace_path(work_dir, event_id)) 精确执行一次, 将对应的 JSON 事件记录原子性地追加到从 work_dir 派生的 trace 目录下的 events.jsonl 中。exit_code 和 error 的值反映最终结果: 如果 _wait_opencode_process 设置了 exit_code 和 error, 则 exit_code 是整数进程退出码, error 是字符串或 None; 如果引发了 subprocess.CallledProcessError, 则 exit_code 是 exc.returncode, error 是 error 或 str(exc); 否则 exit_code 保持 0, error 为 None。任何非 None 的 log_thread 或 stdin_thread 都已被 join (如果在 finally 块中仍存活, 则等待其完成), 确保位于 opencode_log_path 的日志文件被完全写入并关闭。如果 _start_opencode_process 未引发异常而完成, 子进程 proc 已终止, 其退出码得到反映。该函数要么正常返回一个 subprocess.CompletedProcess 实例, 其 args 为 command_argv(command), returncode 为 exit_code (在此情况下始终为 0), 要么传播一个异常, 该异常在检测到 error 或非零

```



```
exit_code 时始终为 subprocess.CallProcessError，或是子进程管理产生的任何其他异常。在所有情况下，除了显式创建的文件（日志、跟踪和事件记录）外，work_dir 和 proj_dir 不被修改。",
"code_evidence": "Line 45: status=\"success\" if exit_code == 0 else \"error\";",
"trigger_condition": "代码仅通过与 0 比较 exit_code 来决定事件状态。规格要求当 exit_code 0 或存在错误时状态为 'error'。此处进程以代码 0 退出，但 _wait_opencode_process 报告错误字符串，因此状态应为 'error'，而非 'success'。"
}
}
```

- Bug Validator result（完整）
- Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-27T18:47:44.333132Z	4178
2	check_post_implies_spec	mismatch	1	0/1	2026-07-27T18:48:27.897344Z	6538

事件 1 — llm_7dd79333e84c48a9a91ea0dfb85529aa

- parsed_json（完整）

```
{
  "post_condition": "在执行 run_opencode_traced 之后，无论其正常返回还是抛出异常，以下条件都成立。已生成一个形如 \"opencode_<hex_uuid>\" 的事件标识符 event_id，并捕获了两个 ISO 8601 UTC 时间戳 started 和 ended，且 ended >= started。调用 record_opencode_call(work_dir=work_dir, event_id=event_id, stage=stage, status='success' if exit_code == 0 else 'error', started=started, ended=ended, command=command, function_ids=function_ids, input_files=input_files, output_files=output_files, exit_code=exit_code, summary=summary, error=error, metadata=metadata, opencode_log_path=_opencode_log_path(work_dir, event_id), opencode_trace_path=_opencode_trace_path(work_dir, event_id)) 恰好执行了一次，原子性地将相应的 JSON 事件记录追加到从 work_dir 派生的跟踪目录中的 events.jsonl 文件中。exit_code 和 error 的值反映最终结果：如果 _wait_opencode_process 设置了 exit_code 和 error，则 exit_code 为整型进程退出码，error 为字符串或 None；如果引发了 subprocess.CallProcessError，则 exit_code 为 exc.returncode，error 为 error 或 str(exc)；否则 exit_code 保持为 0，error 为 None。任何非 None 的 log_thread 或 stdin_thread 都已被 join（如果在 finally 块中仍处于活动状态，则等待其完成），确保位于 opencode_log_path 的日志文件已完全写入并关闭。如果 _start_opencode_process 在未引发异常的情况下完成，则子进程 proc 已终止，其退出码已反映出来。该函数要么正常返回一个 subprocess.CompletedProcess 实例，其 args 为 command_argv(command)，returncode 为 exit_code（在这种情况下始终为 0），要么传播一个异常，该异常在检测到 error 或非零 exit_code 时始终为 subprocess.CallProcessError，或者是由子进程管理产生的任何其他异常。在所有情况下，work_dir 和 proj_dir 均未被修改，除了显式创建的文件（日志、跟踪和事件记录）。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_7dd79333e84c48a9a91ea0dfb85529aa_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_7dd79333e84c48a9a91ea0dfb85529aa_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_7dd79333e84c48a9a91ea0dfb85529aa_response.txt)

事件 2 — llm_e2f5ae65131e43beb1f3e877b60255e1

- parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir='/tmp/proj', work_dir='/tmp/work', command=['true'], stage='test', input_files=['dummy.txt'], 其他可选参数为 None。子进程 'true' 以代码 0 非常迅速地退出，导致负责输入传输的 stdin 线程命中 BrokenPipeError。_wait_opencode_process 检测到这一异常情况并返回 (0, 'Broken pipe')。代码引发 subprocess.CallProcessError(returncode=0)，并在 finally 块中记录一个 status='success' 的事件，因为 exit_code == 0。而根据规范，当发生错误时，即使退出码为 0，status 也必须是 'error'。",
}
```

```
"offending_statements": "第45行:
    status=\"success\" if exit_code == 0 else \"error\",
    \"reason\": \"代码仅通过将 exit_code 与 0 比较来确定事件状态。规范要求当 exit_code≠0 或存在 error 时, status 为
'error'。这里进程以代码 0 退出,但 _wait_opencode_process 报告了一个 error 字符串,因此 status 应为 'error' 而非
'success'。\"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_e2f5ae65131e43beb1f3e877b60255e1_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_e2f5ae65131e43beb1f3e877b60255e1_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_e2f5ae65131e43beb1f3e877b60255e1_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足; 若可满足, 将其转为回归测试并修复实现。

模块: trace_writer

FMA-MISMATCH-182 — src--trace_writer.py--append_event

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块: Phase 4 — LLM Communication & Tracing / trace_writer; 原源码 src/trace_writer.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** 以线程安全的方式将事件字典作为自包含的 JSON 单行 (以换行符结束) 追加到位于 trace_dir 内的 events.jsonl 文件中。写入的 JSON 保留所提供事件字典中的每一个顶级键。该函数不修改事件字典参数。如果 trace_dir 目录层级不存在, 则在写入之前创建。在文件系统写入失败 (例如磁盘已满、权限被拒绝) 时, OSError 将传播给调用者。
- **Actual behavior:** 正常执行: 函数返回 None。锁 _LOCK 被获取并释放。文件 os.path.join(trace_dir, 'events.jsonl') 追加了新的一行, 包含 json.dumps(event, ensure_ascii=False) + '\n'。目录 trace_dir 存在 (如有必要由 _ensure_trace_dirs 创建)。

异常路径:

- _ensure_trace_dirs 抛出 OSError: 函数以 OSError 退出。未获取锁。未发生文件写入。对事件文件无副作用。
- json.dumps 抛出 TypeError: 函数以 TypeError 退出。trace 目录可能已被创建 (如果 _ensure_trace_dirs 成功), 但未获取锁, 也未发生文件写入。
- 文件操作 (open 或 write) 在持有锁时抛出 OSError: 锁被释放 (由 with 语句保证)。trace 目录存在 (如需要已创建)。文件可能已部分写入或未更改; 不保证原子性。OSError 传播。

形式化: post

(returns None

directory_exists(trace_dir)

appended_line(file_path, json.dumps(event, ensure_ascii=False) + '\n')

lock_released(_LOCK)) raises OSError from _ensure_trace_dirs (no file modifications) 从 json.dumps 引发 TypeError (目录可能存在, 未修改文件) raises OSError from file I/O (directory exists, lock released, file state undefined)

- **Code evidence:** Line 4: line = json.dumps(event, ensure_ascii=False)
- **Trigger condition:** 代码无条件地对输入字典调用 json.dumps(event)。如果事件字典包含一个不是有效 JSON 键的顶级键 (例如 tuple), json.dumps 会抛出 TypeError。规范 (B) 只允许在文件系统写入失败时传播 OSError,

未考虑 `TypeError`。输入 `event={1,2}: 'value'}` 是一个有效的字典，但代码的行为（抛出 `TypeError`）违反了指定的后置条件，该条件不包含此异常结果。

- **Validator trigger:** 传入带有非字符串键（例如 `tuple`）的事件字典会导致 `json.dumps` 抛出 `TypeError`，这在规范中未文档化。
- **Probe stdout:** CONFIRMED — `TypeError` raised when `json.dumps` encounters non-serializable dict key `(1, 2)`; spec only permits `OSError`

► SPEC sidecar（完整）

```
{
  "signature": "append_event(trace_dir, event) -> None",
  "pre_condition": "trace_dir 是一个字符串路径。event 是一个字典，至少包含一个非空字符串值的 'event_id' 字段和一个 ISO 8601 格式字符串值的 'timestamp' 字段。",
  "post_condition": "以线程安全的方式将事件字典作为自包含的 JSON 单行（以换行符结束）追加到位于 trace_dir 内的 events.jsonl 文件中。写入的 JSON 保留所提供事件字典中的每一个顶级键。该函数不修改事件字典参数。如果 trace_dir 目录层级不存在，则在写入之前创建。在文件系统写入失败（例如磁盘已满、权限被拒绝）时，OSError 将传播给调用者。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "_ensure_trace_dirs",
      "signature": "_ensure_trace_dirs(trace_dir) -> None",
      "pre_condition": "trace_dir 是一个字符串路径。",
      "post_condition": "如果 trace_dir 不存在，则创建 trace_dir 及任何缺失的父目录。如果 trace_dir 已存在，则无副作用。在目录创建失败（例如权限被拒绝）时，OSError 将传播给调用者。"
    }
  ]
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/trace_writer-py/append_event.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "以线程安全的方式将事件字典作为自包含的 JSON 单行（以换行符结束）追加到位于 trace_dir 内的 events.jsonl 文件中。写入的 JSON 保留所提供事件字典中的每一个顶级键。该函数不修改事件字典参数。如果 trace_dir 目录层级不存在，则在写入之前创建。在文件系统写入失败（例如磁盘已满、权限被拒绝）时，OSError 将传播给调用者。",
    "actual_behavior": "正常执行：函数返回 None。锁 `_LOCK` 被获取并释放。文件 `os.path.join(trace_dir, 'events.jsonl')` 追加了新的一行，包含 `json.dumps(event, ensure_ascii=False) + '\\n'`。目录 `trace_dir` 存在（如有必要由 `_ensure_trace_dirs` 创建）。\\n\\n异常路径：\\n- `_ensure_trace_dirs` 抛出 `OSError`：函数以 `OSError` 退出。未获取锁。未发生文件写入。对事件文件无副作用。\\n- `json.dumps` 抛出 `TypeError`：函数以 `TypeError` 退出。trace 目录可能已被创建（如果 `_ensure_trace_dirs` 成功），但未获取锁，也未发生文件写入。\\n- 文件操作（`open` 或 `write`）在持有锁时抛出 `OSError`：锁被释放（由 `with` 语句保证）。trace 目录存在（如需要已创建）。文件可能已部分写入或未更改；不保证原子性。`OSError` 传播。\\n\\n形式化：\\npost  \\n (returns None  \\n  directory_exists(trace_dir)  \\n  appended_line(file_path, json.dumps(event, ensure_ascii=False) + '\\n')  \\n  lock_released(_LOCK))\\n raises OSError from _ensure_trace_dirs (no file modifications)\\n  raises TypeError from json.dumps (directory may exist, no file modifications)\\n  raises OSError from file I/O (directory exists, lock released, file state undefined)",
    "code_evidence": "Line 4: line = json.dumps(event, ensure_ascii=False)",
    "trigger_condition": "代码无条件地对输入字典调用 json.dumps(event)。如果事件字典包含一个不是有效 JSON 键的顶级键（例如 tuple），json.dumps 会抛出 TypeError。规范 (B) 只允许在文件系统写入失败时传播 OSError，未考虑 TypeError。输入 event={1,2}: 'value'} 是一个有效的字典，但代码的行为（抛出 TypeError）违反了指定的后置条件，该条件不包含此异常结果。"
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--trace_writer-py--append_event",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/trace_writer-py/append_event.py",
  "function_name": "append_event",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--trace_writer-py--append_event.py",
  "detail_file": "fm_agent/bug_validation/src--trace_writer-py--append_event.md",
  "probe_stdout": "CONFIRMED - TypeError raised when json.dumps encounters non-serializable dict key (1, 2); spec only permits OSError\n",
  "trigger_summary": "传入带有非字符串键 (例如 tuple) 的事件字典会导致 json.dumps 抛出 TypeError, 这在规范中未文档化。"
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌, 使用每条事件后的载荷链接打开。

#	用途	状态	尝试	块	开始时间	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:13:06.689906Z	2919
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:13:48.217586Z	7545

事件 1 — [llm_c36e6cc765274ceaa86be26bbb74cf48](#)

► parsed_json (完整)

```
{
  "post_condition": "正常执行: 函数返回 None。锁 `_LOCK` 被获取并释放。文件 `os.path.join(trace_dir, 'events.jsonl')` 追加了新的一行, 包含 `json.dumps(event, ensure_ascii=False) + '\\n'`。目录 `trace_dir` 存在 (如有必要由 `_ensure_trace_dirs` 创建)。\n\n异常路径:\n- `_ensure_trace_dirs` 抛出 `OSError`: 函数以 `OSError` 退出。未获取锁。未发生文件写入。对事件文件无副作用。\n- `json.dumps` 抛出 `TypeError`: 函数以 `TypeError` 退出。trace 目录可能已被创建 (如果 `_ensure_trace_dirs` 成功), 但未获取锁, 也未发生文件写入。\n- 文件操作 (`open` 或 `write`) 在持有锁时抛出 `OSError`: 锁被释放 (由 `with` 语句保证)。trace 目录存在 (如需要已创建)。文件可能已部分写入或未更改; 不保证原子性。`OSError` 传播。\n\n形式化:\npost ≡ \n  (returns None ∧ \n    directory_exists(trace_dir) ∧ \n    appended_line(file_path, json.dumps(event, ensure_ascii=False) + '\\n') ∧ \n    lock_released(_LOCK))\n  v\n  raises OSError from _ensure_trace_dirs (no file modifications)\n  v\n  raises TypeError from json.dumps (directory may exist, no file modifications)\n  v\n  raises OSError from file I/O (directory exists, lock released, file state undefined)"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_c36e6cc765274ceaa86be26bbb74cf48_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_c36e6cc765274ceaa86be26bbb74cf48_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_c36e6cc765274ceaa86be26bbb74cf48_response.txt)

事件 2 — [llm_47e6e11b57454141b87c3ea7d5016927](#)

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "trace_dir='/tmp/test', event={(1,2): 'value'}",
  "offending_statements": "Line 4: line = json.dumps(event, ensure_ascii=False)",
  "reason": "代码无条件地对输入字典调用 json.dumps(event)。如果事件字典包含一个不是有效 JSON 键的顶级键 (例如 tuple), json.dumps 会抛出 TypeError。规范 (B) 只允许在文件系统写入失败时传播 OSError, 未考虑 TypeError。输入 event=
```

```
{(1,2): 'value'} 是一个有效的字典，但代码的行为（抛出 TypeError）违反了指定的后置条件，该条件不包含此异常结果。"
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_47e6e11b57454141b87c3ea7d5016927_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_47e6e11b57454141b87c3ea7d5016927_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_47e6e11b57454141b87c3ea7d5016927_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

Phase 5 — Specification & Verification Utilities

模块: file_utils

FMA-MISMATCH-192 — `src--file_utils.py--_get_incomplete_verification_files`

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 `fm_agent/bug_list.md` 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: `confirmed`; 尝试 1
- Phase / 模块: 第5阶段 — 规范与验证工具 / `file_utils`; 原源码 `src/file_utils.py`
- 证据: [抽取源码](#) · [规范](#) · [信息](#) · [逻辑](#) · [验证器结果](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** 返回一个列表，包含来自 `layer_files` 中需要进一步处理的文件路径，保留原始输入顺序。一个文件需要进一步处理，如果 (a) 其对应的验证结果文件 '`<output_dir>/<file_without_extension>.json`' 无法作为有效的 JSON 读取（文件缺失、不可读或不是有效的 JSON），或者 (b) 验证结果有一个顶层 '`verdict`' 键等于字符串 '`MISMATCH`' 并且对应的 bug 验证结果文件 '`<work_dir>/bug_validation/<bug_id>.result.json`' 不是由 `_json_file_is_valid` 确定的有效 JSON 文件，其中 `bug_id` 是从相对路径派生而来，通过将每一个出现的 '`/`' 和 `os.sep` 替换为 '`--`'。具有可读的验证结果且其 '`verdict`' 不是 '`MISMATCH`' 的文件将被排除在返回的列表之外。
- **Actual behavior:** 正常执行后，函数返回一个列表 `incomplete`，该列表是 `layer_files` 中元素的子序列（按迭代顺序）。对于 `rel` 中的每个相对路径 `layer_files`，`rel` 被包含在 `incomplete` 中当且仅当满足以下任一条件: (a) `os.path.join(output_dir, os.path.splitext(rel)[0] + '.json')` 处的文件不存在、无法打开读取或包含语法无效的 JSON；或 (b) 该文件存在、可读、包含有效的 JSON，加载的对象具有 `result.get('verdict') == 'MISMATCH'`，并且路径 `os.path.join(work_dir, 'bug_validation', (os.path.splitext(rel)[0].replace(os.sep, '--').replace('/', '--')) + '.result.json')` 处的 bug 验证文件根据 `_json_file_is_valid` 无效（即，它不存在、不可读或包含无效的 JSON）。`layer_files` 中的所有其他元素被省略。如果发生任何未处理的异常（例如，来自 `os` 操作），函数可能引发异常而不返回值。形式逻辑：令 `L` 为遍历 `layer_files` 获得的元素序列。那么返回的列表 `incomplete` 满足: `incomplete = [rel for rel in L | (let P = os.path.join(output_dir, os.path.splitext(rel)[0] + '.json') in ((P exists P readable P contains valid JSON))) (P exists P readable P contains valid JSON let result = load_json(P) in result.get('verdict') = 'MISMATCH' let bug_id = os.path.splitext(rel)[0].replace(os.sep, '--').replace('/', '--') in _json_file_is_valid(os.path.join(work_dir, 'bug_validation', bug_id + '.result.json')))]`
- **Code evidence:** Line 14: `bug_id = os.path.splitext(rel)[0].replace(os.sep, '--').replace('/', '--')`
- **Trigger condition:** 规范要求 `bug_id` 从相对路径派生，通过将每个 '`/`' 和 `os.sep` 替换为 '`--`'，而不移除文件扩展名。代码使用 `os.path.splitext` 在替换前剥离扩展名，导致检查不同的 bug 验证文件路径。当规范路径存在且有

效，但代码路径缺失时，代码错误地将文件包含在不完整列表中，违反了规范。

- **Validator trigger:** 代码使用 `os.path.splitext` 在构建 `bug_id` 前剥离文件扩展名，但规范要求包含扩展名；当 bug 验证结果存在于规范路径但不在代码的 `splitext` 派生路径时，函数会错误地将该条目标记为不完整。
- **Probe stdout:** Probe 工作空间: `/tmp/probe_9vajms6m` CONFIRMED — actual: `['foo/bar.py']` | expected: `[]` | `spec_validation_path exists: True`

► SPEC sidecar (完整)

```
{
  "signature": "_get_incomplete_verification_files(layer_files, input_dir, output_dir, work_dir) -> list",
  "pre_condition": "layer_files 是一个可迭代的相对文件路径字符串。output_dir 是一个包含验证结果 JSON 文件的目录的字符串路径。work_dir 是一个可能包含 bug_validation/ 子目录的目录的字符串路径。",
  "post_condition": "返回一个列表，包含来自 layer_files 中需要进一步处理的文件路径，保留原始输入顺序。一个文件需要进一步处理，如果 (a) 其对应的验证结果文件 '<output_dir>/<file_without_extension>.json' 无法作为有效的 JSON 读取（文件缺失、不可读或不是有效的 JSON），或者 (b) 验证结果有一个顶层 'verdict' 键等于字符串 'MISMATCH' 并且对应的 bug 验证结果文件 '<work_dir>/bug_validation/<bug_id>.result.json' 不是由 _json_file_is_valid 确定的有效 JSON 文件，其中 bug_id 是从相对路径派生而来，通过将每一个出现的 '/' 和 os.sep 替换为 '--'。具有可读的验证结果且其 'verdict' 不是 'MISMATCH' 的文件将被排除在返回的列表之外。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_json_file_is_valid",
      "signature": "_json_file_is_valid(file_path) -> bool",
      "pre_condition": "file_path 是一个字符串路径，指向一个其存在性和可读性尚未确定的文件。",
      "post_condition": "如果 file_path 处的文件存在、可读且包含语法有效的 JSON，则返回 True；否则返回 False。对于缺失的文件、权限拒绝的读取或格式错误的 JSON 内容，不会引发异常。"
    }
  ]
}
```

► 逻辑验证结果 (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/file_utils-py/_get_incomplete_verification_files.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个列表，包含来自 layer_files 中需要进一步处理的文件路径，保留原始输入顺序。一个文件需要进一步处理，如果 (a) 其对应的验证结果文件 '<output_dir>/<file_without_extension>.json' 无法作为有效的 JSON 读取（文件缺失、不可读或不是有效的 JSON），或者 (b) 验证结果有一个顶层 'verdict' 键等于字符串 'MISMATCH' 并且对应的 bug 验证结果文件 '<work_dir>/bug_validation/<bug_id>.result.json' 不是由 _json_file_is_valid 确定的有效 JSON 文件，其中 bug_id 是从相对路径派生而来，通过将每一个出现的 '/' 和 os.sep 替换为 '--'。具有可读的验证结果且其 'verdict' 不是 'MISMATCH' 的文件将被排除在返回的列表之外。",
    "actual_behavior": "正常执行后，函数返回一个列表 `incomplete`，该列表是 `layer_files` 中元素的子序列（按迭代顺序）。对于 `layer_files` 中的每个相对路径 `rel`，`rel` 被包含在 `incomplete` 中当且仅当满足以下任一条件：(a) `os.path.join(output_dir, os.path.splitext(rel)[0] + '.json')` 处的文件不存在、无法打开读取或包含语法无效的 JSON；或 (b) 该文件存在、可读、包含有效的 JSON，加载的对象具有 `result.get('verdict') == 'MISMATCH'`，并且路径 `os.path.join(work_dir, 'bug_validation', (os.path.splitext(rel)[0].replace(os.sep, '--').replace('/', '--')) + '.result.json')` 处的 bug 验证文件根据 `_json_file_is_valid` 无效（即，它不存在、不可读或包含无效的 JSON）。`layer_files` 中的所有其他元素被省略。如果发生任何未处理的异常（例如，来自 `os` 操作），函数可能引发异常而不返回值。形式逻辑：令 L 为遍历 `layer_files` 获得的元素序列。那么返回的列表 `incomplete` 满足：`incomplete = [ rel for rel in L | ( let P = os.path.join(output_dir, os.path.splitext(rel)[0] + '.json') in ( (P exists P readable P contains valid JSON) ) ) ( P exists P readable P contains valid JSON let result = load_json(P) in result.get('verdict') = 'MISMATCH' let bug_id = os.path.splitext(rel)[0].replace(os.sep, '--').replace('/', '--') in _json_file_is_valid(os.path.join(work_dir, 'bug_validation', bug_id + '.result.json')) ) ]`",
    "code_evidence": "Line 14: bug_id = os.path.splitext(rel)[0].replace(os.sep, '--').replace('/', '--')",
  }
}
```

```
"trigger_condition": "规范要求 bug_id 从相对路径派生, 通过将每个 '/' 和 os.sep 替换为 '--', 而不移除文件扩展名。代码使用 os.path.splitext 在替换前剥离扩展名, 导致检查不同的 bug 验证文件路径。当规范路径存在且有效, 但代码路径缺失时, 代码错误地将文件包含在不完整列表中, 违反了规范。"
}
```

► Bug 验证器结果（完整）

```
{
  "id": "src--file_utils-py--_get_incomplete_verification_files",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/file_utils-py/_get_incomplete_verification_files.py",
  "function_name": "_get_incomplete_verification_files",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--file_utils-py--_get_incomplete_verification_files.py",
  "detail_file": "fm_agent/bug_validation/src--file_utils-py--_get_incomplete_verification_files.md",
  "probe_stdout": "Probe 工作空间: /tmp/probe_9vajms6m\nCONFIRMED - actual: ['foo/bar.py'] | expected: [] | spec_validation_path exists: True",
  "trigger_summary": "代码使用 os.path.splitext 在构建 bug_id 前剥离文件扩展名, 但规范要求包含扩展名; 当 bug 验证结果存在于规范路径但不在代码的 splitext 派生路径时, 函数会错误地将该条目标记为不完整。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	目的	状态	尝试	块	开始时间	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T03:29:25.645570Z	2602
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:30:01.773481Z	4450

事件 1 — [llm_747b2d5dac8c45b484265daec26809af](#)

► parsed_json（完整）

```
{
  "post_condition": "正常执行后, 函数返回一个列表 `incomplete`, 该列表是 `layer_files` 中元素的子序列（按迭代顺序）。对于 `layer_files` 中的每个相对路径 `rel`, `rel` 被包含在 `incomplete` 中当且仅当满足以下任一条件: (a) `os.path.join(output_dir, os.path.splitext(rel)[0] + '.json')` 处的文件不存在、无法打开读取或包含语法无效的 JSON; 或 (b) 该文件存在、可读、包含有效的 JSON, 加载的对象具有 `result.get('verdict') == 'MISMATCH'`, 并且路径 `os.path.join(work_dir, 'bug_validation', (os.path.splitext(rel)[0].replace(os.sep, '--').replace('/', '--')) + '.result.json')` 处的 bug 验证文件根据 `_json_file_is_valid` 无效（即, 它不存在、不可读或包含无效的 JSON）。`layer_files` 中的所有其他元素被省略。如果发生任何未处理的异常（例如, 来自 `os` 操作）, 函数可能引发异常而不返回值。形式逻辑: 令 L 为遍历 `layer_files` 获得的元素序列。那么返回的列表 `incomplete` 满足: `incomplete = [ rel for rel in L | ( let P = os.path.join(output_dir, os.path.splitext(rel)[0] + '.json') in ( ¬(P exists ∧ P readable ∧ P contains valid JSON) ) ) v ( P exists ∧ P readable ∧ P contains valid JSON ∧ let result = load_json(P) in result.get('verdict') = 'MISMATCH' ∧ let bug_id = os.path.splitext(rel)[0].replace(os.sep, '--').replace('/', '--') in ¬_json_file_is_valid(os.path.join(work_dir, 'bug_validation', bug_id + '.result.json')) ) ]`"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_747b2d5dac8c45b484265daec26809af_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_747b2d5dac8c45b484265daec26809af_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_747b2d5dac8c45b484265daec26809af_response.txt)

事件 2 — [llm_be126ba68545437aad50f27a13c1778c](#)

► parsed_json (完整)

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_be126ba68545437aad50f27a13c1778c_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_be126ba68545437aad50f27a13c1778c_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_be126ba68545437aad50f27a13c1778c_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-193 — `src--file_utils-py--_get_phase_files`

- 四类结论: 可能有 Bug
- 分类依据: probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- Validator: `confirmed`; attempts 1
- Phase / 模块: Phase 5 — Specification & Verification Utilities / `file_utils`; 原源码 `src/file_utils.py`
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** 返回属于指定 phase 的所有提取函数文件的文件标识符（相对于 input_dir 的相对路径）列表。每个标识符都是一个非元数据 sidecar 文件的常规文件的相对路径。返回列表中不得出现重复的文件标识符。每个提取目录中的文件按其名称的字典顺序升序返回。派生提取目录在磁盘上不存在的源文件将被跳过，不会引发错误。
- **Actual behavior:** 如果前置条件成立且 phases_data 格式良好，函数将返回在所选 phase 的源文件对应的提取目录中找到的常规非元数据文件的相对文件路径列表（相对于 input_dir）。对于该 phase 中的每个 module 及每个 module 中的每个源文件，构建 `extracted_dir = os.path.join(input_dir, os.path.dirname(src_file), subdir)`，其中 `subdir` 是基名，如果存在最后一个点号则将其替换为 '-'，否则保持不变。如果该目录存在，则通过 `os.walk` 递归遍历，按字母文件名顺序收集常规文件的路径（排除 `is_metadata_sidecar` 返回 `True` 的文件），并将它们的相对路径追加到结果中。顺序保持 `module/source-file` 迭代顺序，并在每个目录内按字母文件名顺序排列。结构假设被违反（缺失键、不可迭代的值、错误的类型）将导致 `KeyError`、`TypeError`、`AttributeError` 或 `StopIteration`。如果同一文件在多个目录中遇到，返回的列表可能包含重复的相对路径。形式化逻辑: $R = \text{concatenation}\{m \text{ in modules}\} \text{concatenation}_{\{s \text{ in } m[\text{'source_files'}]\}} [\text{os.path.relpath}(\text{os.path.join}(\text{root}, \text{fname}), \text{input_dir}) \text{ for } (\text{root}, _, \text{fnames}) \text{ in } \text{sorted}(\text{os.walk}(\text{extracted_dir}(s))) \text{ for } \text{fname} \text{ in } \text{sorted}(\text{fnames}) \text{ if } \text{os.path.isfile}(\text{os.path.join}(\text{root}, \text{fname})) \text{ and not } \text{is_metadata_sidecar}(\text{fname})]$ ，其中 `extracted_dir(s)` 如上定义。
- **Code evidence:** Line 24: `phase_files.append(os.path.relpath(fpath, input_dir))`
- **Trigger condition:** 该代码从未去除重复项。如果同一提取目录被访问多次（例如，因为 `source_files` 包含重复条目），则同一个相对路径会被多次追加。规范禁止任何重复的文件标识符。
- **Validator trigger:** `source_files` 中的重复条目导致同一提取目录被多次访问，从而将完全相同的相对路径追加到结果列表中，且未进行去重。
- **Probe stdout:** CONFIRMED — actual: `['foo-py/some_func.py', 'foo-py/some_func.py']` | expected: 无重复列表，长度为 1 total entries=2, unique=1, duplicates=1

► SPEC sidecar (完整)

```
{
  "signature": "_get_phase_files(phases_data, phase_num, input_dir) -> list",
  "pre_condition": "phases_data 是完整的 phases 配置字典，其键 'phases' 映射到一个字典列表，其中至少有一个包含等于 phase_num 的 'phase' 键。phase_num 是一个标识 phase 的整数。input_dir 是指向提取函数目录的字符串路径。",
  "post_condition": "返回属于指定 phase 的所有提取函数文件的文件标识符（相对于 input_dir 的相对路径）列表。每个标识符都是一个非元数据 sidecar 文件的常规文件的相对路径。返回列表中不得出现重复的文件标识符。每个提取目录中的文件按其名称的字典顺序升序返回。派生提取目录在磁盘上不存在的源文件将被跳过，不会引发错误。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_is_metadata_sidecar",
      "signature": "_is_metadata_sidecar(fname) -> bool",
      "pre_condition": "fname 是一个表示文件名的字符串（文件路径的基名部分）。",
      "post_condition": "当 fname 标识一个元数据 sidecar 文件（伴随提取函数源文件的 .spec.json 或 .info.json 文件）时返回 True；否则返回 False。"
    }
  ]
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/file_utils-
py/_get_phase_files.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回属于指定 phase 的所有提取函数文件的文件标识符（相对于 input_dir 的相对路径）列表。每个标识符都是一个非元数据 sidecar 文件的常规文件的相对路径。返回列表中不得出现重复的文件标识符。每个提取目录中的文件按其名称的字典顺序升序返回。派生提取目录在磁盘上不存在的源文件将被跳过，不会引发错误。",
    "actual_behavior": "如果前置条件成立且 phases_data 格式良好，函数将返回在所选 phase 的源文件对应的提取目录中找到的常规非元数据文件的相对文件路径列表（相对于 input_dir）。对于该 phase 中的每个 module 及每个 module 中的每个源文件，构建 extracted_dir = os.path.join(input_dir, os.path.dirname(src_file), subdir)，其中 subdir 是基名，如果存在最后一个点号则将其替换为 '-'，否则保持不变。如果该目录存在，则通过 os.walk 递归遍历，按字母文件名顺序收集常规文件的路径（排除 _is_metadata_sidecar 返回 True 的文件），并将它们的相对路径追加到结果中。顺序保持 module/source-file 迭代顺序，并在每个目录内按字母文件名顺序排列。结构假设被违反（缺失键、不可迭代的值、错误的类型）将导致 KeyError、TypeError、AttributeError 或 StopIteration。如果同一文件在多个目录中遇到，返回的列表可能包含重复的相对路径。形式化逻辑：R = concatenation_{m in modules} concatenation_{s in m['source_files']} [ os.path.relpath(os.path.join(root, fname), input_dir) for (root, _, fnames) in sorted(os.walk(extracted_dir(s))) for fname in sorted(fnames) if os.path.isfile(os.path.join(root, fname)) and not _is_metadata_sidecar(fname) ], 其中 extracted_dir(s) 如上定义。",
    "code_evidence": "Line 24: phase_files.append(os.path.relpath(fpath, input_dir))",
    "trigger_condition": "该代码从未去除重复项。如果同一提取目录被访问多次（例如，因为 source_files 包含重复条目），则同一个相对路径会被多次追加。规范禁止任何重复的文件标识符。"
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "src--file_utils-py--_get_phase_files",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/file_utils-
py/_get_phase_files.py",
  "function_name": "_get_phase_files",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--file_utils-py--_get_phase_files.py",
  "detail_file": "fm_agent/bug_validation/src--file_utils-py--_get_phase_files.md",
  "probe_stdout": "CONFIRMED - actual: ['foo-py/some_func.py', 'foo-py/some_func.py'] | expected: 无重复列表, 长度为 1\n total entries=2, unique=1, duplicates=1\n",
  "trigger_summary": "source_files 中的重复条目导致同一提取目录被多次访问，从而将完全相同的相对路径追加到结果列表中，且未进行去重。"
}
```

► Reasoner 流程 (3 条事件)

共 3 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	format_error	1	0/1	2026-07-28T03:36:34.081205Z	4307
2	generate_block_post_condition	success	2	0/1	2026-07-28T03:37:31.521792Z	2455
3	check_post_implies_spec	mismatch	1	0/1	2026-07-28T03:37:43.691716Z	4097

事件 1 — llm_0b61691dcbfd423080865af2fae6cfff4

- parse_error: LLM response contains multiple JSON values
- 载荷: system_prompt · user_prompt · assistant_output

事件 2 — llm_074b998bcf9a40099c77604c9157ecfd

► parsed_json (完整)

```
{
  "post_condition": "如果前置条件成立且 phases_data 格式良好，函数将返回在所选 phase 的源文件对应的提取目录中找到的常规非元数据文件的相对文件路径列表（相对于 input_dir）。对于该 phase 中的每个 module 及每个 module 中的每个源文件，构建 extracted_dir = os.path.join(input_dir, os.path.dirname(src_file), subdir)，其中 subdir 是基名，如果存在最后一个点号则将其替换为 '-'，否则保持不变。如果该目录存在，则通过 os.walk 递归遍历，按字母文件名顺序收集常规文件的路径（排除 _is_metadata_sidecar 返回 True 的文件），并将它们的相对路径追加到结果中。顺序保持 module/source-file 迭代顺序，并在每个目录内按字母文件名顺序排列。结构假设被违反（缺失键、不可迭代的值、错误的类型）将导致 KeyError、TypeError、AttributeError 或 StopIteration。如果同一文件在多个目录中遇到，返回的列表可能包含重复的相对路径。形式化逻辑：R = concatenation_{m in modules} concatenation_{s in m['source_files']} [ os.path.relpath(os.path.join(root, fname), input_dir) for (root, _, fnames) in sorted(os.walk(extracted_dir(s))) for fname in sorted(fnames) if os.path.isfile(os.path.join(root, fname)) and not _is_metadata_sidecar(fname) ], 其中 extracted_dir(s) 如上定义。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_074b998bcf9a40099c77604c9157ecfd_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_074b998bcf9a40099c77604c9157ecfd_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_074b998bcf9a40099c77604c9157ecfd_message_02_assistant.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_074b998bcf9a40099c77604c9157ecfd_message_03_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_074b998bcf9a40099c77604c9157ecfd_response.txt)

事件 3 — llm_7dac04c8afb243c6805e53d4d5b172f8

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "假设 phases_data = {'phases': [{'phase': 1, 'modules': [{'source_files': ['dir/file.cpp', 'dir/file.cpp']}]}, phase_num = 1, input_dir = '/tmp'。目录 '/tmp/dir/file-cpp' 存在且包含常规文件 'func.cpp'，该文件不是元数据 sidecar。代码处理 'dir/file.cpp' 两次，每次遍历同一提取目录，导致 phase_files = ['dir/file-cpp/func.cpp', 'dir/file-cpp/func.cpp']。该列表包含重复标识符，违反了规范中关于不得出现重复文件标识符的要求。",
  "offending_statements": "Line 24: phase_files.append(os.path.relpath(fpath, input_dir))",
  "reason": "该代码从未去除重复项。如果同一提取目录被访问多次（例如，因为 source_files 包含重复条目），则同一个相对路径会被多次追加。规范禁止任何重复的文件标识符。"
}
```


- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_7dac04c8afb243c6805e53d4d5b172f8_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_7dac04c8afb243c6805e53d4d5b172f8_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_7dac04c8afb243c6805e53d4d5b172f8_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-198 — [src--file_utils-py--_json_file_is_valid](#)

- 四类结论：可能有 Bug
- 分类依据：沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: [confirmed](#)；尝试 1
- Phase / 模块：Phase 5 — 规范与验证工具 / [file_utils](#)；原源码 [src/file_utils.py](#)
- 证据：[抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator 结果](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC 声明：** 当 `file_path` 处的文件存在于文件系统中、可以打开以供读取，且其全部内容构成语法有效的 JSON 时，返回 `True`。当文件不存在、无法打开以供读取，或其内容不是语法有效的 JSON 时，返回 `False`。绝不引发异常；每一个错误条件（包括文件缺失、权限不足无法读取以及格式错误的 JSON）都会被捕获，并导致返回 `False`。
- **实际行为：** 执行后，该函数返回一个布尔值。若为 `True`，则 `path` 处的文件在调用时存在、可读，且包含语法有效的 JSON；若为 `False`，则文件不存在、不可读、不包含有效 JSON，或发生了其他 OS/JSON 错误。文件已关闭且未修改。形式化表述为：令 `result = _json_file_is_valid(p)`。则 $(result = True) \Leftrightarrow (exists_file(p) \wedge readable(p) \wedge valid_json(contents(p)))$ ，且 $(result = False) \Leftrightarrow \neg(exists_file(p) \wedge readable(p) \wedge valid_json(contents(p))) \vee raised(OSError \vee JSONDecodeError)$ 。估值在调用时刻被快照。
- **代码证据：** 第 6 行： `except (OSError, json.JSONDecodeError):`
- **触发条件：** 代码仅捕获 `OSError` 和 `json.JSONDecodeError`。以文本模式打开文件（第 3 行）时，如果文件内容无法被默认编码解码，可能会引发 `UnicodeDecodeError`。该异常未被捕获，因此函数引发异常，而不是按规范要求返回 `False`（‘绝不引发异常；每一个错误条件……都会被捕获，并导致返回 `False`’）。
- **验证器触发条件：** 以文本模式打开包含非 UTF-8 字节的文件会引发 `UnicodeDecodeError`，而该异常未被 `except (OSError, json.JSONDecodeError)` 子句捕获。
- **探测标准输出：** 已确认 — `_json_file_is_valid` 引发 `UnicodeDecodeError`，而非返回 `False`

► SPEC sidecar（完整）

```
{
  "signature": "_json_file_is_valid(file_path) -> bool",
  "pre_condition": "file_path 是一个字符串路径，指向一个其存在性和可读性尚未确定的文件。",
  "post_condition": "当 file_path 处的文件存在于文件系统中、可以打开以供读取，且其全部内容构成语法有效的 JSON 时，返回 True。当文件不存在、无法打开以供读取，或其内容不是语法有效的 JSON 时，返回 False。绝不引发异常；每一个错误条件（包括文件缺失、权限不足无法读取以及格式错误的 JSON）都会被捕获，并导致返回 False。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/file_utils-
py/_json_file_is_valid.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "当 file_path 处的文件存在于文件系统中、可以打开以供读取，且其全部内容构成语法有效的 JSON 时，返回
True。当文件不存在、无法打开以供读取，或其内容不是语法有效的 JSON 时，返回 False。绝不引发异常；每一个错误条件（包括文件缺
失、权限不足无法读取以及格式错误的 JSON）都会被捕获，并导致返回 False。",
    "actual_behavior": "执行后，该函数返回一个布尔值。若为 True，则 `path` 处的文件在调用时存在、可读，且包含语法有效的
JSON；若为 False，则文件不存在、不可读、不包含有效 JSON，或发生了其他 OS/JSON 错误。文件已关闭且未修改。形式化表述为：令
result = _json_file_is_valid(p)。则 (result = True) ⇔ (exists_file(p) ∧ readable(p) ∧
valid_json(contents(p)))，且 (result = False) ⇔ ¬(exists_file(p) ∧ readable(p) ∧ valid_json(contents(p)))
v raised(OSError v JSONDecodeError)。估值在调用时刻被快照。",
    "code_evidence": "Line 6:     except (OSError, json.JSONDecodeError):",
    "trigger_condition": "代码仅捕获 OSError 和 json.JSONDecodeError。以文本模式打开文件（第 3 行）时，如果文件内容
无法被默认编码解码，可能会引发 UnicodeDecodeError。该异常未被捕获，因此函数引发异常，而不是按规范要求返回 False（‘绝不引发
异常；每一个错误条件.....都会被捕获，并导致返回 False’）。"
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--file_utils-py--_json_file_is_valid",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/file_utils-
py/_json_file_is_valid.py",
  "function_name": "_json_file_is_valid",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--file_utils-py--_json_file_is_valid.py",
  "detail_file": "fm_agent/bug_validation/src--file_utils-py--_json_file_is_valid.md",
  "probe_stdout": "已确认 - _json_file_is_valid 引发 UnicodeDecodeError，而非返回 False\n",
  "trigger_summary": "以文本模式打开包含非 UTF-8 字节的文件会引发 UnicodeDecodeError，而该异常未被 except
(OSError, json.JSONDecodeError) 子句捕获。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	目的	状态	尝试	块	开始	令牌
1	generate_block_post_condition	成功	1	0/1	2026-07-28T03:35:46.013343Z	1432
2	check_post_implies_spec	不匹配	1	0/1	2026-07-28T03:36:06.857865Z	4895

事件 1 — [llm_49646d1fbacb499695413fa041172583](#)

► parsed_json（完整）

```
{
  "post_condition": "执行后，该函数返回一个布尔值。若为 True，则 `path` 处的文件在调用时存在、可读，且包含语法有效的
JSON；若为 False，则文件不存在、不可读、不包含有效 JSON，或发生了其他 OS/JSON 错误。文件已关闭且未修改。形式化表述为：令
result = _json_file_is_valid(p)。则 (result = True) ⇔ (exists_file(p) ∧ readable(p) ∧
valid_json(contents(p)))，且 (result = False) ⇔ ¬(exists_file(p) ∧ readable(p) ∧ valid_json(contents(p)))
v raised(OSError v JSONDecodeError)。估值在调用时刻被快照。"
}
```

(fm_agent/trace/payloads/llm_49646d1fbacb499695413fa041172583_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_49646d1fbacb499695413fa041172583_response.txt)

事件 2 — **llm_5e69c08b444847259541f838c7eaf5f3**

► **parsed_json** (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "path 指向一个存在且操作系统可读的文件，但其中包含的字节对默认文本编码无效（例如，当区域编码为 UTF-8 时，名为 'broken.txt' 的文件包含字节 0xFF）。",
  "offending_statements": "第 6 行：      except (OSError, json.JSONDecodeError):",
  "reason": "代码仅捕获 OSError 和 json.JSONDecodeError。以文本模式打开文件（第 3 行）时，如果文件内容无法被默认编码解码，可能会引发 UnicodeDecodeError。该异常未被捕获，因此函数引发异常，而不是按规范要求返回 False（‘绝不引发异常；每一个错误条件.....都会被捕获，并导致返回 False’）。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_5e69c08b444847259541f838c7eaf5f3_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_5e69c08b444847259541f838c7eaf5f3_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_5e69c08b444847259541f838c7eaf5f3_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

阶段 6 — 推理与验证引擎

模块: **incremental_pipeline**

FMA-MISMATCH-225 — **src--incremental_reasoner-py--_llm_check_caller_info_update**

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 **fm_agent/bug_list.md** 的逐条审计结论: 沿用旧清单的逐条审计结论。
- **Validator**: **confirmed**; attempts 1
- **Phase / 模块**: Phase 6 — Reasoning & Verification Engine / **incremental_pipeline**; 原源码 **src/incremental_reasoner.py**
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: 当能做出一致性判定时返回一个字典。返回的字典有键 'info_updated' (bool)。当 info_updated 为 true 时，字典还包含键 'new_info'，其值是一个符合 .info.json 模式的字典。在该 new_info 字典中，除 name 等于 callee_name 的项外，每条 callee 项的 'signature'、'pre_condition' 和 'post_condition' 值都与 caller_info_block 中的对应项相同；name 等于 callee_name 的项所具有的前置条件和后置条件不与 callee_new_spec 的前置条件或后置条件中的任何断言矛盾。当 info_updated 为 false 时，caller_info_block 中 callee_name 对应的项已经与 callee_new_spec 一致，无需修订。当无法做出判定时返回 None。
- **Actual behavior**: 成功执行时，该函数委托 _llm_select_json 并返回其结果。除 _llm_select_json 可能产生的副作用（可能包括一次 LLM 调用）外，不产生其他副作用。如果参数 work_dir、comment_prefix 和 proj_dir 的所有隐式前置条件均满足（work_dir 是有效目录路径，comment_prefix 是字符串等），则不抛出异常。

令 result = _llm_check_caller_info_update(proj_dir, work_dir, idx, caller_fqn, callee_name, lang_key, comment_prefix, callee_new_spec, caller_info_block, caller_source)。

自然语言描述：函数返回 `None` 或一个字典。如果所配置的 LLM 生成的响应是有效的 JSON，符合模式 `{"info_updated": boolean, "new_info": object|null}`，并通过了 `_validate_caller_info_update` 验证器的检查，则 `result` 为该解析后的字典；否则 `result` 为 `None`。

形式逻辑：（`result = None`）

`( result Dict hasKeys(result, {"info_updated", "new_info"}) result["info_updated"] Bool ( result["new_info"] = None result["new_info"] Dict ) alt_result ( alt_result = _llm_select_json(...) (alt_result = None result = None) (alt_result None result = alt_result) ) )`（最后一个合取项表明 `result` 恰好是 `_llm_select_json` 的返回值。）

- **Code evidence:** Line 43: `return _llm_select_json(` Line 44: `work_dir,` Line 45: `prompt_content,` Line 46: `stage="update_caller_info",` Line 47: `validator=_validate_caller_info_update,` Line 48: `schema_description={"info_updated": boolean, "new_info": object|null},` Line 49: `trace_meta={"caller_fqn": caller_fqn, "callee_name": callee_name, "idx": idx},` Line 50: `)`
- **Trigger condition:** 该函数仅经过结构验证后无条件返回 `_llm_select_json` 的结果。它并不验证返回的字典是否满足规范所要求的语义约束（例如，当 `info_updated` 为 `false` 时现有条目是否真的是一致，或当 `info_updated` 为 `true` 时 `new_info` 是否保留了其他 `callee` 条目并使指定条目变为一致）。LLM 可能生成结构上合法但违反这些约束的响应，而该函数会将其原样交付，从而破坏 Condition B。
- **Validator trigger:** `_llm_select_json` 的结果仅经过 `_validate_caller_info_update` 的结构检查即通过，没有语义验证去确认其他 `callee` 条目得到保留或指定的 `callee` 条目与新 `spec` 一致。
- **Probe stdout:** CONFIRMED — function returned the structurally valid but semantically wrong response: `other_callee entry was dropped from new_info`

► SPEC sidecar（完整）

```
{
  "signature": "_llm_check_caller_info_update(proj_dir, work_dir, idx, caller_fqn, callee_name, lang_key,
comment_prefix, callee_new_spec, caller_info_block, caller_source) -> dict | None",
  "pre_condition": "idx 是非负整数。caller_fqn 和 callee_name 是非空字符串。lang_key 是标识系统可识别语言集中一种源
语言的字符串。callee_new_spec 是一个字典，恰好包含字符串键 'signature'、'pre_condition' 和 'post_condition'。
caller_info_block 是一个符合 .info.json 模式的字典：它有一个 'callees' 键，其值为字典列表，每个字典均包含
'name'、'signature'、'pre_condition' 和 'post_condition' 字符串键。caller_source 是非空字符串。",
  "post_condition": "当能做出一致性判定时返回一个字典。返回的字典有键 'info_updated' (bool)。当 info_updated 为
true 时，字典还包含键 'new_info'，其值是一个符合 .info.json 模式的字典。在该 new_info 字典中，除 name 等于
callee_name 的项外，每条 callee 项的 'signature'、'pre_condition' 和 'post_condition' 值都与 caller_info_block
中的对应项相同；name 等于 callee_name 的项所具有的前置条件和后置条件不与 callee_new_spec 的前置条件或后置条件中的任何断
言矛盾。当 info_updated 为 false 时，caller_info_block 中 callee_name 对应的项已经与 callee_new_spec 一致，无需修
订。当无法做出判定时返回 None。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "_domain_knowledge_prompt_section",
      "signature": "_domain_knowledge_prompt_section(work_dir) -> str",
      "pre_condition": "work_dir 是一个可能包含通过 --domain-knowledge 流水线参数传入的领域知识 markdown 文件的目
录路径。",
      "post_condition": "当 work_dir 不包含领域知识文件时，返回一个空字符串。当存在领域知识文件时，返回一个适合包含在
LLM 提示中的格式化字符串，该字符串以一个标题行开头，后跟每个文件的内容。"
    },
    {
      "name": "_llm_select_json",
      "signature": "_llm_select_json(work_dir, prompt_content, stage, validator, schema_description,
trace_meta) -> dict | None",
      "pre_condition": "work_dir 是有效目录路径。prompt_content 是非空字符串。stage 是标识流水线阶段的非空字符串。
validator 是一个可调用对象，接受一个字典并返回一个 (bool, str | None) 元组，其中 bool 表示有效性，第二个元素在无效时是错
```

```

    误描述。schema_description 是描述所期望的 JSON 输出形状的非空字符串。trace_meta 是一个字典，其键为字符串，值为 JSON 可序列化对象。",
    "post_condition": "将 prompt_content 发送给已配置的 LLM。当 LLM 返回可解析为 JSON、符合 schema_description 并通过 validator 检查的输出时，返回解析后的字典。当 LLM 无输出、输出不可解析、或不是有效 JSON、不符合 schema_description 或未通过 validator 时，返回 None。该交换过程以 stage 和 trace_meta 作为标识标签进行追踪。"
  }
}
}

```

► Logic verification result（完整）

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_llm_check_caller_info_update.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "当能做出一致性判定时返回一个字典。返回的字典有键 'info_updated' (bool)。当 info_updated 为 true 时，字典还包含键 'new_info'，其值是一个符合 .info.json 模式的字典。在该 new_info 字典中，除 name 等于 callee_name 的项外，每条 callee 项的 'signature'、'pre_condition' 和 'post_condition' 值都与 caller_info_block 中的对应项相同；name 等于 callee_name 的项所具有的前置条件和后置条件不与 callee_new_spec 的前置条件或后置条件中的任何断言矛盾。当 info_updated 为 false 时，caller_info_block 中 callee_name 对应的项已经与 callee_new_spec 一致，无需修订。当无法做出判定时返回 None。",
    "actual_behavior": "成功执行时，该函数委托 _llm_select_json 并返回其结果。除 _llm_select_json 可能产生的副作用（可能包括一次 LLM 调用）外，不产生其他副作用。如果参数 work_dir、comment_prefix 和 proj_dir 的所有隐式前置条件均满足（work_dir 是有效目录路径，comment_prefix 是字符串等），则不抛出异常。\\n\\n令 result = _llm_check_caller_info_update(proj_dir, work_dir, idx, caller_fqn, callee_name, lang_key, comment_prefix, callee_new_spec, caller_info_block, caller_source)。\\n\\n自然语言描述：函数返回 None 或一个字典。如果所配置的 LLM 生成的响应是有效的 JSON，符合模式 {\\\"info_updated\\\": boolean, \\\"new_info\\\": object|null}，并通过了 _validate_caller_info_update 验证器的检查，则 result 为该解析后的字典；否则 result 为 None。\\n\\n形式逻辑：\\n( result = None )\\n\\n( result Dict hasKeys(result, {\\\"info_updated\\\", \\\"new_info\\\"}) )\\n result[\\\"info_updated\\\"] Bool \\n ( result[\\\"new_info\\\"] = None result[\\\"new_info\\\"] Dict )\\n alt_result ( alt_result = _llm_select_json(...) (alt_result = None result = None) (alt_result None result = alt_result) )\\n\\n(最后一个合取项表明 result 恰好是 _llm_select_json 的返回值。)",
    "code_evidence": "Line 43: return _llm_select_json(\\nLine 44:     work_dir,\\nLine 45:     prompt_content,\\nLine 46:     stage=\\\"update_caller_info\\\",\\nLine 47:     validator=_validate_caller_info_update,\\nLine 48:     schema_description='{\\\"info_updated\\\": boolean, \\\"new_info\\\": object|null}',\\nLine 49:     trace_meta={\\\"caller_fqn\\\": caller_fqn, \\\"callee_name\\\": callee_name, \\\"idx\\\": idx},\\nLine 50: )",
    "trigger_condition": "该函数仅经过结构验证后无条件返回 _llm_select_json 的结果。它并不验证返回的字典是否满足规范所要求的语义约束（例如，当 info_updated 为 false 时现有条目是否真的是一致，或当 info_updated 为 true 时 new_info 是否保留了其他 callee 条目并使指定条目变为一致）。LLM 可能生成结构上合法但违反这些约束的响应，而该函数会将其原样交付，从而破坏 Condition B。"
  }
}

```

► Bug Validator result（完整）

```

{
  "id": "src--incremental_reasoner-py--_llm_check_caller_info_update",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/_llm_check_caller_info_update.py",
  "function_name": "_llm_check_caller_info_update",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--_llm_check_caller_info_update.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--_llm_check_caller_info_update.md",
  "probe_stdout": "CONFIRMED - function returned the structurally valid but semantically wrong response: other_callee entry was dropped from new_info\\n",
  "trigger_summary": "_llm_select_json 的结果仅经过 _validate_caller_info_update 的结构检查即通过，没有语义验证去确认其他 callee 条目得到保留或指定的 callee 条目与新 spec 一致。"
}

```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:10:52.307695Z	4000
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:11:29.910903Z	5426

事件 1 — [llm_9e3d1aee841a4b11ab5b1774ba059a6e](#)

► [parsed_json](#) (完整)

- 载荷: [系统提示词](fm_agent/trace/payloads/llm_9e3d1aee841a4b11ab5b1774ba059a6e_message_00_system.txt) · [用户提示词](fm_agent/trace/payloads/llm_9e3d1aee841a4b11ab5b1774ba059a6e_message_01_user.txt) · [助手输出](fm_agent/trace/payloads/llm_9e3d1aee841a4b11ab5b1774ba059a6e_response.txt)

事件 2 — [llm_e1363a618dda47d8905cd563898a8eed](#)

► [parsed_json](#) (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "proj_dir='/tmp', work_dir='/tmp', idx=1, caller_fqn='mod.A', callee_name='my_callee',
  lang_key='python', comment_prefix='#', callee_new_spec={'pre_condition': 'x <= 0', 'post_condition': 'y == 0'}, caller_info_block={'my_callee': {'signature': 'def foo(x: int) -> int', 'pre_condition': 'x > 0',
  'post_condition': 'y == 0'}}, caller_source='def A(): B()'. LLM 通过 _llm_select_json 调用返回字典
  {"info_updated": false, "new_info": null}。验证器 _validate_caller_info_update 接受了它，因为它符合预期的模式。函数返回相同的字典。规范要求，当 info_updated 为 false 时，调用者针对 callee_name 的条目必须已经与被调用者的新规范一致，但此处前置条件 'x > 0' 与 'x <= 0' 矛盾，因此该条目不一致。",
  "offending_statements": "Line 43: return _llm_select_json(\nLine 44:     work_dir,\nLine 45:
  prompt_content,\nLine 46:     stage=\"update_caller_info\", \nLine 47:
  validator=_validate_caller_info_update, \nLine 48:     schema_description='{\"info_updated\": boolean,
  \"new_info\": object|null}', \nLine 49:     trace_meta={\"caller_fqn\": caller_fqn, \"callee_name\":
  callee_name, \"idx\": idx}, \nLine 50: )",
  "reason": "该函数仅经过结构验证后便无条件返回 _llm_select_json 的结果。它没有验证返回的字典是否满足规范所要求的语义约束（例如，当 info_updated 为 false 时，现有条目确实一致；或当 info_updated 为 true 时，new_info 保留其他被调用者条目并使指定条目一致）。LLM 可能生成结构上有效但违反这些约束的响应，而函数将直接交付它，从而破坏条件 B。"
}
```

- 载荷: [系统提示词](fm_agent/trace/payloads/llm_e1363a618dda47d8905cd563898a8eed_message_00_system.txt) · [用户提示词](fm_agent/trace/payloads/llm_e1363a618dda47d8905cd563898a8eed_message_01_user.txt) · [助手输出](fm_agent/trace/payloads/llm_e1363a618dda47d8905cd563898a8eed_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-244 — [src--incremental_reasoner-py--check_last_run_existence](#)

- 四类结论：可能有 Bug
- 分类依据：沿用旧 [fm_agent/bug_list.md](#) 的逐条审计结论：沿用旧清单的逐条审计结论。
- Validator: [confirmed](#)；attempts 1
- Phase / 模块：Phase 6 — Reasoning & Verification Engine / [incremental_pipeline](#)；原源码 [src/incremental_reasoner.py](#)
- 证据：[抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

- **SPEC claim:** 当且仅当以下所有条件同时满足时返回 True: (a) `proj_dir/fm_agent/phases.json` 作为常规文件存在; (b) `proj_dir/fm_agent/extracted_functions/` 作为目录存在, 并且至少包含一个非 `sidecar` 功能文件; (c) 在 `extracted_functions/` 下找到的每个非 `sidecar` 功能文件 (当提供了 `submodules` 时, 限制为那些其相对路径至少以一个 `submodule` 路径开头并跟随分隔符的文件) 都拥有有效的 `.spec.json` 和 `.info.json` `sidecar`。如果条件 (a)、(b) 或 (c) 中的任何一个不满足, 则返回 `False`, 表明上一次运行不完整, 不是增量分析的可靠基础。文件名匹配元数据 `sidecar` 命名模式 (后缀为 `.spec.json` 或 `.info.json`) 的文件不计入功能文件数量和就绪检查。该函数没有副作用: 它仅从磁盘读取。
- **Actual behavior:** 当且仅当 (1) 文件 `<proj_dir>/fm_agent/phases.json` 存在, (2) 目录 `<proj_dir>/fm_agent/extracted_functions` 存在, (3) 在 `extracted_functions` 目录树中至少存在一个不是元数据 `sidecar` (即, 不以 `.spec.json` 或 `.info.json` 结尾) 的常规文件, 并且当提供了 `submodules` 时, 其从 `extracted_functions` 起的相对路径以其中一个 `submodule` 路径开头, 并且紧接着是一个正斜杠, 以及 (4) 每个这样的文件 `f` 都满足 `is_file_ready(f)` (即, `f.spec.json` 和 `f.info.json` 都作为具有所需模式的、可访问的、有效的 JSON 文件存在) 时, 函数返回 `True`。否则, 函数返回 `False`。形式化表示: 令 `work_dir = proj_dir + '/fm_agent'`, `extracted_dir = work_dir + '/extracted_functions'`。令 $E = \{f \mid f \text{ 是 } extracted_dir \text{ 下 (递归地) 的一个文件并且不是 } sidecar(f)\}$ 。定义 $E' = \text{如果 } submodules \text{ 为 None 则为 } E, \text{ 否则为 } \{f \in E \mid is_under_submodules(relpath(f, extracted_dir), submodules)\}$ 。返回值 `V` 满足 $V (is_file(work_dir + '/phases.json') is_dir(extracted_dir) E' f E' : ready(f))$ 。
- **Code evidence:** 第 31 行: `if submodules and not _is_under_submodules(rel, submodules):`
- **Trigger condition:** 规范 (条件 B) 将条件 (b) (至少存在一个非 `sidecar` 功能文件, 不限制位置) 与条件 (c) 中基于 `submodule` 的限制分离开来。代码通过第 31-35 行, 要求所选 `submodules` 内至少有一个就绪的功能文件, 这要求更为严格。当提供了 `submodules` 但没有文件匹配它们, 而此时在其他位置存在就绪文件时, 代码返回 `False`, 而规范预期返回 `True`。
- **Validator trigger:** 当提供了 `submodules`, 但没有提取出的功能文件位于任何指定的 `submodule` 路径下, 而文件存在其他位置时, 代码返回 `False` (在 `saw_function` 计数之前应用了 `submodule` 过滤器), 但规范预期为 `True` (条件 b 仅要求任何文件无条件存在)。
- **Probe stdout:** `CONFIRMED — actual: False | expected: True (submodules=[subdir_a] but file is under subdir_b)`

► SPEC sidecar (完整)

```
{
  "signature": "check_last_run_existence(proj_dir: str, submodules: list | None) -> bool",
  "pre_condition": "proj_dir 是一个现存目录, 曾作为完整流水线运行 (run_pipeline) 的目标。如果提供, submodules 是项目相对子目录路径的非空列表。",
  "post_condition": "当且仅当以下所有条件同时满足时返回 True: (a) proj_dir/fm_agent/phases.json 作为常规文件存在; (b) proj_dir/fm_agent/extracted_functions/ 作为目录存在, 并且至少包含一个非 sidecar 功能文件; (c) 在 extracted_functions/ 下找到的每个非 sidecar 功能文件 (当提供了 submodules 时, 限制为那些其相对路径至少以一个 submodule 路径开头并跟随分隔符的文件) 都拥有有效的 .spec.json 和 .info.json sidecar。如果条件 (a)、(b) 或 (c) 中的任何一个不满足, 则返回 False, 表明上一次运行不完整, 不是增量分析的可靠基础。文件名匹配元数据 sidecar 命名模式 (后缀为 .spec.json 或 .info.json) 的文件不计入功能文件数量和就绪检查。该函数没有副作用: 它仅从磁盘读取。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "is_file_ready",
      "signature": "is_file_ready(fpath: str) -> bool",
      "pre_condition": "fpath 是 extracted_functions/ 目录内一个提取出的功能文件的绝对路径。",
      "post_condition": "当且仅当 {{fpath}}.spec.json 和 {{fpath}}.info.json 都作为分别包含所需 spec 和 info 模式的有效 JSON 文件存在时, 返回 True。如果任何一个 sidecar 缺失、不可解析或不包含预期结构, 则返回 False。"
    },
  ],
}
```

```

"name": "_is_metadata_sidecar",
"signature": "_is_metadata_sidecar(fpath: str) -> bool",
"pre_condition": "fpath 是 extracted_functions/ 目录树内的一个文件路径。",
"post_condition": "如果文件名以 .spec.json 或 .info.json 结尾, 返回 True; 否则返回 False。"
},
{
"name": "_is_under_submodules",
"signature": "_is_under_submodules(rel: str, submodules: list[str]) -> bool",
"pre_condition": "rel 是从 extracted_functions/ 根目录开始的、使用正斜杠分隔的相对路径。submodules 是一个非空的项目相对子目录路径列表, 其中不包含尾部路径分隔符。",
"post_condition": "当且仅当 rel 以 submodules 中的一个路径开头, 并且紧接着一个路径分隔符 (例如, '{submodule_path}/') 时, 返回 True。如果 rel 不以任何 submodule 路径以这种方式开头, 返回 False。"
}
]
}

```

► Logic verification result (完整)

```

{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/incremental_reasoner-py/check_last_run_existence.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "当且仅当以下所有条件同时满足时返回 True: (a) proj_dir/fm_agent/phases.json 作为常规文件存在; (b) proj_dir/fm_agent/extracted_functions/ 作为目录存在, 并且至少包含一个非 sidecar 功能文件; (c) 在 extracted_functions/ 下找到的每个非 sidecar 功能文件 (当提供了 submodules 时, 限制为那些其相对路径至少以一个 submodule 路径开头并跟随分隔符的文件) 都拥有有效的 .spec.json 和 .info.json sidecar。如果条件 (a)、(b) 或 (c) 中的任何一个不满足, 则返回 False, 表明上一次运行不完整, 不是增量分析的可靠基础。文件名匹配元数据 sidecar 命名模式 (后缀为 .spec.json 或 .info.json) 的文件不计入功能文件数量和就绪检查。该函数没有副作用: 它仅从磁盘读取。",
    "actual_behavior": "当且仅当 (1) 文件 <proj_dir>/fm_agent/phases.json 存在, (2) 目录 <proj_dir>/fm_agent/extracted_functions 存在, (3) 在 extracted_functions 目录树中至少存在一个不是元数据 sidecar (即, 不以 .spec.json 或 .info.json 结尾) 的常规文件, 并且当提供了 submodules 时, 其从 extracted_functions 起的相对路径以其中一个 submodule 路径开头, 并且紧接着是一个正斜杠, 以及 (4) 每个这样的文件 f 都满足 is_file_ready(f) (即, f.spec.json 和 f.info.json 都作为具有所需模式的、可访问的、有效的 JSON 文件存在) 时, 函数返回 True。否则, 函数返回 False。形式化表示: 令 work_dir = proj_dir + '/fm_agent', extracted_dir = work_dir + '/extracted_functions'。令 E = { f | f 是 extracted_dir 下 (递归地) 的一个文件并且不是 sidecar(f) }。定义 E' = 如果 submodules 为 None 则为 E, 否则为 { f in E | is_under_submodules(relpath(f, extracted_dir), submodules) }。返回值 V 满足 V ( is_file(work_dir + '/phases.json') is_dir(extracted_dir) E' f E' : ready(f) )。",
    "code_evidence": "Line 31: if submodules and not _is_under_submodules(rel, submodules):",
    "trigger_condition": "规范 (条件 B) 将条件 (b) (至少存在一个非 sidecar 功能文件, 不限制位置) 与条件 (c) 中基于 submodule 的限制分离开来。代码通过第 31-35 行, 要求所选 submodules 内至少有一个就绪的功能文件, 这要求更为严格。当提供了 submodules 但没有文件匹配它们, 而此时在其他位置存在就绪文件时, 代码返回 False, 而规范预期返回 True。"
  }
}

```

► Bug Validator result (完整)

```

{
  "id": "src--incremental_reasoner-py--check_last_run_existence",
  "source_file": "fm_agent/extracted_functions/src/incremental_reasoner-py/check_last_run_existence.py",
  "function_name": "check_last_run_existence",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--incremental_reasoner-py--check_last_run_existence.py",
  "detail_file": "fm_agent/bug_validation/src--incremental_reasoner-py--check_last_run_existence.md",
  "probe_stdout": "CONFIRMED - actual: False | expected: True (submodules=[subdir_a] but file is under subdir_b)",
  "trigger_summary": "当提供了 submodules, 但没有提取出的功能文件位于任何指定的 submodule 路径下, 而文件存在其他位置时, 代码返回 False (在 saw_function 计数之前应用了 submodule 过滤器), 但规范预期为 True (条件 b 仅要求任何文件无条件存在)。"
}

```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:09:14.228843Z	3132
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:09:43.030393Z	9501

事件 1 — llm_de9a5f56a7c44eb2bafcd686a5aa6504

► parsed_json (完整)

```
{
  "post_condition": "当且仅当 (1) 文件 <proj_dir>/fm_agent/phases.json 存在, (2) 目录 <proj_dir>/fm_agent/extracted_functions 存在, (3) 在 extracted_functions 目录树中至少存在一个不是元数据 sidecar (即, 不以 .spec.json 或 .info.json 结尾) 的常规文件, 并且当提供了 submodules 时, 其从 extracted_functions 起的相对路径以其中一个 submodule 路径开头, 并且紧接着是一个正斜杠, 以及 (4) 每个这样的文件 f 都满足 is_file_ready(f) (即, f.spec.json 和 f.info.json 都作为具有所需模式的、可访问的、有效的 JSON 文件存在) 时, 函数返回 True。否则, 函数返回 False。形式化表示: 令 work_dir = proj_dir + '/fm_agent', extracted_dir = work_dir + '/extracted_functions'。令 E = { f | f 是 extracted_dir 下 (递归地) 的一个文件并且不是 sidecar(f) }。定义 E' = 如果 submodules 为 None 则为 E, 否则为 { f in E | is_under_submodules(relpath(f, extracted_dir), submodules) }。返回值 v 满足 v ⇔ ( is_file(work_dir + '/phases.json') ∧ is_dir(extracted_dir) ∧ E' ≠ ∅ ∧ ∀ f ∈ E' : ready(f) )。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_de9a5f56a7c44eb2bafcd686a5aa6504_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_de9a5f56a7c44eb2bafcd686a5aa6504_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_de9a5f56a7c44eb2bafcd686a5aa6504_response.txt)

事件 2 — llm_f47c10a610de45488c89b7c256b79f8a

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "创建一个项目目录 'proj', 其中包含 'proj/fm_agent/phases.json' (常规文件) 和带有有效 'other.py.spec.json' 及 'other.py.info.json' 的 'proj/fm_agent/extracted_functions/other.py' (非 sidecar 文件)。在 'some_submodule' 子目录下不存在文件。调用 check_last_run_existence(proj_dir='proj', submodules=['some_submodule'])。代码返回 False, 因为没有功能文件位于 'some_submodule/' 下, 但规范返回 True: 条件 (b) 因 'other.py' 的存在而满足, 并且条件 (c) 因没有文件匹配 submodule 限制而空真。",
  "offending_statements": "第 31 行: if submodules and not _is_under_submodules(rel, submodules):",
  "reason": "规范 (条件 B) 将条件 (b) (至少存在一个非 sidecar 功能文件, 不限制位置) 与条件 (c) 中基于 submodule 的限制分离开来。代码通过第 31-35 行, 要求所选 submodules 内至少有一个就绪的功能文件, 这要求更为严格。当提供了 submodules 但没有文件匹配它们, 而此时在其他位置存在就绪文件时, 代码返回 False, 而规范预期返回 True。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_f47c10a610de45488c89b7c256b79f8a_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_f47c10a610de45488c89b7c256b79f8a_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_f47c10a610de45488c89b7c256b79f8a_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块: orchestration

- 四类结论：可能有 Bug
- 分类依据：probe 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- Validator: `confirmed`; attempts 1
- Phase / 模块: Phase 6 — Reasoning & Verification Engine / `orchestration`; 原源码 `dashboard.py`
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim:** `self` 反映已按顺序摄入的全部事件的累积内容，现在包括 `ev`。`self.first_event_time` 等于所有摄入事件中最早可解析的 `start_time`（如果没有则为 `None`）。`self.last_event_time` 等于所有摄入事件中最晚可解析的 `end_time`（如果没有则为 `None`）。`self.stage_counts[stage][status]` 等于所有摄入事件中具有该 `(stage, status)` 对的事件计数。对于每个 `ev['type']` 为 `'llm_call'` 的事件：`self.totals` 反映所有 LLM call 事件的输入、输出、缓存读取和缓存写入 token 数量的总和；`self.cost_native` 反映按每个 LLM call 事件计算的累积成本；`self.cache_window` 包含每个总输入 token 超过零的 LLM call 事件恰好一个 `(cache_read, total_input)` 对，按摄入顺序追加；每个事件最多递增 `self.verification_success`、`self.verification_mismatch` 或 `self.verification_error` 中的一个，由该事件的状态及其 `purpose` 是否为 `'check_post_implies_spec'` 决定；为事件追加一条 `recent-event` 条目和一条 LLM-status 条目，携带该事件的时间戳、`stage`、`status` 和 `purpose`。对于每个 `ev['type']` 为 `'opcode_call'` 的事件：为事件追加一条 `recent-event` 条目，携带该事件的时间戳、`stage`、`status` 和 `summary`。追加的条目在连续摄入过程中保持发生顺序。
- **Actual behavior:** 在 `_ingest_event` 正常执行后（无未处理异常），对象 `self` 满足以下情况。

设 `pre` 表示调用前立即的对象状态，`post` 表示调用后立即的状态。设 `stage = ev.get("stage", "?")`，`status = ev.get("status", "?")`，`et = ev.get("type")`。设 `start = _parse_iso(ev.get("start_time"))`，`end = _parse_iso(ev.get("end_time"))`。

1. 最早/最晚时间更新：

- 如果 `post.first_event_time = start` 不为 `None` 且 (`start is not None` 为 `None` 或 `pre.first_event_time is None`)，则 `start < pre.first_event_time`，否则保持 `pre.first_event_time`。
- 如果 `post.last_event_time = end` 不为 `None` 且 (`end is not None` 为 `None` 或 `pre.last_event_time is None`)，则 `end > pre.last_event_time`，否则保持 `pre.last_event_time`。

2. 阶段计数器：

- `post.stage_counts[stage][status] = pre.stage_counts[stage][status] + 1`。

3. 根据事件类型进行条件处理：

(A) 如果 `et == "llm_call"`: 设 `md = ev.get("metadata", {})`，`model = md.get("model")`。a. 如果 `post.model_seen = model` 为真值且 `model` 为真，则 `not pre.model_seen`，否则保持 `pre.model_seen`。b. 设 `usage = md.get("usage") or {}`。如果 `usage` 非空：通过 Anthropic/OpenAI 形状检测计算 token 计数：- `inp0 = usage.get("input_tokens", 0) or 0` - `out0 = usage.get("output_tokens", 0) or 0` - `cr0 = usage.get("cache_read_input_tokens", 0) or 0` - `cw0 = usage.get("cache_creation_input_tokens", 0) or 0` 如果 `inp0`, `out0`, `cr0`, `cw0` 都不为真（全为零/假值）：- `inp1 = usage.get("prompt_tokens", 0) or 0` - `out1 = usage.get("completion_tokens", 0) or 0` - `hit = usage.get("prompt_cache_hit_tokens") -` 如果 `hit is not None`: `cr1 = hit`; `inp1 = usage.get("prompt_cache_miss_tokens", inp1 -`

hit)。- 否则: `cr1 = 0; inp1 = inp1`。- 设置 `inp = inp1, out = out1, cr = cr1, cw = 0`。否则: - 设置 `inp = inp0, out = out0, cr = cr0, cw = cw0`。然后更新合计值: -
`post.totals["input"] = pre.totals["input"] + inp - post.totals["output"] =`
`pre.totals["output"] + out - post.totals["cache_read"] = pre.totals["cache_read"] +`
`cr - post.totals["cache_write"] = pre.totals["cache_write"] + cw - post.cost_native =`
`pre.cost_native + _cost_from_usage(model, usage)` - 设 `total_in = inp + cr + cw`。如果
`total_in > 0`: `post.cache_window = pre.cache_window + [(cr, total_in)]` 否则
`post.cache_window` 不变。否则 (`usage` 为空/假值): `totals`、`cost_native` 和 `cache_window` 不变。c. 验证计
数器: 如果 `status == "success"` 且 `md.get("purpose") == "check_post_implies_spec"`:
`post.verification_success = pre.verification_success + 1` 否则如果 `status ==`
`"mismatch"`: `post.verification_mismatch = pre.verification_mismatch + 1` 否则如果 `status ==`
`"error"`: `post.verification_error = pre.verification_error + 1` 否则: 验证计数器不变。
d. Summary: `summary = ev.get("summary") or md.get("purpose") or "llm_call"`。e. 时间线追
加: - 用时间戳 `post._push_recent`、`stage`、`status` 和 `summary` 调用 `end or start`。这会向 `recent-event`
时间线追加一条新条目。- 用时间戳 `post._push_llm_status`、来源 `end or start`、标签 `"direct"`、
`status`、`model`、代码 `md.get("purpose") or summary` 和详细信息 `_llm_code_for_event(status)` 调
用 `md.get("error")`。这会向 `LLM-status` 时间线追加一条新条目。

(B) 如果 `et == "opcode_call"`: a. `summary = ev.get("summary") or "opcode_call"`。b. 用
时间戳 `post._push_recent`、`stage`、`status` 和 `summary` 调用 `end or start`，追加到 `recent-event` 时间线。
c. 不进行其他属性修改 (`model_seen`、`totals`、验证计数器等保持为 `pre`)。

(C) 其他情况 (`et` 不在 `{"llm_call", "opcode_call"}` 中): 只应用 (1)(2) 描述的最早/最晚时间和阶段计数器更改;
不进行进一步修改。

所有时间线序列 (`recent-events` 和 `LLM-status`) 在调用相应 `push` 方法时各扩展一个元素, 保留之前的条目。

- **Code evidence:** 第 44 行: `elif status == "mismatch"`: 第 46 行: `elif status == "error"`:
- **Trigger condition:** 规范要求验证计数器仅在 `purpose` 为 `'check_post_implies_spec'` 时递增, 但这些分支无条件
递增 `self.verification_mismatch` 和 `self.verification_error`, 因此一个非验证 `llm_call` 事件如果状态为 `'mismatch'` 或
`'error'`, 会错误地递增相应的计数器。
- **Validator trigger:** 状态为 `'mismatch'` 或 `'error'` 的非验证 `llm_call` 事件会错误地递增验证计数器, 因为 `mismatch`
和 `error` 分支缺少 `purpose` 检查 (只有 `success` 分支检查了 `purpose == 'check_post_implies_spec'`)。
- **Probe stdout:** `CONFIRMED — verification_mismatch: 0 -> 1` (非验证事件被错误递增) | `verification_error: 0 -`
`> 1` (非验证事件被错误递增)

► SPEC sidecar (完整)

```
{
  "signature": "_ingest_event(self, ev)",
  "pre_condition": "ev 是一个字典, 从跟踪事件文件中的一个有效 JSON 对象行解析而来。ev 将字符串键映射到 JSON 类型值。",
  "post_condition": "self 反映已按顺序摄入的全部事件的累积内容, 现在包括 ev。self.first_event_time 等于所有摄入事件中  
最早可解析的 start_time (如果没有则为 None)。self.last_event_time 等于所有摄入事件中最晚可解析的 end_time (如果没有则  
为 None)。self.stage_counts[stage][status] 等于所有摄入事件中具有该 (stage, status) 对的事件计数。对于每个  
ev['type'] 为 'llm_call' 的事件: self.totals 反映所有 LLM call 事件的输入、输出、缓存读取和缓存写入 token 数量的总  
和; self.cost_native 反映按每个 LLM call 事件计算的累积成本; self.cache_window 包含每个总输入 token 超过零的 LLM  
call 事件恰好一个 (cache_read, total_input) 对, 按摄入顺序追加; 每个事件最多递增 self.verification_success、  
self.verification_mismatch 或 self.verification_error 中的一个, 由该事件的状态及其 purpose 是否为  
'check_post_implies_spec' 决定; 为事件追加一条 recent-event 条目和一条 LLM-status 条目, 携带该事件的时间戳、stage、  
status 和 purpose。对于每个 ev['type'] 为 'opcode_call' 的事件: 为事件追加一条 recent-event 条目, 携带该事件的时间  
戳、stage、status 和 summary。追加的条目在连续摄入过程中保持发生顺序。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "_parse_iso",
      "signature": "_parse_iso(ts)",
      "pre_condition": "ts 是从 ev 字典的 'start_time' 或 'end_time' 字段提取的字符串值，如果该字段缺失或为假值则为 None。",
      "post_condition": "如果 ts 是有效的 ISO 8601 时间戳字符串，则返回解析后的 datetime 值。如果 ts 为 None 或无法解析为有效时间戳，则返回 None。"
    },
    {
      "name": "_cost_from_usage",
      "signature": "_cost_from_usage(model, usage)",
      "pre_condition": "model 是字符串模型标识符或 None。usage 是包含反映单次 LLM 请求消耗的 token 计数字段的字典。",
      "post_condition": "返回给定模型和 token 计数的美元货币成本。如果 model 为 None 或无法识别，返回 0.0。"
    },
    {
      "name": "_push_recent",
      "signature": "_push_recent(self, ts, stage, status, summary)",
      "pre_condition": "ts 是 datetime 或 None。stage、status 和 summary 是描述事件的字符串。",
      "post_condition": "向 self 的内存中 recent-event 时间线追加一条新条目，记录时间戳、stage、status 和 summary。条目按调用顺序追加。"
    },
    {
      "name": "_push_llm_status",
      "signature": "_push_llm_status(self, ts, source, label, status, model, code, detail)",
      "pre_condition": "ts 是 datetime 或 None。source 是标识来源的字符串（例如 'direct' 或 'opencode'）。label、status、model、code 和 detail 是描述 LLM call 结果的字符串。",
      "post_condition": "向 self 的内存中 LLM-status 时间线追加一条新条目，记录时间戳、source、label、status、model、状态码和详细信息。条目按调用顺序追加。"
    },
    {
      "name": "_llm_code_for_event",
      "signature": "_llm_code_for_event(status)",
      "pre_condition": "status 是表示事件状态值的字符串。",
      "post_condition": "返回与状态值对应的简短显示码字符串。对于没有特定映射的状态值，返回默认码。"
    }
  ]
}
```

► Logic verification result (完整)

► Bug Validator 结果 (完整)

```
{
  "id": "dashboard-py--State::_ingest_event",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/dashboard.py",
  "function_name": "State::_ingest_event",
  "confirmation_status": "confirmed",
  "attempts": "1",
  "probe_script": "fm_agent/bug_validation/probe_dashboard-py--State::_ingest_event.py",
  "detail_file": "fm_agent/bug_validation/dashboard-py--State::_ingest_event.md",
  "probe_stdout": "已确认 - verification_mismatch: 0 -> 1 (非验证事件错误递增) | verification_error: 0 -> 1 (非验证事件错误递增)",
  "trigger_summary": "状态为 'mismatch' 或 'error' 的非验证 llm_call 事件错误递增验证计数器，因为 mismatch 和 error 分支缺少 purpose 检查 (仅 success 分支检查 purpose == 'check_post_implies_spec')。"
}
```

► Reasoner 流程 (2 条事件)

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:25:02.405913Z	8976
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:26:46.925125Z	8397

事件 1 — llm_9b11d7b9daa74b43ae2a38f40450e17a

► parsed_json (完整)

```
{
  "post_condition": "在 `_ingest_event` 正常执行（未发生未处理异常）后，以下内容适用于对象 `self`。\\n\\n令 `pre` 表示调用前瞬间的对象状态，`post` 表示调用后瞬间的状态。令 `stage = ev.get(\\\"stage\\\", \\\"?\\\")`、`status = ev.get(\\\"status\\\", \\\"?\\\")`、`et = ev.get(\\\"type\\\")`。令 `start = _parse_iso(ev.get(\\\"start_time\\\"))`、`end = _parse_iso(ev.get(\\\"end_time\\\"))`。\\n\\n1. 首次/末次时间更新：\\n    - 若 `start is not None` 且 (`pre.first_event_time is None` 或 `start < pre.first_event_time`)，则 `post.first_event_time = start`，否则为 `pre.first_event_time`。\\n    - 若 `end is not None` 且 (`pre.last_event_time is None` 或 `end > pre.last_event_time`)，则 `post.last_event_time = end`，否则为 `pre.last_event_time`。\\n\\n2. 阶段计数器：\\n    - `post.stage_counts[stage][status] = pre.stage_counts[stage][status] + 1`。\\n\\n3. 根据事件类型条件：\\n\\n    (A) 若 `et == \\\"llm_call\\\"`：\\n        令 `md = ev.get(\\\"metadata\\\", {})`、`model = md.get(\\\"model\\\")`。\\n        a. 若 `model` 为真值且 `not pre.model_seen` 为真值，则 `post.model_seen = model`，否则为 `pre.model_seen`。\\n        b. 令 `usage = md.get(\\\"usage\\\") or {}`。若 `usage` 非空：\\n            通过 Anthropic/OpenAI 形状检测计算 token 数量：\\n            - `inp0 = usage.get(\\\"input_tokens\\\", 0) or 0`\\n            - `out0 = usage.get(\\\"output_tokens\\\", 0) or 0`\\n            - `cr0 = usage.get(\\\"cache_read_input_tokens\\\", 0) or 0`\\n            - `cw0 = usage.get(\\\"cache_creation_input_tokens\\\", 0) or 0`\\n            若 `inp0, out0, cr0, cw0` 均非真值（全为零/假值）：\\n            - `inp1 = usage.get(\\\"prompt_tokens\\\", 0) or 0`\\n            - `out1 = usage.get(\\\"completion_tokens\\\", 0) or 0`\\n            - `hit = usage.get(\\\"prompt_cache_hit_tokens\\\")`\\n            - 若 `hit is not None`：`cr1 = hit`；`inp1 =`\\n            `usage.get(\\\"prompt_cache_miss_tokens\\\", inp1 - hit)`。\\n            - 否则：`cr1 = 0`；`inp1 = inp1`。\\n            - 设置 `inp = inp1`、`out = out1`、`cr = cr1`、`cw = 0`。\\n            否则：\\n            - 设置 `inp = inp0`、`out = out0`、`cr = cr0`、`cw = cw0`。\\n            然后更新合计：\\n            - `post.totals[\\\"input\\\"] = pre.totals[\\\"input\\\"] + inp`\\n            - `post.totals[\\\"output\\\"] = pre.totals[\\\"output\\\"] + out`\\n            - `post.totals[\\\"cache_read\\\"] = pre.totals[\\\"cache_read\\\"] + cr`\\n            - `post.totals[\\\"cache_write\\\"] = pre.totals[\\\"cache_write\\\"] + cw`\\n            - `post.cost_native = pre.cost_native + _cost_from_usage(model, usage)`\\n            - 令 `total_in = inp + cr + cw`。若 `total_in > 0`：\\n            - `post.cache_window = pre.cache_window + [(cr, total_in)]`\\n            否则 `post.cache_window` 不变。\\n            否则（`usage` 为空/假值）：`totals`、`cost_native` 和 `cache_window` 不变。\\n            c. 验证计数器：\\n            若 `status == \\\"success\\\"` 且 `md.get(\\\"purpose\\\") == \\\"check_post_implies_spec\\\"`：\\n            - `post.verification_success = pre.verification_success + 1`\\n            否则若 `status == \\\"mismatch\\\"`：\\n            - `post.verification_mismatch = pre.verification_mismatch + 1`\\n            否则若 `status == \\\"error\\\"`：\\n            - `post.verification_error = pre.verification_error + 1`\\n            否则：验证计数器不变。\\n            d. 摘要：`summary = ev.get(\\\"summary\\\") or md.get(\\\"purpose\\\") or \\\"llm_call\\\"`。\\n            e. 时间线追加：\\n            - 调用 `post._push_recent`，参数为时间戳 `end or start`、`stage`、`status` 和 `summary`。此调用将新条目追加到最近事件时间线。\\n            - 调用 `post._push_llm_status`，参数为时间戳 `end or start`、数据源 `\\\"direct\\\"`、标签 `md.get(\\\"purpose\\\") or summary`、`status`、`model`、代码 `_llm_code_for_event(status)` 以及详情 `md.get(\\\"error\\\")`。此调用将新条目追加到 LLM 状态时间线。\\n\\n    (B) 若 `et == \\\"opcode_call\\\"`：\\n        a. `summary = ev.get(\\\"summary\\\") or \\\"opcode_call\\\"`。\\n        b. 调用 `post._push_recent`，参数为时间戳 `end or start`、`stage`、`status` 和 `summary`，追加到最近事件时间线。\\n        c. 无其他属性修改（`model_seen`、`totals`、验证计数器等保持为 `pre` 状态）。\\n\\n    (C) 其他情况（`et` 不在 `{\\\"llm_call\\\", \\\"opcode_call\\\"}` 中）：\\n        仅应用 (1)-(2) 中描述的首次/末次时间和阶段计数器更改；无进一步修改。\\n\\n所有时间线序列（`recent-events` 和 `LLM-status`）在调用相应 `push` 方法时各扩展一个元素，保留先前的条目。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_9b11d7b9daa74b43ae2a38f40450e17a_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_9b11d7b9daa74b43ae2a38f40450e17a_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_9b11d7b9daa74b43ae2a38f40450e17a_response.txt)

事件 2 — llm_d2fd29c382f3492591d801d4e2d6b5ed

► `parsed_json` (完整)

- 载荷: `[system_prompt]`

`(fm_agent/trace/payloads/llm_d2fd29c382f3492591d801d4e2d6b5ed_message_00_system.txt) · [user_prompt]`

`(fm_agent/trace/payloads/llm_d2fd29c382f3492591d801d4e2d6b5ed_message_01_user.txt) · [assistant_output]`

`(fm_agent/trace/payloads/llm_d2fd29c382f3492591d801d4e2d6b5ed_response.txt)` **复核动作**

- 核对 `probe` 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

FMA-MISMATCH-252 — `dashboard-py` - `State::scan_bugs`

- 四类结论：可能有 Bug
- 分类依据： `probe` 复现了具体的状态破坏、数据遗漏、错误路径/对象或并发与安全风险，列为实现缺陷候选。
- **Validator**: `confirmed`; attempts 1
- **Phase / 模块**: Phase 6 — 推演与验证引擎 / `orchestration`; 原源码 `dashboard.py`
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: `self.bugs_confirmed`、`self.bugs_not_confirmed` 和 `self.bugs_pending` 会根据 `self.bug_dir` 的当前状态更新。当该目录存在时，会对每个 `.result.json` 文件进行分类：若文件的 `confirmation_status`（不区分大小写）中包含子字符串 `'confirm'` 且不包含 `'not'`，则 `bugs_confirmed` 自增；任何其他文件则使 `bugs_not_confirmed` 自增。`self.bugs_pending` 取 `max(0, completed_bug_validation_stages - (bugs_confirmed + bugs_not_confirmed))`。当 `self.bug_dir` 不存在时，这三个计数器均为零。
- **实际行为**: 执行后，属性 `self.bugs_confirmed`、`self.bugs_not_confirmed` 和 `self.bugs_pending` 会按以下规则更新，且不会修改其他属性。设 `E` 为方法执行前的 `self.bug_dir.exists()`。如果 `E`，则 `self.bugs_confirmed = 0`、`self.bugs_not_confirmed = 0` 和 `self.bugs_pending = max(0, bv_done)`，其中 `bv_done = sum(self.stage_counts.get('bug_validation', {}).values())`（使用原始值）。如果 `E`，定义：
- $F = \{ p : p \text{ self.bug_dir.glob}(*.result.json) \}$ （与该模式匹配的文件路径集合）
- $S = \{ p \in F : \text{打开并解析 } p \text{ 作为 JSON 成功, 且不引发任何异常} \}$
- 对于每个 `p ∈ S`，令 `status(p) = (d.get('confirmation_status') or '').lower()`，其中 `d` 是解析后的 JSON 对象
- $C = \{ p \in S : 'confirm' \text{ status}(p) \text{ 'not' status}(p) \}$
- $N = S \setminus C$

然后: `self.bugs_confirmed = |C|` `self.bugs_not_confirmed = |N|` `bv_done = self.stage_counts.get('bug_validation', {})` 中值的总和（以方法入口时的值为准）
`self.bugs_pending = max(0, bv_done - |S|)`。

即 `bugs_confirmed` 用于计数那些经解析且其 `confirmation_status` 的小写形式包含 `"confirm"` 但不包含 `"not"` 的文件；`bugs_not_confirmed` 用于计数其余已解析的文件（包含 `"not"` 或两者皆无的文件）；`bugs_pending` 取零与 `'bug_validation'` 阶段计数值之和（若缺失则为 0）减去成功解析的结果文件总数所得差值的最大值。

- **代码证据**: Line 4: `if not self.bug_dir.exists()`: Line 5: `return`
- **触发条件**: 当目录不存在时，代码会提前返回，而未将 `self.bugs_pending` 设为零。条件 A（实际行为）导致 `self.bugs_pending` 取 `max(0, bv_done)`（或保留其之前的值），而条件 B 明确要求这三个计数器都必须为零。

- **Validator trigger:** 当 self.bug_dir 不存在时, scan_bugs() 提前返回而未将 self.bugs_pending 重置为零, 导致残留一个旧值。
- **Probe stdout:** CONFIRMED — bugs_pending: 5 (应为 0), bugs_confirmed: 0, bugs_not_confirmed: 0

► SPEC sidecar (完整)

```
{
  "signature": "State.scan_bugs(self)",
  "pre_condition": "self.bug_dir 是一个 Path 属性。当目录 self.bug_dir 存在时, 可能包含零个或多个 .result.json 文件。每个此类文件在作为有效 JSON 时, 都拥有一个 'confirmation_status' 键, 其值为字符串。",
  "post_condition": "self.bugs_confirmed、self.bugs_not_confirmed 和 self.bugs_pending 会根据 self.bug_dir 的当前状态更新。当该目录存在时, 会对每个 .result.json 文件进行分类: 若文件的 confirmation_status (不区分大小写) 中包含子字符串 'confirm' 且不包含 'not', 则 bugs_confirmed 自增; 任何其他文件则使 bugs_not_confirmed 自增。self.bugs_pending 取 max(0, completed_bug_validation_stages - (bugs_confirmed + bugs_not_confirmed))。当 self.bug_dir 不存在时, 这三个计数器均为零。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result (完整)

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/dashboard-py/State::scan_bugs.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "self.bugs_confirmed、self.bugs_not_confirmed 和 self.bugs_pending 会根据 self.bug_dir 的当前状态更新。当该目录存在时, 会对每个 .result.json 文件进行分类: 若文件的 confirmation_status (不区分大小写) 中包含子字符串 'confirm' 且不包含 'not', 则 bugs_confirmed 自增; 任何其他文件则使 bugs_not_confirmed 自增。self.bugs_pending 取 max(0, completed_bug_validation_stages - (bugs_confirmed + bugs_not_confirmed))。当 self.bug_dir 不存在时, 这三个计数器均为零。",
    "actual_behavior": "执行后, 属性 `self.bugs_confirmed`、`self.bugs_not_confirmed` 和 `self.bugs_pending` 会按以下规则更新, 且不会修改其他属性。设 `E` 为方法执行前的 `self.bug_dir.exists()`。如果 `E`, 则 `self.bugs_confirmed = 0`、`self.bugs_not_confirmed = 0` 和 `self.bugs_pending = max(0, bv_done)`, 其中 `bv_done = sum(self.stage_counts.get('bug_validation', {}).values())` (使用原始值)。如果 `E`, 定义: \n\n- `F` = { p : p self.bug_dir.glob('*.result.json') } (与该模式匹配的文件路径集合)\n- `S` = { p F : 打开并解析 `p` 作为 JSON 成功, 且不引发任何异常 }\n- 对于每个 `p S`, 令 `status(p)` = `(d.get('confirmation_status') or '').lower()`, 其中 `d` 是解析后的 JSON 对象\n- `C` = { p S : 'confirm' status(p) 'not' status(p) }\n- `N` = `S \ C`\n\n然后: \n`self.bugs_confirmed` = |C| \n`self.bugs_not_confirmed` = |N| \n`bv_done` = `self.stage_counts.get('bug_validation', {})` 中值的总和 (以方法入口时的值为准) \n`self.bugs_pending` = max(0, bv_done - |S|) 。\n\n即 `bugs_confirmed` 用于计数那些经解析且其 confirmation_status 的小写形式包含 `confirm` 但不包含 `not` 的文件; `bugs_not_confirmed` 用于计数其余已解析的文件 (包含 `not` 或两者皆无的文件); `bugs_pending` 取零与 `bug_validation` 阶段计数值之和 (若缺失则为 0) 减去成功解析的结果文件总数所得差值的最大值。",
    "code_evidence": "Line 4:         if not self.bug_dir.exists():\nLine 5:             return",
    "trigger_condition": "当目录不存在时, 代码会提前返回, 而未将 self.bugs_pending 设为零。条件 A (实际行为) 导致 self.bugs_pending 取 max(0, bv_done) (或保留其之前的值), 而条件 B 明确要求这三个计数器都必须为零。"
  }
}
```

► Bug Validator result (完整)

```
{
  "id": "dashboard-py--State::scan_bugs",
  "source_file": "dashboard.py",
  "function_name": "scan_bugs",
  "confirmation_status": "confirmed",
}
```



```
"attempts": "1",
"probe_script": "fm_agent/bug_validation/probe_dashboard-py--State::scan_bugs.py",
"detail_file": "fm_agent/bug_validation/dashboard-py--State::scan_bugs.md",
"probe_stdout": "CONFIRMED - bugs_pending: 5 (应为 0), bugs_confirmed: 0, bugs_not_confirmed: 0\n",
"trigger_summary": "当 self.bug_dir 不存在时, scan_bugs() 提前返回而未将 self.bugs_pending 重置为零, 导致残留一个旧值。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:09:39.859966Z	3376
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:10:19.094956Z	5904

事件 1 — llm_497b467ee7954a80b5c1d17290c2a96a

► parsed_json（完整）

```
{
  "post_condition": "执行后, 属性 `self.bugs_confirmed`、`self.bugs_not_confirmed` 和 `self.bugs_pending` 会按以下规则更新, 且不会修改其他属性。设 `E` 为方法执行前的 `self.bug_dir.exists()`。如果 `¬E`, 则 `self.bugs_confirmed = 0`、`self.bugs_not_confirmed = 0` 和 `self.bugs_pending = max(0, bv_done)`，其中 `bv_done = sum(self.stage_counts.get('bug_validation', {}).values())`（使用原始值）。如果 `E`, 定义: `F` = { p : p ∈ self.bug_dir.glob('*.result.json') }（与该模式匹配的文件路径集合）\n- `S` = { p ∈ F : 打开并解析 `p` 作为 JSON 成功, 且不引发任何异常 }\n- 对于每个 `p ∈ S`, 令 `status(p)` = `(d.get('confirmation_status') or '').lower()`，其中 `d` 是解析后的 JSON 对象\n- `C` = { p ∈ S : 'confirm' ∈ status(p) ∧ 'not' ∉ status(p) }\n- `N` = `S \ C`\n\n然后: \n`self.bugs_confirmed` = `|C|`\n`self.bugs_not_confirmed` = `|N|`\n`bv_done` = `self.stage_counts.get('bug_validation', {})` 中值的总和（以方法入口时的值为准）\n`self.bugs_pending` = `max(0, bv_done - |S|)`。 \n\n即 `bugs_confirmed` 用于计数那些经解析且其 `confirmation_status` 的小写形式包含 `"confirm"` 但不包含 `"not"` 的文件; `bugs_not_confirmed` 用于计数其余已解析的文件（包含 `"not"` 或两者皆无的文件）; `bugs_pending` 取零与 `bug_validation` 阶段计数值之和（若缺失则为 0）减去成功解析的结果文件总数所得差值的最大值。"
}
```

- 载荷: [system_prompt]
(fm_agent/trace/payloads/llm_497b467ee7954a80b5c1d17290c2a96a_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_497b467ee7954a80b5c1d17290c2a96a_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_497b467ee7954a80b5c1d17290c2a96a_response.txt)

事件 2 — llm_21cc2d66f96d4c0dbe1bc5044fc1923b

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "self.bug_dir 不存在 (self.bug_dir.exists() 返回 False), self.stage_counts = {'bug_validation': {'completed': 3}}, bv_done = 3, 且 self.bugs_pending 初始值为 10。在 scan_bugs() 执行后, 规范要求 self.bugs_pending 为 0, 但代码（如条件 A 所述）将其设为 max(0, bv_done) = 3（或从字面上看, 保留其原始值不变）, 该值非零, 违反了要求。",
  "offending_statements": "Line 4:         if not self.bug_dir.exists():\nLine 5:         return",
  "reason": "当目录不存在时, 代码会提前返回, 而未将 self.bugs_pending 设为零。条件 A（实际行为）导致 self.bugs_pending 取 max(0, bv_done)（或保留其之前的值）, 而条件 B 明确要求这三个计数器都必须为零。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_21cc2d66f96d4c0dbe1bc5044fc1923b_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_21cc2d66f96d4c0dbe1bc5044fc1923b_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_21cc2d66f96d4c0dbe1bc5044fc1923b_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块: **reasoner**

FMA-MISMATCH-272 — **src--reasoner-py--_compute_brace_depth_per_line**

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: **confirmed**; attempts 1
- Phase / 模块: Phase 6 — Reasoning & Verification Engine / **reasoner**; 原源码 **src/reasoner.py**
- 证据: [抽取源码](#) · [SPEC](#) · [INFO](#) · [Logic](#) · [Validator result](#) · [probe](#) · [详细报告](#)

冲突摘要

- **SPEC claim**: 返回一个与 lines 长度相同的整数列表。对于每个索引 i, 位置 i 处的元素等于在 lines[0] 到 lines[i] 的合并文本中, 未匹配的 '{' 字符净计数减去未匹配的 '}' 字符净计数, 其中, 出现在任何双引号字符串字面量、单引号字符串字面量、行注释 (从 '/' 到行尾) 或块注释 (位于 '/' 和 '/' 之间) 内部的 '{' 和 '}' 均不计入, 无论注释是否跨行。第一行之前的初始深度为 0。
- **Actual behavior**: 该函数返回列表 'depths', 使得 len(depths) == len(lines), 并且对于每个从 0 到 len(lines)-1 的索引 i, depths[i] 是按照以下规则处理 lines[0..i] 后的大括号深度: 初始深度为 0; 对于每一行, 从左到右扫描字符, 并且任何属于双引号字符串字面量 (以未转义的 '"' 开始和结束, 反斜杠转义跳过下一个字符)、单引号字符串字面量 (同理)、行注释 (以 '/' 开始直到行尾) 或块注释 (以 '/' 开始并在同一行以 '/' 结束, 或者如果未闭合, 则为该行剩余部分) 的字符均被忽略。未被忽略的 '{' 使深度加 1, 未被忽略的 '}' 使深度减 1。depths[i] 是第 i 行末尾的深度。跨行块注释不被识别; 各行独立处理。
- **Code evidence**: Line 40: if ch == '/' and i + 1 < len(line) and line[i + 1] == ': Line 41: i += 2 Line 42: while i < len(line): Line 43: if line[i] == " and i + 1 < len(line) and line[i + 1] == ': Line 44: i += 2 Line 45: break Line 46: i += 1 Line 47: # 如果块注释跨行, 我们忽略其中的大括号 (简化处理) Line 48: continue
- **Trigger condition**: 该代码独立处理每一行, 不跨行携带块注释状态, 因此位于多行块注释内部的大括号不会被忽略, 违反了要求所有块注释内部的大括号均需排除 (无论是否跨行) 的规范。
- **Validator trigger**: 多行块注释不会跨行追踪, 因此跨越多个行的 /* ... */ 内部的大括号被当作真实大括号计数, 而非被排除。
- **Probe stdout**: CONFIRMED — actual: [0, 1, 1] | expected: [0, 0, 0]

► SPEC sidecar (完整)

```
{
  "signature": "_compute_brace_depth_per_line(lines: list[str]) -> list[int]",
  "pre_condition": "lines 是一个非空的字符串列表, 每个字符串代表一行源代码。",
  "post_condition": "返回一个与 lines 长度相同的整数列表。对于每个索引 i, 位置 i 处的元素等于在 lines[0] 到 lines[i] 的合并文本中, 未匹配的 '{' 字符净计数减去未匹配的 '}' 字符净计数, 其中, 出现在任何双引号字符串字面量、单引号字符串字面量、行注释 (从 '/' 到行尾) 或块注释 (位于 '/' 和 '/' 之间) 内部的 '{' 和 '}' 均不计入, 无论注释是否跨行。第一行之前的初始深度为 0。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/reasoner-
py/_compute_brace_depth_per_line.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个与 lines 长度相同的整数列表。对于每个索引 i，位置 i 处的元素等于在 lines[0] 到 lines[i] 的
合并文本中，未匹配的 '{' 字符净计数减去未匹配的 '}' 字符净计数，其中，出现在任何双引号字符串字面量、单引号字符串字面量、行注释
（从 '//' 到行尾）或块注释（位于 '/*' 和 '*/' 之间）内部的 '{' 和 '}' 均不计入，无论注释是否跨行。第一行之前的初始深度为
0。",
    "actual_behavior": "该函数返回列表 'depths'，使得 len(depths) == len(lines)，并且对于每个从 0 到
len(lines)-1 的索引 i，depths[i] 是按照以下规则处理 lines[0..i] 后的大括号深度：初始深度为 0；对于每一行，从左到右扫描
字符，并且任何属于双引号字符串字面量（以未转义的 '\"' 开始和结束，反斜杠转义跳过下一个字符）、单引号字符串字面量（同理）、行注释
（以 '//' 开始直到行尾）或块注释（以 '/*' 开始并在同一行以 '*/' 结束，或者如果未闭合，则为该行剩余部分）的字符均被忽略。未
被忽略的 '{' 使深度加 1，未被忽略的 '}' 使深度减 1。depths[i] 是第 i 行末尾的深度。跨行块注释不被识别；各行独立处理。",
    "code_evidence": "Line 40:         if ch == '/' and i + 1 < len(line) and line[i + 1] ==
':':\nLine 41:             i += 2\nLine 42:             while i < len(line):\nLine 43:                 i +=
if line[i] == '*' and i + 1 < len(line) and line[i + 1] == '/':\nLine 44:                     i +=
2\nLine 45:                     break\nLine 46:                     i += 1\nLine 47:                     #
If block comment spans lines, we ignore braces inside it (simplified)\nLine 48:                     continue",
    "trigger_condition": "该代码独立处理每一行，不跨行携带块注释状态，因此位于多行块注释内部的大括号不会被忽略，违反了要求
所有块注释内部的大括号均需排除（无论是否跨行）的规范。"
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--reasoner-py--_compute_brace_depth_per_line",
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/reasoner-
py/_compute_brace_depth_per_line.py",
  "function_name": "_compute_brace_depth_per_line",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--reasoner-py--_compute_brace_depth_per_line.py",
  "detail_file": "fm_agent/bug_validation/src--reasoner-py--_compute_brace_depth_per_line.md",
  "probe_stdout": "CONFIRMED - actual: [0, 1, 1] | expected: [0, 0, 0]\n",
  "trigger_summary": "多行块注释不会跨行追踪，因此跨越多个行的 /* ... */ 内部的大括号被当作真实大括号计数，而非被排除。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T05:12:59.374572Z	1981
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T05:13:20.996312Z	3037

事件 1 — [llm_dc3af6ed2a6b411a9d723bb546d1cb77](#)

► parsed_json（完整）

```
{
  "post_condition": "该函数返回列表 'depths', 使得 len(depths) == len(lines), 并且对于每个从 0 到 len(lines)-1 的索引 i, depths[i] 是按照以下规则处理 lines[0..i] 后的大括号深度: 初始深度为 0; 对于每一行, 从左到右扫描字符, 并且任何属于双引号字符串字面量 (以未转义的 '\"' 开始和结束, 反斜杠转义跳过下一个字符)、单引号字符串字面量 (同理)、行注释 (以 '//' 开始直到行尾) 或块注释 (以 '/*' 开始并在同一行以 '*/' 结束, 或者如果未闭合, 则为该行剩余部分) 的字符均被忽略。未被忽略的 '{' 使深度加 1, 未被忽略的 '}' 使深度减 1。depths[i] 是第 i 行末尾的深度。跨行块注释不被识别; 各行独立处理。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_dc3af6ed2a6b411a9d723bb546d1cb77_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_dc3af6ed2a6b411a9d723bb546d1cb77_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_dc3af6ed2a6b411a9d723bb546d1cb77_response.txt)

事件 2 — llm_ed01aca097ca4f3f836a0444a48d5190

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "lines = ['{', '/* {', '}', '*/', '}]'",
  "offending_statements": "Line 40:           if ch == '/' and i + 1 < len(line) and line[i + 1] ==
'':\nLine 41:           i += 2\nLine 42:           while i < len(line):\nLine 43:
if line[i] == '*' and i + 1 < len(line) and line[i + 1] == '/':\nLine 44:           i +=
2\nLine 45:           break\nLine 46:           i += 1\nLine 47:           #
如果块注释跨行, 我们忽略其中的大括号 (简化处理)\nLine 48:           continue",
  "reason": "该代码独立处理每一行, 不跨行携带块注释状态, 因此位于多行块注释内部的大括号不会被忽略, 违反了要求所有块注释内部的大括号均需排除 (无论是否跨行) 的规范。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_ed01aca097ca4f3f836a0444a48d5190_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_ed01aca097ca4f3f836a0444a48d5190_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_ed01aca097ca4f3f836a0444a48d5190_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足; 若可满足, 将其转为回归测试并修实现。

模块: scope_analysis

FMA-MISMATCH-284 — src--scope-py--_llm_rerank

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块: Phase 6 — Reasoning & Verification Engine / scope_analysis; 原源码 src/scope.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** 成功时: 返回一个函数名字字符串列表, 顺序由 LLM 评估的与 issue 相关性决定, 其长度至多为 top_k, 且无重复名称。每一个返回的名称都出现在 funcs_info 中。失败时 (在 3 次尝试后, LLM 响应不是有效的非空函数名字字符串的 JSON 数组): 返回 None。初始 LLM 调用包含格式化的函数列表、filepath、issue 和 top_k; 当响应不是有效的非空字符串 JSON 数组时, 最多会进行两次额外重试, 并在会话后追加纠正指令, 每次重试之间伴有递增的退避延迟。该函数不抛出异常; 所有尝试中的错误均被捕获, 并导致要么重试, 要么返回 None。

- **Actual behavior:** 该函数要么返回一个从 LLM 响应中的有效 JSON 数组解析出的非空字符串列表（函数名），要么在所有三次尝试均失败时返回 None。输入参数保持不变。所有异常均在内部捕获，不会传播。副作用（日志记录、休眠、本地消息列表修改）不影响调用方。形式化后置条件：(返回值为 None) (返回值为一个字符串列表 返回值中的 s, isinstance(s, str) s.strip() != "")。列表可以为空。如果返回，该列表源自三次尝试内首次成功的 LLM 调用，该调用响应中包含一个可解析的字符串 JSON 数组，且每个字符串非空。否则，在三次失败尝试后，返回 None。
- **Code evidence:** Line 29: names = _parse_json_response(text) Line 30: if not isinstance(names, list) or not all(Line 31: isinstance(name, str) and name.strip() for name in names Line 32:): Line 33: raise ValueError("LLM rerank response must be a JSON array of non-empty function names") Line 34: return [name.strip() for name in names]
- **Trigger condition:** 代码仅确保响应是非空字符串的 JSON 数组。它并未检查返回列表的长度 top_k、是否包含重复项、或是否仅包含 funcs_info 中存在的名称，上述任何一条违反都将违背规格说明。
- **Validator trigger:** Mock LLM 在 top_k=2 时返回 4 个函数名，超出长度限制 —— 该函数未经校验便返回全部 4 个。
- **Probe stdout:** CONFIRMED — 规格要求 len <= 2，但得到 4 个名称：['foo', 'bar', 'baz', 'qux']

► SPEC sidecar（完整）

```
{
  "signature": "_llm_rerank(funcs_info: list[dict], source_lines: list[str], filepath: str, issue: str,
top_k: int, llm_client: Any, model: str) -> list[str] | None",
  "pre_condition": "funcs_info 是一个非空的函数元数据字典列表，每个字典至少包含 'name'（字符串）和 'score'（浮点数）
键，并按分数降序排列。source_lines 是以字符串列表表示的文件内容，每行一个。filepath 是一个非空字符串，标识源文件。issue
是一个非空字符串，描述开发者意图。top_k 是一个正整数，指定要选择的最大函数数量。llm_client 是一个已配置的 LLM API 客户端，
其 chat.completions.create 接受 model、messages、max_tokens 和 temperature 参数。model 是一个非空字符串，用于标识
LLM 模型。",
  "post_condition": "成功时：返回一个函数名字符串列表，顺序由 LLM 评估的与 issue 相关性决定，其长度至多为 top_k，且无重
复名称。每一个返回的名称都出现在 funcs_info 中。失败时（在 3 次尝试后，LLM 响应不是有效的非空函数名字符串的 JSON 数组）：
返回 None。初始 LLM 调用包含格式化的函数列表、filepath、issue 和 top_k；当响应不是有效的非空字符串 JSON 数组时，最多会进
行两次额外重试，并在会话后追加纠正指令，每次重试之间伴有递增的退避延迟。该函数不抛出异常；所有尝试中的错误均被捕获，并导致要么
重试，要么返回 None。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": [
    {
      "name": "_build_func_list_text",
      "signature": "_build_func_list_text(funcs_info: list[dict], source_lines: list[str]) -> str",
      "pre_condition": "funcs_info 是一个函数元数据字典列表，每个字典至少包含 'name' 和 score 键。source_lines 是
一个字符串列表，表示源文件内容，每行一个。",
      "post_condition": "返回 funcs_info 中所有函数的字符串表示，适合嵌入 LLM 提示，包括每个函数的：名称、行范围（起始
到结束），以及从 source_lines 中该行范围摘取的主体文本。"
    },
    {
      "name": "_parse_json_response",
      "signature": "_parse_json_response(text: str) -> list[str]",
      "pre_condition": "text 是一个非空字符串，可能包含一个字符串 JSON 数组，周围可能有 markdown 格式或额外空白。",
      "post_condition": "返回从 text 中第一个找到的 JSON 数组中解析出的非空字符串列表。当 text 不包含可解析的非空字
符串 JSON 数组时，引发异常。"
    }
  ]
}
```

► Logic verification result（完整）


```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/scope-
py/_llm_rerank.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "成功时：返回一个函数名字字符串列表，顺序由 LLM 评估的与 issue 相关性决定，其长度至多为 top_k，且无重复
名称。每一个返回的名称都出现在 funcs_info 中。失败时（在 3 次尝试后，LLM 响应不是有效的非空函数名字字符串的 JSON 数组）：返
回 None。初始 LLM 调用包含格式化的函数列表、filepath、issue 和 top_k；当响应不是有效的非空字符串 JSON 数组时，最多会进行
两次额外重试，并在会话后追加纠正指令，每次重试之间伴有递增的退避延迟。该函数不抛出异常；所有尝试中的错误均被捕获，并导致要么重
试，要么返回 None。",
    "actual_behavior": "该函数要么返回一个从 LLM 响应中的有效 JSON 数组解析出的非空字符串列表（函数名），要么在所有三次
尝试均失败时返回 None。输入参数保持不变。所有异常均在内部捕获，不会传播。副作用（日志记录、休眠、本地消息列表修改）不影响调用
方。格式化后置条件：（返回值为 None）（返回值为一个字符串列表 返回值中的 s, isinstance(s, str) s.strip() != ''）。列
表可以为空。如果返回，该列表源自三次尝试内首次成功的 LLM 调用，该调用响应中包含一个可解析的字符串 JSON 数组，且每个字符串非
空。否则，在三次失败尝试后，返回 None。",
    "code_evidence": "Line 29: names = _parse_json_response(text)\nLine 30: if not isinstance(names, list)
or not all(\nLine 31:     isinstance(name, str) and name.strip() for name in names\nLine 32: ):\nLine 33:
raise ValueError(\"LLM rerank response must be a JSON array of non-empty function names\")\nLine 34:
return [name.strip() for name in names]",
    "trigger_condition": "代码仅确保响应是非空字符串的 JSON 数组。它并未检查返回列表的长度 top_k、是否包含重复项、或是
是否仅包含 funcs_info 中存在的名称，上述任何一条违反都将违背规格说明。"
  }
}
```

► Bug Validator result（完整）

```
{
  "id": "src--scope-py--_llm_rerank",
  "source_file": "src/scope.py",
  "function_name": "_llm_rerank",
  "confirmation_status": "confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--scope-py--_llm_rerank.py",
  "detail_file": "fm_agent/bug_validation/src--scope-py--_llm_rerank.md",
  "probe_stdout": "CONFIRMED - 规格要求 len <= 2, 但得到 4 个名称: ['foo', 'bar', 'baz', 'qux']",
  "trigger_summary": "Mock LLM 在 top_k=2 时返回 4 个函数名, 超出长度限制 — 该函数未经校验便返回全部 4 个。"
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T04:46:20.815655Z	2448
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T04:46:48.139364Z	2696

事件 1 — [llm_d05b76387aee4dfcb0c6d76d3d40ad8a](#)

► parsed_json（完整）

```
{
  "post_condition": "该函数要么返回一个从 LLM 响应中的有效 JSON 数组解析出的非空字符串列表（函数名），要么在所有三次尝试均失败时返回 None。输入参数保持不变。所有异常均在内部捕获，不会传播。副作用（日志记录、休眠、本地消息列表修改）不影响调用方。形式化后置条件：(返回值为 None) ∨ (返回值为一个字符串列表 ∧ ∀ 返回值中的 s, isinstance(s, str) ∧ s.strip() != '')。列表可以为空。如果返回，该列表源自三次尝试内首次成功的 LLM 调用，该调用响应中包含一个可解析的字符串 JSON 数组，且每个字符串非空。否则，在三次失败尝试后，返回 None。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_d05b76387aee4dfcb0c6d76d3d40ad8a_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_d05b76387aee4dfcb0c6d76d3d40ad8a_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_d05b76387aee4dfcb0c6d76d3d40ad8a_response.txt)

事件 2 — llm_f2f94e5a9ba64a42bf35ac0a0c27225a

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "输入: funcs_info = [{'name': 'foo'}, {'name': 'bar'}], top_k = 1, 有效 LLM 响应解析为 ['foo', 'bar']。该函数返回 ['foo', 'bar'], 而规格要求至多 1 个名称。",
  "offending_statements": "Line 29: names = _parse_json_response(text)\nLine 30: if not isinstance(names, list) or not all(\nLine 31:     isinstance(name, str) and name.strip() for name in names\nLine 32: ): \nLine 33:     raise ValueError(\"LLM rerank response must be a JSON array of non-empty function names\")\nLine 34: return [name.strip() for name in names]",
  "reason": "代码仅确保响应是非空字符串的 JSON 数组。它并未检查返回列表的长度 ≤ top_k、是否包含重复项、或是否仅包含 funcs_info 中存在的名称，上述任何一条违反都将违背规格说明。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_f2f94e5a9ba64a42bf35ac0a0c27225a_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_f2f94e5a9ba64a42bf35ac0a0c27225a_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_f2f94e5a9ba64a42bf35ac0a0c27225a_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

模块: topdown_layers

FMA-MISMATCH-301 — src--generate_topdown_layers-py--_get_call_regex

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: confirmed; attempts 1
- Phase / 模块: Phase 6 — Reasoning & Verification Engine / topdown_layers; 源代码 src/generate_topdown_layers.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** 返回一个编译的正则表达式。当应用于已去除注释的源代码时，每次匹配的第一个捕获组会生成一个裸词标识符，该标识符在由 lang_key 标识的语言的语法调用位置处位于左括号 '(' 之前。语言允许的位于标识符和括号之间的任何类型参数语法（例如，尖括号分隔或方括号分隔的泛型参数）均被容忍。标识符、可选类型参数和括号之间允许存在空格。

Actual behavior: 返回一个编译的正则表达式 (re.Pattern)，用于匹配由 lang_key 给定的语言中的函数调用点。返回的正则表达式将函数名捕获到组 1 中，并确保其后跟着一个左括号。具体的模式取决于 lang_key：对于 'cpp'、'c'、'java'、'typescript'、'javascript'、'cuda'、'arkts'，它包含可选的模板参数 <...>；对于 'rust'，它包含可选的 turbofish ::<...>；对于 'go'，它包含可选的类型参数 [...]; 对于任何其他 lang_key（例如 'python'、'ruby'），它匹配一个普通标识符，后跟可选的空格和 '('。形式上：令 L = {'cpp','c','java','typescript','javascript','cuda','arkts'}，则 (lang_key L) return = re.compile(r"\b(\w+)\s*(?:<[^\>]*>)?\s(" (lang_key = 'rust') return = re.compile(r"\b(\w+)\s*(?:::<[^\>]*>)?\s(" (lang_key = 'go') return = re.compile(r"\b(\w+)\s*(?:\[[^\]]*\])?\s(" (lang_key L {'rust','go'}) return = re.compile(r"\b(\w+)\s*(")。

• **Code evidence:** Line 5: return re.compile(r"\b(\w+)\s*(?:<[^\>]*>)?\s(" Line 8: return re.compile(r"\b(\w+)\s*(?:::<[^\>]*>)?\s(" Line 11: return re.compile(r"\b(\w+)\s*(?:\[[^\]]*\])?\s(")

Trigger condition: 这些正则表达式模式使用了 [^\>]* 和 [^\]]*，它们不允许 C++、Java、Rust、Go 等语言中典型的模板/泛型参数嵌套括号，因此无法匹配像 foo<A>(x) 这样带有嵌套类型参数的有效调用点。 **Validator trigger:** 正则表达式模式使用了 [^\>]* 和 [^\]]*，在模板/泛型参数中遇到嵌套括号时会失败（例如 foo<A>(x)）。 **Probe stdout:** CONFIRMED — 嵌套括号正则表达式 bug 已复现（3 处不匹配） [cpp] cpp 嵌套模板参数：输入='foo<A>(x)' 期望='foo' 实际=None [rust] rust 嵌套 turbofish：输入='bar::<Vec>(x)' 期望='bar' 实际=None [go] go 嵌套类型参数：输入='fn map[string]int' 期望='fn' 实际=None 阳性对照 (func(x)) 匹配正确

► SPEC sidecar（完整）

```
{
  "signature": "_get_call_regex(lang_key) -> re.Pattern",
  "pre_condition": "lang_key 是一个字符串键，系统中为其定义了调用点正则表达式。",
  "post_condition": "返回一个编译的正则表达式。当应用于已去除注释的源代码时，每次匹配的第一个捕获组会生成一个裸词标识符，该标识符在由 lang_key 标识的语言的语法调用位置处位于左括号 '(' 之前。语言允许的位于标识符和括号之间的任何类型参数语法（例如，尖括号分隔或方括号分隔的泛型参数）均被容忍。标识符、可选类型参数和括号之间允许存在空格。"
}
```

► INFO / callees sidecar（完整）

```
{
  "callees": []
}
```

► Logic verification result（完整）

```
{
  "function": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/generate_topdown_layers-py/_get_call_regex.py",
  "verdict": "MISMATCH",
  "gaps": {
    "spec_claim": "返回一个编译的正则表达式。当应用于已去除注释的源代码时，每次匹配的第一个捕获组会生成一个裸词标识符，该标识符在由 lang_key 标识的语言的语法调用位置处位于左括号 '(' 之前。语言允许的位于标识符和括号之间的任何类型参数语法（例如，尖括号分隔或方括号分隔的泛型参数）均被容忍。标识符、可选类型参数和括号之间允许存在空格。",
    "actual_behavior": "返回一个编译的正则表达式 (re.Pattern)，用于匹配由 lang_key 给定的语言中的函数调用点。返回的正则表达式将函数名捕获到组 1 中，并确保其后跟着一个左括号。具体的模式取决于 lang_key：对于 'cpp'、'c'、'java'、'typescript'、'javascript'、'cuda'、'arkts'，它包含可选的模板参数 `<...>`；对于 'rust'，它包含可选的 turbofish `::<...>`；对于 'go'，它包含可选的类型参数 `[...]`；对于任何其他 lang_key（例如 'python'、'ruby'），它匹配一个普通标识符后跟可选的空格和 '('。形式上：令 L = {'cpp','c','java','typescript','javascript','cuda','arkts'}，则 (lang_key L) return = re.compile(r"\\b(\\w+)\\s*(?:<[^\>]*>)?\\s*\\(\\)" (lang_key = 'rust') return = re.compile(r"\\b(\\w+)\\s*(?:::<[^\>]*>)?\\s*\\(\\)" (lang_key = 'go') return = re.compile(r"\\b(\\w+)\\s*(?:\\[[^\]]*\\])?\\s*\\(\\)" (lang_key L {'rust','go'}) return = re.compile(r"\\b(\\w+)\\s*\\(\\)".",
    "code_evidence": "Line 5: return re.compile(r"\\b(\\w+)\\s*(?:<[^\>]*>)?\\s*\\(\\)"\\nLine 8: return re.compile(r"\\b(\\w+)\\s*(?:::<[^\>]*>)?\\s*\\(\\)"\\nLine 11: return re.compile(r"\\b(\\w+)\\s*(?:\\[[^\]]*\\])?\\s*\\(\\)",
    "trigger_condition": "这些正则表达式模式使用了 [^\>]* 和 [^\]]*，它们不允许 C++、Java、Rust、Go 等语言中典型的模
```

板/泛型参数嵌套括号，因此无法匹配像 `foo<A<B>>(x)` 这样带有嵌套类型参数的有效调用点。

```
}  
}
```

► Bug Validator result（完整）

```
{  
  "id": "src--generate_topdown_layers-py--_get_call_regex",  
  "source_file": "/home/fancy/Projects_Vault/FM-Agent/fm_agent/extracted_functions/src/generate_topdown_layers-py/_get_call_regex.py",  
  "function_name": "_get_call_regex",  
  "confirmation_status": "confirmed",  
  "attempts": "1",  
  "probe_script": "fm_agent/bug_validation/probe_src--generate_topdown_layers-py--_get_call_regex.py",  
  "detail_file": "fm_agent/bug_validation/src--generate_topdown_layers-py--_get_call_regex.md",  
  "probe_stdout": "CONFIRMED - 嵌套括号正则表达式 bug 已复现 (3 处不匹配)\n [cpp] cpp 嵌套模板参数:  
input='foo<A<B>>(x)' expected='foo' actual=None\n [rust] rust 嵌套 turbofish: input='bar::<Vec<u8>>(x)'  
expected='bar' actual=None\n [go] go 嵌套类型参数: input='fn[map[string]int](x)' expected='fn'  
actual=None\n positive control (func<int>(x)) matched correctly",  
  "trigger_summary": "正则表达式模式使用 [^>]* 和 [^\\]]*，在模板/泛型参数中遇到嵌套括号时会失败（例如 foo<A<B>>(x)>）"  
}
```

► Reasoner 流程（2 条事件）

共 2 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/1	2026-07-28T05:57:04.200840Z	1200
2	check_post_implies_spec	mismatch	1	0/1	2026-07-28T05:57:16.922959Z	4026

事件 1 — [llm_f12643103eb942e0b6992f69407ead37](#)

► parsed_json（完整）

```
{  
  "post_condition": "返回一个编译的正则表达式 (re.Pattern)，用于匹配由 lang_key 给定的语言中的函数调用点。返回的正则表达式将函数名捕获到组 1 中，并确保其后跟着一个左括号。具体的模式取决于 lang_key：对于 'cpp', 'c', 'java', 'typescript', 'javascript', 'cuda', 'arkts'，它包含可选的模板参数 <...>；对于 'rust'，它包含可选的 turbofish ::<...>；对于 'go'，它包含可选的类型参数 [...]；对于任何其他 lang_key（例如 'python', 'ruby'），它匹配一个普通标识符后跟可选的空格和 '('。形式上：令 L = {'cpp', 'c', 'java', 'typescript', 'javascript', 'cuda', 'arkts'}，则 (lang_key ∈ L) ⇒ return = re.compile(r"\\b(\\w+)\\s*(?:<[>]*>?\\s*\\(\\") ∧ (lang_key = 'rust') ⇒ return = re.compile(r"\\b(\\w+)\\s*(?:::<[>]*>?\\s*\\(\\") ∧ (lang_key = 'go') ⇒ return = re.compile(r"\\b(\\w+)\\s*(?:\\[[^\\]]*\\]\\s*\\(\\") ∧ (lang_key ∉ L ∪ {'rust', 'go'}) ⇒ return = re.compile(r"\\b(\\w+)\\s*\\(\\")。"  
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_f12643103eb942e0b6992f69407ead37_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_f12643103eb942e0b6992f69407ead37_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_f12643103eb942e0b6992f69407ead37_response.txt)

事件 2 — [llm_5bc427f2a1fb428892306cc1b26a2ac6](#)

► parsed_json (完整)

```
{
  "verdict": "MISMATCH",
  "counterexample": "lang_key='cpp', source='foo<A<B>>(x)'",
  "offending_statements": "Line 5: return re.compile(r\"\\b(\\w+)\\s*(?:<[>]*>)?\\s*\\(\"\\nLine 8:
return re.compile(r\"\\b(\\w+)\\s*(?:::<[>]*>)?\\s*\\(\"\\nLine 11: return re.compile(r\"\\b(\\w+)\\s*
(?:\\[[^\\]]*\\])?\\s*\\(\"\\n",
  "reason": "这些正则表达式模式使用了 [>]* 和 [^\\]]*, 它们不允许 C++、Java、Rust、Go 等语言中典型的模板/泛型参数嵌套
括号, 因此无法匹配像 foo<A<B>>(x) 这样带有嵌套类型参数的有效调用点。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_5bc427f2a1fb428892306cc1b26a2ac6_message_00_system.txt) · [user_prompt]
(fm_agent/trace/payloads/llm_5bc427f2a1fb428892306cc1b26a2ac6_message_01_user.txt) · [assistant_output]
(fm_agent/trace/payloads/llm_5bc427f2a1fb428892306cc1b26a2ac6_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足; 若可满足, 将其转为回归测试并修实现。

模块: verification

FMA-MISMATCH-310 — src--verification-py--_validate_single_bug

- 四类结论: 可能有 Bug
- 分类依据: 沿用旧 fm_agent/bug_list.md 的逐条审计结论: 沿用旧清单的逐条审计结论。
- Validator: not_confirmed; attempts 1
- Phase / 模块: Phase 6 — Reasoning & Verification Engine / verification; 原源码
src/verification.py
- 证据: 抽取源码 · SPEC · INFO · Logic · Validator result · probe · 详细报告

冲突摘要

- **SPEC claim:** 当对应于 result_json_rel 的差异的结果标记文件在 <work_dir>/bug_validation/<bug_id>.result.json 处不存在, 或 resume 为 False 时, 会分派一个 bug 验证代理来评估 result_json_rel 中记录的规范与代码差异是否构成一个真实、可利用的 bug。只有当该差异被确认为可利用的 bug 时, 才会在 <work_dir>/bug_validation/<bug_id>.md 处生成一份经过验证的 bug 报告。该 bug 报告包含: 规范声称的、而代码违反了的行为断言, 从代码中观察到的实际行为, 附有行号引用的具体代码证据, 触发该违规的条件, 逐步重现指令, 一个演示该违规的探测脚本, 以及执行探测脚本的原始输出。当验证完成时, 会在 <work_dir>/bug_validation/<bug_id>.result.json 处写入一个结果标记文件。当 resume 为 True 且该结果标记文件已包含有效、可解析的 JSON 内容时, 不会重新验证, 也不会产生任何副作用。bug_id 是通过去除标准的验证结果路径前缀并将目录分隔符规范化后, 从 result_json_rel 唯一派生而得的。
- **Actual behavior:** 代码块之后, 将出现以下三种情况之一: (1) 抛出了未处理的异常 (例如, 来自 os.makedirs、文件写入、os.replace 或 build_llm_cli_command)。此时程序可能已中止, 仅产生部分副作用; 在异常点之后, 对变量不做进一步保证。(2) 没有异常发生, 并且函数在第 73 行提前返回, 因为 resume 为 truthy, result_path 存在, 并且文件能被成功读取并解析为 JSON。那么以下成立: 目录 os.path.join(work_dir, 'bug_validation') 存在; 位于 prompt_path 的文件存在, 且其全部内容等于 prompt_content; 临时文件 tmp_path 不再存在; 以下变量均按代码中给出的值定义: prompt_content = '# Bug Validator\n\nTarget result file: {result_json_rel}\nBug ID: {bug_id}\n\n---\n\n' + user_knowledge_section + base_content, prompt_filename = 'fm_agent/bug_validation/bug_validator_{bug_id}.md', prompt_path = os.path.join(proj_dir, prompt_filename), tmp_path = prompt_path + '.tmp', prompt = 'Follow the instructions in the attached file', command = build_llm_cli_command(OPENCODE_BUG_VALIDATION_MODEL, prompt, cwd=proj_dir, files=[prompt_path]), result_relp_path = 'fm_agent/bug_validation/{bug_id}.result.json', result_path = os.path.join(proj_dir,

result_relpath); 函数正常返回。(3) 没有异常发生, 也未提前返回 (resume 为 falsy、result_path 不存在, 或 JSON 解析抛出异常)。那么与场景 (2) 相同的目录和文件副作用成立, 与场景 (2) 相同的变量赋值成立, 并且此外: max_attempts = config.BUG_VALIDATION_MAX_RETRIES; for attempt in range(1, max_attempts + 1) 循环已进入, 第一次迭代以 attempt = 1 且 run_failed = False 开始; 第 80 行的 try 代码块已进入, 但尚未执行任何内部语句。形式化表述为: (exception_raised_at_line(l) | {49,55,56,57,59,60,61,62,63,64,65,66,68,69,70,71}) (exception early_return prompt_path_exists file_content(prompt_path) = prompt_content (v {prompt_content, prompt_filename, prompt_path, tmp_path, prompt, command, result_relpath, result_path} : assigned(v) = value_from_code) return_executed) (exception early_return same_effects_as_above max_attempts = config.BUG_VALIDATION_MAX_RETRIES attempt = 1 run_failed = False)。此处 early_return 定义为 resume file_exists(result_path) json_load_succeeded(result_path)。

- **Code evidence:** 第 41 行到第 80 行: 代码块只做了提示设置和跳过检查; 它从未分派 bug 验证代理, 从未将验证后的 bug 报告写入 <work_dir>/bug_validation/<bug_id>.md, 也从未在验证完成时按照规范要求写入结果标记文件 <work_dir>/bug_validation/<bug_id>.result.json。
- **Trigger condition:** 规范要求 resume 为 False 或结果标记文件不存在时, 分派一个 bug 验证代理, 从而生成 bug 报告和结果标记文件。条件 A 描述了以下后状态: 要么发生异常 (未分派), 要么函数提前返回 (跳过), 要么进入了重试循环但未执行任何分派。在这些场景中, 代码均未执行所要求的副作用。
- **Validator trigger:** 当 resume=False 且不存在结果标记文件时, 函数通过 run_opencode_traced 正确地分派了一个 bug 验证代理——该验证为误报, 原因是不完整的控制流分析停在了第 80 行。
- **Probe stdout:** NOT CONFIRMED — actual: 'dispatch called' | expected: 'dispatch called' | output_files= ['fm_agent/bug_validation/src--verification-py--_validate_single_bug.md', 'fm_agent/bug_validation/src--verification-py--_validate_single_bug.result.json'] | stage=bug_validation | summary=OpenCode bug validation for src--verification-py--_validate_single_bug

► SPEC sidecar (完整)

```
{
  "signature": "_validate_single_bug(result_json_rel, proj_dir, work_dir=None, resume=False,
bug_validator_path=None)",
  "pre_condition": "result_json_rel 是一个非空字符串, 标识从 proj_dir 到一个现有 JSON 文件的相对路径, 该文件包含一个推理判决, 至少包含一个 'verdict' 字段。proj_dir 是一个现有目录。如果提供了 bug_validator_path, 则是一个绝对或相对路径, 指向一个现有文件, 该文件内容规定了自定义的 bug 验证指令。如果 work_dir 为 None, 则默认为 proj_dir。",
  "post_condition": "当对应于 result_json_rel 的差异的结果标记文件在 <work_dir>/bug_validation/<bug_id>.result.json 处不存在, 或 resume 为 False 时, 会分派一个 bug 验证代理来评估 result_json_rel 中记录的规范与代码差异是否构成一个真实、可利用的 bug。只有当该差异被确认为可利用的 bug 时, 才会在 <work_dir>/bug_validation/<bug_id>.md 处生成一份经过验证的 bug 报告。该 bug 报告包含: 规范声称的、而代码违反了的行为断言, 从代码中观察到的实际行为, 附有行号引用的具体代码证据, 触发该违规的条件, 逐步重现指令, 一个演示该违规的探测脚本, 以及执行探测脚本的原始输出。当验证完成时, 会在 <work_dir>/bug_validation/<bug_id>.result.json 处写入一个结果标记文件。当 resume 为 True 且该结果标记文件已包含有效、可解析的 JSON 内容时, 不会重新验证, 也不会产生任何副作用。bug_id 是通过去除标准的验证结果路径前缀并将目录分隔符规范化后, 从 result_json_rel 唯一派生而得的。"
}
```

► INFO / callees sidecar (完整)

```
{
  "callees": [
    {
      "name": "function_id_from_result_path",
      "signature": "function_id_from_result_path(result_json_rel) -> str",
      "pre_condition": "result_json_rel 是一个非空字符串, 表示指向推理判决 JSON 文件的路径。",
      "post_condition": "返回一个从结果路径派生的函数标识符字符串, 适用于将验证结果与其源函数关联。"
    },
    {
      "name": "list_staged_domain_knowledge_relpaths",
      "signature": "list_staged_domain_knowledge_relpaths(work_dir) -> list[str]",
      "pre_condition": "work_dir 是 FM-Agent 的工作目录, 可能包含暂存的领域知识文件。",
      "post_condition": "返回当前在工作目录中暂存的所有用户提供的领域知识 Markdown 文件的相对路径列表 (相对于
```

```
work_dir)。当没有暂存的领域知识文件时，返回空列表。"
},
{
  "name": "format_domain_knowledge_bullets",
  "signature": "format_domain_knowledge_bullets(paths) -> str",
  "pre_condition": "paths 是一个由相对文件路径字符串组成的列表。",
  "post_condition": "返回一个 Markdown 格式的字符串，其中包含一个标题部分和所提供路径的 bullet 列表，适合嵌入到提示中作为上下文引用。当 paths 为空时，返回空字符串。"
},
{
  "name": "build_llm_cli_command",
  "signature": "build_llm_cli_command(model, prompt, cwd, files) -> list[str]",
  "pre_condition": "model 是一个非空字符串，标识 LLM 模型。prompt 是一个非空字符串。cwd 是一个现有目录。files 是一个要附加的文件路径列表。",
  "post_condition": "返回一个 CLI 参数字符串列表，当作为子进程执行时，会使用给定的模型、提示、工作目录和附加文件调用配置的 LLM 后端。"
},
{
  "name": "run_opencode_traced",
  "signature": "run_opencode_traced(proj_dir, work_dir, command, stage, function_ids, input_files, output_files, summary, metadata) -> None",
  "pre_condition": "proj_dir 是一个现有目录。command 是一个非空的 CLI 参数字符串列表。stage 是一个非空字符串，标识流水线阶段。output_files 是一个非空的相对路径列表（相对于 proj_dir），必须在成功执行后存在。",
  "post_condition": "以 proj_dir 为工作目录执行 command 子进程。生成一个结构化的追踪事件，记录执行元数据、时间、输入/输出文件引用和退出代码。当子进程以非零代码退出时，引发 CalledProcessError。output_files 中列出的输出文件必须在成功运行后存在于磁盘上。"
}
]
```

- Logic verification result（完整）
- Bug Validator 结果（完整）

```
{
  "id": "src--verification-py--_validate_single_bug",
  "source_file": "src/verification.py",
  "function_name": "_validate_single_bug",
  "confirmation_status": "not_confirmed",
  "attempts": 1,
  "probe_script": "fm_agent/bug_validation/probe_src--verification-py--_validate_single_bug.py",
  "detail_file": "fm_agent/bug_validation/src--verification-py--_validate_single_bug.md",
  "probe_stdout": "NOT CONFIRMED - 实际: 'dispatch called' | 预期: 'dispatch called' | output_files=[
'fm_agent/bug_validation/src--verification-py--_validate_single_bug.md', 'fm_agent/bug_validation/src--
verification-py--_validate_single_bug.result.json'] | stage=bug_validation | summary=OpenCode bug 验证, 针
对 src--verification-py--_validate_single_bug\\n",
  "trigger_summary": "当 resume=False 且不存在结果标记时，函数正确地通过 run_opencode_traced 调用了 bug 验证代理—
验证报错是由不完整的控制流分析在第 80 行停止造成的。"
}
```

► Reasoner 流程（3 条事件）

共 3 条直接 Reasoner 事件。原始 system/user/assistant 内容不再内嵌，使用每条事件后的载荷链接打开。

#	purpose	status	attempt	block	start	tokens
1	generate_block_post_condition	success	1	0/3	2026-07-28T04:11:29.985956Z	5490
2	generate_block_post_condition	success	1	1/3	2026-07-28T04:12:28.304371Z	6294
3	check_post_implies_spec	mismatch	1	1/3	2026-07-28T04:13:38.312810Z	6276

事件 1 — llm_af91b2775ee34a3abe6bed5f2e929438

► parsed_json (完整)

```
{
  "post_condition": "代码块成功执行后，以下条件成立：变量 `work_dir` 被设置为 `proj_dir`，如果原始的 `work_dir` 参数为 `None`；否则保留原值。`script_dir` 被设置为包含当前文件的目录的父目录，且该目录存在。`bug_id` 是一个字符串，通过从 `result_json_rel` 中去除任何前缀 `\"fm_agent/logic_verification_results/\"` 或 `os.path.join(\"fm_agent\", \"logic_verification_results\")+os.sep`，然后去除文件扩展名，并将所有目录分隔符替换为 `\"--\"` 计算得到。`function_id` 是 `function_id_from_result_path(result_json_rel)` 返回的字符串，该字符串是用于关联验证结果的函数标识符。`base_md_path` 被设置为 `bug_validator_path`，如果该参数为真值（字符串），否则设置为 `os.path.join(script_dir, \"md\", \"bug_validator.md\")`；该文件存在，其全部内容被读入 `base_content`。列表 `user_knowledge_paths` 是调用 `list_staged_domain_knowledge_relpaths(work_dir)` 的结果，可能是一个由相对路径字符串组成的空列表。如果 `user_knowledge_paths` 非空，`user_knowledge_section` 包含一个 markdown 节标题和由 `format_domain_knowledge_bullets` 生成的这些路径的项目符号列表，其后跟随 `\"\\n\\n--\\n\\n\"`；如果列表为空，`user_knowledge_section` 是一个空字符串。\\n\\n形式化：\\n`(W_initial is None → work_dir = P_dir) ∧ (W_initial is not None → work_dir = W_initial)`\\n`script_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__))) ∧ exists(script_dir)`\\n`prefix1 = os.path.join(\"fm_agent\", \"logic_verification_results\") + os.sep`\\n`prefix2 = \"fm_agent/logic_verification_results/\"`\\n`parts = if P_rel.startswith(prefix1) then P_rel[len(prefix1):] else if P_rel.startswith(prefix2) then P_rel[len(prefix2):] else P_rel`\\n`bug_id = os.path.splitext(parts)[0].replace(os.sep, \"--\").replace(\"/\", \"\", \"--\")`\\n`function_id = function_id_from_result_path(P_rel)`\\n`base_md_path = (BVP if BVP else os.path.join(script_dir, \"md\", \"bug_validator.md\")) ∧ file_exists(base_md_path) ∧ base_content = read_file_content(base_md_path)`\\n`user_knowledge_paths = list_staged_domain_knowledge_relpaths(work_dir)` (字符串列表)\\n`(user_knowledge_paths ≠ [] → user_knowledge_section = \"### User-Provided Domain Knowledge\\n\\nRead these Markdown files as additional context for intended behavior, terminology, data encodings, and invariants before validating the candidate bug:\\n\\n\" + format_domain_knowledge_bullets(user_knowledge_paths) + \"\\n\\n--\\n\\n\") ∧ (user_knowledge_paths = [] → user_knowledge_section = \"\")`\\n`
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_af91b2775ee34a3abe6bed5f2e929438_message_00_system.txt) · [user_prompt]

(fm_agent/trace/payloads/llm_af91b2775ee34a3abe6bed5f2e929438_message_01_user.txt) · [assistant_output]

(fm_agent/trace/payloads/llm_af91b2775ee34a3abe6bed5f2e929438_response.txt)

事件 2 — llm_ac4d30153b7a49e49041781b65139bd7

► parsed_json (完整)

```
{
  "post_condition": "代码块执行后，可能出现以下三种情形之一：(1) 发生了未处理的异常（例如，来自 os.makedirs、文件写入、os.replace 或 build_llm_cli_command）。在这种情况下，程序可能中断，只产生部分副作用；异常发生点之后的变量不再有任何保证。(2) 未发生异常，且函数因 resume 为真值、result_path 存在且文件能被成功读取并解析为 JSON 而在第 73 行提前返回。则以下条件成立：目录 os.path.join(work_dir, 'bug_validation') 存在；位于 prompt_path 的文件存在且其内容整体等于 prompt_content；临时文件 tmp_path 不再存在；以下变量已定义且值为代码中给出的值：prompt_content = '# Bug Validator\\n\\n**Target result file:** `{result_json_rel}`\\n**Bug ID:** `{bug_id}`\\n\\n---\\n\\n' + user_knowledge_section + base_content, prompt_filename = 'fm_agent/bug_validation/bug_validator_{bug_id}.md', prompt_path = os.path.join(proj_dir, prompt_filename), tmp_path = prompt_path + '.tmp', prompt = 'Follow the instructions in the attached file', command = build_llm_cli_command(OPENCODE_BUG_VALIDATION_MODEL, prompt, cwd=proj_dir, files=[prompt_path]), result_relp_path = 'fm_agent/bug_validation/{bug_id}.result.json', result_path = os.path.join(proj_dir, result_relp_path)；函数正常返回。(3) 未发生异常且未提前返回（resume 为假值，或 result_path 不存在，或 JSON 解析引发异常）。则与情形（2）相同的目录和文件副作用成立，与情形（2）相同的变量赋值成立，此外：max_attempts = config.BUG_VALIDATION_MAX_RETRIES；for 循环 for attempt in range(1, max_attempts + 1) 已进入，且第一次迭代以 attempt = 1 和 run_failed = False 开始；第 80 行的 try 块已进入，但尚未执行任何内部语句。形式化表示：(exception_raised_at_line(l) ∧ l ∈ {49,55,56,57,59,60,61,62,63,64,65,66,68,69,70,71}) ∨ (¬exception ∧ early_return ∧ prompt_path_exists ∧ file_content(prompt_path) = prompt_content ∧ (∀v ∈ {prompt_content, prompt_filename, prompt_path, tmp_path, prompt, command, result_relp_path, result_path} : assigned(v) = value_from_code) ∧ return_executed) ∨ (¬exception ∧ ¬early_return ∧ same_effects_as_above ∧ max_attempts = config.BUG_VALIDATION_MAX_RETRIES ∧ attempt = 1 ∧ run_failed = False)。这里 early_return 定义为 resume ∧ file_exists(result_path) ∧ json_load_succeeded(result_path)。"
}
```

- 载荷: [system_prompt]

(fm_agent/trace/payloads/llm_ac4d30153b7a49e49041781b65139bd7_message_00_system.txt) · [user_prompt]
 (fm_agent/trace/payloads/llm_ac4d30153b7a49e49041781b65139bd7_message_01_user.txt) · [assistant_output]
 (fm_agent/trace/payloads/llm_ac4d30153b7a49e49041781b65139bd7_response.txt)

事件 3 — llm_1f3e0ef1ab7345cbb2fe3f2780b2e8f5

► parsed_json（完整）

```
{
  "verdict": "MISMATCH",
  "counterexample": "resume=False, result_json_rel='some/path/result.json', bug_id='some_func', work_dir='/tmp/work', proj_dir='/project', user_knowledge_section='', base_content='',
  "offending_statements": "第 41 行至第 80 行：代码块仅进行了提示设置和跳过检查；它从未调度 bug 验证代理，从未将验证后的 bug 报告写入 <work_dir>/bug_validation/<bug_id>.md，从未在验证完成后按规格要求写入结果标记文件 <work_dir>/bug_validation/<bug_id>.result.json。",
  "reason": "规格要求当 resume 为 False 或结果标记不存在时，应调度 bug 验证代理，从而产生 bug 报告和结果标记。条件 A 描述的后置状态要么发生异常（未调度），要么函数提前返回（跳过），要么进入重试循环但未执行任何调度。在这些情形中，代码均未执行所要求的副作用。"
}
```

- 载荷: [system_prompt](fm_agent/trace/payloads/llm_1f3e0ef1ab7345cbb2fe3f2780b2e8f5_message_00_system.txt)
 · [user_prompt](fm_agent/trace/payloads/llm_1f3e0ef1ab7345cbb2fe3f2780b2e8f5_message_01_user.txt) ·
 [assistant_output](fm_agent/trace/payloads/llm_1f3e0ef1ab7345cbb2fe3f2780b2e8f5_response.txt) **复核动作**

- 核对 probe 前置条件是否可由真实调用路径满足；若可满足，将其转为回归测试并修实现。

运行完整性备注

- [trace/events.jsonl](#) 仍有 1 行并发写坏；本报告只索引可解析事件。
- 原始提示词和响应保留在 [fm_agent/trace/payloads/](#)，每条 Reasoner 事件均提供链接。
- 报告拆分不会修改任何验证结果、源码、SPEC 或 probe。

